



NHIBERNATE

NHibernate Reference Documentation

Version: 5.7

Table of Contents

Preface	ix
1. Quick-start with IIS and Microsoft SQL Server	1
1.1. Getting started with NHibernate	1
1.2. First persistent class	2
1.3. Mapping the cat	2
1.4. Playing with cats	3
1.5. Finally	6
2. Architecture	7
2.1. Overview	7
2.2. Instance states	9
2.3. Contextual Sessions	10
3. ISessionFactory Configuration	11
3.1. Programmatic Configuration	11
3.2. Obtaining an ISessionFactory	12
3.3. User provided ADO.NET connection	12
3.4. NHibernate provided ADO.NET connection	12
3.5. Using a custom configuration provider	15
3.6. Optional configuration properties	15
3.6.1. SQL Dialects	25
3.6.2. Outer Join Fetching	28
3.6.3. Custom ICacheProvider	28
3.6.4. Query Language Substitution	28
3.6.5. System level optional properties	28
3.7. Logging	29
3.8. Implementing an INamingStrategy	30
3.9. XML Configuration File	30
4. Persistent Classes	32
4.1. A simple POCO example	32
4.1.1. Declare properties for persistent fields	32
4.1.2. Implement a default constructor	33
4.1.3. Provide an identifier property (optional)	33
4.1.4. Prefer non-sealed classes and virtual methods (optional)	33
4.2. Implementing inheritance	33
4.3. Implementing Equals() and GetHashCode()	33
4.4. Dynamic models	34
4.5. Tuplizers	36
4.6. Lifecycle Callbacks	37
4.7. IValidatable callback	38
5. Basic O/R Mapping	39
5.1. Mapping declaration	39
5.1.1. XML Namespace	39
5.1.2. hibernate-mapping	40
5.1.3. class	40
5.1.4. subselect	44
5.1.5. id	44
5.1.5.1. generator	45
5.1.5.2. Hi/Lo Algorithm	47
5.1.5.3. UUID Hex Algorithm	47

5.1.5.4. UUID String Algorithm	48
5.1.5.5. GUID Algorithms	48
5.1.5.6. Identity columns and Sequences	48
5.1.5.7. Assigned Identifiers	48
5.1.5.8. Primary keys assigned by triggers	49
5.1.5.9. Enhanced identifier generators	49
5.1.6. composite-id	50
5.1.7. discriminator	51
5.1.8. version (optional)	52
5.1.9. timestamp (optional)	53
5.1.10. property	53
5.1.11. many-to-one	57
5.1.12. one-to-one	59
5.1.13. natural-id	60
5.1.14. component, dynamic-component	61
5.1.15. properties	62
5.1.16. subclass	63
5.1.17. joined-subclass	63
5.1.18. union-subclass	64
5.1.19. join	65
5.1.20. key	66
5.1.21. column and formula elements	66
5.1.22. map, set, list, bag	67
5.1.23. import	67
5.1.24. any	67
5.2. NHibernate Types	68
5.2.1. Entities and values	68
5.2.2. Basic value types	69
5.2.3. Custom value types	74
5.3. Mapping a class more than once	75
5.4. SQL quoted identifiers	76
5.5. Modular mapping files	76
5.6. Generated Properties	76
5.7. Auxiliary Database Objects	77
5.8. Meta data	77
6. Collection Mapping	79
6.1. Persistent Collections	79
6.2. Mapping a Collection	80
6.2.1. Collection foreign keys	81
6.2.2. Collection elements	82
6.2.3. Indexed collections	82
6.2.4. Collections of Values and Many-To-Many Associations	83
6.2.5. One-To-Many Associations	85
6.3. Advanced collection mappings	86
6.3.1. Sorted Collections	86
6.3.2. Bidirectional Associations	87
6.3.3. Bidirectional associations with indexed collections	88
6.3.4. Ternary Associations	89
6.3.5. Using an <idbag>	89
6.3.6. Heterogeneous Associations	90
6.4. Lazy Initialization	90
6.5. Collection examples	91

7. Association Mappings	94
7.1. Introduction	94
7.2. Unidirectional associations	94
7.2.1. Many-to-one	94
7.2.2. One-to-one	94
7.2.3. One-to-many	95
7.3. Unidirectional associations with join tables	95
7.3.1. One-to-Many	96
7.3.2. Many-to-one	96
7.3.3. One-to-one	96
7.3.4. Many-to-many	97
7.4. Bidirectional associations	97
7.4.1. One-to-many / many-to-one	97
7.4.2. One-to-one	98
7.5. Bidirectional associations with join tables	99
7.5.1. One-to-many / many-to-one	99
7.5.2. One-to-one	100
7.5.3. Many-to-many	101
7.6. More complex association mappings	101
8. Component Mapping	103
8.1. Dependent objects	103
8.2. Collections of dependent objects	104
8.3. Components as IDictionary indices	105
8.4. Components as composite identifiers	105
8.5. Dynamic components	106
9. Inheritance Mapping	108
9.1. The Three Strategies	108
9.1.1. Table per class hierarchy	108
9.1.2. Table per subclass	109
9.1.3. Table per subclass, using a discriminator	109
9.1.4. Mixing table per class hierarchy with table per subclass	110
9.1.5. Table per concrete class	110
9.1.6. Table per concrete class, using implicit polymorphism	111
9.1.7. Mixing implicit polymorphism with other inheritance mappings	111
9.2. Limitations	112
10. Manipulating Persistent Data	114
10.1. Creating a persistent object	114
10.2. Loading an object	114
10.3. Querying	115
10.3.1. Scalar queries	116
10.3.2. The IQuery interface	117
10.3.3. Filtering collections	118
10.3.4. Criteria queries	118
10.3.5. Queries in native SQL	119
10.4. Updating objects	119
10.4.1. Updating in the same ISession	119
10.4.2. Updating detached objects	119
10.4.3. Reattaching detached objects	121
10.5. Deleting persistent objects	121
10.6. Flush	121
10.7. Checking dirtiness	122
10.8. Ending a Session	122

10.8.1. Flushing the Session	123
10.8.2. Committing the database transaction	123
10.8.3. Closing the ISession	123
10.9. Exception handling	124
10.10. Lifecycles and object graphs	124
10.11. Interceptors	125
10.12. Metadata API	125
11. Read-only entities	127
11.1. Making persistent entities read-only	127
11.1.1. Entities of immutable classes	128
11.1.2. Loading persistent entities as read-only	128
11.1.3. Loading read-only entities from an HQL query/criteria	129
11.1.4. Making a persistent entity read-only	130
11.2. Read-only affect on property type	130
11.2.1. Simple properties	131
11.2.2. Unidirectional associations	132
11.2.2.1. Unidirectional one-to-one and many-to-one	132
11.2.2.2. Unidirectional one-to-many and many-to-many	133
11.2.3. Bidirectional associations	133
11.2.3.1. Bidirectional one-to-one	133
11.2.3.2. Bidirectional one-to-many/many-to-one	134
11.2.3.3. Bidirectional many-to-many	134
12. Transactions And Concurrency	135
12.1. Configurations, Sessions and Factories	135
12.2. Threads and connections	135
12.3. Considering object identity	136
12.4. Optimistic concurrency control	136
12.4.1. Long session with automatic versioning	136
12.4.2. Many sessions with automatic versioning	137
12.4.3. Customizing automatic versioning	137
12.4.4. Application version checking	138
12.5. Session disconnection	138
12.6. Pessimistic Locking	139
12.7. Connection Release Modes	140
12.8. Transaction scopes (System.Transactions)	141
13. Interceptors and events	142
13.1. Interceptors	142
13.2. Event system	143
14. Batch processing	145
14.1. Batch inserts	145
14.2. The StatelessSession interface	146
14.3. DML-style operations	146
15. HQL: The Hibernate Query Language	149
15.1. Case Sensitivity	149
15.2. The from clause	149
15.3. Associations and joins	149
15.4. The select clause	150
15.5. Aggregate functions	151
15.6. Polymorphic queries	151
15.7. The where clause	152
15.8. Expressions	153
15.9. The order by clause	155

15.10. The group by clause	156
15.11. Sub-queries	156
15.12. HQL examples	157
15.13. Tips & Tricks	158
16. Criteria Queries	160
16.1. Creating an ICriteria instance	160
16.2. Narrowing the result set	160
16.3. Ordering the results	161
16.4. Associations	161
16.5. Join entities without association (Entity joins or ad hoc joins)	161
16.6. Dynamic association fetching	162
16.7. Example queries	162
16.8. Projections, aggregation and grouping	163
16.9. Detached queries and sub-queries	164
17. QueryOver Queries	165
17.1. Structure of a Query	165
17.2. Simple Expressions	165
17.3. Additional Restrictions	167
17.4. Associations	167
17.5. Join entities without association (Entity joins or ad hoc joins)	168
17.6. Aliases	168
17.7. Projections	169
17.8. Projection Functions	170
17.9. Entities Projection	170
17.10. Sub-queries	171
18. Linq Queries	172
18.1. Structure of a Query	172
18.2. Parameter types	173
18.3. Supported methods and members	173
18.3.1. Common methods	173
18.3.2. DateTime and DateTimeOffset	174
18.3.3. ICollection, non generic and generic	174
18.3.4. IDictionary, non generic and generic	174
18.3.5. Mathematical functions	175
18.3.6. Nullables	175
18.3.7. Strings	175
18.4. Future results	176
18.5. Fetching associations	176
18.6. Modifying entities inside the database	177
18.6.1. Inserting new entities	177
18.6.2. Updating entities	178
18.6.3. Deleting entities	179
18.7. Query cache	179
18.8. Extending the Linq to NHibernate provider	179
18.8.1. Adding SQL functions	179
18.8.2. Adding a custom generator	180
19. Native SQL	183
19.1. Using an ISQLQuery	183
19.1.1. Scalar queries	183
19.1.2. Entity queries	183
19.1.3. Handling associations and collections	184
19.1.4. Returning multiple entities	184

19.1.4.1. Alias and property references	185
19.1.5. Returning non-managed entities	186
19.1.6. Handling inheritance	186
19.1.7. Parameters	186
19.2. Named SQL queries	187
19.2.1. Using return-property to explicitly specify column/alias names	188
19.2.2. Using stored procedures for querying	189
19.2.2.1. Rules/limitations for using stored procedures	189
19.3. Custom SQL for create, update and delete	190
19.4. Custom SQL for loading	191
20. Filtering data	192
20.1. NHibernate filters	192
21. Improving performance	194
21.1. Fetching strategies	194
21.1.1. Working with lazy associations	194
21.1.2. Tuning fetch strategies	195
21.1.3. Single-ended association proxies	196
21.1.4. Initializing collections and proxies	197
21.1.5. Using batch fetching	198
21.1.6. Using subselect fetching	199
21.2. The Second Level Cache	199
21.2.1. Cache mappings	200
21.2.2. Strategy: read only	200
21.2.3. Strategy: read/write	200
21.2.4. Strategy: nonstrict read/write	200
21.2.5. Strategy: never	201
21.3. Managing the caches	201
21.4. The Query Cache	202
21.5. Understanding Collection performance	203
21.5.1. Taxonomy	203
21.5.2. Lists, maps, idbags and sets are the most efficient collections to update	203
21.5.3. Bags and lists are the most efficient inverse collections	204
21.5.4. One shot delete	204
21.6. Batch updates	205
21.7. Query batch	205
21.8. Future results	207
22. Toolset Guide	209
22.1. Schema Generation	209
22.1.1. Customizing the schema	209
22.1.2. Running the tool	210
23. Example: Parent/Child	212
23.1. A note about collections	212
23.2. Bidirectional one-to-many	212
23.3. Cascading lifecycle	213
23.4. Using cascading Update()	214
23.5. Conclusion	216
24. Example: Weblog Application	217
24.1. Persistent Classes	217
24.2. NHibernate Mappings	217
24.3. NHibernate Code	218
25. Example: Various Mappings	222
25.1. Employer/Employee	222

25.2. Author/Work	223
25.3. Customer/Order/Product	225
25.4. Miscellaneous example mappings	227
25.4.1. "Typed" one-to-one association	227
25.4.2. Composite key example	227
25.4.3. Many-to-many with shared composite key attribute	229
25.4.4. Content based discrimination	229
25.4.5. Associations on alternate keys	230
26. Best Practices	232
I. NHibernateContrib Documentation	234
Preface	CCXXXV
27. NHibernate.Caches	236
27.1. How to use a cache?	237
27.2. Using a custom configuration provider	238
27.3. SysCache Configuration	238
27.4. SysCache2 Configuration	239
27.4.1. Table-based Dependency	240
27.4.2. Command-Based Dependencies	240
27.4.3. Aggregate Dependencies	242
27.4.4. Additional Settings	242
27.5. EnyimMemcached Configuration	243
27.6. RtMemoryCache Configuration	243
27.7. NHibernate.Caches.StackExchangeRedis Configuration	244
27.8. CoreMemoryCache Configuration	247
27.9. CoreDistributedCache Configuration	248
27.9.1. Memcached distributed cache factory	249
27.9.2. Redis distributed cache factory	250
27.9.3. SQL Server distributed cache factory	250
27.9.4. Memory distributed cache factory	251
28. NHibernate.Mapping.Attributes	252
28.1. What's new?	252
28.2. How to use it?	253
28.3. Tips	254
28.4. Known issues and TODOs	256
28.5. Developer Notes	256

Preface

Working with object-oriented software and a relational database can be cumbersome and time consuming in today's enterprise environments. NHibernate is an object/relational mapping tool for .NET environments. The term object/relational mapping (*ORM*) refers to the technique of mapping a data representation from an object model to a relational data model with a SQL-based schema.

NHibernate not only takes care of the mapping from .NET classes to database tables (and from .NET data types to SQL data types), but also provides data query and retrieval facilities and can significantly reduce development time otherwise spent with manual data handling in SQL and ADO.NET.

NHibernate's goal is to relieve the developer from 95 percent of common data persistence related programming tasks. NHibernate may not be the best solution for data-centric applications that only use stored-procedures to implement the business logic in the database, it is most useful with object-oriented domain models and business logic in the .NET-based middle-tier. However, NHibernate can certainly help you to remove or encapsulate vendor-specific SQL code and will help with the common task of result set translation from a tabular representation to a graph of objects.

If you are new to NHibernate and Object/Relational Mapping or even .NET Framework, please follow these steps:

1. Read Chapter 1, *Quick-start with IIS and Microsoft SQL Server* for a 30 minute tutorial, using Internet Information Services (IIS) web server.
2. Read Chapter 2, *Architecture* to understand the environments where NHibernate can be used.
3. Use this reference documentation as your primary source of information. Consider reading *Hibernate in Action* [<https://www.manning.com/books/hibernate-in-action>] (java) or *NHibernate in Action* [<https://www.manning.com/books/nhibernate-in-action>] or *NHibernate 4.x Cookbook - Second Edition* [<https://www.packtpub.com/application-development/nhibernate-40-cookbook>] or *NHibernate 2 Beginner's Guide* [<https://www.packtpub.com/application-development/nhibernate-2-beginners-guide>] if you need more help with application design or if you prefer a step-by-step tutorial. Also visit <http://nhibernate.sourceforge.net/NHibernateEg/> for NHibernate tutorial with examples.
4. FAQs are answered on the NHibernate users group [<https://groups.google.com/forum/#!forum/nhusers>].
5. The Community Area on the NHibernate website [<https://nhibernate.info/>] is a good source for design patterns and various integration solutions (ASP.NET, Windows Forms).

If you have questions, use the NHibernate user forum [<https://groups.google.com/forum/#!forum/nhusers>]. We also provide a GitHub issue tracking system [<https://github.com/nhibernate/nhibernate-core/issues>] for bug reports and feature requests. If you are interested in the development of NHibernate, join the developer mailing list. If you are interested in translating this documentation into your language, contact us on the developer mailing list [<https://groups.google.com/forum/#!forum/nhibernate-development>].

Chapter 1. Quick-start with IIS and Microsoft SQL Server

1.1. Getting started with NHibernate

This tutorial explains a setup of NHibernate 5.0.0 within a Microsoft environment. The tools used in this tutorial are:

1. Microsoft Internet Information Services (IIS) - web server supporting ASP.NET.
2. Microsoft SQL Server 2012 - the database server. This tutorial uses the desktop edition (SQL Express), a free download from Microsoft. Support for other databases is only a matter of changing the NHibernate SQL dialect and driver configuration.
3. Microsoft Visual Studio .NET (at least 2013) - the development environment.

First, we have to create a new Web project. We use the name `QuickStart`. In the project, add a NuGet reference to NHibernate. Visual Studio will automatically copy the library and its dependencies to the project output directory. If you are using a database other than SQL Server, add a reference to its driver assembly to your project.

We now set up the database connection information for NHibernate. To do this, open the file `Web.config` automatically generated for your project and add configuration elements according to the listing below:

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <!-- Add this element -->
  <configSections>
    <section
      name="hibernate-configuration"
      type="NHibernate.Cfg.ConfigurationSectionHandler, NHibernate" />
  </configSections>

  <!-- Add this element -->
  <hibernate-configuration xmlns="urn:hibernate-configuration-2.2">
    <session-factory>
      <property name="dialect">NHibernate.Dialect.MsSql2012Dialect</property>
      <property name="connection.connection_string">
        Server=localhost\SQLEXPRESS;initial catalog=quickstart;Integrated Security=True
      </property>

      <mapping assembly="QuickStart" />
    </session-factory>
  </hibernate-configuration>

  <!-- Leave the other sections unchanged -->
  <system.web>
    ...
  </system.web>
</configuration>
```

The `<configSections>` element contains definitions of sections that follow and handlers to use to process their content. We declare the handler for the configuration section here. The `<hibernate-configuration>` section contains the configuration itself, telling NHibernate that we will use a Microsoft SQL Server 2012 database and connect to it through the specified connection string. The dialect is a required setting, databases differ in their interpretation of the SQL "standard". NHibernate will take care of the differences and comes bundled with dia-

lects for several major commercial and open source databases.

An `ISessionFactory` is NHibernate's concept of a single datastore, multiple databases can be used by creating multiple XML configuration files and creating multiple `Configuration` and `ISessionFactory` objects in your application.

The last element of the `<hibernate-configuration>` section declares `QuickStart` as the name of an assembly containing class declarations and mapping files. The mapping files contain the metadata for the mapping of the POCO class to a database table (or multiple tables). We'll come back to mapping files soon. Let's write the POCO class first and then declare the mapping metadata for it.

1.2. First persistent class

NHibernate works best with the Plain Old CLR Objects (POCOs, sometimes called Plain Ordinary CLR Objects) programming model for persistent classes. A POCO has its data accessible through the standard .NET property mechanisms, shielding the internal representation from the publicly visible interface:

```
namespace QuickStart
{
    public class Cat
    {
        public virtual string Id { get; set; }

        public virtual string Name { get; set; }

        public virtual char Sex { get; set; }

        public virtual float Weight { get; set; }
    }
}
```

NHibernate is not restricted in its usage of property types, all .NET types and primitives (like `string`, `char` and `DateTime`) can be mapped, including classes from the `System.Collections.Generic` namespace. You can map them as values, collections of values, or associations to other entities. The `Id` is a special property that represents the database identifier (primary key) of that class, it is highly recommended for entities like a `Cat`. NHibernate can use identifiers only internally, without having to declare them on the class, but we would lose some of the flexibility in our application architecture.

No special interface has to be implemented for persistent classes nor do we have to subclass from a special root persistent class. NHibernate also doesn't use any build time processing, such as IL manipulation, it relies solely on .NET reflection and runtime class enhancement. So, without any dependency in the POCO class on NHibernate, we can map it to a database table.

For the above mentioned runtime class enhancement to work, NHibernate requires that all public properties of an entity class are declared as `virtual`. It also requires a parameter-less constructor: if you add a constructor having parameters, make sure to add a parameter-less constructor too.

1.3. Mapping the cat

This tutorial directly uses xml mapping files. This is just one among many mapping solutions NHibernate provides, see Section 5.1, "Mapping declaration".

The `Cat.hbm.xml` mapping file contains the metadata required for the object/relational mapping. The metadata includes declaration of persistent classes and the mapping of properties (to columns and foreign key relationships to other entities) to database tables.

Please note that the `Cat.hbm.xml` file should be set to an embedded resource.

```
<?xml version="1.0" encoding="utf-8" ?>
<hibernate-mapping xmlns="urn:hibernate-mapping-2.2"
    namespace="QuickStart" assembly="QuickStart">

    <class name="Cat" table="Cat">

        <!-- A 32 hex character is our surrogate key. It's automatically
            generated by NHibernate with the UUID pattern. -->
        <id name="Id">
            <column name="CatId" sql-type="char(32)" not-null="true"/>
            <generator class="uuid.hex" />
        </id>

        <!-- A cat has to have a name, but it shouldn't be too long. -->
        <property name="Name">
            <column name="Name" length="16" not-null="true" />
        </property>
        <property name="Sex" />
        <property name="Weight" />
    </class>

</hibernate-mapping>
```

Every persistent class should have an identifier attribute (actually, only classes representing entities, not dependent value objects, which are mapped as components of an entity). This property is used to distinguish persistent objects: Two cats are equal if `catA.Id.Equals(catB.Id)` is true, this concept is called *database identity*. NHibernate comes bundled with various identifier generators for different scenarios (including native generators for database sequences, hi/lo identifier tables, and application assigned identifiers). We use the UUID generator (only recommended for testing, as integer surrogate keys generated by the database should be preferred) and also specify the column `CatId` of the table `cat` for the NHibernate generated identifier value (as a primary key of the table).

All other properties of `cat` are mapped to the same table. In the case of the `Name` property, we mapped it with an explicit database column declaration. This is especially useful when the database schema is automatically generated (as SQL DDL statements) from the mapping declaration with NHibernate's *SchemaExport* tool. All other properties are mapped using NHibernate's default settings, which is what you need most of the time. Here the specification of the table name with the attribute `table` is redundant, it default to the class name when not specified. The table `cat` in the database looks like this:

Column	Type	Modifiers
CatId	char(32)	not null, primary key
Name	nvarchar(16)	not null
Sex	nchar(1)	
Weight	real	

You should now create the database and this table manually, and later read Chapter 22, *Toolset Guide* if you want to automate this step with the *SchemaExport* tool. This tool can create a full SQL DDL, including table definition, custom column type constraints, unique constraints and indexes. If you are using SQL Server, you should also make sure the ASPNET user has permissions to use the database.

1.4. Playing with cats

We're now ready to start NHibernate's *ISession*. It is the *persistence manager* interface, we use it to store and retrieve cats to and from the database. But first, we've to get an *ISession* (NHibernate's unit-of-work) from the *ISessionFactory*:

```
ISessionFactory sessionFactory =
    new Configuration().Configure().BuildSessionFactory();
```

An `ISessionFactory` is responsible for one database and may only use one XML configuration file (`Web.config` or `hibernate.cfg.xml`). You can set other properties (and even change the mapping metadata) by accessing the `Configuration` *before* you build the `ISessionFactory` (it is immutable). Where do we create the `ISessionFactory` and how can we access it in our application?

An `ISessionFactory` is usually only built once, e.g. at start-up inside `Application_Start` event handler. This also means you should not keep it in an instance variable in your ASP.NET pages or MVC controllers, but in some other location. Furthermore, we need some kind of *Singleton*, so we can access the `ISessionFactory` easily in application code. The approach shown next solves both problems: configuration and easy access to a `ISessionFactory`.

We implement a `NHibernateHelper` helper class:

```
using System;
using System.Web;
using NHibernate;
using NHibernate.Cfg;

namespace QuickStart
{
    public sealed class NHibernateHelper
    {
        private const string CurrentSessionKey = "nhibernate.current_session";
        private static readonly ISessionFactory _sessionFactory;

        static NHibernateHelper()
        {
            _sessionFactory = new Configuration().Configure().BuildSessionFactory();
        }

        public static ISession GetCurrentSession()
        {
            var context = HttpContext.Current;
            var currentSession = context.Items[CurrentSessionKey] as ISession;

            if (currentSession == null)
            {
                currentSession = _sessionFactory.OpenSession();
                context.Items[CurrentSessionKey] = currentSession;
            }

            return currentSession;
        }

        public static void CloseSession()
        {
            var context = HttpContext.Current;
            var currentSession = context.Items[CurrentSessionKey] as ISession;

            if (currentSession == null)
            {
                // No current session
                return;
            }

            currentSession.Close();
            context.Items.Remove(CurrentSessionKey);
        }

        public static void CloseSessionFactory()
        {
            if (_sessionFactory != null)
            {
            }
        }
    }
}
```

```

        _sessionFactory.Close();
    }
}

```

This class does not only take care of the `ISessionFactory` with its static attribute, but also has code to remember the `ISession` for the current HTTP request.

An `ISessionFactory` is threadsafe, many threads can access it concurrently and request `ISessions`. An `ISession` is a non-threadsafe object that represents a single unit-of-work with the database. `ISessions` are opened by an `ISessionFactory` and are closed when all work is completed:

```

ISession session = NHibernateHelper.GetCurrentSession();
try
{
    using (ITransaction tx = session.BeginTransaction())
    {
        var princess = new Cat
        {
            Name = "Princess",
            Sex = 'F',
            Weight = 7.4f
        };

        session.Save(princess);
        tx.Commit();
    }
}
finally
{
    NHibernateHelper.CloseSession();
}

```

In an `ISession`, every database operation occurs inside a transaction that isolates the database operations (even read-only operations). We use NHibernate's `ITransaction` API to abstract from the underlying transaction strategy (in our case, ADO.NET transactions). Please note that the example above does not handle any exceptions.

Also note that you may call `NHibernateHelper.GetCurrentSession()`; as many times as you like, you will always get the current `ISession` of this HTTP request. You have to make sure the `ISession` is closed after your unit-of-work completes, either in `Application_EndRequest` event handler in your application class, or with a MVC action filter, or in a `HttpModule` before the HTTP response is sent. The nice side effect of the latter is easy lazy initialization: the `ISession` is still open when the view is rendered, so NHibernate can load uninitialized objects while you navigate the graph.

NHibernate has various methods that can be used to retrieve objects from the database. Nowadays the most standard way is using Linq:

```

using(var tx = session.BeginTransaction())
{
    var females = session
        .Query<Cat>()
        .Where(c => c.Sex == 'F')
        .ToList();
    foreach (var cat in females)
    {
        Console.Out.WriteLine("Female Cat: " + cat.Name);
    }

    tx.Commit();
}

```

If you use an older NHibernate, you may have to import the `NHibernate.Linq` namespace.

NHibernate also offers an object-oriented *query by criteria* API that can be used to formulate type-safe queries, the Hibernate Query Language (HQL), which is an easy to learn and powerful object-oriented extension to SQL, as well as a strongly-typed LINQ API which translates internally to HQL. NHibernate of course uses Db-commands and parameter binding for all SQL communication with the database. You may also use NHibernate's direct SQL query feature or get a plain ADO.NET connection from an `ISession` in rare cases.

Since NHibernate 5.0, the session and its queries IO bound methods have async counterparts. Each call to an async method must be awaited before further interacting with the session or its queries.

1.5. Finally

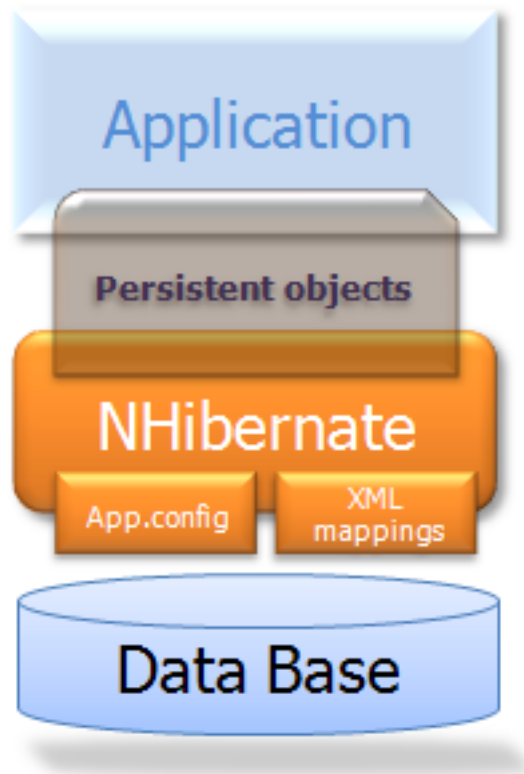
We only scratched the surface of NHibernate in this small tutorial. Please note that we don't include any ASP.NET specific code in our examples. You have to create an ASP.NET page yourself and insert the NHibernate code as you see fit.

Keep in mind that NHibernate, as a data access layer, is tightly integrated into your application. Usually, all other layers depend on the persistence mechanism. Make sure you understand the implications of this design.

Chapter 2. Architecture

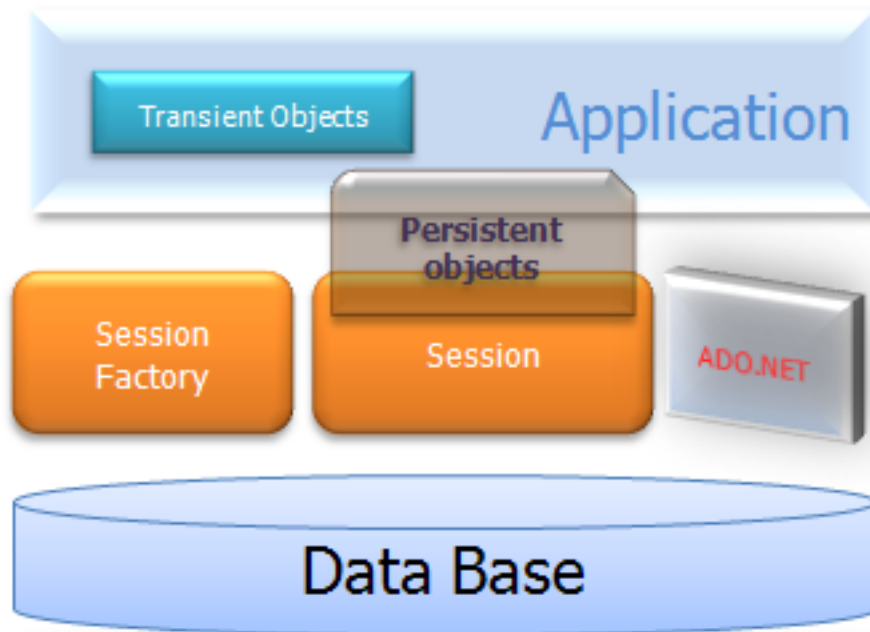
2.1. Overview

A (very) high-level view of the NHibernate architecture:

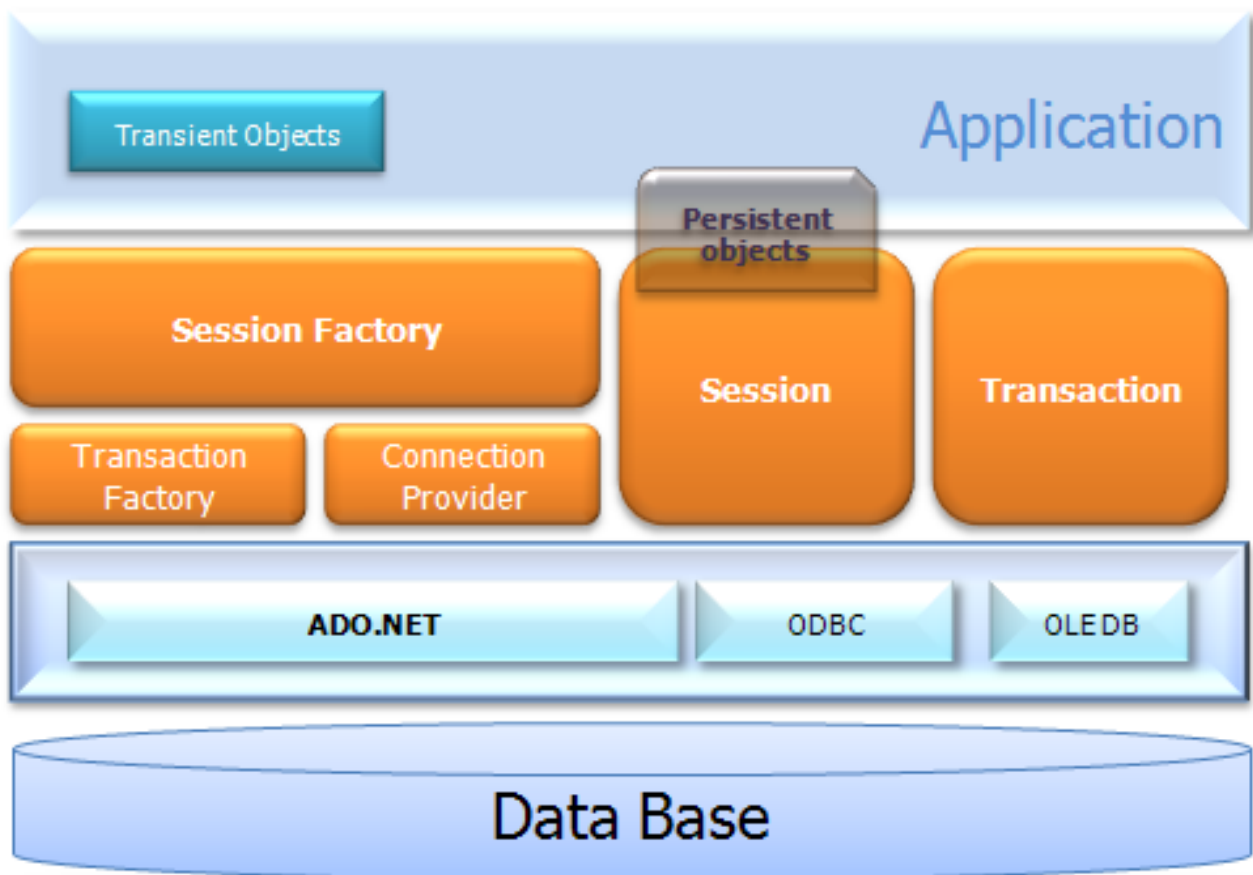


This diagram shows NHibernate using the database and configuration data to provide persistence services (and persistent objects) to the application.

We would like to show a more detailed view of the runtime architecture. Unfortunately, NHibernate is flexible and supports several approaches. We will show the two extremes. The "lite" architecture has the application provide its own ADO.NET connections and manage its own transactions. This approach uses a minimal subset of NHibernate's APIs:



The "full cream" architecture abstracts the application away from the underlying ADO.NET APIs and lets NHibernate take care of the details.



Here are some definitions of the objects in the diagrams:

ISessionFactory (`NHibernate.ISessionFactory`)

A threadsafe (immutable) cache of compiled mappings for a single database. A factory for `ISession` and a client of `IConnectionProvider`. Might hold an optional (second-level) cache of data that is reusable

between transactions, at a process- or cluster-level.

`ISession (NHibernate.ISession)`

A single-threaded, short-lived object representing a conversation between the application and the persistent store. Wraps an ADO.NET connection. Factory for `ITransaction`. Holds a mandatory (first-level) cache of persistent objects, used when navigating the object graph or looking up objects by identifier.

Persistent Objects and Collections

Short-lived, single threaded objects containing persistent state and business function. These might be ordinary POCO's, the only special thing about them is that they are currently associated with (exactly one) `ISession`. As soon as the `Session` is closed, they will be detached and free to use in any application layer (e.g. directly as data transfer objects to and from presentation).

Transient Objects and Collections

Instances of persistent classes that are not currently associated with a `ISession`. They may have been instantiated by the application and not (yet) persisted or they may have been instantiated by a closed `ISession`.

`ITransaction (NHibernate.ITransaction)`

(Optional) A single-threaded, short-lived object used by the application to specify atomic units of work. Abstracts application from underlying ADO.NET transaction. An `ISession` might span several `ITransactions` in some cases. Transaction scopes may be used instead.

`IConectionProvider (NHibernate.Connection.IConectionProvider)`

(Optional) A factory for ADO.NET connections and commands. Abstracts application from the concrete vendor-specific implementations of `DbConnection` and `DbCommand`. Not exposed to application, but can be extended/implemented by the developer.

`IDriver (NHibernate.Driver.IDriver)`

(Optional) An interface encapsulating differences between ADO.NET providers, such as parameter naming conventions and supported ADO.NET features.

`ITransactionFactory (NHibernate.Transaction.ITransactionFactory)`

(Optional) A factory for `ITransaction` instances. Not exposed to the application, but can be extended/implemented by the developer.

Given a "lite" architecture, the application bypasses the `ITransaction/ITransactionFactory` and/or `IConectionProvider` APIs to talk to ADO.NET directly.

2.2. Instance states

An instance of a persistent classes may be in one of three different states, which are defined with respect to a *persistence context*. The `NHibernate ISession` object is the persistence context:

transient

The instance is not, and has never been associated with any persistence context. It has no persistent identity (primary key value).

persistent

The instance is currently associated with a persistence context. It has a persistent identity (primary key value) and, perhaps, a corresponding row in the database. For a particular persistence context, *NHibernate guarantees* that persistent identity is equivalent to CLR identity (in-memory location of the object).

detached

The instance was once associated with a persistence context, but that context was closed, or the instance was serialized to another process. It has a persistent identity and, perhaps, a corresponding row in the database. For detached instances, NHibernate makes no guarantees about the relationship between persistent identity and CLR identity.

2.3. Contextual Sessions

Most applications using NHibernate need some form of "contextual" sessions, where a given session is in effect throughout the scope of a given context. However, across applications the definition of what constitutes a context is typically different; and different contexts define different scopes to the notion of current.

Starting with version 1.2, NHibernate added the `ISessionFactory.GetCurrentSession()` method. The processing behind `ISessionFactory.GetCurrentSession()` is pluggable. An extension interface (`NHibernate.Context.ICurrentSessionContext`) and a new configuration parameter (`current_session_context_class`) have been added to allow pluggability of the scope and context of defining current sessions.

See the API documentation for the `NHibernate.Context.ICurrentSessionContext` interface for a detailed discussion of its contract. It defines a single method, `CurrentSession()`, by which the implementation is responsible for tracking the current contextual session. Out-of-the-box, NHibernate comes with several implementations of this interface:

- `NHibernate.Context.AsyncLocalSessionContext` - current sessions are tracked by current asynchronous flow. You are responsible to bind and unbind an `ISession` instance with static methods of class `CurrentSessionContext`. Binding operations from inner flows will not be propagated to outer or siblings flows. Added in NHibernate 5.0.
- `NHibernate.Context.CallSessionContext` - current sessions are tracked by `CallContext`. You are responsible to bind and unbind an `ISession` instance with static methods of class `CurrentSessionContext`.
- `NHibernate.Context.ThreadStaticSessionContext` - current session is stored in a thread-static variable. This context supports multiple session factory only since NHibernate v5. You are responsible to bind and unbind an `ISession` instance with static methods of class `CurrentSessionContext`.
- `NHibernate.Context.WebSessionContext` - stores the current session in `HttpContext`. You are responsible to bind and unbind an `ISession` instance with static methods of class `CurrentSessionContext`.
- `NHibernate.Context.WcfOperationSessionContext` - current sessions are tracked by WCF `OperationContext`. You need to register the `wcfStateExtension` extension in WCF. You are responsible to bind and unbind an `ISession` instance with static methods of class `CurrentSessionContext`.
- `NHibernate.Context.ManagedWebSessionContext` - current sessions are tracked by `HttpContext`. Removed in NHibernate 4.0 - `NHibernate.Context.WebSessionContext` should be used instead. You are responsible to bind and unbind an `ISession` instance with static methods on this class, it never opens, flushes, or closes an `ISession` itself.

The `current_session_context_class` configuration parameter defines which `NHibernate.Context.ICurrentSessionContext` implementation should be used. Typically, the value of this parameter would just name the implementation class to use (including the assembly name); for the out-of-the-box implementations, however, there are corresponding short names: `async_local`, `call`, `thread_static`, `web` and `wcf_operation`, respectively.

Chapter 3. ISessionFactory Configuration

Because NHibernate is designed to operate in many different environments, there are a large number of configuration parameters. Fortunately, most have sensible default values and NHibernate is distributed with an example `App.config` file (found in `src\NHibernate.Test`) that shows the various options. You usually only have to put that file in your project and customize it.

3.1. Programmatic Configuration

An instance of `NHibernate.Cfg.Configuration` represents an entire set of mappings of an application's .NET types to a SQL database. The `Configuration` is used to build an (immutable) `ISessionFactory`. The mappings are compiled from various XML mapping files.

You may obtain a `Configuration` instance by instantiating it directly. Here is an example of setting up a data-store from mappings defined in two XML configuration files:

```
Configuration cfg = new Configuration()
    .AddFile("Item.hbm.xml")
    .AddFile("Bid.hbm.xml");
```

An alternative (sometimes better) way is to let NHibernate load a mapping file from an embedded resource:

```
Configuration cfg = new Configuration()
    .AddClass(typeof(NHibernate.Auction.Item))
    .AddClass(typeof(NHibernate.Auction.Bid));
```

Then NHibernate will look for mapping files named `NHibernate.Auction.Item.hbm.xml` and `NHibernate.Auction.Bid.hbm.xml` embedded as resources in the assembly that the types are contained in. This approach eliminates any hardcoded filenames.

Another alternative (probably the best) way is to let NHibernate load all of the mapping files contained in an `Assembly`:

```
Configuration cfg = new Configuration()
    .AddAssembly( "NHibernate.Auction" );
```

Then NHibernate will look through the assembly for any resources that end with `.hbm.xml`. This approach eliminates any hardcoded filenames and ensures the mapping files in the assembly get added.

If a tool like Visual Studio .NET or NAnt is used to build the assembly, then make sure that the `.hbm.xml` files are compiled into the assembly as `Embedded Resources`.

A `Configuration` also specifies various optional properties:

```
var props = new Dictionary<string, string>();
...
Configuration cfg = new Configuration()
    .AddClass(typeof(NHibernate.Auction.Item))
    .AddClass(typeof(NHibernate.Auction.Bind))
    .SetProperties(props);
```

A `Configuration` is intended as a configuration-time object, to be discarded once an `ISessionFactory` is built.

3.2. Obtaining an ISessionFactory

When all mappings have been parsed by the Configuration, the application must obtain a factory for ISession instances. This factory is intended to be shared by all application threads:

```
ISessionFactory sessions = cfg.BuildSessionFactory();
```

However, NHibernate does allow your application to instantiate more than one ISessionFactory. This is useful if you are using more than one database.

3.3. User provided ADO.NET connection

An ISessionFactory may open an ISession on a user-provided ADO.NET connection. This design choice frees the application to obtain ADO.NET connections wherever it pleases:

```
var conn = myApp.GetOpenConnection();
var session = sessions.OpenSession(conn);

// do some data access work
```

The application must be careful not to open two concurrent ISessions on the same ADO.NET connection!

3.4. NHibernate provided ADO.NET connection

Alternatively, you can have the ISessionFactory open connections for you. The ISessionFactory must be provided with ADO.NET connection properties in one of the following ways:

1. Pass an instance of IDictionary mapping property names to property values to Configuration.SetProperties().
2. Include <property> elements in a configuration section in the application configuration file. The section should be named hibernate-configuration and its handler set to NHibernate.Cfg.ConfigurationSectionHandler. The XML namespace of the section should be set to urn:hibernate-configuration-2.2.
3. Include <property> elements in hibernate.cfg.xml (discussed later).

If you take this approach, opening an ISession is as simple as:

```
ISession session = sessions.OpenSession(); // open a new Session
// do some data access work, an ADO.NET connection will be used on demand
```

All NHibernate property names and semantics are defined on the class NHibernate.Cfg.Environment. We will now describe the most important settings for ADO.NET connection configuration.

NHibernate will obtain (and pool) connections using an ADO.NET data provider if you set the following properties:

Table 3.1. NHibernate ADO.NET Properties

Property name	Purpose
connection.provider	The type of a custom IConnectionProvider implementation.

Property name	Purpose
	<i>eg. full.classname.of.ConnectionProvider</i> if the Provider is built into NHibernate, or <i>full.classname.of.ConnectionProvider</i>, assembly if using an implementation of <i>IConnectionProvider</i> not included in NHibernate. The default is <i>NHibernate.Connection.DriverConnectionProvider</i>.
<code>connection.driver_class</code>	The type of a custom <i>IDriver</i>, if using <i>DriverConnectionProvider</i>. <i>eg. full.classname.of.Driver</i> if the Driver is built into NHibernate, or <i>full.classname.of.Driver</i>, assembly if using an implementation of <i>IDriver</i> not included in NHibernate. This is usually not needed, most of the time the dialect will take care of setting the <i>IDriver</i> using a sensible default. See the API documentation of the specific dialect for the defaults.
<code>connection.connection_string</code>	Connection string to use to obtain the connection.
<code>connection.connection_string_name</code>	The name of the connection string (defined in <code><connectionStrings></code> configuration file element) to use to obtain the connection.
<code>connection.isolation</code>	Set the ADO.NET transaction isolation level. Check <i>System.Data.IsolationLevel</i> for meaningful values and the database's documentation to ensure that level is supported. <i>eg. Chaos ReadCommitted ReadUncommitted RepeatableRead Serializable Unspecified</i>
<code>connection.release_mode</code>	Specify when NHibernate should release ADO.NET connections. See Section 12.7, “Connection Release Modes”. <i>eg. auto (default) on_close after_transaction</i> Note that for <i>ISessions</i> obtained through <i>ISessionFactory.GetCurrentSession</i>, the <i>ICurrentSessionContext</i> implementation configured for use may control the connection release mode for those <i>ISessions</i>. See Section 2.3, “Contextual Sessions”.
<code>prepare_sql</code>	Specify to prepare <i>DbCommands</i> generated by NHibernate. Defaults to <code>false</code>. <i>eg. true false</i>
<code>command_timeout</code>	Specify the default timeout in seconds of <i>DbCommands</i> generated by NHibernate. Negative values disable it.

Property name	Purpose
	<i>eg. 30</i>
<code>adonet.batch_size</code>	Specify the batch size to use when batching update statements. Setting this to 0 (the default) disables the functionality. See Section 21.6, “Batch updates”. <i>eg. 20</i>
<code>order_inserts</code>	Enable ordering of insert statements for the purpose of more efficient batching. Defaults to <code>true</code> if batching is enabled, <code>false</code> otherwise. <i>eg. true false</i>
<code>order_updates</code>	Enable ordering of update statements for the purpose of more efficient batching. Defaults to <code>true</code> if batching is enabled, <code>false</code> otherwise. <i>eg. true false</i>
<code>adonet.batch_versioned_data</code>	If batching is enabled, specify that versioned data can also be batched. Requires a dialect which batcher correctly returns rows count. Defaults to <code>false</code> . <i>eg. true false</i>
<code>adonet.factory_class</code>	The class name of a <code>IBatcherFactory</code> implementation. This is usually not needed, most of the time the driver will take care of setting the <code>IBatcherFactory</code> using a sensible default according to the database capabilities. <i>eg. classname.of.BatcherFactory, assembly</i>
<code>adonet.wrap_result_sets</code>	Some database vendor data reader implementation have inefficient <code>columnName</code> -to- <code>columnIndex</code> resolution. Enabling this setting allows to wrap them in a data reader that will cache those resolutions. Defaults to <code>false</code> . <i>eg. true false</i>

This is an example of how to specify the database connection properties inside a `web.config`:

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <configSections>
    <section name="hibernate-configuration"
      type="NHibernate.Cfg.ConfigurationSectionHandler, NHibernate" />
  </configSections>

  <hibernate-configuration xmlns="urn:nhibernate-configuration-2.2">
    <session-factory>
      <property name="dialect">NHibernate.Dialect.MsSql2012Dialect</property>
```

```

    <property name="connection.connection_string">
      Server=(local);initial catalog=theDb;Integrated Security=SSPI
    </property>
    <property name="connection.isolation">ReadCommitted</property>
  </session-factory>
</hibernate-configuration>

  <!-- other app specific config follows -->
</configuration>

```

NHibernate relies on the ADO.NET data provider implementation of connection pooling.

You may define your own plug-in strategy for obtaining ADO.NET connections by implementing the interface `NHibernate.Connection.IConnectionProvider`. You may select a custom implementation by setting `connection.provider`.

3.5. Using a custom configuration provider

By default, NHibernate attempts to read the `hibernate-configuration` section through the `.Net ConfigurationManager`. Some environments do not support it, so NHibernate provides a way to set a custom configuration provider, through the `NHibernate.Cfg.ConfigurationProvider.Current` property.

To disable the configuration provider, in case you configure NHibernate entirely programmatically, set this property to `null`.

```
ConfigurationProvider.Current = null;
```

To provide directly the `System.Configuration.Configuration` instance to use, assign the `Current` property with an instance of `NHibernate.Cfg.SystemConfigurationProvider` built with your `Configuration` instance.

```
ConfigurationProvider.Current = new SystemConfigurationProvider(yourConfig);
```

You may also derive a custom provider from `NHibernate.Cfg.ConfigurationProvider`, implements its abstract methods, and assign an instance of your custom provider to the `NHibernate.Cfg.ConfigurationProvider.Current` property.

Changes of the `ConfigurationProvider.Current` property value are to be done very early in the application lifecycle, before any other call on a NHibernate API. Otherwise they may not be taken into account.

3.6. Optional configuration properties

There are a number of other properties that control the behaviour of NHibernate at runtime. All are optional and have reasonable default values.

Table 3.2. NHibernate Configuration Properties

Property name	Purpose
<code>dialect</code>	The class name of a NHibernate <code>Dialect</code> - enables certain platform dependent features. See Section 3.6.1, “SQL Dialects”. <i>eg. full.classname.of.Dialect, assembly</i>

Property name	Purpose
default_catalog	Qualify unqualified table names with the given catalog name in generated SQL. <i>eg. CATALOG_NAME</i>
default_schema	Qualify unqualified table names with the given schema/table-space in generated SQL. <i>eg. SCHEMA_NAME</i>
max_fetch_depth	Set a maximum "depth" for the outer join fetch tree for single-ended associations (one-to-one, many-to-one). A 0 disables default outer join fetching. <i>eg. recommended values between 0 and 3</i>
cache.use_second_level_cache	Enable the second level cache. Requires specifying a <code>cache.provider_class</code> . See Chapter 27, <i>NHibernate.Caches</i> . Defaults to false. <i>eg. true false</i>
cache.provider_class	The class name of a <code>ICacheProvider</code> implementation. <i>eg. classname.of.CacheProvider, assembly</i>
cache.use_minimal_puts	Optimize second-level cache operation to minimize writes, at the cost of more frequent reads (useful for clustered caches). Defaults to false. <i>eg. true false</i>
cache.use_query_cache	Enable the query cache, individual queries still have to be set cacheable. Defaults to false. <i>eg. true false</i>
cache.query_cache_factory	The class name of a custom <code>IQueryCacheFactory</code> implementation. Defaults to the built-in <code>StandardQueryCacheFactory</code> . <i>eg. classname.of.QueryCacheFactory, assembly</i>
cache.read_write_lock_factory	Specify the cache lock factory to use for read-write cache regions. Defaults to the built-in async cache lock factory. <i>eg. async, or sync, or classname.of.CacheLockFactory, assembly with custom implementation of ICacheReadWriteLockFactory</i> async uses a single writer multiple readers locking mechanism supporting asynchronous operations. sync uses a single access locking mechanism which will throw on asynchronous operations but may have better performances than the async provider for ap-

Property name	Purpose
	plications using the .Net Framework (4.8 and below).
cache.region_prefix	A prefix to use for second-level cache region names. <i>eg. prefix</i>
cache.default_expiration	The default expiration delay in seconds for cached entries, for providers supporting this setting. <i>eg. 300</i>
query.substitutions	Mapping from tokens in NHibernate queries to SQL tokens (tokens might be function or literal names, for example). <i>eg. hqlLiteral=SQL_LITERAL, hqlFunction=SQLFUNC</i>
query.default_cast_length	Set the default length used in casting when the target type is length bound and does not specify it. Defaults to 4000, automatically trimmed down according to dialect type registration. <i>eg. 255</i>
query.default_cast_precision	Set the default precision used in casting when the target type is decimal and does not specify it. Defaults to 28, automatically trimmed down according to dialect type registration. <i>eg. 19</i>
query.default_cast_scale	Set the default scale used in casting when the target type is decimal and does not specify it. Defaults to 10, automatically trimmed down according to dialect type registration. <i>eg. 5</i>
query.startup_check	Should named queries be checked during startup (the default is enabled). <i>eg. true false</i>
query.throw_never_cached	Should queries set as cacheable raise an error if they reference an entity using the cache Section 21.2.5, “Strategy: never” (the default is enabled). <i>eg. true false</i> Disabling this setting causes NHibernate to ignore the caching of such queries without raising an error. Furthermore NHibernate will log a warning on cacheable queries referencing an entity using the never cache strategy.
query.factory_class	The class name of a custom IQueryTranslatorFact-

Property name	Purpose
	ory implementation (HQL query parser factory). Defaults to the built-in <code>ASTQueryTranslatorFactory</code>. <i>eg.</i> <code>classname.of.QueryTranslatorFactory</code>, assembly
<code>query.linq_provider_class</code>	The class name of a custom <code>INhQueryProvider</code> implementation (LINQ provider). Defaults to the built-in <code>DefaultQueryProvider</code>. <i>eg.</i> <code>classname.of.LinqProvider</code>, assembly
<code>query.query_model_rewriter_factory</code>	The class name of a custom <code>IQueryModelRewriterFactory</code> implementation (LINQ query model rewriter factory). Defaults to null (no rewriter). <i>eg.</i> <code>classname.of.QueryModelRewriterFactory</code>, assembly
<code>query.pre_transformer_registrar</code>	The class name of the LINQ query pre-transformer registrar, implementing <code>IExpressionTransformerRegistrar</code>. Defaults to null (no registrar). <i>eg.</i> <code>classname.of.ExpressionTransformerRegistrar</code>, assembly
<code>linqtohql.generatorsregistry</code>	The class name of a custom <code>ILinqToHqlGeneratorsRegistry</code> implementation. Defaults to the built-in <code>DefaultLinqToHqlGeneratorsRegistry</code>. See Section 18.8.2, “Adding a custom generator”. <i>eg.</i> <code>classname.of.LinqToHqlGeneratorsRegistry</code>, assembly
<code>linqtohql.legacy_preevaluation</code>	Whether to use the legacy pre-evaluation or not in Linq queries. Defaults to true. <i>eg.</i> <code>true false</code> Legacy pre-evaluation is causing special properties or functions like <code>DateTime.Now</code> or <code>Guid.NewGuid()</code> to be always evaluated with the .Net runtime and replaced in the query by parameter values. The new pre-evaluation allows them to be converted to HQL function calls which will be run on the db side. This allows for example to retrieve the server time instead of the client time, or to generate UUIDs for each row instead of an unique one for all rows. The new pre-evaluation will likely be enabled by default in the next major version (6.0).
<code>linqtohql.fallback_on_preevaluation</code>	When the new pre-evaluation is enabled, should methods which translation is not supported by the

Property name	Purpose
	current dialect fallback to pre-evaluation? Defaults to false. <i>eg. true false</i> When this fallback option is enabled while legacy pre-evaluation is disabled, properties or functions like <code>DateTime.Now</code> or <code>Guid.NewGuid()</code> used in Linq expressions will not fail when the dialect does not support them, but will instead be pre-evaluated. When this fallback option is disabled while legacy pre-evaluation is disabled, properties or functions like <code>DateTime.Now</code> or <code>Guid.NewGuid()</code> used in Linq expressions will fail when the dialect does not support them. This option has no effect if the legacy pre-evaluation is enabled.
<code>sql_exception_converter</code>	The class name of a custom <code>ISQLExceptionConverter</code> implementation. Defaults to <code>Dialect.BuildSQLExceptionConverter()</code>. <i>eg. classname.of.SQLErrorConverter, assembly</i>
<code>escape_backslash_in_strings</code>	Indicates if the database needs to have backslash escaped in string literals. The default value is dialect dependant. That is false for most dialects. <i>eg. true false</i>
<code>show_sql</code>	Write all SQL statements to console. Defaults to false. <i>eg. true false</i>
<code>format_sql</code>	Log formatted SQL. Defaults to false. <i>eg. true false</i>
<code>use_sql_comments</code>	Generate SQL with comments. Defaults to false. <i>eg. true false</i>
<code>hbm2ddl.auto</code>	Automatically export schema DDL to the database when the <code>ISessionFactory</code> is created. With <code>create-drop</code>, the database schema will be dropped when the <code>ISessionFactory</code> is closed explicitly. <i>eg. create create-drop update validate</i>
<code>hbm2ddl.keywords</code>	Automatically import reserved/keywords from the database when the <code>ISessionFactory</code> is created.

Property name	Purpose
	<i>none</i> : disable any operation regarding RDBMS KeyWords (the default). <i>keywords</i> : imports all RDBMS KeyWords where the <code>Dialect</code> can provide the implementation of <code>IDatabaseSchema</code>. <i>auto-quote</i> : imports all RDBMS KeyWords and auto-quote all table-names/column-names. <i>eg.</i> none keywords auto-quote
hbm2ddl.throw_on_update	When hbm2ddl.auto is update, whether to throw or not on schema auto-update failures. Disabled by default. <i>eg.</i> true false
use_proxy_validator	Enables or disables validation of interfaces or classes specified as proxies. Enabled by default. <i>eg.</i> true false
proxyfactory.factory_class	The class name of a custom <code>IProxyFactoryFactory</code> implementation. Defaults to the built-in <code>StaticProxyFactoryFactory</code>. <i>eg.</i> classname.of.ProxyFactoryFactory, assembly
collectiontype.factory_class	The class name of a custom <code>ICollectionTypeFactory</code> implementation. Defaults to the built-in <code>DefaultCollectionTypeFactory</code>. <i>eg.</i> classname.of.CollectionTypeFactory, assembly
transaction.factory_class	The class name of a custom <code>ITransactionFactory</code> implementation. Defaults to the built-in <code>AdoNetWithSystemTransactionFactory</code>. <i>eg.</i> classname.of.TransactionFactory, assembly
transaction.use_connection_on_system_prepare	When a system transaction is being prepared, is using connection during this process enabled? Default is true, for supporting <code>FlushMode.Commit</code> with transaction factories supporting system transactions. But this requires enlisting additional connections, retaining disposed sessions and their connections until transaction end, and may trigger undesired transaction promotions to distributed. Set to false for disabling using connections from system transaction preparation, while still benefiting from <code>FlushMode.Auto</code> on querying.

Property name	Purpose
	See Section 12.8, “Transaction scopes (System.Transactions)”. <i>eg. true false</i>
<code>transaction.system_completion_lock_timeout</code>	Timeout duration in milliseconds for the system transaction completion lock. When a system transaction completes, it may have its completion events running on concurrent threads, after scope disposal. This occurs when the transaction is distributed. This notably concerns <code>ISessionImplementor.AfterTransactionCompletion(bool, ITransaction)</code>. NHibernate protects the session from being concurrently used by the code following the scope disposal with a lock. To prevent any application freeze, this lock has a default timeout of one second. If the application appears to require longer running transaction completion events, this setting allows to raise this timeout. -1 disables the timeout. <i>eg. 10000</i>
<code>transaction.ignore_session_synchronization_failures</code>	Whether session synchronisation failures occurring during finalizations of system transaction should be ignored or not. <code>false</code> by default. When a system transaction terminates abnormally, especially through timeouts, it may have its completion events running on concurrent threads while the session is still performing some processing. To prevent threading concurrency failures, NHibernate then wait for the session to end its processing, up to <code>transaction.system_completion_lock_timeout</code>. If the session processing is still ongoing afterwards, it will by default log an error, perform transaction finalization processing concurrently, then throw a synchronization error. This setting allows to disable that later throw. Disabling the throw can be useful if the used data provider has its own locking mechanism applied during transaction completion, preventing the session to end its processing. It may then be safe to ignore this synchronization failure. In case of threading concurrency failure, you may then need to raise <code>transaction.system_completion_lock_timeout</code>. <i>eg. true false</i>
<code>transaction.auto_join</code>	Should sessions check on every operation whether there is an ongoing system transaction or not, and enlist into it if any?

Property name	Purpose
	Default is true. It can also be controlled at session opening, with <code>ISessionFactory.WithOptions</code>. A session can also be instructed to explicitly join the current transaction by calling <code>ISession.JoinTransaction</code>. This setting has no effect when using a transaction factory that is not system transactions aware. <i>eg. false</i>
<code>default_flush_mode</code>	The default <code>FlushMode</code>, Auto when not specified. See Section 10.6, “Flush”. <i>eg. Manual Commit Auto Always</i>
<code>default_batch_fetch_size</code>	The default batch fetch size to use when lazily loading an entity or collection. Defaults to 1. See Section 21.1.5, “Using batch fetching”. <i>eg. 20</i>
<code>current_session_context_class</code>	The class name of an <code>ICurrentSessionContext</code> implementation. See Section 2.3, “Contextual Sessions”. <i>eg. classname.of.CurrentSessionContext, assembly</i>
<code>id.optimizer.pooled.prefer_lo</code>	When using an enhanced id generator and pooled optimizers (see Section 5.1.5.9, “Enhanced identifier generators”), prefer interpreting the database value as the lower (lo) boundary. The default is to interpret it as the high boundary. <i>eg. true false</i>
<code>generate_statistics</code>	Enable statistics collection within <code>ISessionFactory.Statistics</code> property. Defaults to false. <i>eg. true false</i>
<code>track_session_id</code>	Set whether the session id should be tracked in logs or not. When true, each session will have a unique Guid that can be retrieved with <code>ISessionImplementor.SessionId</code>, otherwise <code>ISessionImplementor.SessionId</code> will be <code>Guid.Empty</code>. Session id is used for logging purpose and can also be retrieved on the static property <code>NHibernate.Impl.SessionIdLoggingContext.SessionId</code>, when tracking is enabled. Disabling tracking by setting <code>track_session_id</code> to false increases performances. Default is true. <i>eg. true false</i>

Property name	Purpose
<code>sql_types.keep_datetime</code>	Since NHibernate v5.0 and if the dialect supports it, <code>DbType.DateTime2</code> is used instead of <code>DbType.DateTime</code>. This may be disabled by setting <code>sql_types.keep_datetime</code> to <code>true</code>. Defaults to <code>false</code>. <i>eg. true false</i>
<code>firebird.disable_parameter_casting</code>	Firebird with <code>FirebirdSql.Data.FirebirdClient</code> may be unable to determine the type of parameters in many circumstances, unless they are explicitly casted in the SQL query. To avoid this trouble, the NHibernate <code>FirebirdClientDriver</code> parses SQL commands for detecting parameters in them and adding an explicit SQL cast around parameters which may trigger the issue. Defaults to <code>false</code>. For disabling this behavior, set this setting to <code>true</code>. <i>eg. true false</i>
<code>oracle.use_n_prefixed_types_for_unicode</code>	Oracle has a dual Unicode support model. Either the whole database use an Unicode encoding, and then all string types will be Unicode. In such case, Unicode strings should be mapped to non N prefixed types, such as <code>Varchar2</code>. This is the default. Or N prefixed types such as <code>NVarchar2</code> are to be used for Unicode strings, the others type are using a non Unicode encoding. In such case this setting needs to be set to <code>true</code>. See Implementing a Unicode Solution in the Database [https://docs.oracle.com/cd/B19306_01/server.102/b14225/ch6unicode.htm#CACHCAHF]. This setting applies only to Oracle dialects and ODP.Net managed or unmanaged driver. <i>eg. true false</i>
<code>oracle.use_binary_floating_point_types</code>	Set whether NHibernate should map .Net double and float to Oracle <code>BINARY_DOUBLE</code> and <code>BINARY_FLOAT</code> types or use Oracle <code>DOUBLE</code> and <code>FLOAT</code> types. Oracle 10g introduced <code>BINARY_DOUBLE</code> and <code>BINARY_FLOAT</code> types which are compatible with .NET double and float types, while Oracle <code>DOUBLE</code> and <code>FLOAT</code> are not. Oracle <code>DOUBLE</code> and <code>FLOAT</code> types do not conform to the IEEE standard as they are internally implemented as <code>NUMBER</code> type, which makes them an

Property name	Purpose
	exact numeric type. <i>eg.</i> true for using Oracle <code>BINARY_DOUBLE</code> and <code>BINARY_FLOAT</code> types false for using Oracle <code>DOUBLE</code> and <code>FLOAT</code> types. false by default. See ANSI SQL data types [https://docs.oracle.com/database/121/TTSQL/types.htm#TTSQL126].
or- <code>acle.suppress_decimal_invalid_cast_exception</code>	This setting specifies whether to suppress or not the <code>InvalidCastException</code> and return a rounded-off 28 precision value if the Oracle <code>NUMBER</code> value has more than a 28 precision. <i>eg.</i> true for suppressing the exception false for letting the exception be raised. false by default. See <code>SuppressGetDecimalInvalidCastException</code> [https://docs.oracle.com/en/database/oracle/oracle-database-19.3/odpnt/DataReaderSuppressGetDecimalInvalidCastException.html]. This setting works only with ODP.NET 19.10 or newer.
<code>odbc.explicit_datetime_scale</code>	This may need to be set to 3 if you are using the <code>OdbcDriver</code> with MS SQL Server 2008+. This is intended to work around issues like: <pre>System.Data.Odbc.OdbcException : ERROR [22008] [Microsoft][SQL Server Native Client 11.0] Datetime field overflow. Fractional second precision exceeds the scale specified in the parameter binding.</pre> <i>eg.</i> 3
<code>sqlite.binaryguid</code>	SQLite can store GUIDs in binary or text form, controlled by the <code>BinaryGuid</code> connection string parameter (default is 'true'). The <code>BinaryGuid</code> setting will affect how to cast GUID to string in SQL. NHibernate will attempt to detect this setting automatically from the connection string, but if the connection or connection string is being handled by the application instead of by NHibernate, you can use the <code>sqlite.binaryguid</code> NHibernate setting to override the behavior. The value can be true or false.
<code>nhibernate-logger</code>	The class name of an <code>ILoggerFactory</code> implementa-

Property name	Purpose
	tion. It allows using another logger than log4net. The default is not defined, which causes NHibernate to search for log4net assembly. If this search succeeds, NHibernate will log with log4net. Otherwise, its internal logging will be disabled. This is a very special system-level property. It can only be set through an appSetting [https://docs.microsoft.com/en-us/dotnet/framework/configure-apps/file-schema/appsettings/] named <code>nhibernate-logger</code> in the application configuration file. It cannot be set neither with <code>NHibernate.Cfg.Environment</code> class, nor be defined in the <code><hibernate-configuration></code> section of the application configuration file, nor supplied by using <code>Configuration.SetProperties</code>, nor set in the <code>hibernate.cfg.xml</code> configuration file. Alternatively to using the setting, a logger factory can be programmatically supplied by using <code>NHibernate.Logger.SetLoggersFactory</code>. <i>eg. <code>classname.of.LoggerFactory</code>, assembly</i>

3.6.1. SQL Dialects

You should always set the `dialect` property to the correct `NHibernate.Dialect.Dialect` subclass for your database. This is not strictly essential unless you wish to use native or sequence primary key generation or pessimistic locking (with, eg. `ISession.Lock()` or `IQuery.SetLockMode()`). However, if you specify a dialect, NHibernate will use sensible defaults for some of the other properties listed above, saving you the effort of specifying them manually.

When a dialect refers to a version of a database engine, that is the minimal version supported by the dialect. The dialect will support later versions of the engine, unless the newer engine version has some incompatible breaking changes (which happens seldomly). Dialects targeting higher versions of an engine are provided when new features in these versions may be used by the dialect, like enabling the use of sequences with SQL Server 2012 or later.

Table 3.3. NHibernate SQL Dialects (dialect)

RDBMS	Dialect	Remarks
DB2 for Linux, UNIX and Windows	<code>NHibernate.Dialect.DB2Dialect</code>	
DB2 for IBM i (formerly iSeries, formerly OS/400)	<code>NHibernate.Dialect.DB2400Dialect</code>	

RDBMS	Dialect	Remarks
Firebird	NHibernate.Dialect.FirebirdDialect	Set driver_class to NHibernate.Driver.FirebirdClientDriver for Firebird ADO.NET provider 2.0.
Informix	NHibernate.Dialect.InformixDialect	
Informix 9.40	NHibernate.Dialect.InformixDialect0940	
Informix 10.00	NHibernate.Dialect.InformixDialect1000	
Ingres	NHibernate.Dialect.IngresDialect	
Ingres 9	NHibernate.Dialect.Ingres9Dialect	
Microsoft SQL Server 7	NHibernate.Dialect.MsSql7Dialect	
Microsoft SQL Server 2000	NHibernate.Dialect.MsSql2000Dialect	
Microsoft SQL Server 2005	NHibernate.Dialect.MsSql2005Dialect	
Microsoft SQL Server 2008	NHibernate.Dialect.MsSql2008Dialect	
Microsoft SQL Azure Server 2008	NHibernate.Dialect.MsSqlAzure2008Dialect	
Microsoft SQL Server 2012	NHibernate.Dialect.MsSql2012Dialect	
Microsoft SQL Server Compact Edition	NHibernate.Dialect.MsSqlCeDialect	
Microsoft SQL Server Compact Edition 4.0	NHibernate.Dialect.MsSqlCe40Dialect	
MySQL 3 or 4	NHibernate.Dialect.MySQLDialect	
MySQL 5	NHibernate.Dialect.MySQL5Dialect	
MySQL 5 InnoDB	NHibernate.Dialect.MySQL5InnoDBDialect	
MySQL 5.5	NHibernate.Dialect.MySQL55Dialect	
MySQL 5.5 InnoDB	NHibernate.Dialect.MySQL55InnoDBDialect	
Oracle	NHibernate.Dialect.Oracle8iDialect	
Oracle 9i	NHibernate.Dialect.Oracle9iDialect	
Oracle 10g, Or-	NHibernate.Dialect.Oracle10gDialect	

RDBMS	Dialect	Remarks
acle 11g		
Oracle 12c	NHibernate.Dialect.Oracle12cDialect	
PostgreSQL	NHibernate.Dialect.PostgreSQLDialect	
PostgreSQL 8.1	NHibernate.Dialect.PostgreSQL81Dialect	This dialect supports FOR UPDATE NOWAIT available in PostgreSQL 8.1.
PostgreSQL 8.2	NHibernate.Dialect.PostgreSQL82Dialect	This dialect supports IF EXISTS keyword in DROP TABLE and DROP SEQUENCE available in PostgreSQL 8.2.
PostgreSQL 8.3	NHibernate.Dialect.PostgreSQL83Dialect	This dialect supports XML type.
SQLite	NHibernate.Dialect.SQLiteDialect	Set driver_class to NHibernate.Driver.SQLite20Driver for System.Data.SQLite provider for .NET 2.0. Due to the behavior of System.Data.SQLite [https://system.data.sqlite.org/index.html/tktview/44a0955ea344a777ffdbcc077831e1adc8b77a36] with DateTime, consider using DateTimeFormatString=yyyy-MM-dd HH:mm:ss.FFFFFFFF; in the SQLite connection string for preventing undesired time shifts with its default configuration.
Sybase Adaptive Server Anywhere 9	NHibernate.Dialect.SybaseASA9Dialect	
Sybase Adaptive Server Enterprise 15	NHibernate.Dialect.SybaseASE15Dialect	
Sybase Adaptive Server Enterprise 16	NHibernate.Dialect.SybaseASE16Dialect	
Sybase SQL Anywhere 10	NHibernate.Dialect.SybaseSQLAnywhere10Dialect	
Sybase SQL Anywhere 11	NHibernate.Dialect.SybaseSQLAnywhere11Dialect	
Sybase SQL Anywhere 12	NHibernate.Dialect.SybaseSQLAnywhere12Dialect	

Additional dialects may be available in the NHibernate.Dialect namespace.

3.6.2. Outer Join Fetching

If your database supports ANSI or Oracle style outer joins, *outer join fetching* might increase performance by limiting the number of round trips to and from the database (at the cost of possibly more work performed by the database itself). Outer join fetching allows a graph of objects connected by many-to-one, one-to-many or one-to-one associations to be retrieved in a single SQL `SELECT`.

By default, the fetched graph when loading an objects ends at leaf objects, collections, objects with proxies, or where circularities occur.

For a *particular association*, fetching may be configured (and the default behaviour overridden) by setting the `fetch` attribute in the XML mapping.

Outer join fetching may be disabled *globally* by setting the property `max_fetch_depth` to 0. A setting of 1 or higher enables outer join fetching for one-to-one and many-to-one associations which have been mapped with `fetch="join"`.

See Section 21.1, “Fetching strategies” for more information.

In NHibernate 1.0, `outer-join` attribute could be used to achieve a similar effect. This attribute is now deprecated in favor of `fetch`.

3.6.3. Custom `ICacheProvider`

You may integrate a process-level (or clustered) second-level cache system by implementing the interface `NHibernate.Cache.ICacheProvider`. You may select the custom implementation by setting `cache.provider_class`. See the Section 21.2, “The Second Level Cache” for more details.

3.6.4. Query Language Substitution

You may define new NHibernate query tokens using `query.substitutions`. For example:

```
query.substitutions true=1, false=0
```

would cause the tokens `true` and `false` to be translated to integer literals in the generated SQL.

```
query.substitutions toLowercase=LOWER
```

would allow you to rename the SQL `LOWER` function.

3.6.5. System level optional properties

Some properties are system-level properties. They can only be set manually by setting static properties of `NHibernate.Cfg.Environment` class or be defined in the `<hibernate-configuration>` section of the application configuration file, as direct sub-elements. These properties can neither be set using `Configuration.SetProperties` or the `hibernate.cfg.xml` configuration file, nor be set as `<session-factory>` properties.

Table 3.4. NHibernate system level properties

Property name	Purpose
bytecode-provider	Specifies the bytecode provider to use to optimize the use of reflection in NHibernate. Use <code>null</code> to disable the optimization completely, <code>lcg</code> to use built-in light-weight code generation, or the assembly qualified class name of a custom <code>IBYTECODEProvider</code> implementation. Defaults to <code>lcg</code>. <pre><bytecode-provider type="lcg null className" /></pre> You can also set this property by code by setting <code>NHibernate.Cfg.Environment.BytecodeProvider</code> before creating any <code>NHibernate.Cfg.Configuration</code> instance.
objects-factory	Specifies a custom objects factory to use for instantiating NHibernate dependencies. Use the assembly qualified class name of a custom <code>IObjectsFactory</code> implementation. <pre><objects-factory type="className" /></pre> You can also set this property by code by setting <code>NHibernate.Cfg.Environment.ObjectsFactory</code> before creating any <code>NHibernate.Cfg.Configuration</code> instance.
reflection-optimizer	Enables use of a runtime-generated class to set or get properties of an entity or component instead of using runtime reflection. The use of the reflection optimizer inflicts a certain startup cost on the application but should lead to better performance in the long run. Defaults to <code>true</code>. <pre><reflection-optimizer use="true false"/></pre> You can also set this property by code by setting <code>NHibernate.Cfg.Environment.UseReflectionOptimizer</code> before creating any <code>NHibernate.Cfg.Configuration</code> instance.

3.7. Logging

NHibernate logs various events using Apache log4net.

You may download log4net from <https://logging.apache.org/log4net/>, or install it with NuGet. To use log4net

you will need a `log4net` configuration section in the application configuration file. An example of the configuration section is distributed with NHibernate in the `src/NHibernate.Test` project.

We strongly recommend that you familiarize yourself with NHibernate's log messages. A lot of work has been put into making the NHibernate log as detailed as possible, without making it unreadable. It is an essential troubleshooting device. Also don't forget to enable SQL logging as described above (`show_sql`), it is your first step when looking for performance problems.

3.8. Implementing an `INamingStrategy`

The interface `NHibernate.Cfg.INamingStrategy` allows you to specify a "naming standard" for database objects and schema elements.

You may provide rules for automatically generating database identifiers from .NET identifiers or for processing "logical" column and table names given in the mapping file into "physical" table and column names. This feature helps reduce the verbosity of the mapping document, eliminating repetitive noise (TBL_ prefixes, for example). The default strategy used by NHibernate is quite minimal.

You may specify a different strategy by calling `Configuration.SetNamingStrategy()` before adding mappings:

```
ISessionFactory sf = new Configuration()
    .SetNamingStrategy(ImprovedNamingStrategy.Instance)
    .AddFile("Item.hbm.xml")
    .AddFile("Bid.hbm.xml")
    .BuildSessionFactory();
```

`NHibernate.Cfg.ImprovedNamingStrategy` is a built-in strategy that might be a useful starting point for some applications.

3.9. XML Configuration File

An alternative approach is to specify a full configuration in a file named `hibernate.cfg.xml`. This file can be used as a replacement for the `<hibernate-configuration>` sections of the application configuration file.

The XML configuration file is by default expected to be in your application directory. Here is an example:

```
<?xml version='1.0' encoding='utf-8'?>
<hibernate-configuration xmlns="urn:hibernate-configuration-2.2">

  <!-- an ISessionFactory instance -->
  <session-factory>

    <!-- properties -->
    <property name="connection.connection_string">
      Server=localhost;initial catalog=nhibernate;User Id=;Password=
    </property>
    <property name="show_sql">false</property>
    <property name="dialect">NHibernate.Dialect.MsSql2012Dialect</property>

    <!-- mapping files -->
    <mapping resource="NHibernate.Auction.Item.hbm.xml" assembly="NHibernate.Auction" />
    <mapping resource="NHibernate.Auction.Bid.hbm.xml" assembly="NHibernate.Auction" />

  </session-factory>

</hibernate-configuration>
```

Configuring NHibernate is then as simple as

```
ISessionFactory sf = new Configuration().Configure().BuildSessionFactory();
```

You can pick a different XML configuration file using

```
ISessionFactory sf = new Configuration()  
    .Configure("/path/to/config.cfg.xml")  
    .BuildSessionFactory();
```

Chapter 4. Persistent Classes

Persistent classes are classes in an application that implement the entities of the business problem (e.g. Customer and Order in an E-commerce application). Persistent classes have, as the name implies, transient and also persistent instance stored in the database.

NHibernate works best if these classes follow some simple rules, also known as the Plain Old CLR Object (POCO) programming model.

4.1. A simple POCO example

Most .NET applications require a persistent class representing felines.

```
using System;
using System.Collections.Generic;

namespace Eg
{
    public class Cat
    {
        long _id;
        // identifier

        public virtual long Id
        {
            get { return _id; }
            protected set { _id = value; }
        }

        public virtual string Name { get; set; }
        public virtual Cat Mate { get; set; }
        public virtual DateTime Birthdate { get; set; }
        public virtual float Weight { get; set; }
        public virtual Color Color { get; set; }
        public virtual ISet<Cat> Kittens { get; set; }
        public virtual char Sex { get; set; }

        // AddKitten not needed by NHibernate
        public virtual void AddKitten(Cat kitten)
        {
            kittens.Add(kitten);
        }
    }
}
```

There are four main rules to follow here:

4.1.1. Declare properties for persistent fields

cat declares properties for all the persistent fields. Many other *ORM tools* directly persist instance variables. We believe it is far better to decouple this implementation detail from the persistence mechanism. NHibernate persists properties, using their getter and setter methods.

Properties need *not* be declared public - NHibernate can persist a property with an internal, protected, protected internal or private visibility.

As shown in the example, both automatic properties and properties with a backing field are supported.

4.1.2. Implement a default constructor

cat has an implicit default (no-argument) constructor. All persistent classes must have a default constructor (which may be non-public) so NHibernate can instantiate them using `Activator.CreateInstance()`.

4.1.3. Provide an identifier property (optional)

cat has a property called `Id`. This property holds the primary key column of a database table. The property might have been called anything, and its type might have been any primitive type, `string` or `System.DateTime`. (If your legacy database table has composite keys, you can even use a user-defined class with properties of these types - see the section on composite identifiers below.)

The identifier property is optional. You can leave it off and let NHibernate keep track of object identifiers internally. However, for many applications it is still a good (and very popular) design decision.

What's more, some functionality is available only to classes which declare an identifier property:

- Cascaded updates (see Section 10.10, “Lifecycles and object graphs”)
- `ISession.SaveOrUpdate()`

We recommend you declare consistently-named identifier properties on persistent classes.

4.1.4. Prefer non-sealed classes and virtual methods (optional)

A central feature of NHibernate, *proxies*, depends upon the persistent class being non-sealed and all its public methods, properties and events declared as virtual. Another possibility is for the class to implement an interface that declares all public members.

You can persist sealed classes that do not implement an interface and don't have virtual members with NHibernate, but you won't be able to use proxies - which will limit your options for performance tuning.

4.2. Implementing inheritance

A subclass must also observe the first and second rules. It inherits its identifier property from `Cat`.

```
using System;
namespace Eg
{
    public class DomesticCat : Cat
    {
        public virtual string Name { get; set; }
    }
}
```

4.3. Implementing `Equals()` and `GetHashCode()`

You have to override the `Equals()` and `GetHashCode()` methods if you intend to mix objects of persistent classes (e.g. in an `ISet`).

This only applies if these objects are loaded in two different `ISessions`, as NHibernate only guarantees identity (`a == b` , the default implementation of `Equals()`) inside a single `ISession`!

Even if both objects *a* and *b* are the same database row (they have the same primary key value as their identifier), we can't guarantee that they are the same object instance outside of a particular *ISession* context.

The most obvious way is to implement `Equals()`/`GetHashCode()` by comparing the identifier value of both objects. If the value is the same, both must be the same database row, they are therefore equal (if both are added to an *ISet*, we will only have one element in the *ISet*). Unfortunately, we can't use that approach. NHibernate will only assign identifier values to objects that are persistent, a newly created instance will not have any identifier value! We recommend implementing `Equals()` and `GetHashCode()` using *Business key equality*.

Business key equality means that the `Equals()` method compares only the properties that form the business key, a key that would identify our instance in the real world (a *natural* candidate key):

```
public class Cat
{
    ...
    public override bool Equals(object other)
    {
        if (this == other) return true;

        Cat cat = other as Cat;
        if (cat == null) return false; // null or not a cat

        if (Name != cat.Name) return false;
        if (!Birthday.Equals(cat.Birthday)) return false;

        return true;
    }

    public override int GetHashCode()
    {
        unchecked
        {
            int result;
            result = Name.GetHashCode();
            result = 29 * result + Birthday.GetHashCode();
            return result;
        }
    }
}
```

Keep in mind that our candidate key (in this case a composite of name and birthday) has to be only valid for a particular comparison operation (maybe even only in a single use case). We don't need the stability criteria we usually apply to a real primary key!

4.4. Dynamic models

Note that the following features are currently considered experimental and may change in the near future.

Persistent entities don't necessarily have to be represented as POCO classes at runtime. NHibernate also supports dynamic models (using *Dictionaries* or *C# dynamic*). With this approach, you don't write persistent classes, only mapping files.

The following examples demonstrates the dynamic model feature. First, in the mapping file, an entity-name has to be declared instead of a class name:

```
<hibernate-mapping>
  <class entity-name="Customer">
    <id name="Id"
      type="long"
```

```

        column="ID">
        <generator class="sequence"/>
    </id>

    <property name="Name"
        column="NAME"
        type="string"/>

    <property name="Address"
        column="ADDRESS"
        type="string"/>

    <many-to-one name="Organization"
        column="ORGANIZATION_ID"
        class="Organization"/>

    <bag name="Orders"
        inverse="true"
        lazy="false"
        cascade="all">
        <key column="CUSTOMER_ID"/>
        <one-to-many class="Order"/>
    </bag>
</class>
</hibernate-mapping>

```

Note that even though associations are declared using target class names, the target type of an associations may also be a dynamic entity instead of a POCO.

At runtime we can work with Dictionaries:

```

using(ISession s = OpenSession())
using(ITransaction tx = s.BeginTransaction())
{
    // Create a customer
    var frank = new Dictionary<string, object>();
    frank["name"] = "Frank";

    // Create an organization
    var foobar = new Dictionary<string, object>();
    foobar["name"] = "Foobar Inc.";

    // Link both
    frank["organization"] = foobar;

    // Save both
    s.Save("Customer", frank);
    s.Save("Organization", foobar);

    tx.Commit();
}

```

Or we can work with dynamic:

```

using(var s = OpenSession())
using(var tx = s.BeginTransaction())
{
    // Create a customer
    dynamic frank = new ExpandoObject();
    frank.Name = "Frank";

    // Create an organization
    dynamic foobar = new ExpandoObject();
    foobar.Name = "Foobar Inc.";

    // Link both
    frank.Organization = foobar;
}

```

```
// Save both
s.Save("Customer", frank);
s.Save("Organization", foobar);

tx.Commit();
}
```

The advantages of a dynamic mapping are quick turnaround time for prototyping without the need for entity class implementation. However, you lose compile-time type checking and will very likely deal with many exceptions at runtime. Thanks to the NHibernate mapping, the database schema can easily be normalized and sound, allowing to add a proper domain model implementation on top later on.

A loaded dynamic entity can be manipulated as an `IDictionary`, an `IDictionary<string, object>` or a C# `dynamic`.

```
using(ISession s = OpenSession())
using(ITransaction tx = s.BeginTransaction())
{
    var customers = s
        .CreateQuery("from Customer")
        .List<IDictionary<string, object>>();
    ...
}
```

```
using System.Linq.Dynamic.Core;

...

using(ISession s = OpenSession())
using(ITransaction tx = s.BeginTransaction())
{
    var customers = s
        .Query<dynamic>("Customer")
        .OrderBy("Name")
        .ToList();
    ...
}
```

4.5. Tuplizers

`NHibernate.Tuple.Tuplizer`, and its sub-interfaces, are responsible for managing a particular representation of a piece of data, given that representation's `NHibernate.EntityMode`. If a given piece of data is thought of as a data structure, then a tuplizer is the thing which knows how to create such a data structure and how to extract values from and inject values into such a data structure. For example, for the POCO entity mode, the corresponding tuplizer knows how create the POCO through its constructor and how to access the POCO properties using the defined property accessors. There are two high-level types of Tuplizers, represented by the `NHibernate.Tuple.Entity.IEntityTypeTuplizer` and `NHibernate.Tuple.Component.IComponentTuplizer` interfaces. `IEntityTypeTuplizers` are responsible for managing the above mentioned contracts in regards to entities, while `IComponentTuplizers` do the same for components.

Users may also plug in their own tuplizers. Perhaps you require that a `IDictionary` / `DynamicObject` implementation other than NHibernate own implementation is used while in the dynamic-map entity-mode; or perhaps you need to define a different proxy generation strategy than the one used by default. Both would be achieved by defining a custom tuplizer implementation. Tuplizers definitions are attached to the entity or component mapping they are meant to manage. Going back to the example of our customer entity:

```
<hibernate-mapping>
  <class entity-name="Customer">
```

```

<!--
    Override the dynamic-map entity-mode
    tuplizer for the customer entity
-->
<tuplizer entity-mode="dynamic-map"
    class="CustomMapTuplizerImpl"/>

<id name="id" type="long" column="ID">
    <generator class="sequence"/>
</id>

<!-- other properties -->
...
</class>
</hibernate-mapping>

public class CustomMapTuplizerImpl : NHibernate.Tuple.Entity.DynamicMapEntityTuplizer
{
    // override the BuildInstantiator() method to plug in our custom map...
    protected override IInstantiator BuildInstantiator(
        NHibernate.Mapping.PersistentClass mappingInfo)
    {
        return new CustomMapInstantiator(mappingInfo);
    }

    private sealed class CustomMapInstantiator : NHibernate.Tuple.DynamicMapInstantiator
    {
        // override the generateMap() method to return our custom map...
        protected override IDictionary GenerateMap()
        {
            return new CustomMap();
        }
    }
}

```

4.6. Lifecycle Callbacks

Optionally, a persistent class might implement the interface `ILifecycle` which provides some callbacks that allow the persistent object to perform necessary initialization/cleanup after save or load and before deletion or update.

The NHibernate `IInterceptor` offers a less intrusive alternative, however.

```

public interface ILifecycle
{
    LifecycleVeto OnSave(ISession s);           (1)
    LifecycleVeto OnUpdate(ISession s);        (2)
    LifecycleVeto OnDelete(ISession s);        (3)
    void OnLoad(ISession s, object id);        (4)
}

```

- (1) `OnSave` - called just before the object is saved or inserted
- (2) `OnUpdate` - called just before an object is updated (when the object is passed to `ISession.Update()`)
- (3) `OnDelete` - called just before an object is deleted
- (4) `OnLoad` - called just after an object is loaded

`OnSave()`, `OnDelete()` and `OnUpdate()` may be used to cascade saves and deletions of dependent objects. This is an alternative to declaring cascaded operations in the mapping file. `OnLoad()` may be used to initialize transient properties of the object from its persistent state. It may not be used to load dependent objects since the `ISession` interface may not be invoked from inside this method. A further intended usage of `OnLoad()`, `OnSave()` and `OnUpdate()` is to store a reference to the current `ISession` for later use.

Note that `onUpdate()` is not called every time the object's persistent state is updated. It is called only when a transient object is passed to `ISession.update()`.

If `onSave()`, `onUpdate()` or `onDelete()` return `LifecycleVeto.Veto`, the operation is silently vetoed. If a `CallbackException` is thrown, the operation is vetoed and the exception is passed back to the application.

Note that `onSave()` is called after an identifier is assigned to the object, except when native key generation is used.

4.7. IValidatable callback

If the persistent class needs to check invariants before its state is persisted, it may implement the following interface:

```
public interface IValidatable
{
    void Validate();
}
```

The object should throw a `ValidationFailure` if an invariant was violated. An instance of `Validatable` should not change its state from inside `Validate()`.

Unlike the callback methods of the `ILifecycle` interface, `validate()` might be called at unpredictable times. The application should not rely upon calls to `validate()` for business functionality.

Chapter 5. Basic O/R Mapping

5.1. Mapping declaration

Object/relational mappings are defined in an XML document. The mapping document is designed to be readable and hand-editable. The mapping language is object-centric, meaning that mappings are constructed around persistent class declarations, not table declarations.

Note that, even though many NHibernate users choose to define XML mappings by hand, a number of tools exist to generate the mapping document, even transparently at runtime. This includes the `NHibernate.Mapping.Attributes` library which allows to directly annotate your entities with mapping declarations, various template-based code generators (CodeSmith, MyGeneration), the built-in `NHibernate.Mapping.ByCode` API available since NHibernate 3.2, or the Fluent NHibernate [<https://github.com/jagregory/fluent-nhibernate>] independent library.

Let's kick off with an example mapping:

```
<?xml version="1.0"?>
<hibernate-mapping xmlns="urn:nhibernate-mapping-2.2" assembly="Eg"
    namespace="Eg">

    <class name="Cat" table="CATS" discriminator-value="C">
        <id name="Id" column="uid" type="Int64">
            <generator class="hilo"/>
        </id>
        <discriminator column="subclass" type="Char"/>
        <property name="BirthDate" type="Date"/>
        <property name="Color" not-null="true"/>
        <property name="Sex" not-null="true" update="false"/>
        <property name="Weight"/>
        <many-to-one name="Mate" column="mate_id"/>
        <set name="Kittens">
            <key column="mother_id"/>
            <one-to-many class="Cat"/>
        </set>
        <subclass name="DomesticCat" discriminator-value="D">
            <property name="Name" type="String"/>
        </subclass>
    </class>

    <class name="Dog">
        <!-- mapping for Dog could go here -->
    </class>

</hibernate-mapping>
```

We will now discuss the content of the mapping document. We will mainly describe the document elements and attributes that are used by NHibernate at runtime. The mapping document also contains some extra optional attributes and elements that affect the database schemas exported by the schema export tool, for example, the column sub-element of a property.

5.1.1. XML Namespace

All XML mappings should declare the XML namespace shown. The actual schema definition may be found in the `src\nhibernate-mapping.xsd` file in the NHibernate distribution.

Tip: to enable IntelliSense for mapping and configuration files, copy the appropriate .xsd files as part of any

project in your solution, (*Build Action* can be "None") or as "Solution Files" or in your "Lib" folder and then add it to the *Schemas* property of the xml file. You can copy it in `<vs installation directory>\Xml\Schemas`, take care because you will have to deal with different version of the xsd for different versions of NHibernate.

5.1.2. hibernate-mapping

This element has several optional attributes. The *schema* and *catalog* attributes specify that tables referred to by this mapping belong to the named schema and/or catalog. If they are specified, table names will be qualified by the given schema and/or catalog name. If they are missing, table names will be unqualified. The *default-cascade* attribute specifies what cascade style should be assumed for properties and collections which do not specify a cascade attribute. The *auto-import* attribute lets us use unqualified class names in the query language, by default. The *assembly* and *namespace* attributes specify the assembly where persistent classes are located and the namespace they are declared in.

```
<hibernate-mapping
  schema="schemaName"                (1)
  catalog="catalogName"              (2)
  default-cascade="none|save-update..." (3)
  auto-import="true|false"           (4)
  assembly="Eg"                      (5)
  namespace="Eg"                    (6)
  default-access="field|property|field.camelcase..." (7)
  default-lazy="true|false"          (8)
/>
```

- (1) *schema* (optional): The name of a database schema.
- (2) *catalog* (optional): The name of a database catalog.
- (3) *default-cascade* (optional - defaults to none): A default cascade style.
- (4) *auto-import* (optional - defaults to true): Specifies whether we can use unqualified class names of classes in this mapping in the query language.
- (5) *assembly* and *namespace* (optional): Specify assembly and namespace to use for unqualified class names in the mapping document.
- (6) *default-access* (optional - defaults to property): The strategy NHibernate should use for accessing a property value. It can be a custom implementation of *IPropertyAccessor*.
- (7) *default-lazy* (optional - defaults to true): The default value for unspecified lazy attributes of class and collection mappings.

If you are not using *assembly* and *namespace* attributes, you have to specify fully-qualified class names, including the name of the assembly that classes are declared in.

If you have two persistent classes with the same (unqualified) name, you should set *auto-import="false"*. NHibernate will throw an exception if you attempt to assign two classes to the same "imported" name.

The *hibernate-mapping* element allows you to nest several persistent `<class>` mappings, as shown above. It is, however, good practice to map only a single persistent class, or a single class hierarchy, in one mapping file and name it after the persistent super-class. For example, `Cat.hbm.xml`, `Dog.hbm.xml`, or if using inheritance, `Animal.hbm.xml`.

5.1.3. class

You may declare a persistent class using the *class* element:

```
<class
  name="className"                (1)
  table="tableName"              (2)
```

```

discriminator-value="discriminatorValue"          (3)
mutable="true|false"                             (4)
schema="owner"                                    (5)
catalog="catalog"                                 (6)
proxy="proxyInterface"                           (7)
dynamic-update="true|false"                       (8)
dynamic-insert="true|false"                       (9)
select-before-update="true|false"                 (10)
polymorphism="implicit|explicit"                  (11)
where="arbitrary sql where condition"              (12)
persister="persisterClass"                        (13)
batch-size="N"                                    (14)
optimistic-lock="none|version|dirty|all"           (15)
lazy="true|false"                                 (16)
abstract="true|false"                             (17)
entity-name="entityName"                         (18)
check="arbitrary sql check condition"              (19)
subselect="SQL expression"                        (20)
schema-action="none|drop|update|export|validate|all|commaSeparatedValues" (21)
rowid="unused"                                    (22)
node="unused"                                     (23)
/>

```

- (1) name: The fully qualified .NET class name of the persistent class (or interface), including its assembly name.
- (2) table(optional - defaults to the unqualified class name): The name of its database table.
- (3) discriminator-value (optional, defaults to the class name): A value that distinguishes individual sub-classes, used for polymorphic behaviour. Acceptable values include null and not null.
- (4) mutable (optional - defaults to true): Specifies that instances of the class are (not) mutable.
- (5) schema (optional): Overrides the schema name specified by the root <hibernate-mapping> element.
- (6) catalog (optional): Overrides the catalog name specified by the root <hibernate-mapping> element.
- (7) proxy (optional): Specifies an interface to use for lazy initializing proxies. You may specify the name of the class itself.
- (8) dynamic-update (optional - defaults to false): Specifies that UPDATE SQL should be generated at runtime and can contain only those columns whose values have changed.
- (9) dynamic-insert (optional - defaults to false): Specifies that INSERT SQL should be generated at runtime and contain only the columns whose values are not null.
- (10) select-before-update (optional - defaults to false): Specifies that NHibernate should *never* perform an SQL UPDATE unless it is certain that an object is actually modified. In certain cases (actually, only when a transient object has been associated with a new session using update()), this means that NHibernate will perform an extra SQL SELECT to determine if an UPDATE is actually required.
- (11) polymorphism (optional - defaults to implicit): Determines whether implicit or explicit query polymorphism is used.
- (12) where (optional) specify an arbitrary SQL WHERE condition to be used when retrieving objects of this class.
- (13) persister (optional): Specifies a custom IClassPersister.
- (14) batch-size (optional - defaults to 1): Specifies a "batch size" for fetching instances of this class by identifier. See Section 21.1.5, "Using batch fetching".
- (15) optimistic-lock (optional - defaults to version): Determines the optimistic locking strategy.
- (16) lazy (optional - defaults to true): Lazy fetching can be disabled by setting lazy="false".
- (17) abstract (optional - defaults to false): Used to mark abstract super-classes in <union-subclass> hierarchies.
- (18) entity-name (optional - defaults to the class name): NHibernate allows a class to be mapped multiple times, potentially to different tables. See Section 5.3, "Mapping a class more than once". It also allows entity mappings that are represented by dictionaries at the .Net level. In these cases, you should provide an explicit arbitrary name for the entity. See Section 4.4, "Dynamic models" for more information.
- (19) check (optional): An SQL expression used to generate a multi-row check constraint for automatic schema generation.

- (20) `subselect` (optional): Maps an immutable and read-only entity to a database sub-select. This is useful if you want to have a view instead of a base table. See Section 5.1.4, “subselect” for more information.
- (21) `schema-action` (optional): Specifies which schema actions should be generated for the class table by the schema export tool. Valid values are `none`, `drop`, `update`, `export`, `validate`, `all` and any combination of them separated by commas (,). Not specifying it is treated as `all`. See Section 22.1, “Schema Generation”.

(23) `rowid` and `node` (optional): These attributes have no usage in NHibernate. The `node` attribute is present on many more mapping elements, and has no usage on them too. It will be omitted from the documentation of these other elements.

It is perfectly acceptable for the named persistent class to be an interface. You would then declare implementing classes of that interface using the `<subclass>` element. You may persist any inner class. You should specify the class name using the standard form ie. Eg. `Foo+Bar`, Eg. Due to an HQL parser limitation inner classes can not be used in queries in NHibernate, unless assigning to them an `entity-name` without the `+` character.

Changes to immutable classes, `mutable="false"`, will not be persisted. This allows NHibernate to make some minor performance optimizations.

The optional `proxy` attribute enables customization of the generated proxy for lazy initialization of persistent instances of the class: by default, the proxy derives from the class. Specifying an interface causes the proxy to implement it and to use `System.Object` as its base class. This enables proxifying a sealed class or a class having non virtual members. The specified interface should declare all the public members of the persistent class. On `ISession.Load` and on many-to-one associations, NHibernate will initially return proxies. The actual persistent object will be loaded when a method or property of the proxy is invoked. See Section 21.1.4, “Initializing collections and proxies”.

Implicit polymorphism means that instances of the class will be returned by a query that names any superclass or implemented interface or the class and that instances of any subclass of the class will be returned by a query that names the class itself. *Explicit* polymorphism means that class instances will be returned only by queries that explicitly name that class and that queries that name the class will return only instances of subclasses mapped inside this `<class>` declaration as a `<subclass>` or `<joined-subclass>`. For most purposes the default, `polymorphism="implicit"`, is appropriate. Explicit polymorphism is useful when two different classes are mapped to the same table (this allows a "lightweight" class that contains a subset of the table columns).

The `persister` attribute lets you customize the persistence strategy used for the class. You may, for example, specify your own subclass of `NHibernate.Persister.EntityPersister` or you might even provide a completely new implementation of the interface `NHibernate.Persister.IClassPersister` that implements persistence via, for example, stored procedure calls, serialization to flat files or LDAP. See `NHibernate.DomainModel.CustomPersister` for a simple example (of "persistence" to a `Hashtable`).

Note that the `dynamic-update` and `dynamic-insert` settings are not inherited by subclasses and so may also be specified on the `<subclass>` or `<joined-subclass>` elements. These settings may increase performance in some cases, but might actually decrease performance in others. Use judiciously.

Use of `select-before-update` will usually decrease performance. It is very useful to prevent a database update trigger being called unnecessarily if you reattach a graph of detached instances to an `ISession`.

If you enable `dynamic-update`, you will have a choice of optimistic locking strategies:

- `version` check the version/timestamp columns
- `all` check all columns
- `dirty` check the changed columns
- `none` do not use optimistic locking

We very strongly recommend that you use version/timestamp columns for optimistic locking with NHibernate. This is the optimal strategy with respect to performance and is the only strategy that correctly handles modifications made outside of the session (ie. when `ISession.Update()` is used).

Beginning with NHibernate 1.2.0, version numbers start with 1, not 0 as in previous versions. This was done to allow using 0 as `unsaved-value` for the version property.

5.1.4. subselect

An alternative to mapping a class to table or view columns is to map a *query*. For that, we use the `<subselect>` element, which is mutually exclusive with `<subclass>`, `<joined-subclass>` and `<union-subclass>`. The subselect may also be specified as a `<class>` attribute. The content of the subselect element is a SQL query:

```
<subselect>
  <![CDATA[
    SELECT cat.ID, cat.NAME, cat.SEX, cat.MATE FROM cat
  ]]>
</subselect>
```

Usually, when mapping a query using subselect you will want to mark the class as not mutable (`mutable="false"`), unless you specify custom SQL for performing the UPDATE, DELETE and INSERT operations.

Also, it makes sense to force synchronization of the tables affected by the query, using one or more `<synchronize>` entries:

```
<subselect>
  <![CDATA[
    SELECT cat.ID, cat.NAME, cat.SEX, cat.MATE FROM cat
  ]]>
</subselect>
<synchronize table="cat"/>
```

This ensures that auto-flush happens correctly and that queries against the derived entity do not return stale data.

You then still have to declare the class id and properties.

5.1.5. id

Mapped classes *must* declare the primary key column of the database table. Most classes will also have a property holding the unique identifier of an instance. The `<id>` element defines the mapping from that property to the primary key column.

```
<id
  name="propertyName"                (1)
  type="typeName"                    (2)
  length="typeLength"                (3)
  column="columnName"                (4)
  unsaved-value="any|none|null|undefined|idValue" (5)
  access="field|property|nosetter|className" (6)
  generator="generatorClass">        (7)
  <generator class="generatorClass"/>
</id>
```

- (1) name (optional): The name of the identifier property.
- (2) type (optional): A name that indicates the NHibernate type.
- (3) length (optional): If the type takes a length and does not already specify it, its length.
- (4) column (optional - defaults to the property name): The name of the primary key column.
- (5) unsaved-value (optional - defaults to a "sensible" value): An identifier property value that indicates that

an instance is newly instantiated (unsaved), distinguishing it from transient instances that were saved or loaded in a previous session.

- (6) **access** (optional - defaults to `property`): The strategy NHibernate should use for accessing the property value.
- (7) **generator**: The name of a generator class to use for generating unique identifiers for instances of the persistent class. See the generator element. Specifying either the attribute or the element is required. The attribute takes precedence over the element.

If the `name` attribute is missing, it is assumed that the class has no identifier property.

The `unsaved-value` attribute is almost never needed in NHibernate.

There is an alternative `<composite-id>` declaration to allow access to legacy data with composite keys. We strongly discourage its use for anything else.

5.1.5.1. generator

The required generator names a .NET class used to generate unique identifiers for instances of the persistent class.

The generator can be declared using the `<generator>` child element. If any parameters are required to configure or initialize the generator instance, they are passed using `<param>` elements.

```
<id name="Id" type="Int64" column="uid">
  <generator class="NHibernate.Id.TableHiLoGenerator">
    <param name="table">uid_table</param>
    <param name="column">next_hi_value_column</param>
  </generator>
</id>
```

If no parameters are required, the generator can be declared using a generator attribute directly on the `<id>` element, as follows:

```
<id name="Id" type="Int64" column="uid" generator="native" />
```

All generators implement the interface `NHibernate.Id.IIdentifierGenerator`. This is a very simple interface; some applications may choose to provide their own specialized implementations. However, NHibernate provides a range of built-in implementations. There are shortcut names for the built-in generators:

increment

generates identifiers of any integral type that are unique only when no other process is inserting data into the same table. *Do not use in a cluster.*

counter, vm

generates identifiers of 64bits integral type constructed from the system time and a counter value. *Do not use in a cluster. May generate colliding identifiers in a bit less than one year.*

identity

supports identity columns in DB2, MySQL, MS SQL Server, Sybase and some other systems. The identifier returned by the database is converted to the property type using `Convert.ChangeType`. Any integral property type is thus supported. See Section 5.1.5.6, “Identity columns and Sequences”.

sequence

uses a sequence in DB2, PostgreSQL, Oracle and some other systems, or a generator in Firebird. The identifier returned by the database is converted to the property type using `Convert.ChangeType`. Any integral

property type is thus supported. See Section 5.1.5.6, “Identity columns and Sequences”.

`hilo`

uses a hi/lo algorithm to efficiently generate identifiers of any integral type, given a table and column (by default `hibernate_unique_key` and `next_hi` respectively) as a source of hi values. The hi/lo algorithm generates identifiers that are unique only for a particular database. *Do not use this generator with an user-supplied connection.* See Section 5.1.5.2, “Hi/Lo Algorithm”.

`seqhilo`

uses a hi/lo algorithm to efficiently generate identifiers of any integral type, given a named database sequence. See Section 5.1.5.2, “Hi/Lo Algorithm”.

`uuid.hex`

uses `System.Guid` and its `ToString(string format)` method to generate identifiers of type `string`. The length of the string returned depends on the configured format. See Section 5.1.5.3, “UUID Hex Algorithm”.

`uuid.string`

uses a new `System.Guid` to create a `byte[]` that is converted to a `string`. See Section 5.1.5.4, “UUID String Algorithm”.

`guid`

uses a new `System.Guid` as the identifier. It is *not equivalent* to the Hibernate `guid` generator. See Section 5.1.5.5, “GUID Algorithms”.

`guid.comb`

uses the algorithm to generate a new `System.Guid` described by Jimmy Nilsson in this article [<https://www.informit.com/articles/article.aspx?p=25862>]. See also Section 5.1.5.5, “GUID Algorithms”.

`guid.native`

uses the database server side `Guid` function.

`native`

picks identity, sequence or hilo depending upon the capabilities of the underlying database. See Section 5.1.5.6, “Identity columns and Sequences”.

`assigned`

lets the application to assign an identifier to the object before `Save()` is called. See Section 5.1.5.7, “Assigned Identifiers”.

`select`

retrieves a primary key, assigned by a database trigger, by selecting the row by some unique key and retrieving the primary key value. See Section 5.1.5.8, “Primary keys assigned by triggers”.

`foreign`

uses the identifier of another associated object. It is usually used in conjunction with a `<one-to-one>` primary key association.

`sequence-identity`

a generator which combines sequence generation with immediate retrieval by attaching an output parameter to the SQL command. In this respect it works much like ANSI-SQL identity generation.

`trigger-identity`

a generator which uses an output parameter to return the identifier generated by the insert on database server side.

enhanced-sequence

uses a sequence with database supporting them, otherwise uses a table for emulating the sequence. See Section 5.1.5.9, “Enhanced identifier generators”. Supports Section 5.1.5.9.1, “Identifier generator optimization”.

enhanced-table

uses a table like the hilo generator, but can handle multiple increment values per table. See Section 5.1.5.9, “Enhanced identifier generators”. Supports Section 5.1.5.9.1, “Identifier generator optimization”.

5.1.5.2. Hi/Lo Algorithm

The hilo and seqhilo generators provide two alternate implementations of the hi/lo algorithm, a favorite approach to identifier generation. The first implementation requires a "special" database table to hold the next available "hi" value. The second uses an Oracle-style sequence (where supported).

Unfortunately, you can't use hilo when supplying your own `DbConnection` to NHibernate. NHibernate must be able to fetch the "hi" value in a new transaction.

```
<id name="Id" type="Int64" column="cat_id">
  <generator class="hilo">
    <param name="table">tableName</param>
    <param name="column">columnName</param>
    <param name="schema">schemaName</param>
    <param name="catalog">catalogName</param>
    <param name="max_lo">100</param>
    <param name="where">arbitraryWhereClause</param>
  </generator>
</id>
```

table defaults to `hibernate_unique_key`, column defaults to `next_hi`, and `max_lo` defaults to 32767. schema, catalog and where are optional and have no default value.

You can use the optional where parameter to specify the row to use in a table. This is useful if you want to use a single table for your identifiers, with different rows for each table. Do not include the where keyword in its value. This is not handled by NHibernate schema generation tooling.

```
<id name="Id" type="Int64" column="cat_id">
  <generator class="seqhilo">
    <param name="sequence">hi_value</param>hibernate_sequence
    <param name="schema">schemaName</param>
    <param name="catalog">catalogName</param>
    <param name="max_lo">100</param>
    <param name="parameters">arbitrarySqlFragment</param>
  </generator>
</id>
```

sequence defaults to `hibernate_sequence`, and `max_lo` defaults to 9. schema, catalog and parameters are optional and have no default value.

You can use the optional parameters parameter to specify some SQL to be appended to the create sequence DDL.

5.1.5.3. UUID Hex Algorithm

```
<id name="Id" type="String" column="cat_id">
  <generator class="uuid.hex">
    <param name="format">format_value</param>
    <param name="separator">separator_value</param>
  </generator>
```

```
</id>
```

The UUID is generated by calling `Guid.NewGuid().ToString(format)`. The valid values for `format` are described in the MSDN documentation. The default separator is `-` and should rarely be modified. The `format` determines if the configured separator can replace the default separator used by the `format`.

5.1.5.4. UUID String Algorithm

The UUID is generated by calling `Guid.NewGuid().ToByteArray()` and then converting the `byte[]` into a `char[]`. The `char[]` is returned as a `String` consisting of 16 characters.

5.1.5.5. GUID Algorithms

The `guid` identifier is generated by calling `Guid.NewGuid()`. To address some of the performance concerns with using `Guids` as primary keys, foreign keys, and as part of indexes with MS SQL the `guid.comb` can be used. The benefit of using the `guid.comb` with other databases that support GUIDs has not been measured.

5.1.5.6. Identity columns and Sequences

For databases which support identity columns (DB2, MySQL, Sybase, MS SQL), you may use identity key generation. For databases that support sequences (DB2, Oracle, PostgreSQL, Interbase, McKoi, SAP DB) you may use sequence style key generation. Both these strategies require two SQL queries to insert a new object.

```
<id name="Id" type="Int64" column="uid">
  <generator class="sequence">
    <param name="sequence">uid_sequence</param>
  </generator>
</id>
```

```
<id name="Id" type="Int64" column="uid">
  <generator class="identity"/>
</id>
```

For cross-platform development, the native strategy will choose from the `identity`, `sequence` and `hilo` strategies, dependent upon the capabilities of the underlying database.

5.1.5.7. Assigned Identifiers

If you want the application to assign identifiers (as opposed to having NHibernate generate them), you may use the assigned generator. This special generator will use the identifier value already assigned to the object's identifier property. Be very careful when using this feature to assign keys with business meaning (almost always a terrible design decision).

Due to its inherent nature, entities that use this generator may, when saved via the `ISession's SaveOrUpdate()` method, cause NHibernate to query the database for checking if the entity exist or not.

To avoid this, either:

- ensure that the identifier `unsaved-value` is not defined or is set to `undefined`, and define an `unsaved-value` on a `<version>` or `<timestamp>` property mapping for the class. (Depending on the version type, its default `unsaved-value` will generally already be suitable.)
- specify to NHibernate if the object should be saved or updated by calling either the `Save()` or `Update()` method of the `ISession`.

- `set unsaved-value="none"` and explicitly `Save()` newly instantiated objects.
- `set unsaved-value="any"` and explicitly `Update()` previously persistent objects.
- implement `IIInterceptor.IsTransient()` for providing your own strategy for distinguishing newly instantiated objects. See Section 13.1, “Interceptors”.

For assigned identifiers, `unsaved-value` default value is `undefined`.

5.1.5.8. Primary keys assigned by triggers

NHibernate does not generate DDL with triggers. It is for legacy schemas only.

```
<id name="id" type="long" column="person_id">
  <generator class="select">
    <param name="key">socialSecurityNumber</param>
  </generator>
</id>
```

In the above example, there is a unique valued property named `socialSecurityNumber`. It is defined by the class, as a property. And a surrogate key named `person_id` has its value generated by a trigger. Its value will be retrieved by querying the entity table filtered by the key property.

If the class maps a natural id, key can be omitted. The natural id will be used when key is omitted.

5.1.5.9. Enhanced identifier generators

Starting with NHibernate release 3.3.0, there are two new generators which represent a re-thinking of two different aspects of identifier generation. The first aspect is database portability; the second is optimization. Optimization means that you do not have to query the database for every request for a new identifier value. These two new generators are intended to take the place of some of the named generators described above.

The first of these new generators is `NHibernate.Id.Enhanced.SequenceStyleGenerator` (short name `enhanced-sequence`) which is intended, firstly, as a replacement for the `sequence` generator and, secondly, as a better portability generator than `native`. This is because `native` generally chooses between `identity` and `sequence` which have largely different semantics that can cause subtle issues in applications eyeing portability. `NHibernate.Id.Enhanced.SequenceStyleGenerator`, however, achieves portability in a different manner. It chooses between a table or a sequence in the database to store its incrementing values, depending on the capabilities of the dialect being used. The difference between this and `native` is that table-based and sequence-based storage have the same exact semantic. In fact, sequences are exactly what NHibernate tries to emulate with its table-based generators. This generator has a number of configuration parameters:

- `sequence_name` (optional - defaults to `hibernate_sequence`): the name of the sequence or table to be used. schema and catalog parameters can be additionally used to qualify the sequence name.
- `initial_value` (optional - defaults to 1): the initial value to be retrieved from the sequence/table. In sequence creation terms, this is analogous to the clause typically named "STARTS WITH".
- `increment_size` (optional - defaults to 1): the value by which subsequent calls to the sequence/table should differ. In sequence creation terms, this is analogous to the clause typically named "INCREMENT BY".
- `force_table_use` (optional - defaults to `false`): should we force the use of a table as the backing structure even though the dialect might support sequence?
- `value_column` (optional - defaults to `next_val`): only relevant for table structures, it is the name of the column on the table which is used to hold the value.
- `optimizer` (optional - defaults to `none` if `increment_size` is 1): See Section 5.1.5.9.1, “Identifier generator optimization”

The second of these new generators is `NHibernate.Id.Enhanced.TableGenerator` (short name `enhanced-table`), which is intended, firstly, as a replacement for the table generator, even though it actually functions much more like `org.hibernate.id.MultipleHiLoPerTableGenerator` (not available in `NHibernate`), and secondly, as a re-implementation of `org.hibernate.id.MultipleHiLoPerTableGenerator` (not available in `NHibernate`) that utilizes the notion of pluggable optimizers. Essentially this generator defines a table capable of holding a number of different increment values simultaneously by using multiple distinctly keyed rows. This generator has a number of configuration parameters:

- `table_name` (optional - defaults to `hibernate_sequences`): the name of the table to be used. schema and catalog parameters can be additionally used to qualify the sequence name.
- `value_column_name` (optional - defaults to `next_val`): the name of the column on the table that is used to hold the value.
- `segment_column_name` (optional - defaults to `sequence_name`): the name of the column on the table that is used to hold the "segment key". This is the value which identifies which increment value to use.
- `segment_value` (optional - defaults to default unless `prefer_entity_table_as_segment_value` is true): The "segment key" value for the segment from which we want to pull increment values for this generator.
- `segment_value_length` (optional - defaults to 255): Used for schema generation; the column size to create this segment key column.
- `prefer_entity_table_as_segment_value` (optional - defaults to false): If `segment_value` is not specified, whether it should default to the entity table name or to default.
- `initial_value` (optional - defaults to 1): The initial value to be retrieved from the table.
- `increment_size` (optional - defaults to 1): The value by which subsequent calls to the table should differ.
- `optimizer` (optional - defaults to none if `increment_size` is 1): See Section 5.1.5.9.1, "Identifier generator optimization".

5.1.5.9.1. Identifier generator optimization

For identifier generators that store values in the database, it is inefficient for them to hit the database on each and every call to generate a new identifier value. Instead, you can group a bunch of them in memory and only hit the database when you have exhausted your in-memory value group. This is the role of the pluggable optimizers. Currently only the two enhanced generators (Section 5.1.5.9, "Enhanced identifier generators" support this operation.

- `none` (generally this is the default if no optimizer was specified): this will not perform any optimizations and hit the database for each and every request.
- `hilo`: applies a hi/lo algorithm around the database retrieved values. The values from the database for this optimizer are expected to be sequential. The values retrieved from the database structure for this optimizer indicates the "group number". The `increment_size` is multiplied by that value in memory to define a group "hi value".
- `pooled`: as with the case of `hilo`, this optimizer attempts to minimize the number of hits to the database. Here, however, we simply store the starting value for the "next group" into the database structure rather than a sequential value in combination with an in-memory grouping algorithm. Here, `increment_size` refers to the values coming from the database.
- `pooled-lo`: similar to `pooled`, except that it's the starting value of the "current group" that is stored into the database structure. Here, `increment_size` refers to the values coming from the database.

5.1.6. composite-id

```
<composite-id
  name="propertyName"
  class="className"
  unsaved-value="any|none"
  access="field|property|nosetter|className"
  mapped="true|false">
```

```

    <key-property name="propertyName" type="typeName" column="columnName"/>
    <key-many-to-one name="propertyName" class="className" column="columnName"/>
    ...
</composite-id>

```

For a table with a composite key, you may map multiple properties of the class as identifier properties. The `<composite-id>` element accepts `<key-property>` property mappings and `<key-many-to-one>` mappings as child elements.

```

<composite-id>
  <key-property name="MedicareNumber"/>
  <key-property name="Dependent"/>
</composite-id>

```

Your persistent class *must* override `Equals()` and `GetHashCode()` to implement composite identifier equality. It must also be marked with the `Serializable` attribute.

Unfortunately, this approach to composite identifiers means that a persistent object is its own identifier. There is no convenient "handle" other than the object itself. You must instantiate an instance of the persistent class itself and populate its identifier properties before you can `Load()` the persistent state associated with a composite key. We call this approach an embedded composite identifier, and discourage it for serious applications.

- `unsaved-value` (optional - default to `undefined`): controls how transient instances are considered. any assumes any transient instance is newly instantiated. In other words, the instance will be saved (inserted in the database). none assumes any transient instance is already persisted. In other words, the instance will be updated. `undefined` does not assume anything: NHibernate may query the database for determining if the entity is already persisted or not.
- `mapped` (optional - defaults to `false`): This attribute has no usage in NHibernate.

A much more convenient approach is to implement the composite identifier as a separate class, as in Section 8.4, "Components as composite identifiers". The attributes described below apply only to this alternative approach:

- `name` (optional, required for this approach): A property of component type that holds the composite identifier (see Section 8.4, "Components as composite identifiers").
- `access` (optional - defaults to `property`): The strategy NHibernate should use for accessing the property value.
- `class` (optional - defaults to the property type determined by reflection): The component class used as a composite identifier.

The second approach, an identifier component, is recommended for almost all applications.

5.1.7. discriminator

The `<discriminator>` element is required for polymorphic persistence using the table-per-class-hierarchy mapping strategy and declares a discriminator column of the table. The discriminator column contains marker values that tell the persistence layer what subclass to instantiate for a particular row. A restricted set of types may be used: `String`, `Char`, `Int32`, `Byte`, `Short`, `Boolean`, `YesNo`, `TrueFalse`.

```

<discriminator
  column="discriminatorColumn"           (1)
  type="discriminatorType"               (2)
  length="typeLength"                   (3)
  not-null="true|false"                  (4)
  force="true|false"                     (5)
  insert="true|false"                    (6)

```

```

    formula="arbitrary SQL expression"          (7)
  />

```

- (1) `column` (optional - defaults to `class`): the name of the discriminator column.
- (2) `type` (optional - defaults to `String`): a name that indicates the NHibernate type
- (3) `length` (optional): if the type takes a length and does not already specify it, its length.
- (4) `not-null` (optional - defaults to `true`): sets the column nullability for DDL generation.
- (5) `force` (optional - defaults to `false`): "force" NHibernate to specify allowed discriminator values even when retrieving all instances of the root class.
- (6) `insert` (optional - defaults to `true`): set this to `false` if your discriminator column is also part of a mapped composite identifier.
- (7) `formula` (optional): an arbitrary SQL expression that is executed when a type has to be evaluated. Allows content-based discrimination.

Actual values of the discriminator column are specified by the `discriminator-value` attribute of the `<class>` and `<subclass>` elements.

The `force` attribute is only useful if the table contains rows with "extra" discriminator values that are not mapped to a persistent class. This will not usually be the case.

Using the `formula` attribute, you can declare an arbitrary SQL expression that will be used to evaluate the type of a row:

```

<discriminator
  formula="case when CLASS_TYPE in ('a', 'b', 'c') then 0 else 1 end"
  type="Int32"/>

```

5.1.8. version (optional)

The `<version>` element is optional and indicates that the table contains versioned data. This is particularly useful if you plan to use *long transactions* (see Section 12.4, "Optimistic concurrency control").

```

<version
  column="versionColumn"          (1)
  name="propertyName"           (2)
  type="typeName"                 (3)
  access="field|property|nosetter|className" (4)
  unsaved-value="null|negative|undefined|value" (5)
  generated="never|always"        (6)
  insert="false|true"             (7)
/>

```

- (1) `column` (optional - defaults to the property name): The name of the column holding the version number.
- (2) `name`: The name of a property of the persistent class.
- (3) `type` (optional - defaults to `Int32`): The type of the version number.
- (4) `access` (optional - defaults to `property`): The strategy NHibernate should use for accessing the property value.
- (5) `unsaved-value` (optional - defaults to a "sensible" value): A version property value that indicates that an instance is newly instantiated (unsaved), distinguishing it from transient instances that were saved or loaded in a previous session. `undefined` specifies that nothing should be assumed from the version property. The identifier check on its own `unsaved-value` will take precedence, unless its own value is `undefined`.
- (6) `generated` (optional - defaults to `never`): Specifies that this version property value is actually generated by the database. See the discussion of generated properties.
- (7) `insert` (optional - defaults to `false`): This attribute has no usage in NHibernate.

Version may be of type `Int64`, `Int32`, `Int16`, `Ticks`, `Timestamp`, `TimeSpan`, `datetimeoffset`, ... (or their nullable counterparts in .NET 2.0 and higher). Any type implementing `IVersionType` is usable as a version.

5.1.9. timestamp (optional)

The optional `<timestamp>` element indicates that the table contains timestamped data. This is intended as an alternative to versioning. Timestamps are by nature a less safe implementation of optimistic locking. However, sometimes the application might use the timestamps in other ways.

```
<timestamp
  column="timestampColumn"           (1)
  name="propertyName"               (2)
  access="field|property|nosetter|className" (3)
  unsaved-value="null|undefined"      (4)
  generated="never|always"            (5)
  source="vm|db"                     (6)
/>
```

- (1) `column` (optional - defaults to the property name): The name of a column holding the timestamp.
- (2) `name`: The name of a property of .NET type `DateTime` of the persistent class.
- (3) `access` (optional - defaults to `property`): The strategy NHibernate should use for accessing the property value.
- (4) `unsaved-value` (optional - defaults to `null`): A timestamp property value that indicates that an instance is newly instantiated (unsaved), distinguishing it from transient instances that were saved or loaded in a previous session. `undefined` specifies that nothing should be assumed from the timestamp value. The identifier check on its own `unsaved-value` will take precedence, unless its own value is `undefined`.
- (5) `generated` (optional - defaults to `never`): Specifies that this timestamp property value is actually generated by the database on insert and updates. See the discussion of generated properties. It should not be confused with `source` value `db`, which mandates NHibernate to retrieve the timestamp from the database in a dedicated query, in order to get the new value to set in the next insert or update.
- (6) `source` (optional - defaults to `vm`): Where should NHibernate retrieve the timestamp value from? From the database, or from the current runtime? Database-based timestamps incur an overhead because NHibernate must hit the database in order to determine the "next value". It is safer to use in clustered environments. Not all dialects are known to support the retrieval of the database's current timestamp. Others may also be unsafe for usage in locking due to lack of precision.

Note that `<timestamp>` is equivalent to `<version type="datetime">`, and `<timestamp source="db">` is equivalent to `<version type="dbtimestamp">`.

5.1.10. property

The `<property>` element declares a persistent property of the class.

```
<property
  name="propertyName"               (1)
  column="columnName"               (2)
  type="typeName"                   (3)
  update="true|false"               (4)
  insert="true|false"               (4)
  formula="arbitrary SQL expression" (5)
  access="field|property|className" (6)
  optimistic-lock="true|false"      (7)
  generated="never|insert|always"    (8)
  lazy="true|false"                 (9)
  lazy-group="groupName"            (10)
  not-null="true|false"              (11)
  unique="true|false"                (12)
/>
```

```

unique-key="uniqueKeyName"           (13)
index="indexName"                    (14)
length="L"                           (15)
precision="P"                        (16)
scale="S"                            (17)
/>

```

- (1) name: the name of the property of your class.
- (2) column (optional - defaults to the property name): the name of the mapped database table column.
- (3) type (optional): a name that indicates the NHibernate type.
- (4) update, insert (optional - defaults to true): specifies that the mapped columns should be included in SQL UPDATE and/or INSERT statements. Setting both to false allows a pure "derived" property whose value is initialized from some other property that maps to the same column(s) or by a trigger or other application.
- (5) formula (optional): an SQL expression that defines the value for a *computed* property. Computed properties do not have a column mapping of their own.
- (6) access (optional - defaults to property): the strategy NHibernate should use for accessing the property value.
- (7) optimistic-lock (optional - defaults to true): specifies that updates to this property do or do not require acquisition of the optimistic lock. In other words, determines if a version increment should occur when this property is dirty.
- (8) generated (optional - defaults to never): specifies that this property value is actually generated by the database. See the discussion of generated properties.
- (9) lazy (optional - defaults to false): specifies that this property is lazy. A lazy property is not loaded when the object is initially loaded, unless the fetch mode has been overridden in a specific query. Values for lazy properties are loaded per lazy-group.

Having lazy properties causes instances of the entity to be loaded as proxies. These proxies ignore the class proxy setting and always derives from the persistent class, requiring its members to be overridable.

- (10) lazy-group (optional - defaults to DEFAULT): if the property is lazy, its lazy-loading group. When a lazy property is accessed, the other lazy properties of the lazy group are also loaded with it.
- (11) not-null (optional - defaults to false): sets the column nullability for DDL generation.
- (12) unique (optional - defaults to false): sets the column uniqueness for DDL generation. Use unique-key instead if the value is unique only in combination with other properties.
- (13) unique-key (optional): a logical name for an unique index for DDL generation. The column will be included in the index, along with other columns sharing the same unique-key logical name. The actual index name depends on the dialect.
- (14) index (optional): a logical name for an index for DDL generation. The column will be included in the index, along with other columns sharing the same index logical name. The actual index name depends on the dialect.
- (15) length (optional): if the type takes a length and does not already specify it, its length.
- (16) precision (optional): if the type takes a precision and does not already specify it, its precision.
- (17) scale (optional): if the type takes a scale and does not already specify it, its scale.

typeName could be:

1. The name of a NHibernate basic type (eg. Int32, String, Char, DateTime, Timestamp, Single, Byte[], Object, ...).
2. The name of a .NET type with a default basic type (eg. System.Int16, System.Single, System.Char, System.String, System.DateTime, System.Byte[], ...).
3. The name of an enumeration type (eg. Eg.Color, Eg).
4. The name of a serializable .NET type.
5. The class name of a custom type (eg. Ilflow.Type.MyCustomType).

Note that you have to specify full *assembly-qualified* names for all except basic NHibernate types (unless you set `assembly` and/or `namespace` attributes of the `<hibernate-mapping>` element).

NHibernate supports .NET 2.0 `Nullable` types. These types are mostly treated the same as plain non-`Nullable` types internally. For example, a property of type `Nullable<Int32>` can be mapped using `type="Int32"` or `type="System.Int32"`.

If you do not specify a type, NHibernate will use reflection upon the named property to take a guess at the correct NHibernate type. NHibernate will try to interpret the name of the return class of the property getter using rules 2, 3, 4 in that order. However, this is not always enough. In certain cases you will still need the `type` attribute. (For example, to distinguish between `NHibernateUtil.DateTime` and `NHibernateUtil.UtcDateTime`, or to specify a custom type.)

See also Section 5.2, “NHibernate Types”.

The `access` attribute lets you control how NHibernate will access the value of the property at runtime. The value of the `access` attribute should be text formatted as `access-strategy.naming-strategy`. The `.naming-strategy` is not always required.

Table 5.1. Access Strategies

Access Strategy Name	Description
<code>property</code>	The default implementation. NHibernate uses the <code>get</code> / <code>set</code> accessors of the property. No naming strategy should be used with this access strategy because the value of the <code>name</code> attribute is the name of the property.
<code>field</code>	NHibernate will access the field directly. NHibernate uses the value of the <code>name</code> attribute as the name of the field. This can be used when a property's getter and setter contain extra actions that you don't want to occur when NHibernate is populating or reading the object. If you want the name of the property and not the field to be what the consumers of your API use with HQL, then a naming strategy is needed.
<code>backfield</code>	NHibernate will access the field of an auto-property directly. This can be used when a property's setter is not accessible.
<code>nosetter</code>	NHibernate will access the field directly when setting the value and will use the <code>Property</code> when getting the value. This can be used when a property only exposes a <code>get</code> accessor because the consumers of your API can't change the value directly. A naming strategy is required because NHibernate uses the value of the <code>name</code> attribute as the property name and needs to be told what the name of the field is.

Access Strategy Name	Description
readonly	Access the mapped property through a Property get to get the value and do nothing to set the value. This is useful to allow calculated properties in the domain that will never be recovered from the DB but can be used for querying.
noop / none	Used to declare properties not represented at the poco level.
className	If NHibernate's built in access strategies are not what is needed for your situation then you can build your own by implementing the interface <code>NHibernate.Property.IPropertyAccessor</code> . The value of the access attribute should be an assembly-qualified name that can be loaded with <code>Activator.CreateInstance(string assemblyQualifiedName)</code> .

Table 5.2. Naming Strategies

Naming Strategy Name	Description
camelcase	The name attribute is converted to camel case to find the field. <code><property name="FooBar" ... ></code> uses the field <code>fooBar</code> .
camelcase-underscore	The name attribute is converted to camel case and prefixed with an underscore to find the field. <code><property name="FooBar" ... ></code> uses the field <code>_fooBar</code> .
camelcase-m-underscore	The name attribute is converted to camel case and prefixed with the character <code>m</code> and an underscore to find the field. <code><property name="FooBar" ... ></code> uses the field <code>m_fooBar</code> .
lowercase	The name attribute is converted to lower case to find the Field. <code><property name="FooBar" ... ></code> uses the field <code>foobar</code> .
lowercase-underscore	The name attribute is converted to lower case and prefixed with an underscore to find the Field. <code><property name="FooBar" ... ></code> uses the field <code>_foobar</code> .
pascalcase-underscore	The name attribute is prefixed with an underscore to find the field. <code><property name="FooBar" ... ></code> uses

Naming Strategy Name	Description
	the field <code>_FooBar</code> .
<code>pascalcase-m</code>	The name attribute is prefixed with the character <code>m</code> to find the field. <code><property name="FooBar" ... ></code> uses the field <code>mFooBar</code> .
<code>pascalcase-m-underscore</code>	The name attribute is prefixed with the character <code>m</code> and an underscore to find the field. <code><property name="FooBar" ... ></code> uses the field <code>m_FooBar</code> .

A powerful feature is derived properties. These properties are by definition read-only. The property value is computed at load time. You declare the computation as an SQL expression. This then translates to a `SELECT` clause subquery in the SQL query that loads an instance:

```
<property name="totalPrice"
  formula="( SELECT SUM (li.quantity*p.price) FROM LineItem li, Product p
            WHERE li.productId = p.productId
            AND li.customerId = customerId
            AND li.orderNumber = orderNumber )"/>
```

You can reference the entity table by not declaring an alias on a particular column. This would be `customerId` in the given example. You can also use the nested `<formula>` mapping element if you do not want to use the attribute.

The nested `<column>` mapping element can also be used instead of the `column` attribute.

5.1.11. many-to-one

An ordinary association to another persistent class is declared using a `many-to-one` element. The relational model is a many-to-one association. (It is really just an object reference.)

```
<many-to-one
  name="propertyName" (1)
  column="columnName" (2)
  class="className" (3)
  cascade="all|none|save-update|delete|delete-orphan|all- (4)delete-orphan"
  fetch="join|select" (5)
  lazy="proxy|no-proxy|false" (6)
  update="true|false" (7)
  insert="true|false" (7)
  property-ref="propertyNameFromAssociatedClass" (8)
  access="field|property|nosetter|className" (9)
  optimistic-lock="true|false" (10)
  not-found="ignore|exception" (11)
  entity-name="entityName" (12)
  formula="arbitrary SQL expression" (13)
  not-null="true|false" (14)
  unique="true|false" (15)
  unique-key="uniqueKeyName" (16)
  index="indexName" (17)
  foreign-key="foreignKeyName" (18)
  outer-join="auto|true|false" (19)
/>
```

- (1) `name`: The name of the property.
- (2) `column` (optional): The name of the column. This can also be specified by nested `<column>` element(s).

- (3) `class` (optional - defaults to the property type determined by reflection): The name of the associated class.
- (4) `cascade` (optional): Specifies which operations should be cascaded from the parent object to the associated object.
- (5) `fetch` (optional - defaults to `select`): Chooses between outer-join fetching or sequential select fetching. join takes precedence over the lazy attribute and causes the association to be eagerly fetched.
- (6) `lazy` (optional - defaults to `proxy`): By default, single point associations are proxied. `lazy="no-proxy"` specifies that the property should be fetched lazily when the instance property is first accessed. It works similarly to lazy properties, and causes the entity owning the association to be proxied instead of the association. `lazy="false"` specifies that the association will be eagerly fetched.
- (7) `update`, `insert` (optional - defaults to `true`): Specifies that the mapped columns should be included in SQL UPDATE and/or INSERT statements. Setting both to `false` allows a pure "derived" association whose value is initialized from some other property that maps to the same column(s) or by a trigger or other application.
- (8) `property-ref` (optional): The name of a property of the associated class that is joined to this foreign key. If not specified, the primary key of the associated class is used.
- (9) `access` (optional - defaults to `property`): The strategy NHibernate should use for accessing the property value.
- (10) `optimistic-lock` (optional - defaults to `true`): Specifies that updates to this property do or do not require acquisition of the optimistic lock. In other words, determines if a version increment should occur when this property is dirty.
- (11) `not-found` (optional - defaults to `exception`): Specifies how foreign keys that reference missing rows will be handled. `ignore` will treat a missing row as a null association.
- (12) `entity-name` (optional): The entity name of the associated class.
- (13) `formula` (optional): An SQL expression that defines the value for a *computed* foreign key.
- (14) `not-null` (optional - defaults to `false`): Enables the DDL generation of a nullability constraint for the foreign key column.
- (15) `unique` (optional - defaults to `false`): Enables the DDL generation of an unique constraint for the foreign-key column.
- (16) `unique-key` (optional): A logical name for an unique index for DDL generation. The column will be included in the index, along with other columns sharing the same unique-key logical name. The actual index name depends on the dialect.
- (17) `index` (optional): A logical name for an index for DDL generation. The column will be included in the index, along with other columns sharing the same index logical name. The actual index name depends on the dialect.
- (18) `foreign-key` (optional): Specifies the name of the foreign key constraint for DDL generation.
- (19) `outer-join` (optional): This attribute is obsoleted in favor of `fetch`. `auto` is equivalent to not specifying `fetch`, `true` is equivalent to `join` and `false` is equivalent to `select`. This obsolete attribute also appears on other elements, with the same usage. It will be omitted from the documentation of these other elements.

Setting a value of the `cascade` attribute to any meaningful value other than `none` will propagate certain operations to the associated object. The meaningful values are divided into three categories. First, basic operations, which include: `persist`, `merge`, `delete`, `save-update`, `evict`, `replicate`, `lock` and `refresh`; second, special values: `delete-orphan`; and third, all comma-separated combinations of operation names: `cascade="persist,merge,evict"` or `cascade="all,delete-orphan"`. See Section 10.10, "Lifecycles and object graphs".

Here is an example of a typical many-to-one declaration:

```
<many-to-one name="product" class="Product" column="PRODUCT_ID"/>
```

The `property-ref` attribute should only be used for mapping legacy data where a foreign key refers to a unique

key of the associated table other than the primary key. This is a complicated and confusing relational model. For example, if the Product class had a unique serial number that is not the primary key, mapped as below:

```
<property name="serialNumber" unique="true" type="string" column="SERIAL_NUMBER"/>
```

The unique attribute controls NHibernate's DDL generation with the SchemaExport tool. Then the mapping for OrderItem might use:

```
<many-to-one name="product" property-ref="serialNumber" column="PRODUCT_SERIAL_NUMBER"/>
```

This is not encouraged, however.

If the referenced unique key comprises multiple properties of the associated entity, you should map the referenced properties inside a named `<properties>` element.

If the referenced unique key is the property of a component, you can specify a property path:

```
<many-to-one name="owner" property-ref="Identity.Ssn" column="OWNER_SSN"/>
```

5.1.12. one-to-one

A one-to-one association to another persistent class is declared using a `one-to-one` element.

```
<one-to-one
  name="propertyName"                                (1)
  class="className"                                  (2)
  cascade="all|none|save-update|delete|delete-orphan|all- (3)delete-orphan"
  constrained="true|false"                            (4)
  fetch="join|select"                                 (5)
  lazy="proxy|no-proxy|false"                         (6)
  property-ref="propertyNameFromAssociatedClass"      (7)
  access="field|property|nosetter|className"          (8)
  formula="any SQL expression"                        (9)
  entity-name="entityName"                            (10)
  foreign-key="foreignKeyName"                        (11)
  optimistic-lock="true|false"                       (12)
/>
```

- (1) `name`: The name of the property.
- (2) `class` (optional - defaults to the property type determined by reflection): The name of the associated class.
- (3) `cascade` (optional): Specifies which operations should be cascaded from the parent object to the associated object.
- (4) `constrained` (optional - default to false): Specifies that a foreign key constraint on the primary key of the mapped table references the table of the associated class. This option affects the order in which `Save()` and `Delete()` are cascaded, and determines whether the association can be proxied. It is also used by the schema export tool.
- (5) `fetch` (optional - defaults to select): Chooses between outer-join fetching or sequential select fetching.
- (6) `lazy` (optional - defaults to proxy): By default, single point associations are proxied. `lazy="no-proxy"` specifies that the property should be fetched lazily when the instance property is first accessed. It works similarly to lazy properties, and causes the entity owning the association to be proxied instead of the association. `lazy="false"` specifies that the association will be eagerly fetched. *Note that if `constrained="false"`, proxying the association is impossible and NHibernate will eagerly fetch the association.*
- (7) `property-ref`: (optional): The name of a property of the associated class that is joined to the primary key of this class (or to the formula of this association). If not specified, the primary key of the associated class

is used.

- (8) `access` (optional - defaults to `property`): The strategy NHibernate should use for accessing the property value.
- (9) `formula` (optional): Almost all one-to-one associations map to the primary key of the owning entity. If this is not the case, you can specify another column or expression to join on using an SQL formula. This can also be specified by nested `<formula>` element(s).
- (10) `entity-name` (optional): The entity name of the associated class.
- (11) `foreign-key` (optional): Specifies the name of the foreign key constraint for DDL generation.
- (12) `optimistic-lock` (optional - defaults to `true`): Specifies that updates to this property do or do not require acquisition of the optimistic lock. In other words, determines if a version increment should occur when this property is dirty.

There are two varieties of one-to-one association:

- primary key associations
- unique foreign key associations

Primary key associations don't need an extra table column; if two rows are related by the association then the two table rows share the same primary key value. So if you want two objects to be related by a primary key association, you must make sure that they are assigned the same identifier value!

For a primary key association, add the following mappings to `Employee` and `Person`, respectively.

```
<one-to-one name="Person" class="Person"/>
```

```
<one-to-one name="Employee" class="Employee" constrained="true"/>
```

Now we must ensure that the primary keys of related rows in the `PERSON` and `EMPLOYEE` tables are equal. We use a special NHibernate identifier generation strategy called `foreign`:

```
<class name="Person" table="PERSON">
  <id name="Id" column="PERSON_ID">
    <generator class="foreign">
      <param name="property">Employee</param>
    </generator>
  </id>
  ...
  <one-to-one name="Employee"
    class="Employee"
    constrained="true"/>
</class>
```

A newly saved instance of `Person` is then assigned the same primary key value as the `Employee` instance referred with the `Employee` property of that `Person`.

Alternatively, a foreign key with a unique constraint, from `Employee` to `Person`, may be expressed as:

```
<many-to-one name="Person" class="Person" column="PERSON_ID" unique="true"/>
```

And this association may be made bidirectional by adding the following to the `Person` mapping:

```
<one-to-one name="Employee" class="Employee" property-ref="Person"/>
```

5.1.13. natural-id

```
<natural-id mutable="true|false"/>
  <property ... />
  <many-to-one ... />
  ...
</natural-id>
```

Even though we recommend the use of surrogate keys as primary keys, you should still try to identify natural keys for all entities. A natural key is a property or combination of properties that is unique and non-null. If it is also immutable, even better. Map the properties of the natural key inside the `<natural-id>` element. NHibernate will generate the necessary unique key and nullability constraints, and your mapping will be more self-documenting.

The `<natural-id>` element can only appear before the other properties, including version.

We strongly recommend that you implement `Equals()` and `GetHashCode()` to compare the natural key properties of the entity.

This mapping is not intended for use with entities with natural primary keys.

- `mutable` (optional, defaults to `false`): By default, natural identifier properties are assumed to be immutable (constant).

5.1.14. component, dynamic-component

The `<component>` element maps properties of a child object to columns of the table of a parent class. Components may, in turn, declare their own properties, components or collections. See Chapter 8, *Component Mapping*.

```
<component
  name="propertyName"                (1)
  class="className"                  (2)
  insert="true|false"                (3)
  update="true|false"                (4)
  access="field|property|nosetter|className" (5)
  optimistic-lock="true|false"       (6)
  lazy="true|false"                  (7)
  lazy-group="groupName"             (8)
  unique="true|false"                (9)

  <property ... />
  <many-to-one ... />
  ...
</component>
```

- (1) `name`: The name of the property.
- (2) `class` (optional - defaults to the property type determined by reflection): The name of the component (child) class.
- (3) `insert` (optional - defaults to `true`): Do the mapped columns appear in SQL INSERTS?
- (4) `update` (optional - defaults to `true`): Do the mapped columns appear in SQL UPDATES?
- (5) `access` (optional - defaults to `property`): The strategy NHibernate should use for accessing the property value.
- (6) `optimistic-lock` (optional - defaults to `true`): Specifies that updates to this component do or do not require acquisition of the optimistic lock. In other words, determines if a version increment should occur when this property is dirty.
- (7) `lazy` (optional - defaults to `false`): Specifies that this component should be fetched lazily when the instance property is first accessed. For more information, see `lazy` on property element.
- (8) `lazy-group` (optional - defaults to `DEFAULT`): If the component is lazy, its lazy-loading group. When a lazy property is accessed on an object, included when the property is a component, the other lazy properties of

the lazy group are also loaded with it.

- (9) **unique** (optional - defaults to `false`): Specifies that an unique constraint exists upon all mapped columns of the component.

The child `<property>` tags map properties of the child class to table columns.

The `<component>` element allows a `<parent>` sub-element that maps a property of the component class as a reference back to the containing entity.

The `<dynamic-component>` element allows an `IDictionary`, `IDictionary<string, object>`, or a C# dynamic to be mapped as a component. When using dictionaries, the property names refer to keys of the dictionary. See Section 8.5, “Dynamic components”.

5.1.15. properties

The `<properties>` element allows the definition of a named, logical grouping of the properties of a class. The most important use of the construct is that it allows a combination of properties to be the target of a property-ref. It is also a convenient way to define a multi-column unique constraint. For example:

```
<properties
  name="logicalName"                (1)
  insert="true|false"               (2)
  update="true|false"               (3)
  optimistic-lock="true|false"      (4)
  unique="true|false">              (5)

  <property .../>
  <many-to-one .../>
  ...
</properties>
```

- (1) **name**: the logical name of the grouping. It is *not* an actual property name.
- (2) **insert** (optional - defaults to `true`): do the mapped columns appear in SQL INSERTS?
- (3) **update** (optional - defaults to `true`): do the mapped columns appear in SQL UPDATES?
- (4) **optimistic-lock** (optional - defaults to `true`): specifies that updates to these properties either do or do not require acquisition of the optimistic lock. It determines if a version increment should occur when these properties are dirty.
- (5) **unique** (optional - defaults to `false`): specifies that a unique constraint exists upon all mapped columns of the component.

For example, if we have the following `<properties>` mapping:

```
<class name="Person">
  <id name="personNumber" />
  <properties name="name" unique="true" update="false">
    <property name="firstName" />
    <property name="lastName" />
    <property name="initial" />
  </properties>
</class>
```

You might have some legacy data association that refers to this unique key of the `Person` table, instead of to the primary key:

```
<many-to-one name="owner" class="Person" property-ref="name">
  <column name="firstName" />
  <column name="lastName" />
  <column name="initial" />
</many-to-one>
```

The use of this outside the context of mapping legacy data is not recommended.

5.1.16. subclass

Polymorphic persistence requires the declaration of each subclass of the root persistent class. For the (recommended) table-per-class-hierarchy mapping strategy, the `<subclass>` declaration is used.

```
<subclass
  name="className"                      (1)
  discriminator-value="discriminatorValue" (2)
  extends="superclassName"              (3)
  ...>                                  (4)

  <property ... />
  <properties ... />
  ...
</subclass>
```

- (1) name: The fully qualified .NET class name of the subclass, including its assembly name.
- (2) discriminator-value (optional - defaults to the class name): A value that distinguishes individual subclasses.
- (3) extends (optional if the `<subclass>` element is nested into its superclass mapping declaration): Specifies the name of a mapped class as the superclass for the subclass.
- (4) Many attributes available on the `<class>` mapping element are also available on `<subclass>` with the same usage: proxy, dynamic-update, dynamic-insert, select-before-update, persister, batch-size, lazy, entity-name, abstract. See Section 5.1.3, “class”.

Each subclass should declare its own persistent properties and subclasses. `<version>` and `<id>` properties are assumed to be inherited from the root class. Each subclass in a hierarchy must define a unique `discriminator-value`. If none is specified, the fully qualified .NET class name is used.

For information about inheritance mappings, see Chapter 9, *Inheritance Mapping*.

5.1.17. joined-subclass

Each subclass can also be mapped to its own table. This is called the table-per-subclass mapping strategy. An inherited state is retrieved by joining with the table of the superclass. To do this you use the `<joined-subclass>` element.

```
<joined-subclass
  name="className"                      (1)
  table="tableName"                    (2)
  extends="superclassName"              (3)
  ...>                                  (4)

  <key ... />

  <property ... />
  <properties ... />
  ...
</joined-subclass>
```

- (1) name: The fully qualified class name of the subclass.
- (2) table (optional - defaults to the unqualified subclass name): The name of the subclass table.
- (3) extends (optional if the `<joined-subclass>` element is nested into its superclass mapping declaration): Specifies the name of a mapped class as the superclass for the subclass.
- (4) Many attributes available on the `<class>` mapping element are also available on `<joined-subclass>` with

the same usage: schema, catalog, proxy, subselect, dynamic-update, dynamic-insert, select-before-update, persister, batch-size, lazy, entity-name, abstract, check, schema-action. See Section 5.1.3, “class”.

No discriminator column is required for this mapping strategy. Each subclass must, however, declare a table column holding the object identifier using the <key> element. The mapping at the start of the chapter would be re-written as:

```
<?xml version="1.0"?>
<hibernate-mapping xmlns="urn:nhibernate-mapping-2.2" assembly="Eg"
  namespace="Eg">

  <class name="Cat" table="CATS">
    <id name="Id" column="uid" type="Int64">
      <generator class="hilo"/>
    </id>
    <property name="BirthDate" type="Date"/>
    <property name="Color" not-null="true"/>
    <property name="Sex" not-null="true"/>
    <property name="Weight"/>
    <many-to-one name="Mate"/>
    <set name="Kittens">
      <key column="MOTHER"/>
      <one-to-many class="Cat"/>
    </set>
    <joined-subclass name="DomesticCat" table="DOMESTIC_CATS">
      <key column="CAT"/>
      <property name="Name" type="String"/>
    </joined-subclass>
  </class>

  <class name="Dog">
    <!-- mapping for Dog could go here -->
  </class>

</hibernate-mapping>
```

For information about inheritance mappings, see Chapter 9, *Inheritance Mapping*.

5.1.18. union-subclass

A third option is to map only the concrete classes of an inheritance hierarchy to tables, (the table-per-concrete-class strategy) where each table defines all persistent state of the class, including inherited state. In NHibernate, it is not absolutely necessary to explicitly map such inheritance hierarchies. You can simply map each class with a separate <class> declaration. However, if you wish use polymorphic associations (e.g. an association to the superclass of your hierarchy), you need to use the <union-subclass> mapping.

```
<union-subclass
  name="className"                      (1)
  table="tableName"                    (2)
  extends="superclassName"             (3)
  ...>                                  (4)

  <property ... />
  <properties ... />
  ...
</union-subclass>
```

- (1) name: The fully qualified class name of the subclass.
- (2) table (optional - defaults to the unqualified subclass name): The name of the subclass table.
- (3) extends (optional if the <union-subclass> element is nested into its superclass mapping declaration): Specifies the name of a mapped class as the superclass for the subclass.

- (4) Many attributes available on the `<class>` mapping element are also available on `<union-subclass>` with the same usage: `schema`, `catalog`, `proxy`, `subselect`, `dynamic-update`, `dynamic-insert`, `select-before-update`, `persister`, `batch-size`, `lazy`, `entity-name`, `abstract`, `check`. See Section 5.1.3, “class”.

No discriminator column or key column is required for this mapping strategy.

For information about inheritance mappings, see Chapter 9, *Inheritance Mapping*.

5.1.19. join

Using the `<join>` element, it is possible to map properties of one class to several tables, when there's a 1-to-1 relationship between the tables.

```
<join
  table="tableName"                (1)
  schema="owner"                  (2)
  catalog="catalog"               (3)
  fetch="join|select"             (4)
  inverse="true|false"            (5)
  optional="true|false"           (6)
  subselect="SQL expression">    (7)

  <key ... />

  <property ... />
  ...
</join>
```

- (1) `table`: The name of the joined table.
- (2) `schema` (optional): Overrides the schema name specified by the root `<hibernate-mapping>` element.
- (3) `catalog` (optional): Overrides the catalog name specified by the root `<hibernate-mapping>` element.
- (4) `fetch` (optional - defaults to `join`): If set to `join`, the default, NHibernate will use an inner join to retrieve a `<join>` defined by a class or its superclasses and an outer join for a `<join>` defined by a subclass. If set to `select` then NHibernate will use a sequential select for a `<join>` defined on a subclass, which will be issued only if a row turns out to represent an instance of the subclass. Inner joins will still be used to retrieve a `<join>` defined by the class and its superclasses.
- (5) `inverse` (optional - defaults to `false`): If enabled, NHibernate will not try to insert or update the properties defined by this join.
- (6) `optional` (optional - defaults to `false`): If enabled, NHibernate will insert a row only if the properties defined by this join are non-null and will always use an outer join to retrieve the properties.
- (7) `subselect` (optional): Maps an immutable and read-only join to a database sub-select. This is useful if you want to have a view instead of a table. See Section 5.1.4, “subselect” for more information.

For example, the address information for a person can be mapped to a separate table (while preserving value type semantics for all properties):

```
<class name="Person" table="PERSON">
  <id name="id" column="PERSON_ID">...</id>

  <join table="ADDRESS">
    <key column="ADDRESS_ID"/>
    <property name="address"/>
    <property name="zip"/>
    <property name="country"/>
  </join>
  ...
</class>
```

This feature is often only useful for legacy data models, we recommend fewer tables than classes and a fine-grained domain model. However, it is useful for switching between inheritance mapping strategies in a single

hierarchy, as explained later.

5.1.20. key

The `<key>` element has featured a few times within this guide. It appears anywhere the parent mapping element defines a join to a new table that references the primary key of the original table. It also defines the foreign key in the joined table:

```
<key
  column="columnName"                (1)
  on-delete="noaction|cascade"       (2)
  property-ref="propertyName"       (3)
  not-null="true|false"              (4)
  update="true|false"                (5)
  unique="true|false"                (6)
  foreign-key="foreignKeyName"       (7)
/>
```

- (1) `column`: the name of the foreign key column. This can also be specified by nested `<column>` element(s).
- (2) `on-delete` (optional - defaults to `noaction`): specifies whether the foreign key constraint has database-level cascade delete enabled.
- (3) `property-ref` (optional): specifies that the foreign key refers to columns that are not the primary key of the original table. It is provided for legacy data.
- (4) `not-null` (optional): specifies that the foreign key columns are not nullable. This is implied whenever the foreign key is also part of the primary key.
- (5) `update` (optional): specifies that the foreign key should never be updated. This is implied whenever the foreign key is also part of the primary key.
- (6) `unique` (optional): specifies that the foreign key should have a unique constraint. This is implied whenever the foreign key is also the primary key.
- (7) `foreign-key` (optional): specifies the name of the foreign key constraint for DDL generation.

For systems where delete performance is important, we recommend that all keys should be defined `on-delete="cascade"`. NHibernate uses a database-level `ON CASCADE DELETE` constraint, instead of many individual `DELETE` statements. Be aware that this feature bypasses NHibernate's usual optimistic locking strategy for versioned data.

The `not-null` and `update` attributes are useful when mapping an unidirectional one-to-many association. If you map an unidirectional one-to-many association to a non-nullable foreign key, you *must* declare the key column using `<key not-null="true">`.

5.1.21. column and formula elements

Mapping elements which accept a `column` attribute will alternatively accept a `<column>` subelement. Likewise, `<formula>` is an alternative to the `formula` attribute. For example:

```
<column
  name="columnName"
  length="N"
  precision="N"
  scale="N"
  not-null="true|false"
  unique="true|false"
  unique-key="multicolumnUniqueKeyName"
  index="indexName"
  sql-type="sqlTypeName"
  check="SQL expression"
  default="SQL expression"/>
```

```
<formula>SQL expression</formula>
```

Most of the attributes on `column` provide a means of tailoring the DDL during automatic schema generation. See Section 22.1.1, “Customizing the schema”.

The `column` and `formula` elements can even be combined within the same property or association mapping to express, for example, exotic join conditions.

```
<many-to-one name="HomeAddress" class="Address"
    insert="false" update="false">
    <column name="person_id" not-null="true" length="10"/>
    <formula>'MAILING'</formula>
</many-to-one>
```

5.1.22. map, set, list, bag

Collections are discussed in Chapter 6, *Collection Mapping*.

5.1.23. import

Suppose your application has two persistent classes with the same name, and you don't want to specify the fully qualified name in NHibernate queries. Classes may be "imported" explicitly, rather than relying upon `auto-import="true"`. You may even import classes and interfaces that are not explicitly mapped.

```
<import class="System.Object" rename="Universe"/>
```

```
<import
    class="className"                (1)
    rename="shortName"              (2)
/>
```

- (1) `class`: The fully qualified class name of any .NET class, including its assembly name.
- (2) `rename` (optional - defaults to the unqualified class name): A name that may be used in the query language.

5.1.24. any

There is one more type of property mapping. The `<any>` mapping element defines a polymorphic association to classes from multiple tables. This type of mapping requires more than one column. The first column contains the type of the associated entity. The remaining columns contain the identifier. It is impossible to specify a foreign key constraint for this kind of association. This is not the usual way of mapping polymorphic associations and you should use this only in special cases. For example, for audit logs, user session data, etc.

```
<any name="AnyEntity" id-type="long" meta-type="class">
    <column name="type"/>
    <column name="id"/>
</any>
```

The `meta-type` attribute allows the application to specify a custom type that maps database column values to persistent classes that have identifier properties of the type specified by `id-type`. If the `meta-type` is `class`, nothing else is required. The class full name will be persisted in the database as the type of the associated entity. On the other hand, if it is a basic type like `int` or `char`, you must specify the mapping from values to classes. Here is an example with `string`.

```

<any name="being" id-type="long" meta-type="string">
  <meta-value value="ANIMAL_TYPE" class="Animal"/>
  <meta-value value="HUMAN_TYPE" class="Human"/>
  <meta-value value="ALIEN_TYPE" class="Alien"/>
  <column name="type"/>
  <column name="id"/>
</any>

```

String types are a special case: they can be used without meta-values, in which case they will behave much like the class meta-type.

```

<any
  name="propertyName"                (1)
  id-type="idTypeName"                (2)
  meta-type="metaTypeName"           (3)
  cascade="cascadeStyle"              (4)
  access="field|property|className"   (5)
  optimistic-lock="true|false"        (6)
  lazy="true|false"                   (7)
  update="true|false"                 (8)
  insert="true|false"                 (9)
  index="indexName">                 (10)

  <meta-value ... />
  <meta-value ... />
  ...
  <column ... />
  <column ... />
  ...
</any>

```

- (1) name: the property name.
- (2) id-type: the identifier type.
- (3) meta-type (optional - defaults to string): a type that is allowed for a discriminator mapping, or class.
- (4) cascade (optional - defaults to none): the cascade style.
- (5) access (optional - defaults to property): the strategy Hibernate uses for accessing the property value.
- (6) optimistic-lock (optional - defaults to true): specifies that updates to this property either do or do not require acquisition of the optimistic lock. It defines whether a version increment should occur if this property is dirty.
- (7) lazy (optional - defaults to false): specifies that this property should be fetched lazily when the it is first accessed. For more informations, see lazy on property element.
- (8) update (optional - defaults to true): Do the mapped columns appear in SQL UPDATES?
- (9) insert (optional - defaults to true): Do the mapped columns appear in SQL INSERTS?
- (10) index (optional): a logical name for an index for DDL generation. The column will be included in the index, along with other columns sharing the same index logical name. The actual index name depends on the dialect.

Do not confuse <meta-value> elements with <meta> elements. <meta> elements have no functionnal impact on <any>, see Section 5.8, “Meta data” for more information on them.

5.2. NHibernate Types

5.2.1. Entities and values

To understand the behaviour of various .NET language-level objects with respect to the persistence service, we need to classify them into two groups:

An *entity* exists independently of any other objects holding references to the entity. Contrast this with the usual .NET model where an unreferenced object is garbage collected. Entities must be explicitly saved and deleted (except that saves and deletions may be *cascaded* from a parent entity to its children). This is different from the ODMG model of object persistence by reachability - and corresponds more closely to how application objects are usually used in large systems. Entities support circular and shared references. They may also be versioned.

An entity's persistent state consists of references to other entities and instances of *value* types. Values are primitives, collections, components and certain immutable objects. Unlike entities, values (in particular collections and components) *are* persisted and deleted by reachability. Since value objects (and primitives) are persisted and deleted along with their containing entity they may not be independently versioned. Values have no independent identity, so they cannot be shared by two entities or collections.

All NHibernate types except collections support null semantics if the .NET type is nullable (i.e. not derived from `System.ValueType`).

Up until now, we've been using the term "persistent class" to refer to entities. We will continue to do that. Strictly speaking, however, not all user-defined classes with persistent state are entities. A *component* is a user defined class with value semantics.

The challenge is to map the .Net type system, and the developers' definition of entities and value types, to the SQL/database type system. The bridge between both systems is provided by NHibernate. For entities, `<class>`, `<subclass>` and so on are used. For value types we use `<property>`, `<component>`, etc., that usually have a type attribute. The value of this attribute is the name of a NHibernate mapping type. NHibernate provides a range of mappings for standard .Net value types out of the box. You can write your own mapping types and implement your own custom conversion strategies.

5.2.2. Basic value types

The *basic types* may be roughly categorized into three groups - `System.ValueType` types, `System.Object` types, and `System.Object` types for large objects. Just like Columns for `System.ValueType` types can handle null values only if the entity property is properly typed with a `Nullable<T>`. Otherwise null will be replaced by the default value for the type when reading, and then will be overwritten by it when persisting the entity, potentially leading to phantom updates.

Table 5.3. System.ValueType Mapping Types

NHibernate Type	.NET Type	Database Type	Remarks
AnsiChar	System.Char	DbType.AnsiStringFixedLength - 1 char	type="AnsiChar" must be specified.
Boolean	System.Boolean	DbType.Boolean	Default when no type attribute specified.
Byte	System.Byte	DbType.Byte	Default when no type attribute specified.
Char	System.Char	DbType.StringFixedLength - 1 char	Default when no type attribute specified.
Currency	System.Decimal	DbType.Currency	type="Currency" must be specified.

NHibernate Type	.NET Type	Database Type	Remarks
Date	System.DateTime	DbType.Date	type="Date" must be specified.
DateTime	System.DateTime	DbType.DateTime / DbType.DateTime2(1)	Default when no type attribute specified. Does no longer ignore fractional seconds since NHibernate v5.0.
DateTimeNoMs	System.DateTime	DbType.DateTime / DbType.DateTime2(1)	type="DateTimeNoMs" must be specified. Ignores fractional seconds. Available since NHibernate v5.0.
DateTime2	System.DateTime	DbType.DateTime2	type="DateTime2" must be specified. Obsolete since NHibernate v5.0, use DateTime instead.
DateTimeOffset	System.DateTimeOffset	DbType.DateTimeOffset	Default when no type attribute specified.
DbTimestamp	System.DateTime	DbType.DateTime / DbType.DateTime2(1)	type="DbTimestamp" must be specified. When used as a version field, uses the database's current time retrieved in dedicated queries, rather than the client's current time. In case of lack of database support, it falls back on the client's current time.
Decimal	System.Decimal	DbType.Decimal	Default when no type attribute specified.
Double	System.Double	DbType.Double	Default when no type attribute specified.
Guid	System.Guid	DbType.Guid	Default when no type attribute specified.
Int16	System.Int16	DbType.Int16	Default when no type attribute specified.
Int32	System.Int32	DbType.Int32	Default when no type attribute specified.
Int64	System.Int64	DbType.Int64	Default when no type attribute specified.
LocalDateTime	System.DateTime	DbType.DateTime / DbType.DateTime2(1)	type="LocalDateTime" must be specified. Ensures the DateTimeKind is

NHibernate Type	.NET Type	Database Type	Remarks
			set to DateTimeKind.Local. Throws if set with a date having another kind. Does no longer ignore fractional seconds since NHibernate v5.0.
LocalDateTimeNoMs	System.DateTime	DbType.DateTime / DbType.DateTime2(1)	type="LocalDateTimeNoMs" must be specified. Similar to LocalDateTime but ignores fractional seconds. Available since NHibernate v5.0.
PersistentEnum	A System.Enum	The DbType for the underlying value.	Do not specify type="PersistentEnum" in the mapping. Instead specify the Assembly Qualified Name of the Enum or let NHibernate use Reflection to "guess" the Type. The UnderlyingType of the Enum is used to determine the correct DbType.
SByte	System.SByte	DbType.SByte	Default when no type attribute specified.
Single	System.Single	DbType.Single	Default when no type attribute specified.
Ticks	System.DateTime	DbType.Int64	type="Ticks" must be specified. This is the recommended way to "timestamp" a column, along with UtcTicks.
Time	System.DateTime	DbType.Time	type="Time" must be specified.
TimeAsTimeSpan	System.TimeSpan	DbType.Time	type="TimeAsTimeSpan" must be specified.
TimeSpan	System.TimeSpan	DbType.Int64	Default when no type attribute specified.
Timestamp	System.DateTime	DbType.DateTime / DbType.DateTime2(1)	Obsolete, its Timestamp alias will be remapped to DateTime in a future version.
TrueFalse	System.Boolean	DbType.AnsiStringFixedL	type="TrueFalse" must be specified.

NHibernate Type	.NET Type	Database Type	Remarks
		length - 1 char either 'T' or 'F'	
UInt16	System.UInt16	DbType.UInt16	Default when no type attribute specified.
UInt32	System.UInt32	DbType.UInt32	Default when no type attribute specified.
UInt64	System.UInt64	DbType.UInt64	Default when no type attribute specified.
UtcDateTime	System.DateTime	DbType.DateTime / DbType.DateTime2(1)	Ensures the DateTimeKind is set to DateTimeKind.Utc. Throws if set with a date having another kind. Does no longer ignore fractional seconds since NHibernate v5.0.
UtcDateTimeNoMs	System.DateTime	DbType.DateTime / DbType.DateTime2(1)	type="UtcDateTimeNoMs" must be specified. Similar to UtcDateTime but ignores fractional seconds. Available since NHibernate v5.0.
UtcDbTimestamp	System.DateTime	DbType.DateTime / DbType.DateTime2(1)	type="UtcDbTimestamp" must be specified. When used as a version field, uses the database's current UTC time retrieved in dedicated queries, rather than the client's current time. In case of lack of database support, it falls back on the client's current time.
UtcTicks	System.DateTime	DbType.Int64	type="UtcTicks" must be specified. This is the recommended way to "timestamp" a column, along with Ticks. Ensures the DateTimeKind is set to DateTimeKind.Utc. Throws if set with a date having another kind.
YesNo	System.Boolean	DbType.AnsiStringFixedLength - 1 char either 'Y' or 'N'	type="YesNo" must be specified.

- (1) Since NHibernate v5.0 and if the dialect supports it, `DbType.DateTime2` is used instead of `DbType.DateTime`. This may be disabled by setting `sql_types.keep_datetime` to `true`.

Table 5.4. System.Object Mapping Types

NHibernate Type	.NET Type	Database Type	Remarks
AnsiString	System.String	DbType.AnsiString	<code>type="AnsiString"</code> must be specified.
CultureInfo	System.Globalization.CultureInfo	DbType.String - 5 chars for culture by default; can be modified by the length mapping attribute.	Default when no type attribute specified.
Binary	System.Byte[]	DbType.Binary	Default when no type attribute specified.
Type	System.Type	DbType.String holding Assembly Qualified Name.	Default when no type attribute specified.
String	System.String	DbType.String	Default when no type attribute specified.
Uri	System.Uri	DbType.String	Default when no type attribute specified.

String types use by default .Net default string equality, which is case sensitive and culture insensitive. When a string type is used as an identifier, if the underlying database string equality semantic differs, it may cause issues. For example, loading a children collection by a string parent key stored in a case insensitive column may cause matching children having their parent key differing by the case to be ignored by NHibernate by default.

String types comparer can be set for matching the database or column behavior. To set the default comparer for all string types, affect by code a `StringComparer` to `AbstractStringType.DefaultComparer` static property. To set the comparer used only for a specific `<property>` or `<id>`, specify its type as a sub-element and supply `IgnoreCase` and/or `ComparerCulture` parameters.

```
<id name="Id" generator="assigned">
  <type name="string">
    <param name="ComparerCulture">en-US</param>      (1)
    <param name="IgnoreCase">true</param>           (2)
  </type>
</id>
```

- (1) `IgnoreCase`: `true` for case insensitive comparisons. Any other value will result in case sensitive comparisons.
- (2) `ComparerCulture`: `Current` for using the application current culture, `Invariant` for using the invariant culture, `Ordinal` for using ordinal comparison, or any valid culture name for using another culture. By default, ordinal comparisons are used.

If you have many properties to map with these parameters, consider using `<typedef>`. See Section 5.2.3, “Custom value types” for more information.

These settings should be used in order to match the database or column behavior. They are not taken into account by the `hbm2ddl` tool for generating the database schema. (In other words, it will not generate matching

collate statements for SQL-Server.)

Table 5.5. Large Object Mapping Types

NHibernate Type	.NET Type	Database Type	Remarks
StringClob	System.String	DbType.String	type="StringClob" must be specified. Entire field is read into memory.
BinaryBlob	System.Byte[]	DbType.Binary	type="BinaryBlob" must be specified. Entire field is read into memory.
Serializable	Any System.Object that is marked with SerializableAttribute.	DbType.Binary	type="Serializable" should be specified. This is the fallback type if no NHibernate Type can be found for the Property.
XDoc	System.Xml.Linq.XDocument	DbType.Xml	Default when no type attribute specified. Entire field is read into memory.
XmlDoc	System.Xml.XmlDocument	DbType.Xml	Default when no type attribute specified. Entire field is read into memory.

NHibernate supports some additional type names for compatibility with Java's Hibernate (useful for those coming over from Hibernate or using some of the tools to generate hbm.xml files). A type="integer" or type="int" will map to an Int32 NHibernate type, type="short" to an Int16 NHibernateType. To see all of the conversions you can view the source of static constructor of the class NHibernate.Type.TypeFactory.

Default NHibernate types used when no type attribute is specified can be overridden by using the NHibernate.Type.TypeFactory.RegisterType static method before configuring and building session factories.

5.2.3. Custom value types

It is relatively easy for developers to create their own value types. For example, you might want to persist properties of type Int64 to VARCHAR columns. NHibernate does not provide a built-in type for this. But custom types are not limited to mapping a property (or collection element) to a single table column. So, for example, you might have a property Name { get; set; } of type String that is persisted to the columns FIRST_NAME, INITIAL, SURNAME.

To implement a custom type, implement either NHibernate.UserTypes.IUserType or NHibernate.UserTypes.ICompositeUserType and declare properties using the fully qualified name of the type. Check out NHibernate.DomainModel.DoubleStringType to see the kind of things that are possible.

```
<property name="TwoStrings"
  type="NHibernate.DomainModel.DoubleStringType, NHibernate.DomainModel">
  <column name="first_string"/>
  <column name="second_string"/>
</property>
```

Notice the use of <column> tags to map a property to multiple columns.

The `IEnhancedUserType`, `IUserVersionType`, and `IUserCollectionType` interfaces provide support for more specialized uses. The later is to be used with collections, see Section 6.1, “Persistent Collections”.

You may even supply parameters to an `IUserType` in the mapping file. To do this, your `IUserType` must implement the `NHibernate.UserTypes.IParameterizedType` interface. To supply parameters to your custom type, you can use the `<type>` element in your mapping files.

```
<property name="priority">
  <type name="MyCompany.UserTypes.DefaultValueIntegerType">
    <param name="default">0</param>
  </type>
</property>
```

The `IUserType` can now retrieve the value for the parameter named `default` from the `IDictionary` object passed to it.

If you use a certain `UserType` very often, it may be useful to define a shorter name for it. You can do this using the `<typedef>` element. Typedefs assign a name to a custom type, and may also contain a list of default parameter values if the type is parameterized.

```
<typedef class="MyCompany.UserTypes.DefaultValueIntegerType" name="default_zero">
  <param name="default">0</param>
</typedef>
```

```
<property name="priority" type="default_zero"/>
```

It is also possible to override the parameters supplied in a typedef on a case-by-case basis by using type parameters on the property mapping.

Even though NHibernate's rich range of built-in types and support for components means you will very rarely *need* to use a custom type, it is nevertheless considered good form to use custom types for (non-entity) classes that occur frequently in your application. For example, a `MonetaryAmount` class is a good candidate for an `ICompositeUserType`, even though it could easily be mapped as a component. One motivation for this is abstraction. With a custom type, your mapping documents would be future-proofed against possible changes in your way of representing monetary values.

5.3. Mapping a class more than once

It is possible to provide more than one mapping for a particular persistent class. In this case, you must specify an *entity name* to disambiguate between instances of the two mapped entities. By default, the entity name is the same as the class name. NHibernate lets you specify the entity name when working with persistent objects, when writing queries, or when mapping associations to the named entity.

```
<class name="Contract" table="Contracts"
  entity-name="CurrentContract">
  ...
  <set name="History" inverse="true"
    order-by="effectiveEndDate desc">
    <key column="currentContractId"/>
    <one-to-many entity-name="HistoricalContract"/>
  </set>
</class>

<class name="Contract" table="ContractHistory"
  entity-name="HistoricalContract">
  ...
  <many-to-one name="CurrentContract"
    column="currentContractId">
```

```
entity-name="CurrentContract"/>
</class>
```

Associations are now specified using `entity-name` instead of `class`.

5.4. SQL quoted identifiers

You may force NHibernate to quote an identifier in the generated SQL by enclosing the table or column name in back-ticks in the mapping document. NHibernate will use the correct quotation style for the SQL dialect (usually double quotes, but brackets for SQL Server and back-ticks for MySQL).

```
<class name="LineItem" table="`Line Item`">
  <id name="Id" column="`Item Id`">
    <generator class="assigned"/>
  </id>
  <property name="ItemNumber" column="`Item #`"/>
  ...
</class>
```

Quoting column identifiers is required if a table contains two columns differing only by case. Ensure you use consistent casing when quoting identifiers.

5.5. Modular mapping files

It is possible to define subclass and joined-subclass mappings in separate mapping documents, directly beneath `hibernate-mapping`. This allows you to extend a class hierarchy just by adding a new mapping file. You must specify an `extends` attribute in the subclass mapping, naming a previously mapped superclass.

```
<hibernate-mapping>
  <subclass name="Eg.Subclass.DomesticCat, Eg"
    extends="Eg.Cat, Eg" discriminator-value="D">
    <property name="name" type="string"/>
  </subclass>
</hibernate-mapping>
```

5.6. Generated Properties

Generated properties are properties which have their values generated by the database. Typically, NHibernate applications needed to refresh objects which contain any properties for which the database was generating values. Marking properties as generated, however, lets the application delegate this responsibility to NHibernate. Essentially, whenever NHibernate issues an SQL INSERT or UPDATE for an entity which has defined generated properties, it immediately issues a select afterwards to retrieve the generated values.

Properties marked as generated must additionally be non-insertable and non-updatable. Only versions, timestamps, and simple properties can be marked as generated.

`never` (the default) - means that the given property value is not generated within the database.

`insert` - states that the given property value is generated on insert, but is not regenerated on subsequent updates. Things like created-date would fall into this category. Note that even though version and timestamp properties can be marked as generated, this option is not available for them.

always - states that the property value is generated both on insert and on update.

5.7. Auxiliary Database Objects

Auxiliary database objects allow CREATE and DROP of arbitrary database objects. In conjunction with NHibernate's schema evolution tools, they have the ability to fully define a user schema within the NHibernate mapping files. Although designed specifically for creating and dropping things like triggers or stored procedures, any SQL command that can be run via a `DbCommand.ExecuteNonQuery()` method is valid (for example, ALTERs, INSERTS, etc.). There are essentially two modes for defining auxiliary database objects:

The first mode is to explicitly list the CREATE and DROP commands in the mapping file:

```
<hibernate-mapping>
...
  <database-object>
    <create>CREATE TRIGGER my_trigger ...</create>
    <drop>DROP TRIGGER my_trigger</drop>
  </database-object>
</hibernate-mapping>
```

The second mode is to supply a custom class that constructs the CREATE and DROP commands. This custom class must implement the `NHibernate.Mapping.IAuxiliaryDatabaseObject` interface.

```
<hibernate-mapping>
...
  <database-object>
    <definition class="MyTriggerDefinition, MyAssembly"/>
  </database-object>
</hibernate-mapping>
```

You may also specify parameters to be passed to the database object:

```
<hibernate-mapping>
...
  <database-object>
    <definition class="MyTriggerDefinition, MyAssembly">
      <param name="parameterName">parameterValue</param>
    </definition>
  </database-object>
</hibernate-mapping>
```

NHibernate will call `IAuxiliaryDatabaseObject.SetParameterValues` passing it a dictionary of parameter names and values.

Additionally, these database objects can be optionally scoped such that they only apply when certain dialects are used.

```
<hibernate-mapping>
...
  <database-object>
    <definition class="MyTriggerDefinition"/>
    <dialect-scope name="NHibernate.Dialect.Oracle9iDialect"/>
    <dialect-scope name="NHibernate.Dialect.Oracle8iDialect"/>
  </database-object>
</hibernate-mapping>
```

5.8. Meta data

You may declare additional metadata in your mappings by using the `meta` element. This element can be added to the main mapping elements.

```
<meta
  attribute="metaName"           (1)
  inherit="true|false"         (2)
>metaData</meta>
```

- (1) `attribute`: The name of the metadata.
- (2) `inherit` (optional - defaults to `true`): Whether the metadata is inherited by mapping sub-elements or not.
- (3) The text content of the `meta` element is its data.

The metadata can be retrieved at runtime from the Section 10.12, “Metadata API”.

Chapter 6. Collection Mapping

6.1. Persistent Collections

NHibernate requires that persistent collection-valued fields be declared as a generic interface type, for example:

```
public class Product
{
    public virtual ISet<Part> Parts { get; set; } = new HashSet<Part>();
    public virtual string SerialNumber { get; set; }
}
```

The actual interface might be `System.Collections.Generic.ICollection<T>`, `System.Collections.Generic.IList<T>`, `System.Collections.Generic.IDictionary<K, V>`, `System.Collections.Generic.ISet<T>` or ... anything you like! (Where "anything you like" means you will have to write an implementation of `NHibernate.UserType.IUserCollectionType`.)

Notice how we initialized the instance variable with an instance of `HashSet<T>`. This is the best way to initialize collection valued properties of newly instantiated (non-persistent) instances. When you make the instance persistent - by calling `Save()`, for example - NHibernate will actually replace the `HashSet<T>` with an instance of NHibernate's own implementation of `ISet<T>`. Watch out for errors like this:

```
Cat cat = new DomesticCat();
Cat kitten = new DomesticCat();
...
ISet<Cat> kittens = new HashSet<Cat>();
kittens.Add(kitten);
cat.Kittens = kittens;
session.Save(cat);
kittens = cat.Kittens; //Okay, kittens collection is an ISet
HashSet<Cat> hs = (HashSet<Cat>) cat.Kittens; //Error!
```

Collection instances have the usual behavior of value types. They are automatically persisted when referenced by a persistent object and automatically deleted when unreferenced. If a collection is passed from one persistent object to another, its elements might be moved from one table to another. Two entities may not share a reference to the same collection instance. Due to the underlying relational model, collection-valued properties do not support null value semantics; NHibernate does not distinguish between a null collection reference and an empty collection.

You shouldn't have to worry much about any of this. Just use NHibernate's collections the same way you use ordinary .NET collections, but make sure you understand the semantics of bidirectional associations (discussed later) before using them.

Collection instances are distinguished in the database by a foreign key to the owning entity. The collection key is mapped by the `<key>` element.

Collections may contain almost any other NHibernate type, including all basic types, custom types, entity types and components. See Section 6.2.2, "Collection elements".

All collection types except `ISet` and `bag` have an *index* column - a column that maps to an array or `IList` index or `IDictionary` key. See Section 6.2.3, "Indexed collections" for more information.

There are quite a range of mappings that can be generated for collections, covering many common relational models. We suggest you experiment with the schema generation tool to get a feeling for how various mapping

declarations translate to database tables.

6.2. Mapping a Collection

The NHibernate mapping element used for mapping a collection depends upon the type of interface. By example, a `<set>` element is used for mapping properties of type `ISet`.

```
<class name="Product">
  <id name="SerialNumber" column="productSerialNumber"/>
  <set name="Parts">
    <key column="productSerialNumber" not-null="true"/>
    <one-to-many class="Part"/>
  </set>
</class>
```

Apart from `<set>`, there is also `<list>`, `<map>`, `<bag>`, `<array>` and `<primitive-array>` mapping elements. The `<map>` element is representative:

```
<map
  name="propertyName" (1)
  table="tableName" (2)
  schema="schemaName" (3)
  catalog="catalogName" (4)
  lazy="true|false|extra" (5)
  inverse="true|false" (6)
  cascade="all|none|save-update|delete|all-delete-orphan" (7)
  sort="unsorted|natural|comparatorClass" (8)
  order-by="columnName asc|desc" (9)
  where="arbitrary sql where condition" (10)
  fetch="select|join|subselect" (11)
  batch-size="N" (12)
  access="field|property|className" (13)
  optimistic-lock="true|false" (14)
  mutable="true|false" (15)
  subselect="SQL expression" (16)
  check="arbitrary sql check condition" (17)
  persister="persisterClass" (18)
  collection-type="collectionClass" (19)
  outer-join="auto|true|false" (20)
  generic="true|false"> (21)

  <key ... />
  <map-key ... />
  <element ... />
</map>
```

- (1) name: the collection property name.
- (2) table (optional - defaults to property name): the name of the collection table. It is not used for one-to-many associations.
- (3) schema (optional): overrides the schema name specified by the root `<hibernate-mapping>` element.
- (4) catalog (optional): overrides the catalog name specified by the root `<hibernate-mapping>` element.
- (5) lazy (optional - defaults to true): may be used to disable lazy fetching and specify that the association is always eagerly fetched. Using extra fetches only the elements that are needed - see Section 21.1, “Fetching strategies” for more information.
- (6) inverse (optional - defaults to false): marks this collection as the "inverse" end of a bidirectional association.
- (7) cascade (optional - defaults to none): enables operations to cascade to child entities.
- (8) sort (optional): specifies a sorted collection with a natural sort order, or a given comparator class. See Section 6.3.1, “Sorted Collections”.
- (9) order-by (optional): specifies a table column (or columns) that defines the iteration order of the `IDic-`

tionary, `ISet` or `bag`, together with an optional `asc` or `desc`.

- (10) `where` (optional): specifies an arbitrary SQL `WHERE` condition to be used when retrieving or removing the collection. It is useful if the collection should contain only a subset of the available data.
- (11) `fetch` (optional, defaults to `select`): chooses between outer-join fetching, fetching by sequential select, and fetching by sequential subselect - see Section 21.1.6, “Using subselect fetching”. `join` takes precedence over the `lazy` attribute and causes the association to be eagerly fetched.
- (12) `batch-size` (optional, defaults to `1`): specifies a “batch size” for lazily fetching instances of this collection. See Section 21.1.5, “Using batch fetching”.
- (13) `access` (optional - defaults to `property`): the strategy NHibernate should use for accessing the property value.
- (14) `optimistic-lock` (optional - defaults to `true`): specifies that changes to the state of the collection results in increment of the owning entity's version. For one to many associations, you may want to disable this setting.
- (15) `mutable` (optional - defaults to `true`): a value of `false` specifies that the elements of the collection never change. This allows for minor performance optimization in some cases.
- (16) `subselect` (optional): maps an immutable and read-only collection to a database sub-select. This is useful if you want to have a view instead of a base table. It is not used for one-to-many associations.
- (17) `check` (optional): an SQL expression used to generate a multi-row check constraint for automatic schema generation. It is not used for one-to-many associations.
- (18) `persister` (optional): specifies a custom `ICollectionPersister`.
- (19) `collection-type` (optional): the fully qualified type name of a class implementing `IUserCollectionType`.
- (20) `outer-join` (optional): This attribute is obsoleted in favor of `fetch`. `auto` is equivalent to not specifying `fetch`, `true` is equivalent to `join` and `false` is equivalent to `select`. It has no value matching the subselect `fetch` value.
- (21) `generic` (optional, defaults according to the reflected property type): if `false`, NHibernate will type the collection elements as `object`. Otherwise NHibernate will use reflection to determine the element type to use.

The mapping of an `IList` or array requires a separate table column holding the array or list index (the `i` in `foo[i]`). If your relational model doesn't have an index column, use an unordered `ISet` instead. This seems to put people off who assume that `IList` should just be a more convenient way of accessing an unordered collection. NHibernate collections strictly obey the actual semantics attached to the `ISet`, `IList` and `IDictionary` interfaces. `IList` elements don't just spontaneously rearrange themselves!

On the other hand, people who planned to use the `IList` to emulate *bag* semantics have a legitimate grievance here. A bag is an unordered, unindexed collection which may contain the same element multiple times. The .NET collections framework lacks an `IBag` interface, hence you have to emulate it with an `IList`. NHibernate lets you map properties of type `IList` or `ICollection` with the `<bag>` element. Note that bag semantics are not really part of the `ICollection` contract and they actually conflict with the semantics of the `IList` contract (however, you can sort the bag arbitrarily, discussed later in this chapter).

Note: large NHibernate bags mapped with `inverse="false"` are inefficient and should be avoided. NHibernate can't create, delete or update rows individually, because there is no key that may be used to identify an individual row.

6.2.1. Collection foreign keys

Collection instances are distinguished in the database by the foreign key of the entity that owns the collection. This foreign key is referred to as the *collection key column*, or columns, of the collection table. The collection key column is mapped by the `<key>` element.

There can be a nullability constraint on the foreign key column. For most collections, this is implied. For uni-

directional one-to-many associations, the foreign key column is nullable by default, so you may need to specify `not-null="true"`.

```
<key column="productSerialNumber" not-null="true"/>
```

The foreign key constraint can use `ON DELETE CASCADE`.

```
<key column="productSerialNumber" on-delete="cascade"/>
```

See Section 5.1.20, “key” for a full definition of the `<key>` element.

6.2.2. Collection elements

Collections can contain almost any other NHibernate type, including: basic types, custom types, components and references to other entities. This is an important distinction. An object in a collection might be handled with “value” semantics (its life cycle fully depends on the collection owner), or it might be a reference to another entity with its own life cycle. In the latter case, only the “link” between the two objects is considered to be a state held by the collection.

The contained type is referred to as the collection element type. Collection elements are mapped by `<element>` or `<composite-element>`, or in the case of entity references, with `<one-to-many>` or `<many-to-many>`. The first two map elements with value semantics, the next two are used to map entity associations. A last one exist for exotic entity associations, `<many-to-any>`, similar to the `<any>` mapping element.

6.2.3. Indexed collections

All collection mappings, except those with set and bag semantics, need an *index column* in the collection table. An index column is a column that maps to an array index, or `IList` index, or `IDictionary` key. The index of an `IDictionary` may be of any basic type, mapped with `<map-key>`. It can be an entity reference mapped with `<map-key-many-to-many>` (or in exotic cases `<index-many-to-any>`, similar to the `<any>` mapping element), or it can be a composite type mapped with `<composite-map-key>`. The index of an array or list is always of type `Int32` and is mapped using the `<list-index>` element. The mapped column contains sequential integers that are numbered from zero by default.

```
<list-index
  column="columnName"           (1)
  base="0|1|..."             (2)
/>
```

- (1) `column` (required): the name of the column holding the collection index values.
- (2) `base` (optional - defaults to 0): the value of the index column that corresponds to the first element of the list or array.

```
<map-key
  column="columnName"           (1)
  formula="any SQL expression" (2)
  type="typeName"              (3)
  length="N"                   (4)
/>
```

- (1) `column` (required unless `formula` is used): the name of the column holding the map key values.
- (2) `formula` (optional): a SQL formula used to evaluate the key of the map.
- (3) `type` (required): the type of the map keys.
- (4) `length` (optional): If the type takes a length and does not already specify it, its length.

```

<map-key-many-to-many
  column="columnName"           (1)
  formula="any SQL expression" (2)
  class="className"            (3)
  entity-name="entityName"     (4)
  foreign-key="foreignKeyName" (5)
/>

```

- (1) column (required unless formula is used): the name of the column holding the map key values.
- (2) formula (optional): a SQL formula used to evaluate the key of the map.
- (3) class (required): the entity class used as the map key.
- (4) entity-name (optional): the entity name of the associated class.
- (5) foreign-key (optional): specifies the name of the foreign key constraint for DDL generation.

```

<composite-map-key
  class="className">           (1)

  <key-property .../>          (2)
  <key-many-to-one .../>       (3)
  ...
/>

```

- (1) class (required): the component class used as the map key. Make sure you override `GetHashCode()` and `Equals()` correctly on the component class.
- (2) `<key-property>`: maps a component property.
- (3) `<key-many-to-one>`: maps a component many-to-one association.

If your table does not have an index column, and you still wish to use `IList` as the property type, you can map the property as a NHibernate `<bag>`. A bag does not retain its order when it is retrieved from the database, but it can be optionally sorted or ordered. Also consider using an `ISet` as the property type.

The `<index>`, `<index-many-to-many>` and `<composite-index>` elements are obsoleted predecessors to `<map-key>`/`<list-index>`, `<map-key-many-to-many>` and `<composite-map-key>` respectively.

6.2.4. Collections of Values and Many-To-Many Associations

Any collection of values or many-to-many associations requires a dedicated *collection table* with a foreign key column or columns, *collection element column* or columns, and possibly an index column or columns.

For a collection of values, use the `<element>` tag.

```

<element
  column="columnName"           (1)
  formula="any SQL expression" (2)
  type="typeName"              (3)
  length="L"                   (4)
  precision="P"                 (5)
  scale="S"                    (6)
  not-null="true|false"        (7)
  unique="true|false"          (8)
/>

```

- (1) column (required unless formula is used): the name of the column holding the collection element values.
- (2) formula (optional): an SQL formula used to evaluate the element.
- (3) type (required): the type of the collection element.
- (4) length (optional): if the type takes a length and does not already specify it, its length.
- (5) precision (optional): if the type takes a precision and does not already specify it, its precision.
- (6) scale (optional): if the type takes a scale and does not already specify it, its scale.

- (7) not-null (optional - defaults to false): sets the column nullability for DDL generation.
- (8) unique (optional - defaults to false): sets the column uniqueness for DDL generation.

A *many-to-many* association is specified using the <many-to-many> element.

```
<many-to-many
  column="columnName" (1)
  formula="any SQL expression" (2)
  class="className" (3)
  fetch="join|select" (4)
  lazy="false|proxy" (5)
  not-found="ignore|exception" (6)
  entity-name="entityName" (7)
  property-ref="propertyNameFromAssociatedClass" (8)
  order-by="columnName asc|desc" (9)
  where="arbitrary sql where condition" (10)
  unique="true|false" (11)
  foreign-key="foreignKeyName" (12)
  outer-join="auto|true|false" (13)
/>
```

- (1) column (required unless formula is used): the name of the element foreign key column.
- (2) formula (optional): an SQL formula used to evaluate the element foreign key value.
- (3) class (required): the name of the associated class.
- (4) fetch (optional, defaults to join): enables outer-join or sequential select fetching for this association. This is a special case; for full eager fetching (in a single SELECT) of an entity and its many-to-many relationships to other entities, you would enable join fetching not only of the collection itself, but also with this attribute on the <many-to-many> nested element.
- (5) lazy (optional, defaults to proxy): may be used to disable lazy fetching and specify that the associated entities are always eagerly fetched. This is a special case; disabling laziness of the collection itself only would only eagerly load the list of ids of the associated entities. For fully loading the associated entities instead, while keeping fetch value as select, you would also disable laziness with this attribute on the <many-to-many> nested element.
- (6) not-found (optional - defaults to exception): specifies how foreign keys that reference missing rows will be handled. ignore will treat a missing row as a null association.
- (7) entity-name (optional): the entity name of the associated class.
- (8) property-ref (optional): the name of a property of the associated class that is joined to this foreign key. If not specified, the primary key of the associated class is used.
- (9) order-by (optional): specifies a table column (or columns) that defines the iteration order of the collection. This is a special case; the order-by on the collection could order only by the foreign key. The order-by on the <many-to-many> nested element allows to order by the associated entity columns.
- (10) where (optional): specifies an arbitrary SQL WHERE condition to be used when retrieving or removing the collection. This is a special case; the where on the collection could filter only by the foreign key. The where on the <many-to-many> nested element allows to filter by the associated entity columns.
- (11) unique (optional): enables the DDL generation of a unique constraint for the foreign-key column. This makes the association multiplicity effectively one-to-many.
- (12) foreign-key (optional): specifies the name of the foreign key constraint for DDL generation.
- (13) outer-join (optional): This attribute is obsoleted in favor of fetch. auto is equivalent to not specifying fetch, true is equivalent to join and false is equivalent to select.

Here are some examples.

A set of strings:

```
<set name="Names" table="NAMES">
  <key column="GROUPID"/>
  <element column="NAME" type="String"/>
</set>
```

A bag containing integers with an iteration order determined by the order-by attribute:

```
<bag name="Sizes" table="SIZES" order-by="SIZE ASC">
  <key column="OWNER"/>
  <element column="SIZE" type="Int32"/>
</bag>
```

An array of entities, in this case, a many-to-many association (note that the entities are lifecycle objects, cascade="all", see Section 10.10, “Lifecycles and object graphs”):

```
<array name="Addresses" table="PersonAddress" cascade="all">
  <key column="personId"/>
  <list-index column="sortOrder"/>
  <many-to-many column="addressId" class="Address"/>
</array>
```

A map from string indices to dates:

```
<map name="Holidays" table="holidays" schema="dbo" order-by="hol_name asc">
  <key column="id"/>
  <map-key column="hol_name" type="string"/>
  <element column="hol_date" type="Date"/>
</map>
```

A list of components (discussed in Chapter 8, *Component Mapping*):

```
<list name="CarComponents" table="car_components">
  <key column="car_id"/>
  <list-index column="sortOrder"/>
  <composite-element class="Eg.Car.CarComponent">
    <property name="Price"/>
    <property name="Type"/>
    <property name="SerialNumber" column="serial_no"/>
  </composite-element>
</list>
```

6.2.5. One-To-Many Associations

A *one-to-many association* links the tables of two classes via a foreign key with no intervening collection table. This mapping loses certain semantics of normal .NET collections:

- No null values may be contained in a dictionary, set or list (unless using not-found value ignore).
- An instance of the contained entity class cannot belong to more than one instance of the collection.
- An instance of the contained entity class cannot appear at more than one value of the collection index.

An association from Product to Part requires the existence of a foreign key column and possibly an index column into the Part table. A <one-to-many> tag indicates that this is a one-to-many association.

```
<one-to-many
  class="className" (1)
  not-found="ignore|exception" (2)
  entity-name="entityName" (3)
/>
```

- (1) class (required): the name of the associated class.
- (2) not-found (optional - defaults to exception): specifies how foreign keys that reference missing rows will be handled. ignore will treat a missing row as a null association.
- (3) entity-name (optional): the entity name of the associated class.

The `<one-to-many>` element does not need to declare any columns. Nor is it necessary to specify the table name anywhere.

Very Important Note: If the foreign key column of an `<one-to-many>` association is declared `NOT NULL`, you must declare the `<key>` mapping `not-null="true"` or use a *bidirectional association* with the collection mapping marked `inverse="true"`. See Section 6.3.2, “Bidirectional Associations” for more information.

Example:

```
<set name="Parts">
  <key column="product_id"/>
  <one-to-many class="Eg.Part, Eg"/>
</set>
```

6.3. Advanced collection mappings

6.3.1. Sorted Collections

NHibernate supports collections implemented by `System.Collections.Generic.SortedList<T>` and `System.Collections.Generic.SortedSet<T>`. You must specify a comparer in the mapping file:

```
<set name="Aliases" table="person_aliases" sort="natural">
  <key column="person"/>
  <element column="name" type="String"/>
</set>

<map name="Holidays" sort="My.Custom.HolidayComparer, MyAssembly">
  <key column="year_id"/>
  <map-key column="hol_name" type="String"/>
  <element column="hol_date" type="Date"/>
</map>
```

Allowed values of the `sort` attribute are `unsorted`, `natural` and the name of a class implementing `System.Collections.Generic.IComparer<T>`.

If you want the database itself to order the collection elements use the `order-by` attribute of `set`, `bag` or `map` mappings. This performs the ordering in the SQL query, not in memory.

Setting the `order-by` attribute tells NHibernate to use `Iesi.Collections.Generic.LinkedHashSet` class internally for sets, maintaining the order of the elements. It is not supported on maps, unless specifying also a custom collection-type implementing `IUserCollectionType`.

```
<set name="Aliases" table="person_aliases" order-by="name asc">
  <key column="person"/>
  <element column="name" type="String"/>
</set>

<map name="Holidays" order-by="hol_date, hol_name" collection-type="...">
  <key column="year_id"/>
  <map-key column="hol_name" type="String"/>
  <element column="hol_date" type="Date"/>
</map>
```

Note that the value of the `order-by` attribute is an SQL ordering, not a HQL ordering!

Associations can even be sorted by some arbitrary criteria at runtime using a `CreateFilter()`.

```
sortedUsers = s
```

```
.CreateFilter(group.Users, "order by this.Name")
.List<User>();
```

6.3.2. Bidirectional Associations

A *bidirectional association* allows navigation from both "ends" of the association. Two kinds of bidirectional association are supported:

one-to-many

set or bag valued at one end, single-valued at the other

many-to-many

set or bag valued at both ends

You can specify a bidirectional many-to-many association simply by mapping two many-to-many associations to the same database table and declaring one end as *inverse*. You cannot select an indexed collection.

Here is an example of a bidirectional many-to-many association that illustrates how each category can have many items and each item can be in many categories:

```
<class name="Category">
  <id name="Id" column="CATEGORY_ID"/>
  ...
  <bag name="Items" table="CATEGORY_ITEM">
    <key column="CATEGORY_ID"/>
    <many-to-many class="Item" column="ITEM_ID"/>
  </bag>
</class>

<class name="Item">
  <id name="Id" column="ITEM_ID"/>
  ...

  <!-- inverse end -->
  <bag name="Categories" table="CATEGORY_ITEM" inverse="true">
    <key column="ITEM_ID"/>
    <many-to-many class="Category" column="CATEGORY_ID"/>
  </bag>
</class>
```

Changes made only to the inverse end of the association are *not* persisted. This means that NHibernate has two representations in memory for every bidirectional association, one link from A to B and another link from B to A. This is easier to understand if you think about the .NET object model and how we create a many-to-many relationship in C#:

```
category.Items.Add(item);      // The category now "knows" about the relationship
item.Categories.Add(category); // The item now "knows" about the relationship

session.Update(item);          // No effect, nothing will be saved!
session.Update(category);      // The relationship will be saved
```

The non-inverse side is used to save the in-memory representation to the database. We would get an unnecessary INSERT/UPDATE and probably even a foreign key violation if both would trigger changes! The same is of course also true for bidirectional one-to-many associations.

You can map a bidirectional one-to-many association by mapping a one-to-many association to the same table column(s) as a many-to-one association and declaring the many-valued end *inverse="true"*.

```

<class name="Parent">
  <id name="Id" column="id"/>
  ...
  <set name="Children" inverse="true">
    <key column="parent_id"/>
    <one-to-many class="Child"/>
  </set>
</class>

<class name="Child">
  <id name="Id" column="id"/>
  ...
  <many-to-one name="Parent" class="Parent" column="parent_id"/>
</class>

```

Mapping one end of an association with `inverse="true"` doesn't affect the operation of cascades as these are orthogonal concepts.

6.3.3. Bidirectional associations with indexed collections

A bidirectional association where one end is represented as a `<list>` or `<map>`, requires special consideration. If there is a property of the child class that maps to the index column you can use `inverse="true"` on the collection mapping:

```

<class name="Parent">
  <id name="Id" column="parent_id"/>
  ...
  <map name="Children" inverse="true">
    <key column="parent_id"/>
    <map-key column="name"
      type="string"/>
    <one-to-many class="Child"/>
  </map>
</class>

<class name="Child">
  <id name="Id" column="child_id"/>
  ...
  <property name="Name" column="name"
    not-null="true"/>
  <many-to-one name="Parent"
    class="Parent"
    column="parent_id"
    not-null="true"/>
</class>

```

If there is no such property on the child class, the association cannot be considered truly bidirectional. That is, there is information available at one end of the association that is not available at the other end. In this case, you cannot map the collection `inverse="true"`. Instead, you could use the following mapping:

```

<class name="Parent">
  <id name="Id" column="parent_id"/>
  ...
  <map name="Children">
    <key column="parent_id"
      not-null="true"/>
    <map-key column="name"
      type="string"/>
    <one-to-many class="Child"/>
  </map>
</class>

<class name="Child">
  <id name="Id" column="child_id"/>
  ...

```

```

    <many-to-one name="Parent"
      class="Parent"
      column="parent_id"
      insert="false"
      update="false"
      not-null="true"/>
  </class>

```

Note that in this mapping, the collection-valued end of the association is responsible for updates to the foreign key.

6.3.4. Ternary Associations

There are three possible approaches to mapping a ternary association. One approach is to use an `IDictionary` with an association as its index:

```

<map name="Contracts">
  <key column="employer_id"/>
  <map-key-many-to-many column="employee_id" class="Employee"/>
  <one-to-many class="Contract"/>
</map>

```

```

<map name="Connections">
  <key column="incoming_node_id"/>
  <map-key-many-to-many column="outgoing_node_id" class="Node"/>
  <many-to-many column="connection_id" class="Connection"/>
</map>

```

A second approach is to remodel the association as an entity class. This is the most common approach.

A final alternative is to use composite elements, see Section 8.2, “Collections of dependent objects”.

6.3.5. Using an `<idbag>`

If you've fully embraced our view that composite keys are a bad thing and that entities should have synthetic identifiers (surrogate keys), then you might find it a bit odd that the many to many associations and collections of values that we've shown so far all map to tables with composite keys! Now, this point is quite arguable; a pure association table doesn't seem to benefit much from a surrogate key (though a collection of composite values *might*). Nevertheless, NHibernate provides a feature that allows you to map many-to-many associations and collections of values to a table with a surrogate key.

The `<idbag>` element lets you map a `List` (or `Collection`) with bag semantics.

```

<idbag name="Lovers" table="LOVERS">
  <collection-id column="ID" type="Int64">
    <generator class="hilo"/>
  </collection-id>
  <key column="PERSON1"/>
  <many-to-many column="PERSON2" class="Person" fetch="join"/>
</idbag>

```

An `<idbag>` has a synthetic id generator, just like an entity class. A different surrogate key is assigned to each collection row. NHibernate does not, however, provide any mechanism to discover the surrogate key value of a particular row.

The update performance of an `<idbag>` supersedes a regular `<bag>`. NHibernate can locate individual rows efficiently and update or delete them individually, similar to a `<list>`, `<map>` or `<set>`.

As of version 2.0, the native identifier generation strategy is supported for <idbag> collection identifiers.

6.3.6. Heterogeneous Associations

The <many-to-any> and <index-many-to-any> elements provide for true heterogeneous associations. These mapping elements work in the same way than the <any> element - and should also be used rarely, if ever.

6.4. Lazy Initialization

Collections (other than arrays) may be lazily initialized, meaning they load their state from the database only when the application needs to access it. Initialization happens transparently to the user so the application would not normally need to worry about this (in fact, transparent lazy initialization is the main reason why NHibernate needs its own collection implementations). However, if the application tries something like this:

```
IDictionary<string, int> permissions;
using (s = sessions.OpenSession())
using (ITransaction tx = sessions.BeginTransaction())
{
    var u = s.Load<User>(userId);
    permissions = u.Permissions;
    tx.Commit();
}

int accessLevel = permissions["accounts"]; // Error!
```

It could be in for a nasty surprise. Since the permissions collection was not initialized when the `ISession` was committed, the collection will never be able to load its state. The fix is to move the line that reads from the collection to just before the commit. (There are other more advanced ways to solve this problem, however.)

Alternatively, use a non-lazy collection. However, it is intended that lazy initialization be used for almost all collections, especially for collections of entities (for reasons of efficiency).

Exceptions that occur while lazily initializing a collection are wrapped in a `LazyInitializationException`.

Disable lazy initialization of collection by using the optional `lazy` attribute:

```
<set name="Names" table="NAMES" lazy="false">
  <key column="group_id"/>
  <element column="NAME" type="String"/>
</set>
```

In some application architectures, particularly where the code that accesses data using NHibernate, and the code that uses it are in different application layers, it can be a problem to ensure that the `ISession` is open when a collection is initialized. There are two basic ways to deal with this issue:

- In a web-based application, an event handler can be used to close the `ISession` only at the very end of a user request, once the rendering of the view is complete. Of course, this places heavy demands upon the correctness of the exception handling of your application infrastructure. It is vitally important that the `ISession` is closed and the transaction ended before returning to the user, even when an exception occurs during rendering of the view. The event handler has to be able to access the `ISession` for this approach. We recommend that the current `ISession` is stored in the `HttpContext.Items` collection (see chapter 1, Section 1.4, “Playing with cats”, for an example implementation).
- In an application with a separate business tier, the business logic must “prepare” all collections that will be

needed by the web tier before returning. This means that the business tier should load all the data and return all the data already initialized to the presentation/web tier that is required for a particular use case. Usually, the application calls `NHibernateUtil.Initialize()` for each collection that will be needed in the web tier (this call must occur before the session is closed) or retrieves the collection eagerly using a NHibernate query with a `FETCH` clause.

- You may also attach a previously loaded object to a new `ISession` with `Update()` or `Lock()` before accessing uninitialized collections (or other proxies). NHibernate can not do this automatically, as it would introduce ad hoc transaction semantics!

You can use the `CreateFilter()` method of the NHibernate `ISession` API to get the size of a collection without initializing it:

```
var count = s
    .CreateFilter(collection, "select count(*)")
    .UniqueResult<long>();
```

`CreateFilter()` is also used to efficiently retrieve subsets of a collection without needing to initialize the whole collection.

6.5. Collection examples

This section covers collection examples.

The following class has a collection of `Child` instances:

```
using System;
using System.Collections.Generic;

namespace Eg
{
    public class Parent
    {
        public virtual long Id { get; set; }

        public virtual ISet<Child> Children { get; set; }

        ...
    }
}
```

If each child has at most one parent, the most natural mapping is a one-to-many association:

```
<hibernate-mapping xmlns="urn:hibernate-mapping-2.2"
    assembly="Eg" namespace="Eg">

    <class name="Parent">
        <id name="Id">
            <generator class="sequence"/>
        </id>
        <set name="Children">
            <key column="parent_id"/>
            <one-to-many class="Child"/>
        </set>
    </class>

    <class name="Child">
        <id name="Id">
            <generator class="sequence"/>
        </id>
        <property name="Name"/>
    </class>
</hibernate-mapping>
```

```

    </class>
</hibernate-mapping>

```

This maps to the following table definitions:

```

create table parent (Id bigint not null primary key)
create table child (Id bigint not null primary key, Name varchar(255), parent_id bigint)
alter table child add constraint childfk0 (parent_id) references parent

```

The child class has no property referencing its parent.

If the parent is *required*, use a bidirectional one-to-many association:

```

<hibernate-mapping xmlns="urn:hibernate-mapping-2.2"
  assembly="Eg" namespace="Eg">

  <class name="Parent">
    <id name="Id">
      <generator class="sequence"/>
    </id>
    <set name="Children" inverse="true">
      <key column="parent_id"/>
      <one-to-many class="Child"/>
    </set>
  </class>

  <class name="Child">
    <id name="Id">
      <generator class="sequence"/>
    </id>
    <property name="Name"/>
    <many-to-one name="Parent" class="Parent" column="parent_id" not-null="true"/>
  </class>

</hibernate-mapping>

```

Notice the NOT NULL constraint:

```

create table parent ( Id bigint not null primary key )
create table child ( Id bigint not null
                    primary key,
                    Name varchar(255),
                    parent_id bigint not null )
alter table child add constraint childfk0 (parent_id) references parent

```

Here, the Child class has a Parent property.

Alternatively, if this association must be unidirectional you can declare the NOT NULL constraint on the <key> mapping:

```

<hibernate-mapping xmlns="urn:hibernate-mapping-2.2"
  assembly="Eg" namespace="Eg">

  <class name="Parent">
    <id name="Id">
      <generator class="sequence"/>
    </id>
    <set name="Children">
      <key column="parent_id" not-null="true"/>
      <one-to-many class="Child"/>
    </set>
  </class>

  <class name="Child">

```

```

        <id name="Id">
            <generator class="sequence"/>
        </id>
        <property name="Name"/>
    </class>

</hibernate-mapping>

```

On the other hand, if a child might have multiple parents, a many-to-many association is appropriate:

```

<hibernate-mapping xmlns="urn:nhibernate-mapping-2.2"
    assembly="Eg" namespace="Eg">

    <class name="Parent">
        <id name="Id">
            <generator class="sequence"/>
        </id>
        <set name="Children" table="childset">
            <key column="parent_id"/>
            <many-to-many class="Child" column="child_id"/>
        </set>
    </class>

    <class name="eg.Child">
        <id name="Id">
            <generator class="sequence"/>
        </id>
        <property name="Name"/>
    </class>

</hibernate-mapping>

```

Table definitions:

```

create table parent ( Id bigint not null primary key )
create table child ( Id bigint not null primary key, name varchar(255) )
create table childset ( parent_id bigint not null,
                        child_id bigint not null,
                        primary key ( parent_id, child_id ) )
alter table childset add constraint childsetfk0 (parent_id) references parent
alter table childset add constraint childsetfk1 (child_id) references child

```

For more examples and a complete explanation of a parent/child relationship mapping, see Chapter 7, *Association Mappings* and Chapter 23, *Example: Parent/Child*.

Chapter 7. Association Mappings

7.1. Introduction

Association mappings are often the most difficult thing to implement correctly. In this section we examine some canonical cases one by one, starting with unidirectional mappings and then bidirectional cases. We will use Person and Address in all the examples.

Associations will be classified by multiplicity and whether or not they map to an intervening join table.

Nullable foreign keys are not considered to be good practice in traditional data modelling, so our examples do not use nullable foreign keys. This is not a requirement of NHibernate, and the mappings will work if you drop the nullability constraints.

7.2. Unidirectional associations

7.2.1. Many-to-one

An *unidirectional many-to-one association* is the most common kind of unidirectional association.

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <many-to-one name="Address"
    column="addressId"
    not-null="true"/>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
</class>
```

```
create table Person (
  personId bigint not null primary key, addressId bigint not null
)
create table Address ( addressId bigint not null primary key )
```

7.2.2. One-to-one

An *unidirectional one-to-one association on a foreign key* is almost identical to a many-to-one. The only difference is the column unique constraint.

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <many-to-one name="Address"
    column="addressId"
    unique="true"
    not-null="true"/>
</class>
```

```
<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
</class>
```

```
create table Person (
  personId bigint not null primary key, addressId bigint not null unique
)
create table Address ( addressId bigint not null primary key )
```

An *unidirectional one-to-one association on a primary key* usually uses a special id generator. In this example, however, we have reversed the direction of the association:

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
</class>

<class name="Address">
  <id name="Id" column="personId">
    <generator class="foreign">
      <param name="property">Person</param>
    </generator>
  </id>
  <one-to-one name="Person" constrained="true"/>
</class>
```

```
create table Person ( personId bigint not null primary key )
create table Address ( personId bigint not null primary key )
```

7.2.3. One-to-many

An *unidirectional one-to-many association on a foreign key* is an unusual case, and is not recommended.

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <set name="Addresses">
    <key column="personId"
      not-null="true"/>
    <one-to-many class="Address"/>
  </set>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
</class>
```

```
create table Person ( personId bigint not null primary key )
create table Address (
  addressId bigint not null primary key, personId bigint not null
)
```

7.3. Unidirectional associations with join tables

7.3.1. One-to-Many

An *unidirectional one-to-many association on a join table* is the preferred option. Specifying `unique="true"` changes the multiplicity from many-to-many to one-to-many.

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <set name="Addresses" table="PersonAddress">
    <key column="personId"/>
    <many-to-many column="addressId"
      unique="true"
      class="Address"/>
  </set>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
</class>
```

```
create table Person ( personId bigint not null primary key )
create table PersonAddress (
  personId not null, addressId bigint not null primary key
)
create table Address ( addressId bigint not null primary key )
```

7.3.2. Many-to-one

An *unidirectional many-to-one association on a join table* is common when the association is optional (while avoiding nullable foreign key).

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <join table="PersonAddress"
    optional="true">
    <key column="personId" unique="true"/>
    <many-to-one name="Address"
      column="addressId"
      not-null="true"/>
  </join>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
</class>
```

```
create table Person ( personId bigint not null primary key )
create table PersonAddress (
  personId bigint not null primary key, addressId bigint not null
)
create table Address ( addressId bigint not null primary key )
```

7.3.3. One-to-one

An *unidirectional one-to-one association on a join table* is possible, but extremely unusual.

```

<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <join table="PersonAddress"
    optional="true">
    <key column="personId"
      unique="true"/>
    <many-to-one name="Address"
      column="addressId"
      not-null="true"
      unique="true"/>
  </join>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
</class>

```

```

create table Person ( personId bigint not null primary key )
create table PersonAddress (
  personId bigint not null primary key, addressId bigint not null unique
)
create table Address ( addressId bigint not null primary key )

```

7.3.4. Many-to-many

Finally, here is an example of an *unidirectional many-to-many association*.

```

<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <set name="Addresses" table="PersonAddress">
    <key column="personId"/>
    <many-to-many column="addressId"
      class="Address"/>
  </set>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
</class>

```

```

create table Person ( personId bigint not null primary key )
create table PersonAddress (
  personId bigint not null, addressId bigint not null,
  primary key (personId, addressId)
)
create table Address ( addressId bigint not null primary key )

```

7.4. Bidirectional associations

7.4.1. One-to-many / many-to-one

A *bidirectional many-to-one association* is the most common kind of association. The following example illustrates the standard parent/child relationship.

```

<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <many-to-one name="Address"
    column="addressId"
    not-null="true"/>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
  <set name="People" inverse="true">
    <key column="addressId"/>
    <one-to-many class="Person"/>
  </set>
</class>

```

```

create table Person (
  personId bigint not null primary key, addressId bigint not null
)
create table Address ( addressId bigint not null primary key )

```

If you use an `ICollection`, or other indexed collection, set the key column of the foreign key to `not null`. NHibernate will manage the association from the collections side to maintain the index of each element, making the other side virtually inverse by setting `update="false"` and `insert="false"`:

```

<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <many-to-one name="Address"
    column="addressId"
    not-null="true"
    insert="false"
    update="false"/>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
  <list name="People">
    <key column="addressId" not-null="true"/>
    <list-index column="peopleIdx"/>
    <one-to-many class="Person"/>
  </list>
</class>

```

```

create table Person (
  personId bigint not null primary key,
  addressId bigint not null,
  peopleIdx int not null
)
create table Address ( addressId bigint not null primary key )

```

When the underlying foreign key column is `NOT NULL`, it is important that you define `not-null="true"` on the `<key>` element of the collection mapping. Do not only declare `not-null="true"` on a possible nested `<column>` element, but always declare it on the `<key>` element too.

7.4.2. One-to-one

A *bidirectional one-to-one association on a foreign key* is common.

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <many-to-one name="Address"
    column="addressId"
    unique="true"
    not-null="true"/>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
  <one-to-one name="Person"
    property-ref="Address"/>
</class>
```

```
create table Person (
  personId bigint not null primary key, addressId bigint not null unique
)
create table Address ( addressId bigint not null primary key )
```

A *bidirectional one-to-one association on a primary key* uses the special foreign id generator.

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <one-to-one name="Address"/>
</class>

<class name="Address">
  <id name="Id" column="personId">
    <generator class="foreign">
      <param name="property">Person</param>
    </generator>
  </id>
  <one-to-one name="Person"
    constrained="true"/>
</class>
```

```
create table Person ( personId bigint not null primary key )
create table Address ( personId bigint not null primary key )
```

7.5. Bidirectional associations with join tables

7.5.1. One-to-many / many-to-one

The following is an example of a *bidirectional one-to-many association on a join table*. The `inverse="true"` can go on either end of the association, on the collection, or on the join.

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <set name="Addresses"
    table="PersonAddress">
    <key column="personId"/>
    <many-to-many column="addressId"
```

```

        unique="true"
        class="Address"/>
    </set>
</class>

<class name="Address">
    <id name="Id" column="addressId">
        <generator class="native"/>
    </id>
    <join table="PersonAddress"
        inverse="true"
        optional="true">
        <key column="addressId"/>
        <many-to-one name="Person"
            column="personId"
            not-null="true"/>
    </join>
</class>

```

```

create table Person ( personId bigint not null primary key )
create table PersonAddress (
    personId bigint not null, addressId bigint not null primary key
)
create table Address ( addressId bigint not null primary key )

```

7.5.2. One-to-one

A *bidirectional one-to-one association on a join table* is possible, but extremely unusual.

```

<class name="Person">
    <id name="Id" column="personId">
        <generator class="native"/>
    </id>
    <join table="PersonAddress"
        optional="true">
        <key column="personId"
            unique="true"/>
        <many-to-one name="Address"
            column="addressId"
            not-null="true"
            unique="true"/>
    </join>
</class>

<class name="Address">
    <id name="Id" column="addressId">
        <generator class="native"/>
    </id>
    <join table="PersonAddress"
        optional="true"
        inverse="true">
        <key column="addressId"
            unique="true"/>
        <many-to-one name="Person"
            column="personId"
            not-null="true"
            unique="true"/>
    </join>
</class>

```

```

create table Person ( personId bigint not null primary key )
create table PersonAddress (
    personId bigint not null primary key, addressId bigint not null unique
)
create table Address ( addressId bigint not null primary key )

```

7.5.3. Many-to-many

Here is an example of a *bidirectional many-to-many association*.

```
<class name="Person">
  <id name="Id" column="personId">
    <generator class="native"/>
  </id>
  <set name="Addresses" table="PersonAddress">
    <key column="personId"/>
    <many-to-many column="addressId"
      class="Address"/>
  </set>
</class>

<class name="Address">
  <id name="Id" column="addressId">
    <generator class="native"/>
  </id>
  <set name="People" inverse="true" table="PersonAddress">
    <key column="addressId"/>
    <many-to-many column="personId"
      class="Person"/>
  </set>
</class>
```

```
create table Person ( personId bigint not null primary key )
create table PersonAddress (
  personId bigint not null, addressId bigint not null,
  primary key (personId, addressId)
)
create table Address ( addressId bigint not null primary key )
```

7.6. More complex association mappings

More complex association joins are *extremely* rare. NHibernate handles more complex situations by using SQL fragments embedded in the mapping document. For example, if a table with historical account information data defines AccountNumber, EffectiveEndDate and EffectiveStartDate columns, it would be mapped as follows:

```
<properties name="CurrentAccountKey">
  <property name="AccountNumber" type="string" not-null="true"/>
  <property name="CurrentAccount" type="boolean">
    <formula>case when EffectiveEndDate is null then 1 else 0 end</formula>
  </property>
</properties>
<property name="EffectiveEndDate" type="date"/>
<property name="EffectiveStateDate" type="date" not-null="true"/>
```

You can then map an association to the *current* instance, the one with null EffectiveEndDate, by using:

```
<many-to-one name="CurrentAccountInfo"
  property-ref="CurrentAccountKey"
  class="AccountInfo">
  <column name="AccountNumber"/>
  <formula>'1'</formula>
</many-to-one>
```

In a more complex example, imagine that the association between Employee and Organization is maintained in an Employment table full of historical employment data. An association to the employee's *most recent* employer, the one with the most recent startDate, could be mapped in the following way:

```
<join>
  <key column="employeeId"/>
  <subselect>
    select employeeId, orgId
    from Employments
    group by orgId
    having startDate = max(startDate)
  </subselect>
  <many-to-one name="MostRecentEmployer"
    class="Organization"
    column="orgId"/>
</join>
```

This functionality allows a degree of creativity and flexibility, but it is more practical to handle these kinds of cases by using queries.

Chapter 8. Component Mapping

The notion of a *component* is re-used in several different contexts, for different purposes, throughout NHibernate.

8.1. Dependent objects

A component is a contained object that is persisted as a value type and not an entity reference. The term "component" refers to the object-oriented notion of composition and not to architecture-level components. For example, you can model a person like this:

```
public class Person
{
    public virtual string Key { get; set; }

    public virtual DateTime Birthday { get; set; }

    public virtual Name Name { get; set; }

    ...
}
```

```
public class Name
{
    public string First { get; set; }

    public string Last { get; set; }

    public char Initial { get; set; }
}
```

Now Name may be persisted as a component of Person. Name defines getter and setter methods for its persistent properties, but it does not need to declare any interfaces or identifier properties.

Our NHibernate mapping would look like:

```
<class name="Eg.Person, Eg" table="person">
  <id name="Key" column="pid" type="string">
    <generator class="uuid.hex"/>
  </id>
  <property name="Birthday" type="date"/>
  <component name="Name" class="Eg.Name, Eg"> <!-- class attribute optional -->
    <property name="Initial"/>
    <property name="First"/>
    <property name="Last"/>
  </component>
</class>
```

The person table would have the columns pid, Birthday, Initial, First and Last.

Like value types, components do not support shared references. In other words, two persons could have the same name, but the two person objects would contain two independent name objects that were only "the same" by value. The null value semantics of a component are *ad hoc*. When reloading the containing object, NHibernate will assume that if all component columns are null, then the entire component is null. This is suitable for most purposes.

The properties of a component can be of any NHibernate type (collections, many-to-one associations, other components, etc). Nested components should *not* be considered an exotic usage. NHibernate is intended to sup-

port a fine-grained object model.

The `<component>` element allows a `<parent>` sub-element that maps a property of the component class as a reference back to the containing entity.

```
<class name="Eg.Person, Eg" table="person">
  <id name="Key" column="pid" type="string">
    <generator class="uuid.hex"/>
  </id>
  <property name="Birthday" type="date"/>
  <component name="Name" class="Eg.Name, Eg">
    <parent name="NamedPerson"/> <!-- reference back to the Person -->
    <property name="Initial"/>
    <property name="First"/>
    <property name="Last"/>
  </component>
</class>
```

8.2. Collections of dependent objects

Collections of components are supported (eg. an array of type `Name`). Declare your component collection by replacing the `<element>` tag with a `<composite-element>` tag.

```
<set name="SomeNames" table="some_names">
  <key column="id"/>
  <composite-element class="Eg.Name, Eg"> <!-- class attribute required -->
    <property name="Initial"/>
    <property name="First"/>
    <property name="Last"/>
  </composite-element>
</set>
```

Note: if you define an `ISet` of composite elements, it is very important to implement `Equals()` and `GetHashCode()` correctly.

Composite elements can contain components but not collections. If your composite element contains components, use the `<nested-composite-element>` tag. This case is a collection of components which themselves have components. You may want to consider if a one-to-many association is more appropriate. Remodel the composite element as an entity, but be aware that even though the .Net model is the same, the relational model and persistence semantics are still slightly different.

A special case of a composite element is a composite element with a nested `<many-to-one>` element. This mapping allows you to map extra columns of a many-to-many association table to the composite element class. The following is a many-to-many association from `Order` to `Item` where `PurchaseDate`, `Price` and `Quantity` are properties of the association:

```
<class name="Order" ... >
  ...
  <set name="PurchasedItems" table="purchase_items">
    <key column="order_id">
    <composite-element class="Purchase">
      <property name="PurchaseDate"/>
      <property name="Price"/>
      <property name="Quantity"/>
      <many-to-one name="Item" class="Item"/> <!-- class attribute is optional -->
    </composite-element>
  </set>
</class>
```

There cannot be a reference to the purchase on the other side for bidirectional association navigation. Compon-

ents are value types and do not allow shared references. A single `Purchase` can be in the set of an `Order`, but it cannot be referenced by the `Item` at the same time.

Even ternary (or quaternary, etc) associations are possible:

```
<class name="Order" ... >
  ...
  <set name="PurchasedItems" table="purchase_items">
    <key column="order_id">
      <composite-element class="OrderLine">
        <many-to-one name="PurchaseDetails" class="Purchase"/>
        <many-to-one name="Item" class="Item"/>
      </composite-element>
    </set>
  </class>
```

Composite elements can appear in queries using the same syntax as associations to other entities.

8.3. Components as IDictionary indices

The `<composite-map-key>` element lets you map a component class as the key of an `IDictionary`. Make sure you override `GetHashCode()` and `Equals()` correctly on the component class. See Section 6.2.3, “Indexed collections” for more information on the `<composite-map-key>` element.

8.4. Components as composite identifiers

You can use a component as an identifier of an entity class. Your component class must satisfy certain requirements:

- It must be marked with the `Serializable` attribute.
- It must re-implement `Equals()` and `GetHashCode()`, consistently with the database's notion of composite key equality.
- It should re-implement `ToString()` if you consider using the second level cache. See Section 27.1, “How to use a cache?”.

You cannot use an `IIentifierGenerator` to generate composite keys. Instead the application must assign its own identifiers.

Since a composite identifier must be assigned to the object before saving it, you cannot use `unsaved-value` of the identifier to distinguish between newly instantiated instances and instances saved in a previous session. See Section 5.1.5.7, “Assigned Identifiers” for more information.

Use the `<composite-id>` tag, with nested `<key-property>` or `<key-many-to-one>` elements, in place of the usual `<id>` declaration. For example, the following `OrderLine` class has a primary key that depends upon the (composite) primary key of `Order`.

```
<class name="OrderLine">
  <composite-id name="Id" class="OrderLineId">
    <key-property name="lineId"/>
    <key-property name="orderId"/>
    <key-property name="customerId"/>
  </composite-id>

  <property name="Name"/>

  <many-to-one name="Order" class="Order"
    insert="false" update="false">
```

```

        <column name="orderId"/>
        <column name="customerId"/>
    </many-to-one>
    ...
</class>

```

Any foreign keys referencing the `OrderLine` table are now composite. Declare this in your mappings for other classes. An association to `OrderLine` is mapped like this:

```

<many-to-one name="OrderLine" class="OrderLine">
<!-- the "class" attribute is optional, as usual -->
    <column name="lineId"/>
    <column name="orderId"/>
    <column name="customerId"/>
</many-to-one>

```

The `<column>` element is an alternative to the `column` attribute everywhere. Using the `<column>` element is required for composite keys, but also gives more declaration options, which are mostly useful when using `hbm2ddl`. See Section 5.1.21, “column and formula elements”.

A many-to-many association to `OrderLine` also uses the composite foreign key:

```

<set name="UndeliveredOrderLines">
    <key column name="warehouseId"/>
    <many-to-many class="OrderLine">
        <column name="lineId"/>
        <column name="orderId"/>
        <column name="customerId"/>
    </many-to-many>
</set>

```

The collection of `OrderLine` in `Order` would use:

```

<set name="OrderLines" inverse="true">
    <key>
        <column name="orderId"/>
        <column name="customerId"/>
    </key>
    <one-to-many class="OrderLine"/>
</set>

```

The `<one-to-many>` element declares no columns.

If `OrderLine` itself owns a collection, it also has a composite foreign key.

```

<class name="OrderLine">
    ...
    <list name="DeliveryAttempts">
        <key> <!-- a collection inherits the composite key type -->
            <column name="lineId"/>
            <column name="orderId"/>
            <column name="customerId"/>
        </key>
        <list-index column="attemptId" base="1"/>
        <composite-element class="DeliveryAttempt">
            ...
        </composite-element>
    </set>
</class>

```

8.5. Dynamic components

You can also map a property of type `IDictionary` or `IDictionary<string, object>`, or declared as a C# `dynamic`:

```
<dynamic-component name="UserAttributes">
  <property name="Foo" column="F00"/>
  <property name="Bar" column="BAR"/>
  <many-to-one name="Baz" class="Baz" column="BAZ_ID"/>
</dynamic-component>
```

The semantics of a `<dynamic-component>` mapping are identical to `<component>`. The advantage of this kind of mapping is the ability to determine the actual properties of the component at deployment time, just by editing the mapping document. Runtime manipulation of the mapping document is also possible, using a DOM parser. You can also access, and change, NHibernate's configuration-time metamodel via the `Configuration` object.

Chapter 9. Inheritance Mapping

9.1. The Three Strategies

NHibernate supports the three basic inheritance mapping strategies:

- table per class hierarchy
- table per subclass
- table per concrete class

In addition, NHibernate supports a fourth, slightly different kind of polymorphism:

- implicit polymorphism

It is possible to use different mapping strategies for different branches of the same inheritance hierarchy. You can then make use of implicit polymorphism to achieve polymorphism across the whole hierarchy. However, NHibernate does not support mixing `<subclass>`, and `<joined-subclass>` and `<union-subclass>` mappings under the same root `<class>` element. It is possible to mix together the table per hierarchy and table per subclass strategies, under the the same `<class>` element, by combining the `<subclass>` and `<join>` elements (see Section 9.1.4, “Mixing table per class hierarchy with table per subclass”).

It is possible to define subclass, union-subclass, and joined-subclass mappings in separate mapping documents directly beneath `hibernate-mapping`. This allows you to extend a class hierarchy by adding a new mapping file. You must specify an `extends` attribute in the subclass mapping, naming a previously mapped superclass.

```
<hibernate-mapping>
  <subclass name="DomesticCat" extends="Cat" discriminator-value="D">
    <property name="name" type="string"/>
  </subclass>
</hibernate-mapping>
```

9.1.1. Table per class hierarchy

Suppose we have an interface `IPayment`, with implementors `CreditCardPayment`, `CashPayment`, `ChequePayment`. The table-per-hierarchy mapping would display in the following way:

```
<class name="IPayment" table="PAYMENT">
  <id name="Id" type="Int64" column="PAYMENT_ID">
    <generator class="native"/>
  </id>
  <discriminator column="PAYMENT_TYPE" type="String"/>
  <property name="Amount" column="AMOUNT"/>
  ...
  <subclass name="CreditCardPayment" discriminator-value="CREDIT">
    <property name="CreditCardType" column="CCTYPE"/>
    ...
  </subclass>
  <subclass name="CashPayment" discriminator-value="CASH">
    ...
  </subclass>
  <subclass name="ChequePayment" discriminator-value="CHEQUE">
```

```

    ...
</subclass>
</class>

```

Exactly one table is required. There is a limitation of this mapping strategy: columns declared by the subclasses, such as CCTYPE, cannot have NOT NULL constraints.

9.1.2. Table per subclass

A table-per-subclass mapping would look like:

```

<class name="IPayment" table="PAYMENT">
  <id name="Id" type="Int64" column="PAYMENT_ID">
    <generator class="native"/>
  </id>
  <property name="Amount" column="AMOUNT"/>
  ...
  <joined-subclass name="CreditCardPayment" table="CREDIT_PAYMENT">
    <key column="PAYMENT_ID"/>
    <property name="CreditCardType" column="CCTYPE"/>
    ...
  </joined-subclass>
  <joined-subclass name="CashPayment" table="CASH_PAYMENT">
    <key column="PAYMENT_ID"/>
    ...
  </joined-subclass>
  <joined-subclass name="ChequePayment" table="CHEQUE_PAYMENT">
    <key column="PAYMENT_ID"/>
    ...
  </joined-subclass>
</class>

```

Four tables are required. The three subclass tables have primary key associations to the superclass table so the relational model is actually a one-to-one association.

9.1.3. Table per subclass, using a discriminator

NHibernate's implementation of table per subclass does not require a discriminator column. Other object/relational mappers use a different implementation of table per subclass that requires a type discriminator column in the superclass table. The approach taken by NHibernate is much more difficult to implement, but arguably more correct from a relational point of view. If you want to use a discriminator column with the table per subclass strategy, you can combine the use of <subclass> and <join>, as follows:

```

<class name="Payment" table="PAYMENT">
  <id name="Id" type="Int64" column="PAYMENT_ID">
    <generator class="native"/>
  </id>
  <discriminator column="PAYMENT_TYPE" type="string"/>
  <property name="Amount" column="AMOUNT"/>
  ...
  <subclass name="CreditCardPayment" discriminator-value="CREDIT">
    <join table="CREDIT_PAYMENT">
      <key column="PAYMENT_ID"/>
      <property name="CreditCardType" column="CCTYPE"/>
      ...
    </join>
  </subclass>
  <subclass name="CashPayment" discriminator-value="CASH">
    <join table="CASH_PAYMENT">
      <key column="PAYMENT_ID"/>
      ...
    </join>
  ...
</class>

```

```

</subclass>
<subclass name="ChequePayment" discriminator-value="CHEQUE">
  <join table="CHEQUE_PAYMENT" fetch="select">
    <key column="PAYMENT_ID"/>
    ...
  </join>
</subclass>
</class>

```

The optional `fetch="select"` declaration tells NHibernate not to fetch the `ChequePayment` subclass data using an outer join when querying the superclass.

9.1.4. Mixing table per class hierarchy with table per subclass

You can even mix the table per hierarchy and table per subclass strategies using the following approach:

```

<class name="Payment" table="PAYMENT">
  <id name="Id" type="Int64" column="PAYMENT_ID">
    <generator class="native"/>
  </id>
  <discriminator column="PAYMENT_TYPE" type="string"/>
  <property name="Amount" column="AMOUNT"/>
  ...
  <subclass name="CreditCardPayment" discriminator-value="CREDIT">
    <join table="CREDIT_PAYMENT">
      <property name="CreditCardType" column="CCTYPE"/>
      ...
    </join>
  </subclass>
  <subclass name="CashPayment" discriminator-value="CASH">
    ...
  </subclass>
  <subclass name="ChequePayment" discriminator-value="CHEQUE">
    ...
  </subclass>
</class>

```

For any of these mapping strategies, a polymorphic association to `IPayment` is mapped using `<many-to-one>`.

```

<many-to-one name="Payment" column="PAYMENT" class="IPayment"/>

```

9.1.5. Table per concrete class

There are two ways we can map the table per concrete class strategy. First, you can use `<union-subclass>`.

```

<class name="Payment">
  <id name="Id" type="Int64" column="PAYMENT_ID">
    <generator class="sequence"/>
  </id>
  <property name="Amount" column="AMOUNT"/>
  ...
  <union-subclass name="CreditCardPayment" table="CREDIT_PAYMENT">
    <property name="CreditCardType" column="CCTYPE"/>
    ...
  </union-subclass>
  <union-subclass name="CashPayment" table="CASH_PAYMENT">
    ...
  </union-subclass>
  <union-subclass name="ChequePayment" table="CHEQUE_PAYMENT">
    ...
  </union-subclass>
</class>

```

Three tables are involved for the subclasses. Each table defines columns for all properties of the class, including inherited properties.

The limitation of this approach is that if a property is mapped on the superclass, the column name must be the same on all subclass tables. The identity generator strategy is not allowed in union subclass inheritance. The primary key seed has to be shared across all unioned subclasses of a hierarchy.

If your superclass is abstract, map it with `abstract="true"`. If it is not abstract, an additional table (it defaults to `PAYMENT` in the example above), is needed to hold instances of the superclass.

9.1.6. Table per concrete class, using implicit polymorphism

An alternative approach is to make use of implicit polymorphism:

```
<class name="CreditCardPayment" table="CREDIT_PAYMENT">
  <id name="Id" type="Int64" column="CREDIT_PAYMENT_ID">
    <generator class="native"/>
  </id>
  <property name="Amount" column="CREDIT_AMOUNT"/>
  <property name="CreditCardType" column="CCTYPE"/>
  ...
</class>

<class name="CashPayment" table="CASH_PAYMENT">
  <id name="Id" type="Int64" column="CASH_PAYMENT_ID">
    <generator class="native"/>
  </id>
  <property name="Amount" column="CASH_AMOUNT"/>
  ...
</class>

<class name="ChequePayment" table="CHEQUE_PAYMENT">
  <id name="Id" type="Int64" column="CHEQUE_PAYMENT_ID">
    <generator class="native"/>
  </id>
  <property name="Amount" column="CHEQUE_AMOUNT"/>
  ...
</class>
```

Notice that the `IPayment` interface is not mentioned explicitly. Also notice that properties of `IPayment` are mapped in each of the subclasses. If you want to avoid duplication, consider using XML entities (by example [`<!ENTITY allproperties SYSTEM "allproperties.xml">`] in the `DOCTYPE` declaration and `&allproperties;` in the mapping).

The disadvantage of this approach is that `NHibernate` does not generate `SQL UNIONS` when performing polymorphic queries.

For this mapping strategy, a polymorphic association to `IPayment` is usually mapped using `<any>`.

```
<any name="Payment" meta-type="string" id-type="Int64">
  <meta-value value="CREDIT" class="CreditCardPayment"/>
  <meta-value value="CASH" class="CashPayment"/>
  <meta-value value="CHEQUE" class="ChequePayment"/>
  <column name="PAYMENT_CLASS"/>
  <column name="PAYMENT_ID"/>
</any>
```

9.1.7. Mixing implicit polymorphism with other inheritance mappings

Since the subclasses are each mapped in their own `<class>` element and since `IPayment` is just an interface,

each of the subclasses could easily be part of another inheritance hierarchy. You can still use polymorphic queries against the `IPayment` interface.

```
<class name="CreditCardPayment" table="CREDIT_PAYMENT">
  <id name="Id" type="Int64" column="CREDIT_PAYMENT_ID">
    <generator class="native"/>
  </id>
  <discriminator column="CREDIT_CARD" type="String"/>
  <property name="Amount" column="CREDIT_AMOUNT"/>
  ...
  <subclass name="MasterCardPayment" discriminator-value="MDC"/>
  <subclass name="VisaPayment" discriminator-value="VISA"/>
</class>

<class name="NonelectronicTransaction" table="NONELECTRONIC_TXN">
  <id name="Id" type="Int64" column="TXN_ID">
    <generator class="native"/>
  </id>
  ...
  <joined-subclass name="CashPayment" table="CASH_PAYMENT">
    <key column="PAYMENT_ID"/>
    <property name="Amount" column="CASH_AMOUNT"/>
    ...
  </joined-subclass>
  <joined-subclass name="ChequePayment" table="CHEQUE_PAYMENT">
    <key column="PAYMENT_ID"/>
    <property name="Amount" column="CHEQUE_AMOUNT"/>
    ...
  </joined-subclass>
</class>
```

Once again, `IPayment` is not mentioned explicitly. If we execute a query against the `IPayment` interface - for example, from `IPayment` - NHibernate automatically returns instances of `CreditCardPayment` (and its subclasses, since they also implement `IPayment`), `CashPayment` and `ChequePayment` but not instances of `NonelectronicTransaction` (provided it does not implement `IPayment`).

9.2. Limitations

There are limitations to the "implicit polymorphism" approach to the table per concrete-class mapping strategy. There are somewhat less restrictive limitations to `<union-subclass>` mappings.

The following table shows the limitations of table per concrete-class mappings, and of implicit polymorphism, in NHibernate.

Table 9.1. Features of inheritance mappings

Inheritance strategy	Poly-morphic many-to-one	Poly-morphic one-to-one	Poly-morphic one-to-many	Poly-morphic many-to-many	Poly-morphic Load()/Get()	Poly-morphic queries	Poly-morphic joins
table per class-hierarchy	<code><many-to-one></code>	<code><one-to-one></code>	<code><one-to-many></code>	<code><many-to-many></code>	<code>s.Get<IPayment>(id)</code>	from <code>IPayment p</code>	from <code>Order o join o.Payment p</code>
table per subclass	<code><many-to-one></code>	<code><one-to-one></code>	<code><one-to-many></code>	<code><many-to-many></code>	<code>s.Get<IPayment>(id)</code>	from <code>IPayment p</code>	from <code>Order o join o.Payment p</code>

Inherit- ance strategy	Poly- morphic many- to-one	Poly- morphic one-to-one	Poly- morphic one- to-many	Poly- morphic many- to-many	Poly- morphic Load()/Get ()	Poly- morphic queries	Poly- morphic joins
table per concrete- class (union-subc lass)	<many-to-one>	<one-to-one>	<one-to-many> (for in- verse="true" only)	<many-to-many>	s.Get<IPay- ment>(id)	from IPay- ment p	from Order o join o.Payment p
table per concrete class (implicit polymorph- ism)	<any>	<i>not supported</i>	<i>not supported</i>	<many-to-any>	<i>use a query</i>	from IPay- ment p	<i>not supported</i>

Chapter 10. Manipulating Persistent Data

10.1. Creating a persistent object

An object (entity instance) is either *transient* or *persistent* with respect to a particular `ISession`. Newly instantiated objects are, of course, transient. The session offers services for saving (ie. persisting) transient instances:

```
DomesticCat fritz = new DomesticCat();
fritz.Color = Color.Ginger;
fritz.Sex = 'M';
fritz.Name = "Fritz";
long generatedId = (long) sess.Save(fritz);
```

```
DomesticCat pk = new DomesticCat();
pk.Color = Color.Tabby;
pk.Sex = 'F';
pk.Name = "PK";
pk.Kittens = new HashSet<Cat>();
pk.AddKitten(fritz);
sess.Save( pk, 1234L );
```

The single-argument `Save()` generates and assigns a unique identifier to `fritz`. The two-argument form attempts to persist `pk` using the given identifier. We generally discourage the use of the two-argument form since it may be used to create primary keys with business meaning.

Associated objects may be made persistent in any order you like unless you have a `NOT NULL` constraint upon a foreign key column. There is never a risk of violating foreign key constraints. However, you might violate a `NOT NULL` constraint if you `Save()` the objects in the wrong order.

10.2. Loading an object

The `Load()` methods of `ISession` give you a way to retrieve a persistent instance if you already know its identifier. One version takes a class object and will load the state into a newly instantiated object. The second version allows you to supply an instance into which the state will be loaded. The form which takes an instance is only useful in special circumstances (DIY instance pooling etc.)

```
Cat fritz = sess.Load<Cat>(generatedId);
```

```
long pkId = 1234;
DomesticCat pk = sess.Load<DomesticCat>(pkId);
```

```
Cat cat = new DomesticCat();
// load pk's state into cat
sess.Load( cat, pkId );
var kittens = cat.Kittens;
```

Note that `Load()` will throw an unrecoverable exception if there is no matching database row. If the class is mapped with a proxy, `Load()` returns an object that is an uninitialized proxy and does not actually hit the database until you invoke a method of the object. This behaviour is very useful if you wish to create an association to an object without actually loading it from the database.

If you are not certain that a matching row exists, you should use the `Get()` method, which hits the database immediately and returns null if there is no matching row.

```
Cat cat = sess.Get<Cat>(id);
if (cat==null) {
    cat = new Cat();
    sess.Save(cat, id);
}
return cat;
```

You may also load an objects using an SQL `SELECT ... FOR UPDATE`. See the next section for a discussion of NHibernate `LockModes`.

```
Cat cat = sess.Get<Cat>(id, LockMode.Upgrade);
```

Note that any associated instances or contained collections are *not* selected `FOR UPDATE`.

It is possible to re-load an object and all its collections at any time, using the `Refresh()` method. This is useful when database triggers are used to initialize some of the properties of the object.

```
sess.Save(cat);
sess.Flush(); //force the SQL INSERT
sess.Refresh(cat); //re-read the state (after the trigger executes)
```

An important question usually appears at this point: How much does NHibernate load from the database and how many SQL `SELECTS` will it use? This depends on the *fetching strategy* and is explained in Section 21.1, “Fetching strategies”.

10.3. Querying

If you don't know the identifier(s) of the object(s) you are looking for, use the `CreateQuery()` method of `ISession`. NHibernate supports a simple but powerful object oriented query language.

```
IList<Cat> cats = sess
    .CreateQuery("from Cat as cat where cat.Birthdate = ?")
    .SetDateTime(0, date)
    .List<Cat>();

var mates = sess
    .CreateQuery("select mate from Cat as cat join cat.Mate as mate " +
        "where cat.name = ?")
    .SetString(0, name)
    .List<Cat>();

var cats = sess
    .CreateQuery("from Cat as cat where cat.Mate.Birthdate is null")
    .List<Cat>();

var moreCats = sess
    .CreateQuery("from Cat as cat where " +
        "cat.Name = 'Fritz' or cat.id = ? or cat.id = ?")
    .SetInt64(0, id1)
    .SetParameter(1, id2, NHibernateUtil.Int64)
    .List<Cat>();

var mates = sess
    .CreateQuery("from Cat as cat where cat.Mate = ?")
    .SetEntity(0, izi)
    .List<Cat>();
);

var problems = sess
    .CreateQuery("from GoldFish as fish " +
        "where fish.Birthday > fish.Deceased or fish.Birthday is null")
    .List<GoldFish>();
```

These given Set parameters are used to bind the given values to the ? query placeholders (which map to input parameters of an ADO.NET DbCommand). Just as in ADO.NET, you should use this binding mechanism in preference to string manipulation.

The `NHibernateUtil` class defines a number of static methods and constants, providing access to most of the built-in types, as instances of `NHibernate.Type.OfType`.

If you expect your query to return a very large number of objects, but you don't expect to use them all, you might get better performance from the `Enumerable()` method, which return a `IEnumerable`. The iterator will load objects on demand, using the identifiers returned by an initial SQL query (n+1 selects total).

```
// fetch ids
IEnumerable<Qux> en = sess
    .CreateQuery("from eg.Qux q order by q.Likeliness")
    .Enumerable<Qux>();
foreach (Qux qux in en)
{
    // something we couldnt express in the query
    if ( qux.CalculateComplicatedAlgorithm() ) {
        // dont need to process the rest
        break;
    }
}
```

The `Enumerable()` method also performs better if you expect that many of the objects are already loaded and cached by the session, or if the query results contain the same objects many times. (When no data is cached or repeated, `CreateQuery()` is almost always faster.) Here is an example of a query that should be called using `Enumerable()`:

```
var en = sess
    .CreateQuery(
        "select customer, product " +
        "from Customer customer, " +
        "Product product " +
        "join customer.Purchases purchase " +
        "where product = purchase.Product")
    .Enumerable<object[]>();
```

Calling the previous query using `CreateQuery()` would return a very large ADO.NET result set containing the same data many times.

NHibernate queries sometimes return tuples of objects, in which case each tuple is returned as an array:

```
var foosAndBars = sess
    .CreateQuery(
        "select foo, bar from Foo foo, Bar bar " +
        "where bar.Date = foo.Date")
    .Enumerable<object[]>();
foreach (object[] tuple in foosAndBars)
{
    Foo foo = tuple[0]; Bar bar = tuple[1];
    ....
}
```

10.3.1. Scalar queries

Queries may specify a property of a class in the select clause. They may even call SQL aggregate functions. Properties or aggregates are considered "scalar" results.

```

var results = sess
    .CreateQuery(
        "select cat.Color, min(cat.Birthdate), count(cat) from Cat cat " +
        "group by cat.Color")
    .Enumerable<object[]>();
foreach (object[] row in results)
{
    Color type = (Color) row[0];
    DateTime oldest = (DateTime) row[1];
    int count = (int) row[2];
    .....
}

```

```

var en = sess
    .CreateQuery(
        "select cat.Type, cat.Birthdate, cat.Name from DomesticCat cat")
    .Enumerable<object[]>();

```

```

IList<object[]> list = sess
    .CreateQuery("select cat, cat.Mate.Name from DomesticCat cat")
    .List<object[]>();

```

10.3.2. The IQuery interface

If you need to specify bounds upon your result set (the maximum number of rows you want to retrieve and / or the first row you want to retrieve) you should obtain an instance of `NHibernate.IQuery`:

```

IQuery q = sess.CreateQuery("from DomesticCat cat");
q.SetFirstResult(20);
q.SetMaxResults(10);
var cats = q.List<Cat>();

```

You may even define a named query in the mapping document. (Remember to use a CDATA section if your query contains characters that could be interpreted as markup.)

```

<query name="Eg.DomesticCat.by.name.and.minimum.weight"><![CDATA[
    from Eg.DomesticCat as cat
    where cat.Name = ?
    and cat.Weight > ?
] ]></query>

```

```

IQuery q = sess.GetNamedQuery("Eg.DomesticCat.by.name.and.minimum.weight");
q.SetString(0, name);
q.SetInt32(1, minWeight);
var cats = q.List<Cat>();

```

Named queries are by default validated at startup time, allowing to catch errors more easily than having to test all the application features using HQL queries. In case of validation errors, the details of failing queries are logged and a validation error is raised.

Named queries accepts a number of attributes matching settings available on the `IQuery` interface.

- `flush-mode` - override the session flush mode just for this query.
- `cacheable` - allow the query results to be cached by the second level cache. See Chapter 27, *NHibernate.Caches*.
- `cache-region` - specify the cache region of the query.
- `cache-mode` - specify the cache mode of the query.
- `fetch-size` - set a fetch size for the underlying ADO query.
- `timeout` - set the query timeout in seconds.

- read-only - true switches yielded entities to read-only. See Chapter 11, *Read-only entities*.
- comment - add a custom comment to the generated SQL.

The query interface supports the use of named parameters. Named parameters are identifiers of the form `:name` in the query string. There are methods on `IQuery` for binding values to named or positional parameters. NHibernate numbers parameters from zero. The advantages of named parameters are:

- named parameters are insensitive to the order they occur in the query string
- they may occur multiple times in the same query
- they are self-documenting

```
//named parameter (preferred)
IQuery q = sess.CreateQuery("from DomesticCat cat where cat.Name = :name");
q.SetString("name", "Fritz");
var cats = q.Enumerable<DomesticCat>();
```

```
//positional parameter
IQuery q = sess.CreateQuery("from DomesticCat cat where cat.Name = ?");
q.SetString(0, "Izi");
var cats = q.Enumerable<DomesticCat>();
```

```
//named parameter list
var names = new List<string>();
names.Add("Izi");
names.Add("Fritz");
IQuery q = sess.CreateQuery("from DomesticCat cat where cat.Name in (:namesList)");
q.SetParameterList("namesList", names);
var cats = q.List<DomesticCat>();
```

10.3.3. Filtering collections

A collection *filter* is a special type of query that may be applied to a persistent collection or array. The query string may refer to this, meaning the current collection element.

```
var blackKittens = session
    .CreateFilter(pk.Kittens, "where this.Color = ?")
    .SetEnum(0, Color.Black)
    .List<Cat>();
```

The returned collection is considered a bag.

Observe that filters do not require a `from` clause (though they may have one if required). Filters are not limited to returning the collection elements themselves.

```
var blackKittenMates = session
    .CreateFilter(pk.Kittens,
        "select this.Mate where this.Color = Eg.Color.Black")
    .List<Cat>();
```

10.3.4. Criteria queries

HQL is extremely powerful but some people prefer to build queries dynamically, using an object oriented API, rather than embedding strings in their .NET code. For these people, NHibernate provides an intuitive `ICriteria` query API.

```
ICriteria crit = session.CreateCriteria<Cat>();
crit.Add(Expression.Eq("color", Eg.Color.Black));
crit.SetMaxResults(10);
```

```
var cats = crit.List<Cat>();
```

If you are uncomfortable with SQL-like syntax, this is perhaps the easiest way to get started with NHibernate. This API is also more extensible than HQL. Applications might provide their own implementations of the `ICriterion` interface.

10.3.5. Queries in native SQL

You may express a query in SQL, using `createSQLQuery()`. You must enclose SQL aliases in braces.

```
var cats = session
    .CreateSQLQuery("SELECT {cat.*} FROM CAT {cat} WHERE ROWNUM<10")
    .AddEntity("cat", typeof(Cat))
    .List<Cat>();
```

```
var cats = session
    .CreateSQLQuery(
        "SELECT {cat}.ID AS {cat.Id}, {cat}.SEX AS {cat.Sex}, " +
        "{cat}.MATE AS {cat.Mate}, {cat}.SUBCLASS AS {cat.class}, ... " +
        "FROM CAT {cat} WHERE ROWNUM<10")
    .AddEntity("cat", typeof(Cat))
    .List<Cat>();
```

SQL queries may contain named and positional parameters, just like NHibernate queries.

10.4. Updating objects

10.4.1. Updating in the same `ISession`

Transactional persistent instances (ie. objects loaded, saved, created or queried by the `ISession`) may be manipulated by the application and any changes to persistent state will be persisted when the `ISession` is *flushed* (discussed later in this chapter). So the most straightforward way to update the state of an object is to `Load()` it, and then manipulate it directly, while the `ISession` is open:

```
DomesticCat cat = sess.Load<DomesticCat>(69L);
cat.Name = "PK";
sess.Flush(); // changes to cat are automatically detected and persisted
```

Sometimes this programming model is inefficient since it would require both an SQL `SELECT` (to load an object) and an SQL `UPDATE` (to persist its updated state) in the same session. Therefore NHibernate offers an alternate approach.

10.4.2. Updating detached objects

Many applications need to retrieve an object in one transaction, send it to the UI layer for manipulation, then save the changes in a new transaction. (Applications that use this kind of approach in a high-concurrency environment usually use versioned data to ensure transaction isolation.) This approach requires a slightly different programming model to the one described in the last section. NHibernate supports this model by providing the method `ISession.Update()`.

```
// in the first session
Cat cat = firstSession.Load<Cat>(catId);
Cat potentialMate = new Cat();
```

```

firstSession.Save(potentialMate);

// in a higher tier of the application
cat.Mate = potentialMate;

// later, in a new session
secondSession.Update(cat); // update cat

```

If the Cat with identifier catId had already been loaded by secondSession when the application tried to update it, an exception would have been thrown.

The application should individually update() transient instances reachable from the given transient instance if and *only* if it wants their state also updated. (Except for lifecycle objects, see Section 10.10, “Lifecycle objects and object graphs”.)

NHibernate users have requested a general purpose method that either saves a transient instance by generating a new identifier or update the persistent state associated with its current identifier. The SaveOrUpdate() method now implements this functionality.

NHibernate distinguishes "new" (unsaved) instances from "existing" (saved or loaded in a previous session) instances by the value of their identifier (or version, or timestamp) property. The unsaved-value attribute of the <id> (or <version>, or <timestamp>) mapping specifies which values should be interpreted as representing a "new" instance.

```

<id name="Id" type="Int64" column="uid" unsaved-value="0">
  <generator class="hilo"/>
</id>

```

The allowed values of unsaved-value are:

- any - always save
- none - always update
- null - save when identifier is null
- valid identifier value - save when identifier is null or the given value
- undefined - do not infer the operation from the identifier (or version, or timestamp). Ultimately, NHibernate may query the database for determining the operation it has to perform.

If unsaved-value is not specified for a class, NHibernate will attempt to guess it by creating an instance of the class using the no-argument constructor and reading the property value from the instance.

```

// in the first session
Cat cat = firstSession.Load<Cat>(catID);

// in a higher tier of the application
Cat mate = new Cat();
cat.Mate = mate;

// later, in a new session
secondSession.SaveOrUpdate(cat); // update existing state (cat has a non-null id)
secondSession.SaveOrUpdate(mate); // save the new instance (mate has a null id)

```

The usage and semantics of SaveOrUpdate() seems to be confusing for new users. Firstly, so long as you are not trying to use instances from one session in another new session, you should not need to use Update() or SaveOrUpdate(). Some whole applications will never use either of these methods.

Usually update() or SaveOrUpdate() are used in the following scenario:

- the application loads an object in the first session

- the object is passed up to the UI tier
- some modifications are made to the object
- the object is passed back down to the business logic tier
- the application persists these modifications by calling `Update()` in a second session

`SaveOrUpdate()` does the following:

- if the object is already persistent in this session, do nothing
- if the object has no identifier property, `Save()` it
- if the object's identifier matches the criteria specified by `unsaved-value`, `Save()` it
- if the object is versioned (version or timestamp), and its version matches the criteria specified by `unsaved-value`, `Save()` it
- if another object associated with the session has the same identifier, throw an exception

The last case can be avoided by using `Merge(Object o)`. This method copies the state of the given object onto the persistent object with the same identifier. If there is no persistent instance currently associated with the session, it will be loaded. The method returns the persistent instance. If the given instance is unsaved or does not exist in the database, NHibernate will save it and return it as a newly persistent instance. Otherwise, the given instance does not become associated with the session. In most applications with detached objects, you need both methods, `SaveOrUpdate()` and `Merge()`.

10.4.3. Reattaching detached objects

The `Lock()` method allows the application to re-associate an unmodified object with a new session.

```
//just reassociate:
sess.Lock(fritz, LockMode.None);
//do a version check, then reassociate:
sess.Lock(izi, LockMode.Read);
//do a version check, using SELECT ... FOR UPDATE, then reassociate:
sess.Lock(pk, LockMode.Upgrade);
```

10.5. Deleting persistent objects

`ISession.Delete()` will remove an object's state from the database. Of course, your application might still hold a reference to it. So it's best to think of `Delete()` as making a persistent instance transient.

```
sess.Delete(cat);
```

You may also delete many objects at once by passing a NHibernate query string to `Delete()`.

```
sess.Delete("from Cat");
```

You may now delete objects in any order you like, without risk of foreign key constraint violations. Of course, it is still possible to violate a `NOT NULL` constraint on a foreign key column by deleting objects in the wrong order.

10.6. Flush

From time to time the `ISession` will execute the SQL statements needed to synchronize the ADO.NET connection's state with the state of objects held in memory. This process, *flush*, occurs by default at the following points

- from some invocations of IQuery methods such as List or Enumerable, and from similar methods of other querying API.
- from NHibernate.ITransaction.Commit()
- from ISession.Flush()

The SQL statements are issued in the following order

1. all entity insertions, in the same order the corresponding objects were saved using ISession.Save()
2. all entity updates
3. all collection deletions
4. all collection element deletions, updates and insertions
5. all collection insertions
6. all entity deletions, in the same order the corresponding objects were deleted using ISession.Delete()

(An exception is that objects using identity ID generation are inserted when they are saved.)

Except when you explicitly Flush(), there are absolutely no guarantees about *when* the Session executes the ADO.NET calls, only the *order* in which they are executed. However, NHibernate does guarantee that the queries methods will never return stale data; nor will they return the wrong data.

It is possible to change the default behavior so that flush occurs less frequently. The FlushMode class defines three different modes: only flush at commit time (and only when the NHibernate ITransaction API is used, or inside a transaction scope with a legacy option enabled - see Section 12.8, “Transaction scopes (System.Transactions)”), flush automatically using the explained routine (will only work inside an explicit NHibernate ITransaction, or inside a transaction scope with limitations for flushes on commit - see Section 12.8, “Transaction scopes (System.Transactions)”), or never flush unless Flush() is called explicitly. The last mode is useful for long running units of work, where an ISession is kept open and disconnected for a long time (see Section 12.4, “Optimistic concurrency control”).

```
sess = sf.OpenSession();
using (ITransaction tx = sess.BeginTransaction())
{
    // allow queries to return stale state
    sess.FlushMode = FlushMode.Commit;
    Cat izi = sess.Load<Cat>(id);
    izi.Name = "iznizi";
    // execute some queries....
    sess.CreateQuery("from Cat as cat left outer join cat.Kittens kitten")
        .List<object[]>();
    // change to izi is not flushed!
    ...
    tx.Commit(); // flush occurs
}
```

10.7. Checking dirtiness

ISession.IsDirty() will return whether the session hold any pending change to flush or not. Be cautious when using this method, its default implementation may have the following effects:

- Dirty checks all the loaded entities. NHibernate does not instrument the entities for being notified of changes done on loaded ones. Instead, it stores their initial state and compare them to it. If session has loaded a lot of entities, the dirty checking will have a significant impact.
- Triggers pending cascade operations. This includes any pending Save of, by example, children added to a collection having the Save cascade enabled. Depending on the entities ID generators (see Section 5.1.5.1, “generator”), this may trigger calls to the database, or even entity insertions if they are using the identity generator.

10.8. Ending a Session

Ending a session involves four distinct phases:

- flush the session
- commit the transaction
- close the session
- handle exceptions

10.8.1. Flushing the Session

If you happen to be using the `ITransaction` API, you don't need to worry about this step. It will be performed implicitly when the transaction is committed. Otherwise you should call `ISession.Flush()` to ensure that all changes are synchronized with the database.

10.8.2. Committing the database transaction

If you are using the NHibernate `ITransaction` API, this looks like:

```
tx.Commit(); // flush the session and commit the transaction
```

If you are managing ADO.NET transactions yourself you should manually `Commit()` the ADO.NET transaction.

```
sess.Flush();  
currentTransaction.Commit();
```

If you decide *not* to commit your changes:

```
tx.Rollback(); // rollback the transaction
```

or:

```
currentTransaction.Rollback();
```

If you rollback the transaction you should immediately close and discard the current session to ensure that NHibernate's internal state is consistent.

10.8.3. Closing the ISession

A call to `ISession.Close()` marks the end of a session. The main implication of `Close()` is that the ADO.NET connection will be relinquished by the session.

```
tx.Commit();  
sess.Close();
```

```
sess.Flush();  
currentTransaction.Commit();  
sess.Close();
```

If you provided your own connection, `Close()` returns a reference to it, so you can manually close it or return it to the pool. Otherwise `Close()` returns it to the pool.

10.9. Exception handling

NHibernate use might lead to exceptions, usually `HibernateException`. This exception can have a nested inner exception (the root cause), use the `InnerException` property to access it.

If the `ISession` throws an exception you should immediately rollback the transaction, call `ISession.Close()` and discard the `ISession` instance. Certain methods of `ISession` will *not* leave the session in a consistent state.

For exceptions thrown by the data provider while interacting with the database, NHibernate will wrap the error in an instance of `ADOException`. The underlying exception is accessible by calling `ADOException.InnerException`. NHibernate converts the `DbException` into an appropriate `ADOException` subclass using the `ISQLExceptionConverter` attached to the `SessionFactory`. By default, the `ISQLExceptionConverter` is defined by the configured dialect; however, it is also possible to plug in a custom implementation (see the api-docs for the `ISQLExceptionConverter` class for details).

The following exception handling idiom shows the typical case in NHibernate applications:

```
using (ISession sess = factory.OpenSession())
using (ITransaction tx = sess.BeginTransaction())
{
    // do some work
    ...
    tx.Commit();
}
```

Or, when manually managing ADO.NET transactions:

```
ISession sess = factory.openSession();
try
{
    // do some work
    ...
    sess.Flush();
    currentTransaction.Commit();
}
catch (Exception e)
{
    currentTransaction.Rollback();
    throw;
}
finally
{
    sess.Close();
}
```

10.10. Lifecycles and object graphs

To save or update all objects in a graph of associated objects, you must either

- `Save()`, `SaveOrUpdate()` or `Update()` each individual object OR
- map associated objects using `cascade="all"` or `cascade="save-update"`.

Likewise, to delete all objects in a graph, either

- `Delete()` each individual object OR
- map associated objects using `cascade="all"`, `cascade="all-delete-orphan"` or `cascade="delete"`.

Recommendation:

- If the child object's lifespan is bounded by the lifespan of the of the parent object make it a *lifecycle object* by specifying `cascade="all"`.
- Otherwise, `Save()` and `Delete()` it explicitly from application code. If you really want to save yourself some extra typing, use `cascade="save-update"` and explicit `Delete()`.

Mapping an association (many-to-one, one-to-one or collection) with `cascade="all"` marks the association as a *parent/child* style relationship where save/update/deletion of the parent results in save/update/deletion of the child(ren). Furthermore, a mere reference to a child from a persistent parent will result in save / update of the child. The metaphor is incomplete, however. A child which becomes unreferenced by its parent is *not* automatically deleted, except in the cases of <one-to-many> and <one-to-one> associations that have been mapped with `cascade="all-delete-orphan"` or `cascade="delete-orphan"`. The precise semantics of cascading operations are as follows:

- If a parent is saved, all children are passed to `SaveOrUpdate()`
- If a parent is passed to `Update()` or `SaveOrUpdate()`, all children are passed to `SaveOrUpdate()`
- If a transient child becomes referenced by a persistent parent, it is passed to `SaveOrUpdate()`
- If a parent is deleted, all children are passed to `Delete()`
- If a transient child is dereferenced by a persistent parent, *nothing special happens* (the application should explicitly delete the child if necessary) unless `cascade="all-delete-orphan"` or `cascade="delete-orphan"`, in which case the "orphaned" child is deleted.

NHibernate does not fully implement "persistence by reachability", which would imply (inefficient) persistent garbage collection. However, due to popular demand, NHibernate does support the notion of entities becoming persistent when referenced by another persistent object. Associations marked `cascade="save-update"` behave in this way. If you wish to use this approach throughout your application, it's easier to specify the default-cascade attribute of the <hibernate-mapping> element.

10.11. Interceptors

The `IInterceptor` interface provides callbacks from the session to the application allowing the application to inspect and / or manipulate properties of a persistent object before it is saved, updated, deleted or loaded. One possible use for this is to track auditing information. See Section 13.1, "Interceptors" for more information.

10.12. Metadata API

NHibernate requires a very rich meta-level model of all entity and value types. From time to time, this model is very useful to the application itself. For example, the application might use NHibernate's metadata to implement a "smart" deep-copy algorithm that understands which objects should be copied (eg. mutable value types) and which should not (eg. immutable value types and, possibly, associated entities).

NHibernate exposes metadata via the `IClassMetadata` and `ICollectionMetadata` interfaces and the `IType` hierarchy. Instances of the metadata interfaces may be obtained from the `ISessionFactory`.

```
Cat fritz = .....;
IClassMetadata catMeta = sessionFactory.GetClassMetadata(typeof(Cat));
long id = (long) catMeta.GetIdentifier(fritz);
object[] propertyValues = catMeta.GetPropertyValues(fritz);
string[] propertyNames = catMeta.PropertyNames;
IType[] propertyTypes = catMeta.PropertyTypes;

// get an dictionary of all properties which are not collections or associations
// TODO: what about components?

var namedValues = new Dictionary<string, object>();
for (int i = 0; i < propertyNames.Length; i++)
```

```
{  
    if (!propertyTypes[i].IsEntityType && !propertyTypes[i].IsCollectionType)  
    {  
        namedValues[propertyNames[i]] = propertyValues[i];  
    }  
}
```

Chapter 11. Read-only entities

Important

NHibernate's treatment of *read-only* entities may differ from what you may have encountered elsewhere. Incorrect usage may cause unexpected results.

When an entity is read-only:

- NHibernate does not dirty-check the entity's simple properties or single-ended associations;
- NHibernate will not update simple properties or updatable single-ended associations;
- NHibernate will not update the version of the read-only entity if only simple properties or single-ended updatable associations are changed;

In some ways, NHibernate treats read-only entities the same as entities that are not read-only:

- NHibernate cascades operations to associations as defined in the entity mapping.
- NHibernate updates the version if the entity has a collection with changes that dirties the entity;
- A read-only entity can be deleted.

Even if an entity is not read-only, its collection association can be affected if it contains a read-only entity.

For details about the affect of read-only entities on different property and association types, see Section 11.2, “Read-only affect on property type”.

For details about how to make entities read-only, see Section 11.1, “Making persistent entities read-only”

NHibernate does some optimizing for read-only entities:

- It saves execution time by not dirty-checking simple properties or single-ended associations.
- It saves memory by deleting database snapshots.

11.1. Making persistent entities read-only

Only persistent entities can be made read-only. Transient and detached entities must be put in persistent state before they can be made read-only.

NHibernate provides the following ways to make persistent entities read-only:

- you can map an entity class as *immutable*; when an entity of an immutable class is made persistent, NHibernate automatically makes it read-only. see Section 11.1.1, “Entities of immutable classes” for details
- you can change a default so that entities loaded into the session by NHibernate are automatically made read-only; see Section 11.1.2, “Loading persistent entities as read-only” for details
- you can make an HQL query or criteria read-only so that entities loaded when the query or criteria executes,

or iterates, are automatically made read-only; see Section 11.1.3, “Loading read-only entities from an HQL query/criteria” for details

- you can make a persistent entity that is already in the in the session read-only; see Section 11.1.4, “Making a persistent entity read-only” for details

11.1.1. Entities of immutable classes

When an entity instance of an immutable class is made persistent, NHibernate automatically makes it read-only.

An entity of an immutable class can created and deleted the same as an entity of a mutable class.

NHibernate treats a persistent entity of an immutable class the same way as a read-only persistent entity of a mutable class. The only exception is that NHibernate will not allow an entity of an immutable class to be changed so it is not read-only.

11.1.2. Loading persistent entities as read-only

Note

Entities of immutable classes are automatically loaded as read-only.

To change the default behavior so NHibernate loads entity instances of mutable classes into the session and automatically makes them read-only, call:

```
Session.DefaultReadOnly = true;
```

To change the default back so entities loaded by NHibernate are not made read-only, call:

```
Session.DefaultReadOnly = false;
```

You can determine the current setting by using the property:

```
Session.DefaultReadOnly;
```

If `Session.DefaultReadOnly` property returns true, entities loaded by the following are automatically made read-only:

- `Session.Load()` and `Session.Load<T>`
- `Session.Get()` and `Session.Get<T>`
- `Session.Merge()`
- executing, or iterating HQL queries and criteria; to override this setting for a particular HQL query or criteria see Section 11.1.3, “Loading read-only entities from an HQL query/criteria”

Changing this default has no effect on:

- persistent entities already in the session when the default was changed

- persistent entities that are refreshed via `Session.Refresh()`; a refreshed persistent entity will only be read-only if it was read-only before refreshing
- persistent entities added by the application via `Session.Persist()`, `Session.Save()`, `Session.Update()` and `Session.SaveOrUpdate()`

11.1.3. Loading read-only entities from an HQL query/criteria

Note

Entities of immutable classes are automatically loaded as read-only.

If `Session.DefaultReadOnly` returns `false` (the default) when an HQL query or criteria executes, then entities and proxies of mutable classes loaded by the query will not be read-only.

You can override this behavior so that entities and proxies loaded by an HQL query or criteria are automatically made read-only.

For an HQL query, call:

```
Query.SetReadOnly(true);
```

`Query.SetReadOnly(true)` must be called before `Query.List()`, `Query.UniqueResult()`, or `Query.Enumerable()`

For an HQL criteria, call:

```
Criteria.SetReadOnly(true);
```

`Criteria.SetReadOnly(true)` must be called before `Criteria.List()`, or `Criteria.UniqueResult()`

Entities and proxies that exist in the session before being returned by an HQL query or criteria are not affected.

Uninitialized persistent collections returned by the query are not affected. Later, when the collection is initialized, entities loaded into the session will be read-only if `Session.DefaultReadOnly` returns `true`.

Using `Query.SetReadOnly(true)` or `Criteria.SetReadOnly(true)` works well when a single HQL query or criteria loads all the entities and initializes all the proxies and collections that the application needs to be read-only.

When it is not possible to load and initialize all necessary entities in a single query or criteria, you can temporarily change the session default to load entities as read-only before the query is executed. Then you can explicitly initialize proxies and collections before restoring the session default.

```
using (ISession session = factory.OpenSession())
using (ITransaction tx = session.BeginTransaction())
{
    session.DefaultReadOnly = true;
    Contract contract = session
        .CreateQuery("from Contract where CustomerName = 'Sherman'")
        .UniqueResult<Contract>();
    NHibernate.Initialize(contract.Plan);
    NHibernate.Initialize(contract.Variations);
    NHibernate.Initialize(contract.Notes);
    session.DefaultReadOnly = false;
    ...
    tx.Commit();
}
```

If `Session.DefaultReadOnly` returns `true`, then you can use `Query.SetReadOnly(false)` and `Criteria.SetReadOnly(false)` to override this session setting and load entities that are not read-only.

11.1.4. Making a persistent entity read-only

Note

Persistent entities of immutable classes are automatically made read-only.

To make a persistent entity or proxy read-only, call:

```
Session.SetReadOnly(entityOrProxy, true)
```

To change a read-only entity or proxy of a mutable class so it is no longer read-only, call:

```
Session.SetReadOnly(entityOrProxy, false)
```

Important

When a read-only entity or proxy is changed so it is no longer read-only, NHibernate assumes that the current state of the read-only entity is consistent with its database representation. If this is not true, then any non-flushed changes made before or while the entity was read-only, will be ignored.

To throw away non-flushed changes and make the persistent entity consistent with its database representation, call:

```
Session.Refresh(entity);
```

To flush changes made before or while the entity was read-only and make the database representation consistent with the current state of the persistent entity:

```
// evict the read-only entity so it is detached
session.Evict(entity);

// make the detached entity (with the non-flushed changes) persistent
session.Update(entity);

// now entity is no longer read-only and its changes can be flushed
s.Flush();
```

11.2. Read-only affect on property type

The following table summarizes how different property types are affected by making an entity read-only.

Table 11.1. Affect of read-only entity on property types

Property/Association Type	Changes flushed to DB?
Simple (Section 11.2.1, “Simple properties”)	no*
Unidirectional one-to-one	no*

Property/Association Type	Changes flushed to DB?
Unidirectional many-to-one (Section 11.2.2.1, “Unidirectional one-to-one and many-to-one”)	no*
Unidirectional one-to-many Unidirectional many-to-many (Section 11.2.2.2, “Unidirectional one-to-many and many-to-many”)	yes yes
Bidirectional one-to-one (Section 11.2.3.1, “Bidirectional one-to-one”)	only if the owning entity is not read-only*
Bidirectional one-to-many/many-to-one inverse collection non-inverse collection (Section 11.2.3.2, “Bidirectional one-to-many/many-to-one”)	only added/removed entities that are not read-only* yes
Bidirectional many-to-many (Section 11.2.3.3, “Bidirectional many-to-many”)	yes

* Behavior is different when the entity having the property/association is read-only, compared to when it is not read-only.

11.2.1. Simple properties

When a persistent object is read-only, NHibernate does not dirty-check simple properties.

NHibernate will not synchronize simple property state changes to the database. If you have automatic versioning, NHibernate will not increment the version if any simple properties change.

```
using (ISession session = factory.OpenSession())
using (ITransaction tx = session.BeginTransaction())
{
    // get a contract and make it read-only
    Contract contract = session.Get<Contract>(contractId);
    session.SetReadOnly(contract, true);

    // contract.CustomerName is "Sherman"
    contract.CustomerName = "Yogi";
    tx.Commit();

    tx = session.BeginTransaction();

    contract = session.Get<Contract>(contractId);
    // contract.CustomerName is still "Sherman"
```

```

    ...
    tx.Commit();
}

```

11.2.2. Unidirectional associations

11.2.2.1. Unidirectional one-to-one and many-to-one

NHibernate treats unidirectional one-to-one and many-to-one associations in the same way when the owning entity is read-only.

We use the term *unidirectional single-ended association* when referring to functionality that is common to unidirectional one-to-one and many-to-one associations.

NHibernate does not dirty-check unidirectional single-ended associations when the owning entity is read-only.

If you change a read-only entity's reference to a unidirectional single-ended association to null, or to refer to a different entity, that change will not be flushed to the database.

Note

If an entity is of an immutable class, then its references to unidirectional single-ended associations must be assigned when that entity is first created. Because the entity is automatically made read-only, these references can not be updated.

If automatic versioning is used, NHibernate will not increment the version due to local changes to unidirectional single-ended associations.

In the following examples, Contract has a unidirectional many-to-one association with Plan. Contract cascades save and update operations to the association.

The following shows that changing a read-only entity's many-to-one association reference to null has no effect on the entity's database representation.

```

// get a contract with an existing plan;
// make the contract read-only and set its plan to null
using (var tx = session.BeginTransaction())
{
    Contract contract = session.Get<Contract>(contractId);
    session.SetReadOnly(contract, true);
    contract.Plan = null;
    tx.Commit();
}

// get the same contract
using (var tx = session.BeginTransaction())
{
    Contract contract = session.Get<Contract>(contractId);

    // contract.Plan still refers to the original plan;

    tx.Commit();
}
session.Close();

```

The following shows that, even though an update to a read-only entity's many-to-one association has no effect on the entity's database representation, flush still cascades the save-update operation to the locally changed association.

```
// get a contract with an existing plan;
// make the contract read-only and change to a new plan
Contract contract;
Plan newPlan;
using (var tx = session.BeginTransaction())
{
    contract = session.Get<Contract>(contractId);
    session.SetReadOnly(contract, true);
    newPlan = new Plan("new plan");
    contract.Plan = newPlan;
    tx.Commit();
}

// get the same contract
using (var tx = session.BeginTransaction())
{
    contract = session.Get<Contract>(contractId);
    newPlan = session.Get<Plan>(newPlan.Id);

    // contract.Plan still refers to the original plan;
    // newPlan is non-null because it was persisted when
    // the previous transaction was committed;

    tx.Commit();
}
session.Close();
```

11.2.2.2. Unidirectional one-to-many and many-to-many

NHibernate treats unidirectional one-to-many and many-to-many associations owned by a read-only entity the same as when owned by an entity that is not read-only.

NHibernate dirty-checks unidirectional one-to-many and many-to-many associations;

The collection can contain entities that are read-only, as well as entities that are not read-only.

Entities can be added and removed from the collection; changes are flushed to the database.

If automatic versioning is used, NHibernate will update the version due to changes in the collection if they dirty the owning entity.

11.2.3. Bidirectional associations

11.2.3.1. Bidirectional one-to-one

If a read-only entity owns a bidirectional one-to-one association:

- NHibernate does not dirty-check the association.
- updates that change the association reference to null or to refer to a different entity will not be flushed to the database.
- If automatic versioning is used, NHibernate will not increment the version due to local changes to the association.

Note

If an entity is of an immutable class, and it owns a bidirectional one-to-one association, then its reference must be assigned when that entity is first created. Because the entity is automatically made read-

only, these references cannot be updated.

When the owner is not read-only, NHibernate treats an association with a read-only entity the same as when the association is with an entity that is not read-only.

11.2.3.2. Bidirectional one-to-many/many-to-one

A read-only entity has no impact on a bidirectional one-to-many/many-to-one association if:

- the read-only entity is on the one-to-many side using an inverse collection;
- the read-only entity is on the one-to-many side using a non-inverse collection;
- the one-to-many side uses a non-inverse collection that contains the read-only entity

When the one-to-many side uses an inverse collection:

- a read-only entity can only be added to the collection when it is created;
- a read-only entity can only be removed from the collection by an orphan delete or by explicitly deleting the entity.

11.2.3.3. Bidirectional many-to-many

NHibernate treats bidirectional many-to-many associations owned by a read-only entity the same as when owned by an entity that is not read-only.

NHibernate dirty-checks bidirectional many-to-many associations.

The collection on either side of the association can contain entities that are read-only, as well as entities that are not read-only.

Entities are added and removed from both sides of the collection; changes are flushed to the database.

If automatic versioning is used, NHibernate will update the version due to changes in both sides of the collection if they dirty the entity owning the respective collections.

Chapter 12. Transactions And Concurrency

NHibernate is not itself a database. It is a lightweight object-relational mapping tool. Transaction management is delegated to the underlying database connection. If the connection is enlisted with a distributed transaction, operations performed by the `ISession` are atomically part of the wider distributed transaction. NHibernate can be seen as a thin adapter to ADO.NET, adding object-oriented semantics.

12.1. Configurations, Sessions and Factories

An `ISessionFactory` is an expensive-to-create, threadsafe object intended to be shared by all application threads. An `ISession` is an inexpensive, non-threadsafe object that should be used once, for a single business process, and then discarded. For example, when using NHibernate in an ASP.NET application, pages could obtain an `ISessionFactory` using:

```
ISessionFactory sf = Global.SessionFactory;
```

Each call to a service method could create a new `ISession`, `Flush()` it, `Commit()` its transaction, `Close()` it and finally discard it. (The `ISessionFactory` may also be kept in a static *Singleton* helper variable.)

We use the NHibernate `ITransaction` API as discussed previously, a single `Commit()` of a NHibernate `ITransaction` flushes the state and commits any underlying database connection. Transaction scopes can be used instead, which allows the transaction to be distributed. See Section 12.8, “Transaction scopes (System.Transactions)”.

NHibernate may be used without managing transactions neither with the `ITransaction` API nor with transaction scopes. We advise against such a pattern, because this causes each single interaction of the session with the database to be a transaction on its own. This causes overhead, breaks the unit of work in case of errors during flushing, and causes some features to be disabled like auto-flush and second level cache. If the application encounters locks contention or deadlocks, investigate lowering the isolation level (like enabling read committed snapshot with SQL Server) or shortening your transactions rather than removing them.

Ensure you understand the semantics of `Flush()`. Flushing synchronizes the persistent store with in-memory changes but *not* vice-versa. Note that for all NHibernate ADO.NET connections/transactions, the transaction isolation level for that connection applies to all operations executed by NHibernate!

The next few sections will discuss alternative approaches that utilize versioning to ensure transaction atomicity. These are considered "advanced" approaches to be used with care.

12.2. Threads and connections

You should observe the following practices when creating NHibernate Sessions:

- Never create more than one concurrent `ISession` or `ITransaction` instance per database connection.
- Be extremely careful when creating more than one `ISession` per database per transaction. The `ISession` itself keeps track of updates made to loaded objects, so a different `ISession` might see stale data.
- The `ISession` is *not* threadsafe! Never access the same `ISession` in two concurrent threads. An `ISession` is usually only a single unit-of-work!

Since NHibernate 5.0, the session and its queries IO bound methods have async counterparts. Each call to an async method must be awaited before further interacting with the session or its queries.

12.3. Considering object identity

The application may concurrently access the same persistent state in two different units-of-work. However, an instance of a persistent class is never shared between two `ISession` instances. Hence there are two different notions of identity:

Database Identity

```
foo.Id.Equals( bar.Id )
```

CLR Identity

```
foo == bar
```

Then for objects attached to a *particular* Session, the two notions are equivalent. However, while the application might concurrently access the "same" (persistent identity) business object in two different sessions, the two instances will actually be "different" (CLR identity).

This approach leaves NHibernate and the database to worry about concurrency. The application never needs to synchronize on any business object, as long as it sticks to a single thread per `ISession` or object identity (within an `ISession` the application may safely use `==` to compare objects).

12.4. Optimistic concurrency control

Many business processes require a whole series of interactions with the user interleaved with database accesses. In web and enterprise applications it is not acceptable for a database transaction to span a user interaction.

Maintaining isolation of business processes becomes the partial responsibility of the application tier, hence we call this process a long running *application transaction*. A single application transaction usually spans several database transactions. It will be atomic if only one of these database transactions (the last one) stores the updated data, all others simply read data.

The only approach that is consistent with high concurrency and high scalability is optimistic concurrency control with versioning. NHibernate provides for three possible approaches to writing application code that uses optimistic concurrency.

12.4.1. Long session with automatic versioning

A single `ISession` instance and its persistent instances are used for the whole application transaction.

The `ISession` uses optimistic locking with versioning to ensure that many database transactions appear to the application as a single logical application transaction. The `ISession` is disconnected from any underlying ADO.NET connection when waiting for user interaction. This approach is the most efficient in terms of database access. The application need not concern itself with version checking or with reattaching detached instances.

```
// foo is an instance loaded earlier by the Session
session.Reconnect();
using (var transaction = session.BeginTransaction())
{
    foo.Property = "bar";
    session.Flush();
    transaction.Commit();
}
session.Disconnect();
```

The `foo` object still knows which `ISession` it was loaded in. As soon as the `ISession` has an ADO.NET connection, we commit the changes to the object.

This pattern is problematic if our `ISession` is too big to be stored during user think time, e.g. an `HttpSession` should be kept as small as possible. As the `ISession` is also the (mandatory) first-level cache and contains all loaded objects, we can probably use this strategy only for a few request/response cycles. This is indeed recommended, as the `ISession` will soon also have stale data.

12.4.2. Many sessions with automatic versioning

Each interaction with the persistent store occurs in a new `ISession`. However, the same persistent instances are reused for each interaction with the database. The application manipulates the state of detached instances originally loaded in another `ISession` and then "re-associates" them using `ISession.Update()` or `ISession.SaveOrUpdate()`.

```
// foo is an instance loaded by a previous Session
foo.Property = "bar";
using (var session = factory.OpenSession())
using (var transaction = session.BeginTransaction())
{
    session.SaveOrUpdate(foo);
    session.Flush();
    transaction.Commit();
}
```

You may also call `Lock()` instead of `Update()` and use `LockMode.Read` (performing a version check, bypassing all caches) if you are sure that the object has not been modified.

12.4.3. Customizing automatic versioning

You may disable NHibernate's automatic version increment for particular properties and collections by setting the `optimistic-lock` mapping attribute to `false`. NHibernate will then no longer increment versions if the property is dirty.

Legacy database schemas are often static and can't be modified. Or, other applications might also access the same database and don't know how to handle version numbers or even timestamps. In both cases, versioning can't rely on a particular column in a table. To force a version check without a version or timestamp property mapping, with a comparison of the state of all fields in a row, turn on `optimistic-lock="all"` in the `<class>` mapping. Note that this conceptually only works if NHibernate can compare the old and new state, i.e. if you use a single long `ISession` and not `session-per-request-with-detached-objects`.

Sometimes concurrent modification can be permitted as long as the changes that have been made don't overlap. If you set `optimistic-lock="dirty"` when mapping the `<class>`, NHibernate will only compare dirty fields during flush.

In both cases, with dedicated version/timestamp columns or with full/dirty field comparison, NHibernate uses a single `UPDATE` statement (with an appropriate `WHERE` clause) per entity to execute the version check and update the information. If you use transitive persistence to cascade reattachment to associated entities, NHibernate might execute unnecessary updates. This is usually not a problem, but *on update* triggers in the database might be executed even when no changes have been made to detached instances. You can customize this behavior by setting `select-before-update="true"` in the `<class>` mapping, forcing NHibernate to `SELECT` the instance to ensure that changes did actually occur, before updating the row.

12.4.4. Application version checking

Each interaction with the database occurs in a new `ISession` that reloads all persistent instances from the database before manipulating them. This approach forces the application to carry out its own version checking to ensure application transaction isolation. (Of course, NHibernate will still *update* version numbers for you.) This approach is the least efficient in terms of database access.

```
// foo is an instance loaded by a previous Session
using (var session = factory.OpenSession())
using (var transaction = session.BeginTransaction())
{
    int oldVersion = foo.Version;
    session.Load( foo, foo.Key );
    if ( oldVersion != foo.Version ) throw new StaleObjectStateException();
    foo.Property = "bar";
    session.Flush();
    transaction.Commit();
}
```

Of course, if you are operating in a low-data-concurrency environment and don't require version checking, you may use this approach and just skip the version check.

12.5. Session disconnection

The first approach described above is to maintain a single `ISession` for a whole business process that spans user think time. (For example, a servlet might keep an `ISession` in the user's `HttpSession`.) For performance reasons you should

1. commit the `ITransaction` and then
2. disconnect the `ISession` from the ADO.NET connection

before waiting for user activity. The method `ISession.Disconnect()` will disconnect the session from the ADO.NET connection and return the connection to the pool (unless you provided the connection).

`ISession.Reconnect()` obtains a new connection (or you may supply one) and restarts the session. After reconnection, to force a version check on data you aren't updating, you may call `ISession.Lock()` on any objects that might have been updated by another transaction. You don't need to lock any data that you *are* updating.

Here is an example:

```
ISessionFactory sessions;
IList<Foo> fooList;
Bar bar;
....
ISession s = sessions.OpenSession();
ITransaction tx = null;

try
{
    tx = s.BeginTransaction()

    fooList = s
        .CreateQuery(
            "select foo from Eg.Foo foo where foo.Date = current date"
            // uses db2 date function
        ).List<Foo>();

    bar = new Bar();
    s.Save(bar);
}
```

```

        tx.Commit();
    }
    catch (Exception)
    {
        if (tx != null) tx.Rollback();
        s.Close();
        throw;
    }
    s.Disconnect();

```

Later on:

```

s.Reconnect();

try
{
    tx = s.BeginTransaction();

    bar.FooTable = new HashMap();
    foreach (Foo foo in fooList)
    {
        s.Lock(foo, LockMode.Read);    //check that foo isn't stale
        bar.FooTable.Put( foo.Name, foo );
    }

    tx.Commit();
}
catch (Exception)
{
    if (tx != null) tx.Rollback();
    throw;
}
finally
{
    s.Close();
}

```

You can see from this how the relationship between `ITransactions` and `ISessions` is many-to-one, An `ISession` represents a conversation between the application and the database. The `ITransaction` breaks that conversation up into atomic units of work at the database level.

12.6. Pessimistic Locking

It is not intended that users spend much time worrying about locking strategies. It's usually enough to specify an isolation level for the ADO.NET connections and then simply let the database do all the work. However, advanced users may sometimes wish to obtain exclusive pessimistic locks, or re-obtain locks at the start of a new transaction.

NHibernate will always use the locking mechanism of the database, never lock objects in memory!

The `LockMode` class defines the different lock levels that may be acquired by NHibernate. A lock is obtained by the following mechanisms:

- `LockMode.Write` is acquired automatically when NHibernate updates or inserts a row.
- `LockMode.Upgrade` may be acquired upon explicit user request using `SELECT ... FOR UPDATE` on databases which support that syntax.
- `LockMode.UpgradeNowait` may be acquired upon explicit user request using a `SELECT ... FOR UPDATE NOWAIT` under Oracle.
- `LockMode.Read` is acquired automatically when NHibernate reads data under Repeatable Read or Serializable isolation level. May be re-acquired by explicit user request.

- `LockMode.None` represents the absence of a lock. All objects switch to this lock mode at the end of an `ITransaction`. Objects associated with the session via a call to `Update()` or `SaveOrUpdate()` also start out in this lock mode.

The "explicit user request" is expressed in one of the following ways:

- A call to `ISession.Load()`, specifying a `LockMode`.
- A call to `ISession.Lock()`.
- A call to `IQuery.SetLockMode()`.

If `ISession.Load()` is called with `Upgrade` or `UpgradeNowait`, and the requested object was not yet loaded by the session, the object is loaded using `SELECT ... FOR UPDATE`. If `Load()` is called for an object that is already loaded with a less restrictive lock than the one requested, NHibernate calls `Lock()` for that object.

`ISession.Lock()` performs a version number check if the specified lock mode is `Read`, `Upgrade` or `UpgradeNowait`. (In the case of `Upgrade` or `UpgradeNowait`, `SELECT ... FOR UPDATE` is used.)

If the database does not support the requested lock mode, NHibernate will use an appropriate alternate mode (instead of throwing an exception). This ensures that applications will be portable.

12.7. Connection Release Modes

The legacy (1.0.x) behavior of NHibernate in regards to ADO.NET connection management was that a `ISession` would obtain a connection when it was first needed and then hold unto that connection until the session was closed. NHibernate introduced the notion of connection release modes to tell a session how to handle its ADO.NET connections. Note that the following discussion is pertinent only to connections provided through a configured `IConnectionProvider`; user-supplied connections are outside the breadth of this discussion. The different release modes are identified by the enumerated values of `NHibernate.ConnectionReleaseMode`:

- `onClose` - is essentially the legacy behavior described above. The NHibernate session obtains a connection when it first needs to perform some database access and holds unto that connection until the session is closed.
- `AfterTransaction` - says to release connections after a `NHibernate.ITransaction` has completed.

The configuration parameter `connection.release_mode` is used to specify which release mode to use. The possible values:

- `auto` (the default) - equivalent to `after_transaction` in the current release. It is rarely a good idea to change this default behavior as failures due to the value of this setting tend to indicate bugs and/or invalid assumptions in user code.
- `on_close` - says to use `ConnectionReleaseMode.OnClose`. This setting is left for backwards compatibility, but its use is highly discouraged.
- `after_transaction` - says to use `ConnectionReleaseMode.AfterTransaction`. Note that with `ConnectionReleaseMode.AfterTransaction`, if a session is considered to be in auto-commit mode (i.e. no transaction was started) connections will be released after every operation.

If your application manages transactions through .NET APIs such as `System.Transactions` library while not using a compatible transaction factory (see `transaction.factory_class` in Section 3.6, "Optional configuration properties"), `ConnectionReleaseMode.AfterTransaction` may cause NHibernate to open and close several connections during one transaction, leading to unnecessary overhead and transaction promotion from local to distributed. Specifying `ConnectionReleaseMode.OnClose` will revert to the legacy behavior and prevent this problem from occurring.

12.8. Transaction scopes (System.Transactions)

Instead of using NHibernate `ITransaction`, `TransactionScope` can be used. Please do not use both simultaneously. Using `TransactionScope` requires using a compatible transaction factory (see `transaction.factory_class` in Section 3.6, “Optional configuration properties”). The default transaction factory supports scopes.

When using `TransactionScope` with NHibernate, you need to be aware of following points:

- The session will enlist with the first scope in which the session is used (or opened). As of NHibernate v5.0, it will enlist its connection in the transaction regardless of connection string `Enlist` setting. Prior to v5.0, it was relying on that setting being considered true, and on acquiring the connection within the scope.

Sub-scopes are not supported. The session will be enlisted in the first scope within which it was used, until this scope is committed or rollback. If auto-enlistment is enabled on the connection and the session used on others scopes than the one in which it is currently enlisted, the connection may enlist in another scope, and the session will then fail to use it.

Session auto-enlistment can be controlled from the session builder obtained with `ISessionFactory.WithOptions()`, using the `AutoJoinTransaction` option. It can also be controlled at the configuration level, see `transaction.auto_join` in Section 3.6, “Optional configuration properties”. When auto-join is disabled, the connection may still enlist itself if connection string `Enlist` setting is not false. A session can explicitly join the current system transaction by calling `ISession.JoinTransaction()`.

- As of NHibernate v5.0, `FlushMode.Commit` requires the configuration setting `transaction.use_connection_on_system_prepare` to be true for flushing from transaction scope commit. Otherwise, it will be your responsibility to flush the session before completing the scope.

Using `transaction.use_connection_on_system_prepare` can cause undesired transaction promotions to distributed: it requires using a dedicated connection for flushing, and it delays session disposal (if done inside the scope) to the scope disposal. If you want to avoid this, set this setting to false and manually flush your sessions.

For new applications, it is recommended to set `transaction.use_connection_on_system_prepare` to false, and to flush explicitly your sessions before scope completion. For old applications, consider checking how sessions are flushed, and if possible switch it to false too.

- As of NHibernate v5.0, `ConnectionReleaseMode.AfterTransaction` has no more by default an "immediate" effect with transaction scopes. Previously, it was releasing the connection from transaction completion events. But this is not officially supported by Microsoft and this can cause issues especially with distributed transactions.

Since v5.0, by default, the connection will be actually released after the scope disposal at the first session usage involving a connection, or at the session closing, whichever come first. Alternatively, you may `Disconnect()` the session. (Requires `Reconnect()` before re-using the session.)

When using `transaction.use_connection_on_system_prepare`, if the session is disposed within the scope, the connection releasing will still occurs from transaction completion event.

- As of NHibernate v5.0, using transaction scope and trying to use the session connection within `AfterTransactionCompletion` is forbidden and will raise an exception. If the setting `transaction.use_connection_on_system_prepare` is false, it will forbid any connection usage from `BeforeTransactionCompletion` event too, when this event is triggered by a transaction scope commit or rollback.

Chapter 13. Interceptors and events

It is often useful for the application to react to certain events that occur inside NHibernate. This allows implementation of certain kinds of generic functionality, and extension of NHibernate functionality.

13.1. Interceptors

The `IInterceptor` interface provides callbacks from the session to the application allowing the application to inspect and/or manipulate properties of a persistent object before it is saved, updated, deleted or loaded. One possible use for this is to track auditing information. For example, the following `IInterceptor` automatically sets the `createTimestamp` when an `IAuditable` is created and updates the `lastUpdateTimestamp` property when an `IAuditable` is updated.

You may either implement `IInterceptor` directly or (better) extend `EmptyInterceptor`.

```
using System;

using NHibernate;
using NHibernate.Type;

public class AuditInterceptor : EmptyInterceptor {

    private int updates;
    private int creates;
    private int loads;

    public override void OnDelete(object entity,
                                object id,
                                object[] state,
                                string[] propertyNames,
                                IType[] types)

    {
        // do nothing
    }

    public override bool OnFlushDirty(object entity,
                                    object id,
                                    object[] currentState,
                                    object[] previousState,
                                    string[] propertyNames,
                                    IType[] types)

    {
        if ( entity is IAuditable ) {
            updates++;
            for ( int i=0; i < propertyNames.Length; i++ ) {
                if ( "lastUpdateTimestamp".Equals( propertyNames[i] ) ) {
                    currentState[i] = new DateTime();
                    return true;
                }
            }
        }
        return false;
    }

    public override bool OnLoad(object entity,
                               object id,
                               object[] state,
                               string[] propertyNames,
                               IType[] types)

    {
        if ( entity is IAuditable ) {
            loads++;
        }
        return false;
    }
}
```

```

    }

    public override bool OnSave(object entity,
                               object id,
                               object[] state,
                               string[] propertyNames,
                               IType[] types)
    {
        if ( entity is IAuditable ) {
            creates++;
            for ( int i=0; i<propertyNames.Length; i++ ) {
                if ( "createTimestamp".Equals( propertyNames[i] ) ) {
                    state[i] = new DateTime();
                    return true;
                }
            }
        }
        return false;
    }

    public override void AfterTransactionCompletion(ITransaction tx)
    {
        if ( tx.WasCommitted ) {
            System.Console.WriteLine(
                "Creations: " + creates +
                ", Updates: " + updates +
                ", Loads: " + loads);
        }
        updates=0;
        creates=0;
        loads=0;
    }
}

```

Interceptors come in two flavors: `ISession`-scoped and `ISessionFactory`-scoped.

An `ISession`-scoped interceptor is specified when a session is opened using one of the overloaded `ISessionFactory.OpenSession()` methods accepting an `IInterceptor`.

```
ISession session = sf.OpenSession( new AuditInterceptor() );
```

An `ISessionFactory`-scoped interceptor is registered with the `Configuration` object prior to building the `ISessionFactory`. In this case, the supplied interceptor will be applied to all sessions opened from that `ISessionFactory`; this is true unless a session is opened explicitly specifying the interceptor to use. `ISessionFactory`-scoped interceptors must be thread safe, taking care to not store session-specific state since multiple sessions will use this interceptor (potentially) concurrently.

```
new Configuration().SetInterceptor( new AuditInterceptor() );
```

13.2. Event system

If you have to react to particular events in your persistence layer, you may also use the `NHibernate2` *event* architecture. The event system can be used in addition or as a replacement for interceptors.

Essentially all of the methods of the `ISession` interface correlate to an event. You have a `LoadEvent`, a `FlushEvent`, etc (consult the XML configuration-file XSD or the `NHibernate.Event` namespace for the full list of defined event types). When a request is made of one of these methods, the `ISession` generates an appropriate event and passes it to the configured event listeners for that type. Out-of-the-box, these listeners implement the same processing in which those methods always resulted. However, you are free to implement a customiza-

tion of one of the listener interfaces (i.e., the `LoadEvent` is processed by the registered implementation of the `ILoadEventListener` interface), in which case their implementation would be responsible for processing any `Load()` requests made of the `ISession`.

The listeners should be considered effectively singletons; meaning, they are shared between requests, and thus should not save any state as instance variables.

A custom listener should implement the appropriate interface for the event it wants to process and/or extend one of the convenience base classes (or even the default event listeners used by NHibernate out-of-the-box as their methods are declared virtual for this purpose). Custom listeners can either be registered programmatically through the `Configuration` object, or specified in the NHibernate configuration XML. Here's an example of a custom load event listener:

```
public class MyLoadListener : ILoadEventListener
{
    // this is the single method defined by the LoadEventListener interface
    public void OnLoad(LoadEvent theEvent, LoadType loadType)
    {
        if ( !MySecurity.IsAuthorized( theEvent.EntityClassName, theEvent.EntityId ) ) {
            throw new MySecurityException("Unauthorized access");
        }
    }
}
```

You also need a configuration entry telling NHibernate to use the listener in addition to the default listener:

```
<hibernate-configuration>
  <session-factory>
    ...
    <event type="load">
      <listener class="MyLoadListener"/>
      <listener class="NHibernate.Event.Default.DefaultLoadEventListener"/>
    </event>
  </session-factory>
</hibernate-configuration>
```

Instead, you may register it programmatically:

```
Configuration cfg = new Configuration();
ILoadEventListener[] stack =
    new ILoadEventListener[] { new MyLoadListener(), new DefaultLoadEventListener() };
cfg.EventListeners.LoadEventListeners = stack;
```

Listeners registered declaratively cannot share instances. If the same class name is used in multiple `<listener/>` elements, each reference will result in a separate instance of that class. If you need the capability to share listener instances between listener types you must use the programmatic registration approach.

Why implement an interface and define the specific type during configuration? Well, a listener implementation could implement multiple event listener interfaces. Having the type additionally defined during registration makes it easier to turn custom listeners on or off during configuration.

Chapter 14. Batch processing

A naive approach to inserting 100 000 rows in the database using NHibernate might look like this:

```
using (ISession session = sessionFactory.OpenSession())
using (ITransaction tx = session.BeginTransaction())
{
    for (int i = 0; i < 100000; i++)
    {
        Customer customer = new Customer(.....);
        session.Save(customer);
    }
    tx.Commit();
}
```

This would fall over with an `OutOfMemoryException` somewhere around the 50 000th row. That's because NHibernate caches all the newly inserted `Customer` instances in the session-level cache.

In this chapter we'll show you how to avoid this problem. First, however, if you are doing batch processing, it is absolutely critical that you enable the use of ADO batching, if you intend to achieve reasonable performance. Set the ADO batch size to a reasonable number (say, 10-50):

```
adonet.batch_size 20
```

Note that NHibernate disables insert batching at the ADO level transparently if you use an identity identifier generator.

You also might like to do this kind of work in a process where interaction with the second-level cache is completely disabled:

```
cache.use_second_level_cache false
```

However, this is not absolutely necessary, since we can explicitly set the `CacheMode` to disable interaction with the second-level cache.

14.1. Batch inserts

When making new objects persistent, you must `Flush()` and then `Clear()` the session regularly, to control the size of the first-level cache.

```
using (ISession session = sessionFactory.openSession())
using (ITransaction tx = session.BeginTransaction())
{
    for (int i = 0; i < 100000; i++)
    {
        Customer customer = new Customer(.....);
        session.Save(customer);
        // 20, same as the ADO batch size
        if (i % 20 == 0)
        {
            // flush a batch of inserts and release memory:
            session.Flush();
            session.Clear();
        }
    }

    tx.Commit();
}
```

14.2. The StatelessSession interface

Alternatively, NHibernate provides a command-oriented API that may be used for streaming data to and from the database in the form of detached objects. A `IStatelessSession` has no persistence context associated with it and does not provide many of the higher-level life cycle semantics. In particular, a stateless session does not implement a first-level cache nor interact with any second-level or query cache. It does not implement transactional write-behind or automatic dirty checking. Operations performed using a stateless session do not ever cascade to associated instances. Collections are ignored by a stateless session. Operations performed via a stateless session bypass NHibernate's event model and interceptors. Stateless sessions are vulnerable to data aliasing effects, due to the lack of a first-level cache. A stateless session is a lower-level abstraction, much closer to the underlying ADO.

```
using (IStatelessSession session = sessionFactory.OpenStatelessSession())
using (ITransaction tx = session.BeginTransaction())
{
    var customers = session.GetNamedQuery("GetCustomers")
        .Enumerable<Customer>();
    while (customers.MoveNext())
    {
        Customer customer = customers.Current;
        customer.updateStuff(...);
        session.Update(customer);
    }

    tx.Commit();
}
```

Note that in this code example, the `Customer` instances returned by the query are immediately detached. They are never associated with any persistence context.

The `insert()`, `update()` and `delete()` operations defined by the `StatelessSession` interface are considered to be direct database row-level operations, which result in immediate execution of a SQL `INSERT`, `UPDATE` or `DELETE` respectively. Thus, they have very different semantics to the `Save()`, `SaveOrUpdate()` and `Delete()` operations defined by the `ISession` interface.

14.3. DML-style operations

As already discussed, automatic and transparent object/relational mapping is concerned with the management of object state. This implies that the object state is available in memory, hence manipulating (using the SQL Data Manipulation Language (DML) statements: `INSERT`, `UPDATE`, `DELETE`) data directly in the database will not affect in-memory state. However, NHibernate provides methods for bulk SQL-style DML statement execution which are performed through the Hibernate Query Language (HQL). A Linq implementation is available too.

The pseudo-syntax for `UPDATE` and `DELETE` statements is: `( UPDATE | DELETE ) FROM? EntityName (WHERE where_conditions)?`. Some points to note:

- In the from-clause, the `FROM` keyword is optional
- There can only be a single entity named in the from-clause; it can optionally be aliased. If the entity name is aliased, then any property references must be qualified using that alias; if the entity name is not aliased, then it is illegal for any property references to be qualified.
- No joins (either implicit or explicit) can be specified in a bulk HQL query. Sub-queries may be used in the where-clause; the sub-queries, themselves, may contain joins.
- The where-clause is also optional.

As an example, to execute an HQL UPDATE, use the `IQuery.ExecuteUpdate()` method:

```
using (ISession session = sessionFactory.OpenSession())
using (ITransaction tx = session.BeginTransaction())
{
    string hqlUpdate = "update Customer c set c.name = :newName where c.name = :oldName";
    // or string hqlUpdate = "update Customer set name = :newName where name = :oldName";
    int updatedEntities = s.CreateQuery(hqlUpdate)
        .SetString("newName", newName)
        .SetString("oldName", oldName)
        .ExecuteUpdate();
    tx.Commit();
}
```

HQL UPDATE statements, by default do not effect the version or the timestamp property values for the affected entities. However, you can force NHibernate to properly reset the version or timestamp property values through the use of a versioned update. This is achieved by adding the `VERSIONED` keyword after the `UPDATE` keyword.

```
using (ISession session = sessionFactory.OpenSession())
using (ITransaction tx = session.BeginTransaction())
{
    string hqlVersionedUpdate =
        "update versioned Customer set name = :newName where name = :oldName";
    int updatedEntities = s.CreateQuery(hqlUpdate)
        .SetString("newName", newName)
        .SetString("oldName", oldName)
        .ExecuteUpdate();
    tx.Commit();
}
```

Note that custom version types (`NHibernate.UserType.IUserVersionType`) are not allowed in conjunction with a `update versioned` statement.

To execute an HQL DELETE, use the same `IQuery.ExecuteUpdate()` method:

```
using (ISession session = sessionFactory.OpenSession())
using (ITransaction tx = session.BeginTransaction())
{
    string hqlDelete = "delete Customer c where c.name = :oldName";
    // or String hqlDelete = "delete Customer where name = :oldName";
    int deletedEntities = s.CreateQuery(hqlDelete)
        .SetString("oldName", oldName)
        .ExecuteUpdate();
    tx.Commit();
}
```

The `int` value returned by the `IQuery.ExecuteUpdate()` method indicate the number of entities effected by the operation. Consider this may or may not correlate to the number of rows effected in the database. An HQL bulk operation might result in multiple actual SQL statements being executed, for joined-subclass, for example. The returned number indicates the number of actual entities affected by the statement. Going back to the example of joined-subclass, a delete against one of the subclasses may actually result in deletes against not just the table to which that subclass is mapped, but also the "root" table and potentially joined-subclass tables further down the inheritance hierarchy.

The pseudo-syntax for INSERT statements is: `INSERT INTO EntityName properties_list select_statement`. Some points to note:

- Only the `INSERT INTO ... SELECT ...` form is supported; not the `INSERT INTO ... VALUES ...` form.

The `properties_list` is analogous to the `column specification` in the SQL `INSERT` statement. For entities

involved in mapped inheritance, only properties directly defined on that given class-level can be used in the `properties_list`. Superclass properties are not allowed; and subclass properties do not make sense. In other words, `INSERT` statements are inherently non-polymorphic.

- `select_statement` can be any valid HQL select query, with the caveat that the return types must match the types expected by the insert. Currently, this is checked during query compilation rather than allowing the check to be relegated to the database. Note however that this might cause problems between NHibernate Types which are *equivalent* as opposed to *equal*. This might cause issues with mismatches between a property defined as a `NHibernate.Type.DateType` and a property defined as a `NHibernate.Type.TimestampType`, even though the database might not make a distinction or might be able to handle the conversion.
- For the `id` property, the insert statement gives you two options. You can either explicitly specify the `id` property in the `properties_list` (in which case its value is taken from the corresponding select expression) or omit it from the `properties_list` (in which case a generated value is used). This latter option is only available when using `id` generators that operate in the database; attempting to use this option with any "in memory" type generators will cause an exception during parsing. Note that for the purposes of this discussion, in-database generators are considered to be `NHibernate.Id.SequenceGenerator` (and its subclasses) and any implementors of `NHibernate.Id.IPostInsertIdentifierGenerator`. The most notable exception here is `NHibernate.Id.TableHiLoGenerator`, which cannot be used because it does not expose a selectable way to get its values.
- For properties mapped as either `version` or `timestamp`, the insert statement gives you two options. You can either specify the property in the `properties_list` (in which case its value is taken from the corresponding select expressions) or omit it from the `properties_list` (in which case the `seed` value defined by the `NHibernate.Type.IVersionType` is used).

An example HQL `INSERT` statement execution:

```
using (ISession session = sessionFactory.OpenSession())
using (ITransaction tx = session.BeginTransaction())
{
    var hqlInsert =
        "insert into DelinquentAccount (id, name) " +
        "select c.id, c.name from Customer c where ...";
    int createdEntities = s.CreateQuery(hqlInsert)
        .ExecuteUpdate();
    tx.Commit();
}
```

Chapter 15. HQL: The Hibernate Query Language

NHibernate is equipped with an extremely powerful query language that (quite intentionally) looks very much like SQL. But don't be fooled by the syntax; HQL is fully object-oriented, understanding notions like inheritance, polymorphism and association.

15.1. Case Sensitivity

Queries are case-insensitive, except for names of .NET classes and properties. So `SeLeCT` is the same as `seLeCT` is the same as `SELECT` but `Eg.F00` is not `Eg.Foo` and `foo.barSet` is not `foo.BARSET`.

This manual uses lowercase HQL keywords. Some users find queries with uppercase keywords more readable, but we find this convention ugly when embedded in C# code.

15.2. The from clause

The simplest possible NHibernate query is of the form:

```
from Eg.Cat
```

which simply returns all instances of the class `Eg.Cat`.

Most of the time, you will need to assign an *alias*, since you will want to refer to the `cat` in other parts of the query.

```
from Eg.Cat as cat
```

This query assigns the alias `cat` to `cat` instances, so we could use that alias later in the query. The `as` keyword is optional; we could also write:

```
from Eg.Cat cat
```

Multiple classes may appear, resulting in a cartesian product or "cross" join.

```
from Formula, Parameter
```

```
from Formula as form, Parameter as param
```

It is considered good practice to name query aliases using an initial lowercase, consistent with naming standards for local variables (eg. `domesticCat`).

15.3. Associations and joins

We may also assign aliases to associated entities, or even to elements of a collection of values, using a join.

```
from Eg.Cat as cat
    inner join cat.Mate as mate
    left outer join cat.Kittens as kitten

from Eg.Cat as cat left join cat.Mate.Kittens as kittens
```

```
from Formula form full join form.Parameter param
```

The supported join types are borrowed from ANSI SQL

- `inner join`
- `left outer join`
- `right outer join`
- `full join` (not usually useful)

The `inner join`, `left outer join` and `right outer join` constructs may be abbreviated.

```
from Eg.Cat as cat
    join cat.Mate as mate
    left join cat.Kittens as kitten
```

In addition, a "fetch" join allows associations or collections of values to be initialized along with their parent objects, using a single select. This is particularly useful in the case of a collection. It effectively overrides the outer join and lazy declarations of the mapping file for associations and collections. See Section 21.1, "Fetching strategies" for more information.

```
from Eg.Cat as cat
    inner join fetch cat.Mate
    left join fetch cat.Kittens
```

The associated objects are not returned directly in the query results. Instead, they may be accessed via the parent object.

It is possible to create a cartesian product by join fetching more than one collection in a query, so take care in this case. Join fetching multiple collection roles is also disabled for bag mappings. Note also that the `fetch` construct may not be used in queries called using `Enumerable()`. Finally, note that `full join fetch` and `right join fetch` are not meaningful.

15.4. The select clause

The select clause picks which objects and properties to return in the query result set. Consider:

```
select mate
from Eg.Cat as cat
    inner join cat.Mate as mate
```

The query will select Mates of other Cats. Actually, you may express this query more compactly as:

```
select cat.Mate from Eg.Cat cat
```

You may even select collection elements, using the special `elements` function. The following query returns all kittens of any cat.

```
select elements(cat.Kittens) from Eg.Cat cat
```

Queries may return properties of any value type including properties of component type:

```
select cat.Name from Eg.DomesticCat cat
where cat.Name like 'fri%'

select cust.Name.FirstName from Customer as cust
```

Queries may return multiple objects and/or properties as an array of type `object[]`

```
select mother, offspr, mate.Name
from Eg.DomesticCat as mother
     inner join mother.Mate as mate
     left outer join mother.Kittens as offspr
```

or as an actual type-safe object

```
select new Family(mother, mate, offspr)
from Eg.DomesticCat as mother
     join mother.Mate as mate
     left join mother.Kittens as offspr
```

assuming that the class `Family` has an appropriate constructor.

15.5. Aggregate functions

HQL queries may even return the results of aggregate functions on properties:

```
select avg(cat.Weight), sum(cat.Weight), max(cat.Weight), count(cat)
from Eg.Cat cat
```

Collections may also appear inside aggregate functions in the select clause.

```
select cat, count( elements(cat.Kittens) )
from Eg.Cat cat group by cat.Id, cat.Weight, ...
```

The supported aggregate functions are

- `avg(...)`, `sum(...)`, `min(...)`, `max(...)`
- `count(*)`
- `count(...)`, `count(distinct ...)`, `count(all...)`

The `distinct` and `all` keywords may be used and have the same semantics as in SQL.

```
select distinct cat.Name from Eg.Cat cat

select count(distinct cat.Name), count(cat) from Eg.Cat cat
```

15.6. Polymorphic queries

A query like:

```
from Eg.Cat as cat
```

returns instances not only of `Cat`, but also of subclasses like `DomesticCat`. NHibernate queries may name *any* .NET class or interface in the `from` clause. The query will return instances of all persistent classes that extend that class or implement the interface. The following query would return all persistent objects:

```
from System.Object o
```

The interface `INamed` might be implemented by various persistent classes:

```
from Eg.Named n, Eg.Named m where n.Name = m.Name
```

Note that these last two queries will require more than one SQL SELECT. This means that the `order by` clause does not correctly order the whole result set.

In order to use non-mapped base classes or interfaces in HQL queries, they have to be imported. See Section 5.1.23, “import” for more information.

15.7. The where clause

The where clause allows you to narrow the list of instances returned.

```
from Eg.Cat as cat where cat.Name='Fritz'
```

returns instances of Cat named 'Fritz'.

```
select foo
from Eg.Foo foo, Eg.Bar bar
where foo.StartDate = bar.Date
```

will return all instances of Foo for which there exists an instance of Bar with a Date property equal to the StartDate property of the Foo. Compound path expressions make the where clause extremely powerful. Consider:

```
from Eg.Cat cat where cat.Mate.Name is not null
```

This query translates to an SQL query with a table (inner) join. If you were to write something like

```
from Eg.Foo foo
where foo.Bar.Baz.Customer.Address.City is not null
```

you would end up with a query that would require four table joins in SQL.

The `=` operator may be used to compare not only properties, but also instances:

```
from Eg.Cat cat, Eg.Cat rival where cat.Mate = rival.Mate

select cat, mate
from Eg.Cat cat, Eg.Cat mate
where cat.Mate = mate
```

The special property (lowercase) `id` may be used to reference the unique identifier of an object. (You may also use its property name.)

```
from Eg.Cat as cat where cat.id = 123

from Eg.Cat as cat where cat.Mate.id = 69
```

The second query is efficient. No table join is required!

Properties of composite identifiers may also be used. Suppose Person has a composite identifier consisting of Country and MedicareNumber.

```
from Bank.Person person
where person.id.Country = 'AU'
and person.id.MedicareNumber = 123456
```

```
from Bank.Account account
where account.Owner.id.Country = 'AU'
      and account.Owner.id.MedicareNumber = 123456
```

Once again, the second query requires no table join.

Likewise, the special property `class` accesses the discriminator value of an instance in the case of polymorphic persistence. A .Net class name embedded in the where clause will be translated to its discriminator value.

```
from Eg.Cat cat where cat.class = Eg.DomesticCat
```

You may also specify properties of components or composite user types (and of components of components, etc). Never try to use a path-expression that ends in a property of component type (as opposed to a property of a component). For example, if `store.Owner` is an entity with a component `Address`

```
store.Owner.Address.City    // okay
store.Owner.Address         // error!
```

An "any" type has the special properties `id` and `class`, allowing us to express a join in the following way (where `AuditLog.Item` is a property mapped with `<any>`).

```
from Eg.AuditLog log, Eg.Payment payment
where log.Item.class = 'Eg.Payment, Eg, Version=...' and log.Item.id = payment.id
```

Notice that `log.Item.class` and `payment.class` would refer to the values of completely different database columns in the above query.

15.8. Expressions

Expressions allowed in the where clause include most of the kind of things you could write in SQL:

- mathematical operators `+`, `-`, `*`, `/`
- binary comparison operators `=`, `>=`, `<=`, `<>`, `!=`, `like`
- logical operations `and`, `or`, `not`
- string concatenation `||`
- SQL scalar functions like `upper()` and `lower()`
- Parentheses `( )` indicate grouping
- `in`, `between`, `is null`
- positional parameters `?`
- named parameters `:name`, `:start_date`, `:x1`
- SQL literals `'foo'`, `69`, `'1970-01-01 10:00:01.0'`
- Enumeration values and constants `Eg.Color.Tabby`

`in` and `between` may be used as follows:

```
from Eg.DomesticCat cat where cat.Name between 'A' and 'B'

from Eg.DomesticCat cat where cat.Name in ( 'Foo', 'Bar', 'Baz' )
```

and the negated forms may be written

```
from Eg.DomesticCat cat where cat.Name not between 'A' and 'B'

from Eg.DomesticCat cat where cat.Name not in ( 'Foo', 'Bar', 'Baz' )
```

Likewise, `is null` and `is not null` may be used to test for null values.

Booleans may be easily used in expressions by declaring HQL query substitutions in NHibernate configuration:

```
<property name="query.substitutions">true 1, false 0</property>
```

This will replace the keywords `true` and `false` with the literals `1` and `0` in the translated SQL from this HQL:

```
from Eg.Cat cat where cat.Alive = true
```

You may test the size of a collection with the special property `size`, or the special `size()` function.

```
from Eg.Cat cat where cat.Kittens.size > 0
from Eg.Cat cat where size(cat.Kittens) > 0
```

For indexed collections, you may refer to the minimum and maximum indices using `minIndex` and `maxIndex`. Similarly, you may refer to the minimum and maximum elements of a collection of basic type using `minElement` and `maxElement`.

```
from Calendar cal where cal.Holidays.maxElement > current date
```

There are also functional forms (which, unlike the constructs above, are not case sensitive):

```
from Order order where maxindex(order.Items) > 100
from Order order where minelement(order.Items) > 10000
```

The SQL functions `any`, `some`, `all`, `exists`, `in` are supported when passed the element or index set of a collection (elements and indices functions) or the result of a sub-query (see below).

```
select mother from Eg.Cat as mother, Eg.Cat as kit
where kit in elements(mother.Kittens)

select p from Eg.NameList list, Eg.Person p
where p.Name = some elements(list.Names)

from Eg.Cat cat where exists elements(cat.Kittens)

from Eg.Player p where 3 > all elements(p.Scores)

from Eg.Show show where 'fizard' in indices(show.Acts)
```

Note that these constructs - `size`, `elements`, `indices`, `minIndex`, `maxIndex`, `minElement`, `maxElement` - have certain usage restrictions:

- in a `where` clause: only for databases with sub-selects
- in a `select` clause: only `elements` and `indices` make sense

Elements of indexed collections (arrays, lists, maps) may be referred to by index (in a `where` clause only):

```
from Order order where order.Items[0].id = 1234

select person from Person person, Calendar calendar
where calendar.Holidays['national day'] = person.Birthday
and person.Nationality.Calendar = calendar

select item from Item item, Order order
where order.Items[ order.DeliveredItemIndices[0] ] = item and order.id = 11
```

```
select item from Item item, Order order
where order.Items[ maxindex(order.items) ] = item and order.id = 11
```

The expression inside [] may even be an arithmetic expression.

```
select item from Item item, Order order
where order.Items[ size(order.Items) - 1 ] = item
```

HQL also provides the built-in `index()` function, for elements of a one-to-many association or collection of values.

```
select item, index(item) from Order order
join order.Items item
where index(item) < 5
```

Scalar SQL functions supported by the underlying database may be used

```
from Eg.DomesticCat cat where upper(cat.Name) like 'FRI%'
```

If you are not yet convinced by all this, think how much longer and less readable the following query would be in SQL:

```
select cust
from Product prod,
Store store
inner join store.Customers cust
where prod.Name = 'widget'
and store.Location.Name in ( 'Melbourne', 'Sydney' )
and prod = all elements(cust.CurrentOrder.LineItems)
```

Hint: something like

```
SELECT cust.name, cust.address, cust.phone, cust.id, cust.current_order
FROM customers cust,
stores store,
locations loc,
store_customers sc,
product prod
WHERE prod.name = 'widget'
AND store.loc_id = loc.id
AND loc.name IN ( 'Melbourne', 'Sydney' )
AND sc.store_id = store.id
AND sc.cust_id = cust.id
AND prod.id = ALL(
    SELECT item.prod_id
    FROM line_items item, orders o
    WHERE item.order_id = o.id
    AND cust.current_order = o.id
)
```

15.9. The order by clause

The list returned by a query may be ordered by any property of a returned class or components:

```
from Eg.DomesticCat cat
order by cat.Name asc, cat.Weight desc, cat.Birthdate
```

The optional `asc` or `desc` indicate ascending or descending order respectively.

15.10. The group by clause

A query that returns aggregate values may be grouped by any property of a returned class or components:

```
select cat.Color, sum(cat.Weight), count(cat)
from Eg.Cat cat
group by cat.Color

select foo.id, avg( elements(foo.Names) ), max( indices(foo.Names) )
from Eg.Foo foo
group by foo.id
```

Note: You may use the `elements` and `indices` constructs inside a select clause, even on databases with no sub-selects.

A `having` clause is also allowed.

```
select cat.color, sum(cat.Weight), count(cat)
from Eg.Cat cat
group by cat.Color
having cat.Color in (Eg.Color.Tabby, Eg.Color.Black)
```

SQL functions and aggregate functions are allowed in the `having` and `order by` clauses, if supported by the underlying database (ie. not in MySQL).

```
select cat
from Eg.Cat cat
    join cat.Kittens kitten
group by cat.Id, cat.Name, cat.Other, cat.Properties
having avg(kitten.Weight) > 100
order by count(kitten) asc, sum(kitten.Weight) desc
```

Note that neither the `group by` clause nor the `order by` clause may contain arithmetic expressions. Also note that NHibernate currently does not expand a grouped entity, so you can't write `group by cat` if all properties of `cat` are non-aggregated. You have to list all non-aggregated properties explicitly.

15.11. Sub-queries

For databases that support sub-selects, NHibernate supports sub-queries within queries. A sub-query must be surrounded by parentheses (often by an SQL aggregate function call). Even correlated sub-queries (sub-queries that refer to an alias in the outer query) are allowed.

```
from Eg.Cat as fatcat
where fatcat.Weight > (
    select avg(cat.Weight) from Eg.DomesticCat cat
)

from Eg.DomesticCat as cat
where cat.Name = some (
    select name.NickName from Eg.Name as name
)

from Eg.Cat as cat
where not exists (
    from eg.Cat as mate where mate.Mate = cat
)

from Eg.DomesticCat as cat
where cat.Name not in (
    select name.NickName from Eg.Name as name
```

)

15.12. HQL examples

NHibernate queries can be quite powerful and complex. In fact, the power of the query language is one of NHibernate's main selling points. Here are some example queries very similar to queries that I used on a recent project. Note that most queries you will write are much simpler than these!

The following query returns the order id, number of items and total value of the order for all unpaid orders for a particular customer and given minimum total value, ordering the results by total value. In determining the prices, it uses the current catalog. The resulting SQL query, against the ORDER, ORDER_LINE, PRODUCT, CATALOG and PRICE tables has four inner joins and an (uncorrelated) subselect.

```
select order.id, sum(price.Amount), count(item)
from Order as order
  join order.LineItems as item
  join item.Product as product,
  Catalog as catalog
  join catalog.Prices as price
where order.Paid = false
  and order.Customer = :customer
  and price.Product = product
  and catalog.EffectiveDate < sysdate
  and catalog.EffectiveDate >= all (
    select cat.EffectiveDate
    from Catalog as cat
    where cat.EffectiveDate < sysdate
  )
group by order
having sum(price.Amount) > :minAmount
order by sum(price.Amount) desc
```

What a monster! Actually, in real life, I'm not very keen on sub-queries, so my query was really more like this:

```
select order.id, sum(price.amount), count(item)
from Order as order
  join order.LineItems as item
  join item.Product as product,
  Catalog as catalog
  join catalog.Prices as price
where order.Paid = false
  and order.Customer = :customer
  and price.Product = product
  and catalog = :currentCatalog
group by order
having sum(price.Amount) > :minAmount
order by sum(price.Amount) desc
```

The next query counts the number of payments in each status, excluding all payments in the AwaitingApproval status where the most recent status change was made by the current user. It translates to an SQL query with two inner joins and a correlated subselect against the PAYMENT, PAYMENT_STATUS and PAYMENT_STATUS_CHANGE tables.

```
select count(payment), status.Name
from Payment as payment
  join payment.CurrentStatus as status
  join payment.StatusChanges as statusChange
where payment.Status.Name <> PaymentStatus.AwaitingApproval
  or (
    statusChange.TimeStamp = (
      select max(change.TimeStamp)
```

```

        from PaymentStatusChange change
        where change.Payment = payment
    )
    and statusChange.User <> :currentUser
)
group by status.Name, status.SortOrder
order by status.SortOrder

```

If I would have mapped the `StatusChanges` collection as a list, instead of a set, the query would have been much simpler to write.

```

select count(payment), status.Name
from Payment as payment
    join payment.CurrentStatus as status
where payment.Status.Name <> PaymentStatus.AwaitingApproval
    or payment.StatusChanges[ maxIndex(payment.StatusChanges) ].User <> :currentUser
group by status.Name, status.SortOrder
order by status.SortOrder

```

The next query uses the MS SQL Server `isNull()` function to return all the accounts and unpaid payments for the organization to which the current user belongs. It translates to an SQL query with three inner joins, an outer join and a subselect against the `ACCOUNT`, `PAYMENT`, `PAYMENT_STATUS`, `ACCOUNT_TYPE`, `ORGANIZATION` and `ORG_USER` tables.

```

select account, payment
from Account as account
    left outer join account.Payments as payment
where :currentUser in elements(account.Holder.Users)
    and PaymentStatus.Unpaid = isNull(payment.CurrentStatus.Name, PaymentStatus.Unpaid)
order by account.Type.SortOrder, account.AccountNumber, payment.DueDate

```

For some databases, we would need to do away with the (correlated) subselect.

```

select account, payment
from Account as account
    join account.Holder.Users as user
    left outer join account.Payments as payment
where :currentUser = user
    and PaymentStatus.Unpaid = isNull(payment.CurrentStatus.Name, PaymentStatus.Unpaid)
order by account.Type.SortOrder, account.AccountNumber, payment.DueDate

```

15.13. Tips & Tricks

You can count the number of query results without actually returning them:

```

var count = session.CreateQuery("select count(*) from ...").UniqueResult<long>();

```

To order a result by the size of a collection, use the following query:

```

select usr.id, usr.Name
from User as usr
    left join usr.Messages as msg
group by usr.id, usr.Name
order by count(msg)

```

If your database supports sub-selects, you can place a condition upon selection size in the where clause of your query:

```

from User usr where size(usr.Messages) >= 1

```

If your database doesn't support sub-selects, use the following query:

```
select usr.id, usr.Name
from User usr
      join usr.Messages msg
group by usr.id, usr.Name
having count(msg) >= 1
```

As this solution can't return a User with zero messages because of the inner join, the following form is also useful:

```
select usr.id, usr.Name
from User as usr
      left join usr.Messages as msg
group by usr.id, usr.Name
having count(msg) = 0
```

Properties of an object can be bound to named query parameters:

```
IQuery q =
    s.CreateQuery("from foo in class Foo where foo.Name=:Name and foo.Size=:Size");
q.SetProperties(fooBean); // fooBean has properties Name and Size
var foos = q.List<Foo>();
```

Collections are pageable by using the IQuery interface with a filter:

```
IQuery q = s.CreateFilter( collection, "" ); // the trivial filter
q.setMaxResults(PageSize);
q.setFirstResult(PageSize * pageNumber);
var page = q.List<Cat>();
```

Collection elements may be ordered or grouped using a query filter:

```
var orderedCollection = s
    .CreateFilter(collection, "order by this.Amount")
    .List<Cat>();
var counts = s
    .CreateFilter(collection,
        "select this.Type, count(this) group by this.Type")
    .List<object[]>();
```

Chapter 16. Criteria Queries

NHibernate features an intuitive, extensible criteria query API.

16.1. Creating an `ICriteria` instance

The interface `NHibernate.ICriteria` represents a query against a particular persistent class. The `ISession` is a factory for `ICriteria` instances.

```
ICriteria crit = sess.CreateCriteria<Cat>();
crit.SetMaxResults(50);
var cats = crit.List<Cat>();
```

16.2. Narrowing the result set

An individual query criterion is an instance of the interface `NHibernate.Expression.ICriterion`. The class `NHibernate.Expression.Expression` defines factory methods for obtaining certain built-in `ICriterion` types.

```
var cats = sess.CreateCriteria<Cat>()
    .Add( Expression.Like("Name", "Fritz%") )
    .Add( Expression.Between("Weight", minWeight, maxWeight) )
    .List<Cat>();
```

Expressions may be grouped logically.

```
var cats = sess.CreateCriteria<Cat>()
    .Add( Expression.Like("Name", "Fritz%") )
    .Add( Expression.Or(
        Expression.Eq( "Age", 0 ),
        Expression.IsNull("Age")
    ) )
    .List<Cat>();
```

```
var cats = sess.CreateCriteria<Cat>()
    .Add( Expression.In( "Name", new String[] { "Fritz", "Izi", "Pk" } ) )
    .Add( Expression.Disjunction()
        .Add( Expression.IsNull("Age") )
        .Add( Expression.Eq("Age", 0 ) )
        .Add( Expression.Eq("Age", 1 ) )
        .Add( Expression.Eq("Age", 2 ) )
    ) )
    .List<Cat>();
```

There are quite a range of built-in criterion types (`Expression` subclasses), but one that is especially useful lets you specify SQL directly.

```
// Create a string parameter for the SqlString below
var cats = sess.CreateCriteria<Cat>()
    .Add(Expression.Sql("lower({alias}.Name) like lower(?)",
        "Fritz%", NHibernateUtil.String))
    .List<Cat>();
```

The `{alias}` placeholder will be replaced by the row alias of the queried entity.

16.3. Ordering the results

You may order the results using `NHibernate.Expression.Order`.

```
var cats = sess.CreateCriteria<Cat>()
    .Add( Expression.Like("Name", "F%") )
    .AddOrder( Order.Asc("Name") )
    .AddOrder( Order.Desc("Age") )
    .SetMaxResults(50)
    .List<Cat>();
```

16.4. Associations

You may easily specify constraints upon related entities by navigating associations using `CreateCriteria()`.

```
var cats = sess.CreateCriteria<Cat>()
    .Add( Expression.Like("Name", "F%") )
    .CreateCriteria("Kittens")
        .Add( Expression.Like("Name", "F%") )
    .List<Cat>();
```

Note that the second `CreateCriteria()` returns a new instance of `ICriteria`, which refers to the elements of the kittens collection.

The following, alternate form is useful in certain circumstances.

```
var cats = sess.CreateCriteria<Cat>()
    .CreateAlias("Kittens", "kt")
    .CreateAlias("Mate", "mt")
    .Add( Expression.EqProperty("kt.Name", "mt.Name") )
    .List<Cat>();
```

(`CreateAlias()` does not create a new instance of `ICriteria`.)

Note that the kittens collections held by the cat instances returned by the previous two queries are *not* pre-filtered by the criteria! If you wish to retrieve just the kittens that match the criteria, you must use `SetResultTransformer(Transformers.AliasToEntityMap)`.

```
var cats = sess.CreateCriteria<Cat>()
    .CreateCriteria("Kittens", "kt")
        .Add( Expression.Eq("Name", "F%") )
    .SetResultTransformer(Transformers.AliasToEntityMap)
    .List<IDictionary>();
foreach ( IDictionary map in cats )
{
    Cat cat = (Cat) map[CriteriaSpecification.RootAlias];
    Cat kitten = (Cat) map["kt"];
}
```

Note that for retrieving just kittens you can also use an entity projection. See Section 16.8, “Projections, aggregation and grouping” for more information.

16.5. Join entities without association (Entity joins or ad hoc joins)

In criteria you have the ability to define a join to any entity, not just through a mapped association. To achieve it, use `CreateEntityAlias` and `CreateEntityCriteria`. By example:

```

IList<Cat> uniquelyNamedCats = sess.CreateCriteria<Cat>("c")
    .CreateEntityAlias(
        "joinedCat",
        Restrictions.And(
            Restrictions.EqProperty("c.Name", "joinedCat.Name"),
            Restrictions.NotEqProperty("c.Id", "joinedCat.Id")),
        JoinType.LeftOuterJoin,
        typeof(Cat).FullName)
    .Add(Restrictions.IsNull("joinedCat.Id"))
    .List();

```

16.6. Dynamic association fetching

You may specify association fetching semantics at runtime using `Fetch()`.

```

var cats = sess.CreateCriteria<Cat>()
    .Add( Expression.Like("Name", "Fritz%") )
    .Fetch(SelectMode.Fetch, "Mate")
    .Fetch(SelectMode.Fetch, "Kittens")
    .List<Cat>();

```

This query will fetch both `Mate` and `Kittens` by outer join. See Section 21.1, “Fetching strategies” for more information.

16.7. Example queries

The class `NHibernate.Expression.Example` allows you to construct a query criterion from a given instance.

```

Cat cat = new Cat();
cat.Sex = 'F';
cat.Color = Color.Black;
var results = session.CreateCriteria<Cat>()
    .Add( Example.Create(cat) )
    .List<Cat>();

```

Version properties, identifiers and associations are ignored. By default, null-valued properties and properties which return an empty string from the call to `ToString()` are excluded.

You can adjust how the `Example` is applied.

```

Example example = Example.Create(cat)
    .ExcludeZeroes()           //exclude null- or zero-valued properties
    .ExcludeProperty("Color") //exclude the property named "color"
    .IgnoreCase()             //perform case insensitive string comparisons
    .EnableLike();            //use like for string comparisons
var results = session.CreateCriteria<Cat>()
    .Add(example)
    .List<Cat>();

```

You can even use examples to place criteria upon associated objects.

```

var results = session.CreateCriteria<Cat>()
    .Add( Example.Create(cat) )
    .CreateCriteria("Mate")
        .Add( Example.Create( cat.Mate ) )
    .List<Cat>();

```

16.8. Projections, aggregation and grouping

The class `NHibernate.Expression.Projections` is a factory for `IProjection` instances. We apply a projection to a query by calling `SetProjection()`.

```
var results = session.CreateCriteria<Cat>()
    .SetProjection( Projections.RowCount() )
    .Add( Expression.Eq("Color", Color.BLACK) )
    .List<int>();
```

```
var results = session.CreateCriteria<Cat>()
    .SetProjection( Projections.ProjectionList()
        .Add( Projections.RowCount() )
        .Add( Projections.Avg("Weight") )
        .Add( Projections.Max("Weight") )
        .Add( Projections.GroupProperty("Color") )
    )
    .List<object[]>();
```

There is no explicit "group by" necessary in a criteria query. Certain projection types are defined to be *grouping projections*, which also appear in the SQL `group by` clause.

An alias may optionally be assigned to a projection, so that the projected value may be referred to in restrictions or orderings. Here are two different ways to do this:

```
var results = session.CreateCriteria<Cat>()
    .SetProjection( Projections.Alias( Projections.GroupProperty("Color"), "colr" ) )
    .AddOrder( Order.Asc("colr") )
    .List<string>();
```

```
var results = session.CreateCriteria<Cat>()
    .SetProjection( Projections.GroupProperty("Color").As("colr") )
    .AddOrder( Order.Asc("colr") )
    .List<string>();
```

The `Alias()` and `As()` methods simply wrap a projection instance in another, aliased, instance of `IProjection`. As a shortcut, you can assign an alias when you add the projection to a projection list:

```
var results = session.CreateCriteria<Cat>()
    .SetProjection( Projections.ProjectionList()
        .Add( Projections.RowCount(), "catCountByColor" )
        .Add( Projections.Avg("Weight"), "avgWeight" )
        .Add( Projections.Max("Weight"), "maxWeight" )
        .Add( Projections.GroupProperty("Color"), "color" )
    )
    .AddOrder( Order.Desc("catCountByColor") )
    .AddOrder( Order.Desc("avgWeight") )
    .List<object[]>();
```

```
var results = session.CreateCriteria(typeof(DomesticCat), "cat")
    .CreateAlias("kittens", "kit")
    .SetProjection( Projections.ProjectionList()
        .Add( Projections.Property("cat.Name"), "catName" )
        .Add( Projections.Property("kit.Name"), "kitName" )
    )
    .AddOrder( Order.Asc("catName") )
    .AddOrder( Order.Asc("kitName") )
    .List<object[]>();
```

You can also add an entity projection to a criteria query:

```
var kittens = sess.CreateCriteria<Cat>()
    .CreateCriteria("Kittens", "kt")
    .Add(Expression.Eq("Name", "F%"))
    .SetProjection(Projections.Entity(typeof(Cat), "kt"))
    .List();
```

```
var cats = sess.CreateCriteria<Cat>()
    .CreateCriteria("Kittens", "kt")
    .Add(Expression.Eq("Name", "F%"))
    .SetProjection(
        Projections.RootEntity(),
        Projections.Entity(typeof(Cat), "kt"))
    .List<object[]>();

foreach (var objs in cats)
{
    Cat cat = (Cat) objs[0];
    Cat kitten = (Cat) objs[1];
}
```

See Section 17.9, “Entities Projection” for more information.

16.9. Detached queries and sub-queries

The `DetachedCriteria` class lets you create a query outside the scope of a session, and then later execute it using some arbitrary `ISession`.

```
DetachedCriteria query = DetachedCriteria.For<Cat>()
    .Add( Expression.Eq("sex", 'F') );

using (ISession session = ...)
using (ITransaction txn = session.BeginTransaction())
{
    var results = query.GetExecutableCriteria(session).SetMaxResults(100).List<Cat>();
    txn.Commit();
}
```

A `DetachedCriteria` may also be used to express a sub-query. `ICriterion` instances involving sub-queries may be obtained via `Subqueries`.

```
DetachedCriteria avgWeight = DetachedCriteria.For<Cat>()
    .SetProjection( Projections.Avg("Weight") );
session.CreateCriteria<Cat>()
    .Add( Subqueries.PropertyGt("Weight", avgWeight) )
    .List<Cat>();
```

```
DetachedCriteria weights = DetachedCriteria.For<Cat>()
    .SetProjection( Projections.Property("Weight") );
session.CreateCriteria<Cat>()
    .Add( Subqueries.PropertyGeAll("Weight", weights) )
    .List<Cat>();
```

Even correlated sub-queries are possible:

```
DetachedCriteria avgWeightForSex = DetachedCriteria.For<Cat>("cat2")
    .SetProjection( Projections.Avg("Weight") )
    .Add( Expression.EqProperty("cat2.Sex", "cat.Sex") );
session.CreateCriteria(typeof(Cat), "cat")
    .Add( Subqueries.PropertyGt("Weight", avgWeightForSex) )
    .List<Cat>();
```

Chapter 17. QueryOver Queries

The ICriteria API is NHibernate's implementation of Query Object. NHibernate 3.0 introduces the QueryOver API, which combines the use of Extension Methods and Lambda Expressions (both new in .Net 3.5) to provide a statically type-safe wrapper round the ICriteria API.

QueryOver uses Lambda Expressions to provide some extra syntax to remove the 'magic strings' from your ICriteria queries.

So, for example:

```
.Add(Expression.Eq("Name", "Smith"))
```

becomes:

```
.Where<Person>(p => p.Name == "Smith")
```

With this kind of syntax there are no 'magic strings', and refactoring tools like 'Find All References', and 'Refactor->Rename' work perfectly.

Note: QueryOver is intended to remove the references to 'magic strings' from the ICriteria API while maintaining its opaqueness. It is not a LINQ provider; NHibernate has a built-in Linq provider for this.

17.1. Structure of a Query

Queries are created from an ISession using the syntax:

```
IList<Cat> cats =  
    session.QueryOver<Cat>()  
        .Where(c => c.Name == "Max")  
        .List();
```

Detached QueryOver (analogous to DetachedCriteria) can be created, and then used with an ISession using:

```
QueryOver<Cat> query =  
    QueryOver.Of<Cat>()  
        .Where(c => c.Name == "Paddy");  
  
IList<Cat> cats =  
    query.GetExecutableQueryOver(session)  
        .List();
```

Queries can be built up to use restrictions, projections, and ordering using a fluent inline syntax:

```
var catNames =  
    session.QueryOver<Cat>()  
        .WhereRestrictionOn(c => c.Age).IsBetween(2).And(8)  
        .Select(c => c.Name)  
        .OrderBy(c => c.Name).Asc  
        .List<string>();
```

17.2. Simple Expressions

The Restrictions class (used by ICriteria) has been extended to include overloads that allow Lambda Expression syntax. The Where() method works for simple expressions (<, <=, ==, !=, >, >=) so instead of:

```
ICriterion equalCriterion = Restrictions.Eq("Name", "Max")
```

You can write:

```
ICriterion equalCriterion = Restrictions.Where<Cat>(c => c.Name == "Max")
```

Since the QueryOver class (and IQueryOver interface) is generic and knows the type of the query, there is an inline syntax for restrictions that does not require the additional qualification of class name. So you can also write:

```
var cats =
    session.QueryOver<Cat>()
        .Where(c => c.Name == "Max")
        .And(c => c.Age > 4)
        .List();
```

Note, the methods Where() and And() are semantically identical; the And() method is purely to allow QueryOver to look similar to HQL/SQL.

Boolean comparisons can be made directly instead of comparing to true/false:

```
.Where(p => p.IsParent)
.And(p => !p.IsRetired)
```

Simple expressions can also be combined using the || and && operators. So ICriteria like:

```
.Add(Restrictions.And(
    Restrictions.Eq("Name", "test name"),
    Restrictions.Or(
        Restrictions.Gt("Age", 21),
        Restrictions.Eq("HasCar", true))))
```

Can be written in QueryOver as:

```
.Where(p => p.Name == "test name" && (p.Age > 21 || p.HasCar))
```

Each of the corresponding overloads in the QueryOver API allows the use of regular ICriterion to allow access to private properties.

```
.Where(Restrictions.Eq("Name", "Max"))
```

It is worth noting that the QueryOver API is built on top of the ICriteria API. Internally the structures are the same, so at runtime the statement below, and the statement above, are stored as exactly the same ICriterion. The actual Lambda Expression is not stored in the query.

```
.Where(c => c.Name == "Max")
```

17.3. Additional Restrictions

Some SQL operators/functions do not have a direct equivalent in C#. (e.g., the SQL `where name like '%anna%'`). These operators have overloads for QueryOver in the Restrictions class, so you can write:

```
.Where(Restrictions.On<Cat>(c => c.Name).IsLike("%anna%"))
```

There is also an inline syntax to avoid the qualification of the type:

```
.WhereRestrictionOn(c => c.Name).IsLike("%anna%")
```

While simple expressions (see above) can be combined using the `||` and `&&` operators, this is not possible with the other restrictions. So this ICriteria:

```
.Add(Restrictions.Or(
    Restrictions.Gt("Age", 5)
    Restrictions.In("Name", new string[] { "Max", "Paddy" })))
```

Would have to be written as:

```
.Add(Restrictions.Or(
    Restrictions.Where<Cat>(c => c.Age > 5)
    Restrictions.On<Cat>(c => c.Name).IsIn(new string[] { "Max", "Paddy" })))
```

However, in addition to the additional restrictions factory methods, there are extension methods to allow a more concise inline syntax for some of the operators. So this:

```
.WhereRestrictionOn(c => c.Name).IsLike("%anna%")
```

May also be written as:

```
.Where(c => c.Name.IsLike("%anna%"))
```

17.4. Associations

QueryOver can navigate association paths using `JoinQueryOver()` (analogous to `ICriteria.CreateCriteria()` to create sub-criteria).

The factory method `QueryOver<T>()` on `ISession` returns an `IQueryOver<T>`. More accurately, it returns an `IQueryOver<T,T>` (which inherits from `IQueryOver<T>`).

An `IQueryOver` has two types of interest; the root type (the type of entity that the query returns), and the type of the 'current' entity being queried. For example, the following query uses a join to create a sub-`QueryOver` (analogous to creating sub-criteria in the `ICriteria` API):

```
IQueryOver<Cat,Kitten> catQuery =
    session.QueryOver<Cat>()
        .JoinQueryOver(c => c.Kittens)
        .Where(k => k.Name == "Tiddles");
```

The JoinQueryOver returns a new instance of the IQueryOver than has its root at the Kittens collection. The default type for restrictions is now Kitten (restricting on the name 'Tiddles' in the above example), while calling .List() will return an IList<Cat>. The type IQueryOver<Cat,Kitten> inherits from IQueryOver<Cat>.

Note, the overload for JoinQueryOver takes an IEnumerable<T>, and the C# compiler infers the type from that. If your collection type is not IEnumerable<T>, then you need to qualify the type of the sub-criteria:

```
IQueryOver<Cat,Kitten> catQuery =
    session.QueryOver<Cat>()
        .JoinQueryOver<Kitten>(c => c.Kittens)
            .Where(k => k.Name == "Tiddles");
```

The default join is an inner-join. Each of the additional join types can be specified using the methods .Inner, .Left, .Right, or .Full. For example, to left outer-join on Kittens use:

```
IQueryOver<Cat,Kitten> catQuery =
    session.QueryOver<Cat>()
        .Left.JoinQueryOver(c => c.Kittens)
            .Where(k => k.Name == "Tiddles");
```

17.5. Join entities without association (Entity joins or ad hoc joins)

In QueryOver you have the ability to define a join to any entity, not just through a mapped association. To achieve it, use JoinEntityAlias and JoinEntityQueryOver. By example:

```
Cat cat = null;
Cat joinedCat = null;

var uniquelyNamedCats = sess.QueryOver<Cat>(() => cat)
    .JoinEntityAlias(
        () => joinedCat,
        () => cat.Name == joinedCat.Name && cat.Id != joinedCat.Id,
        JoinType.LeftOuterJoin)
    .Where(() => joinedCat.Id == null)
    .List();
```

17.6. Aliases

In the traditional ICriteria interface aliases are assigned using 'magic strings', however their value does not correspond to a name in the object domain. For example, when an alias is assigned using .CreateAlias("Kitten", "kittenAlias"), the string "kittenAlias" does not correspond to a property or class in the domain.

In QueryOver, aliases are assigned using an empty variable. The variable can be declared anywhere (but should be null at runtime). The compiler can then check the syntax against the variable is used correctly, but at runtime the variable is not evaluated (it's just used as a placeholder for the alias).

Each Lambda Expression function in QueryOver has a corresponding overload to allow use of aliases, and a .JoinAlias function to traverse associations using aliases without creating a sub-QueryOver.

```
Cat catAlias = null;
Kitten kittenAlias = null;
```

```
IQueryOver<Cat,Cat> catQuery =
    session.QueryOver<Cat>(() => catAlias)
        .JoinAlias(() => catAlias.Kittens, () => kittenAlias)
        .Where(() => catAlias.Age > 5)
        .And(() => kittenAlias.Name == "Tiddles");
```

17.7. Projections

Simple projections of the properties of the root type can be added using the `.Select` method which can take multiple Lambda Expression arguments:

```
var selection =
    session.QueryOver<Cat>()
        .Select(
            c => c.Name,
            c => c.Age)
        .List<object[]>();
```

Because this query no longer returns a `Cat`, the return type must be explicitly specified. If a single property is projected, the return type can be specified using:

```
IList<int> ages =
    session.QueryOver<Cat>()
        .Select(c => c.Age)
        .List<int>();
```

However, if multiple properties are projected, then the returned list will contain object arrays, as per a projection in `ICriteria`. This could be fed into an anonymous type using:

```
var catDetails =
    session.QueryOver<Cat>()
        .Select(
            c => c.Name,
            c => c.Age)
        .List<object[]>()
        .Select(properties => new {
            CatName = (string)properties[0],
            CatAge = (int)properties[1],
        });

Console.WriteLine(catDetails[0].CatName);
Console.WriteLine(catDetails[0].CatAge);
```

Note that the second `.Select` call in this example is an extension method on `IEnumerable<T>` supplied in `System.Linq`; it is not part of `NHibernate`.

`QueryOver` allows arbitrary `IProjection` to be added (allowing private properties to be projected). The `Projections` factory class also has overloads to allow Lambda Expressions to be used:

```
var selection =
    session.QueryOver<Cat>()
        .Select(Projections.ProjectionList()
            .Add(Projections.Property<Cat>(c => c.Name))
            .Add(Projections.Avg<Cat>(c => c.Age)))
        .List<object[]>();
```

In addition there is an inline syntax for creating projection lists that does not require the explicit class qualification:

```
var selection =
    session.QueryOver<Cat>()
        .SelectList(list => list
            .Select(c => c.Name)
            .SelectAvg(c => c.Age))
        .List<object[]>();
```

Projections can also have arbitrary aliases assigned to them to allow result transformation. If there is a CatSummary DTO class defined as:

```
public class CatSummary
{
    public string Name { get; set; }
    public int AverageAge { get; set; }
}
```

... then aliased projections can be used with the AliasToBean<T> transformer:

```
CatSummary summaryDto = null;
IList<CatSummary> catReport =
    session.QueryOver<Cat>()
        .SelectList(list => list
            .SelectGroup(c => c.Name).WithAlias(() => summaryDto.Name)
            .SelectAvg(c => c.Age).WithAlias(() => summaryDto.AverageAge))
        .TransformUsing(Transformers.AliasToBean<CatSummary>())
        .List<CatSummary>();
```

17.8. Projection Functions

In addition to projecting properties, there are extension methods to allow certain common dialect-registered functions to be applied. For example you can write the following to get 3 letters named people.

```
.Where(p => p.FirstName.StrLength() == 3)
```

The functions can also be used inside projections:

```
.Select(
    p => Projections.Concat(p.LastName, " ", p.FirstName),
    p => p.Height.Abs())
```

17.9. Entities Projection

You can add entity projections via the AsEntity() extension.

```
Cat mate = null;

var catAndMateNameList = sess.QueryOver<Cat>()
    .JoinAlias(c => c.Mate, () => mate)
    .Select(c => c.AsEntity(), c => mate.Name)
    .List<object[]>();
```

Or it can be done via the Projections.RootEntity and Projections.Entity methods if more control over

loaded entities is required. For instance, entity projections can be lazy loaded or fetched with lazy properties:

```
.Select(
    Projections.Entity(() => alias1).SetLazy(true),
    Projections.Entity(() => alias2).SetFetchLazyProperties(true),
    Projections.RootEntity()
)
```

17.10. Sub-queries

The Sub-queries factory class has overloads to allow Lambda Expressions to express sub-query restrictions. For example:

```
QueryOver<Cat> maximumAge =
    QueryOver.Of<Cat>()
        .SelectList(p => p.SelectMax(c => c.Age));

IList<Cat> oldestCats =
    session.QueryOver<Cat>()
        .Where(Subqueries.WhereProperty<Cat>(c => c.Age).Eq(maximumAge))
        .List();
```

The inline syntax allows you to use sub-queries without re-qualifying the type:

```
IList<Cat> oldestCats =
    session.QueryOver<Cat>()
        .WithSubquery.WhereProperty(c => c.Age).Eq(maximumAge)
        .List();
```

There is an extension method `As()` on (a detached) `QueryOver` that allows you to cast it to any type. This is used in conjunction with the overloads `where()`, `whereAll()`, and `whereSome()` to allow use of the built-in C# operators for comparison, so the above query can be written as:

```
IList<Cat> oldestCats =
    session.QueryOver<Cat>()
        .WithSubquery.Where(c => c.Age == maximumAge.As<int>())
        .List();
```

Chapter 18. Linq Queries

NHibernate 3.0 introduces the Linq to NHibernate provider, which allows the use of the Linq API for querying with NHibernate.

IQueryable queries are obtained with the Query methods used on the ISession or IStatelessSession. (Prior to NHibernate 5.0, these methods were extensions defined in the NHibernate.Linq namespace.) A number of NHibernate Linq extensions giving access to NHibernate specific features are defined in the NHibernate.Linq namespace. Of course, the Linq namespace is still needed too.

```
using System.Linq;
using NHibernate.Linq;
```

Note: NHibernate has another querying API which uses lambda, QueryOver. It should not be confused with a Linq provider.

18.1. Structure of a Query

Queries are created from an ISession using the syntax:

```
IList<Cat> cats =
    session.Query<Cat>()
        .Where(c => c.Color == "white")
        .ToList();
```

The Query<TEntity> function yields an IQueryable<TEntity>, with which Linq extension methods or Linq syntax can be used. When executed, the IQueryable<TEntity> will be translated to a SQL query on the database.

It is possible to query a specific sub-class while still using a queryable of the base class.

```
IList<Cat> cats =
    session.Query<Cat>("Eg.DomesticCat, Eg")
        .Where(c => c.Name == "Max")
        .ToList();
```

Starting with NHibernate 5.0, queries can also be created from an entity collection, with the standard Linq extension AsQueryable available from System.Linq namespace.

```
IList<Cat> whiteKittens =
    cat.Kittens.AsQueryable()
        .Where(k => k.Color == "white")
        .ToList();
```

This will be executed as a query on that cat's kittens without loading the entire collection.

If the collection is a map, call AsQueryable on its Values property.

```
IList<Cat> whiteKittens =
    cat.Kittens.Values.AsQueryable()
        .Where(k => k.Color == "white")
        .ToList();
```

A client timeout for the query can be defined. As most others NHibernate specific features for Linq, this is available through an extension defined in `NHibernate.Linq` namespace.

```
IList<Cat> cats =  
    session.Query<Cat>()  
        .Where(c => c.Color == "black")  
        // Allows 10 seconds only.  
        .SetOptions(o => o.SetTimeout(10))  
        .ToList();
```

18.2. Parameter types

Query parameters get extracted from the Linq expression. Their types are selected according to NHibernate types default for .Net types.

The `MappedAs` extension method allows to override the default type.

```
IList<Cat> cats =  
    session.Query<Cat>()  
        .Where(c => c.BirthDate == DateTime.Today.MappedAs(NHibernateUtil.Date))  
        .ToList();
```

```
IList<Cat> cats =  
    session.Query<Cat>()  
        .Where(c => c.Name == "Max".MappedAs(TypeFactory.Basic("AnsiString(200)")))  
        .ToList();
```

18.3. Supported methods and members

Many methods and members of common .Net types are supported by the Linq to NHibernate provider. They will be translated to the appropriate SQL, provided they are called on an entity property (or expression deriving from) or at least one of their arguments references an entity property. (Otherwise, their return values will be evaluated with .Net runtime before query execution.)

18.3.1. Common methods

The .Net 4 `CompareTo` method of strings and numerical types is translated to a case statement yielding `-1|0|1` according to the result of the comparison.

Many type conversions are available. For all of them, .Net overloads with more than one argument are not supported.

Numerical types can be converted to other numerical types or parsed from strings, using following methods:

- `Convert.ToDecimal`
- `Convert.ToDouble`
- `Convert.ToInt32`

- `Decimal.Parse`
- `Double.Parse`
- `Int32.Parse`

Strings can be converted to `Boolean` and `DateTime` with `Convert.ToBoolean` or `Boolean.Parse` and `Convert.ToDateTime` or `DateTime.Parse` respectively.

On all types supporting string conversion, `ToString` method can be called.

```
IList<string> catBirthDates =  
    session.Query<Cat>()  
        .Select(c => c.BirthDate.ToString())  
        .ToList();
```

`Equals` methods taking a single argument with the same type can be used. Of course, `==` is supported too.

18.3.2. `DateTime` and `DateTimeOffset`

Date and time parts properties can be called on `DateTime` and `DateTimeOffset`. Those properties are:

- `Date`
- `Day`
- `Hour`
- `Minute`
- `Month`
- `Second`
- `Year`

18.3.3. `ICollection`, non generic and generic

`Collections.Contains` methods are supported.

```
IList<Cat> catsWithWrongKitten =  
    session.Query<Cat>()  
        .Where(c => c.Kittens.Contains(c))  
        .ToList();
```

18.3.4. `IDictionary`, non generic and generic

Dictionaries `Item` getter are supported. This enables referencing a dictionary item value in a where condition, as it can be done with HQL expressions.

Non generic dictionary method `Contains` and generic dictionary method `ContainsKey` are translated to corresponding indices HQL expressions. Supposing Acts in following HQL example is generic,

```
from Eg.Show show where 'fizard' in indices(show.Acts)
```

it could be written with Linq:

```
IList<Show> shows =  
    session.Query<Show>()  
        .Where(s => s.Acts.ContainsKey("fizard"))  
        .ToList();
```

18.3.5. Mathematical functions

The following list of mathematical functions from `System.Math` is handled:

- Trigonometric functions: `Acos`, `Asin`, `Atan`, `Atan2`, `Cos`, `Cosh`, `Sin`, `Sinh`, `Tan`, `Tanh`
- `Abs` (all overloads)
- `Ceiling` (both overloads)
- `Floor` (both overloads)
- `Pow`
- `Round` (only overloads without a mode argument)
- `Sign` (all overloads)
- `Sqrt`
- `Truncate` (both overloads)

18.3.6. Nullables

On `Nullable<>` types, `GetValueOrDefault` methods, with or without a provided default value, are supported.

18.3.7. Strings

The following properties and methods are supported on strings:

- `Contains`
- `EndsWith` (without additional parameters)
- `IndexOf` (only overloads taking a character or a string, and optionally a start index)
- `Length`
- `Replace` (both overloads)
- `StartsWith` (without additional parameters)
- `Substring` (both overloads)

- `ToLower` (without additional parameters) and `ToLowerInvariant`, both translated to the same database lower function.
- `ToUpper` (without additional parameters) and `ToUpperInvariant`, both translated to the same database upper function.
- `Trim` (both overloads)
- `TrimEnd`
- `TrimStart`

Furthermore, a string `Like` extension methods allows expressing SQL like conditions.

```
IList<DomesticCat> cats =  
    session.Query<DomesticCat>()  
        .Where(c => c.Name.Like("L%l%l"))  
        .ToList();
```

This `Like` extension method is a Linq to NHibernate method only. Trying to call it in another context is not supported.

If you want to avoid depending on the `NHibernate.Linq` namespace, you can define your own replica of the `Like` methods. Any 2 or 3 arguments method named `Like` in a class named `SqlMethods` will be translated.

18.4. Future results

Future results are supported by the Linq provider. They are not evaluated till one gets executed. At that point, all defined future results are evaluated in one single round-trip to the database.

```
// Define queries  
IFutureEnumerable<Cat> cats =  
    session.Query<Cat>()  
        .Where(c => c.Color == "black")  
        .ToFuture();  
IFutureValue<int> catCount =  
    session.Query<Cat>()  
        .ToFutureValue(q => q.Count());  
// Execute them  
foreach(Cat cat in cats.GetEnumerable())  
{  
    // Do something  
}  
if (catCount.Value > 10)  
{  
    // Do something  
}
```

See Section 21.8, “Future results” for more information.

18.5. Fetching associations

A Linq query may load associated entities or collection of entities. Once the query is defined, using `Fetch` allows fetching a related entity, and `FetchMany` allows fetching a collection. These methods are defined as exten-

sions in `NHibernate.Linq` namespace.

```

IList<Cat> oldCats =
    session.Query<Cat>()
        .Where(c => c.BirthDate.Year < 2010)
        .Fetch(c => c.Mate)
        .FetchMany(c => c.Kittens)
        .ToList();

```

Issuing many `FetchMany` on the same query may cause a cartesian product over the fetched collections. This can be avoided by splitting the fetches among future queries.

```

IQueryable<Cat> oldCatsQuery =
    session.Query<Cat>()
        .Where(c => c.BirthDate.Year < 2010);
oldCatsQuery
    .Fetch(c => c.Mate)
    .FetchMany(c => c.Kittens)
    .ToFuture();
IList<Cat> oldCats =
    oldCatsQuery
        .FetchMany(c => c.AnotherCollection)
        .ToFuture()
        .GetEnumerator()
        .ToList();

```

Use `ThenFetch` and `ThenFetchMany` for fetching associations of the previously fetched association.

```

IList<Cat> oldCats =
    session.Query<Cat>()
        .Where(c => c.BirthDate.Year < 2010)
        .Fetch(c => c.Mate)
        .FetchMany(c => c.Kittens)
        .ThenFetch(k => k.Mate)
        .ToList();

```

18.6. Modifying entities inside the database

Beginning with `NHibernate 5.0`, Linq queries can be used for inserting, updating or deleting entities. The query defines the data to delete, update or insert, and then `Delete`, `Update`, `UpdateBuilder`, `InsertInto` and `InsertBuilder` queryable extension methods allow to delete it, or instruct in which way it should be updated or inserted. Those queries happen entirely inside the database, without extracting corresponding entities out of the database.

These operations are a Linq implementation of Section 14.3, “DML-style operations”, with the same abilities and limitations.

18.6.1. Inserting new entities

`InsertInto` and `InsertBuilder` method extensions expect a `NHibernate` queryable defining the data source of the insert. This data can be entities or a projection. Then they allow specifying the target entity type to insert, and how to convert source data to those target entities. Three forms of target specification exist.

Using projection to target entity:

```

session.Query<Cat>()
    .Where(c => c.BodyWeight > 20)

```

```
.InsertInto(c => new Dog { Name = c.Name + "dog", BodyWeight = c.BodyWeight });
```

Projections can be done with an anonymous object too, but it requires supplying explicitly the target type, which in turn requires re-specifying the source type:

```
session.Query<Cat>()
    .Where(c => c.BodyWeight > 20)
    .InsertInto<Cat, Dog>(c => new { Name = c.Name + "dog", BodyWeight = c.BodyWeight });
```

Or using assignments:

```
session.Query<Cat>()
    .Where(c => c.BodyWeight > 20)
    .InsertBuilder()
    .Into<Dog>()
    .Value(d => d.Name, c => c.Name + "dog")
    .Value(d => d.BodyWeight, c => c.BodyWeight)
    .Insert();
```

In all cases, unspecified properties are not included in the resulting SQL insert. version and timestamp properties are exceptions. If not specified, they are inserted with their seed value.

For more information on Insert limitations, please refer to Section 14.3, “DML-style operations”.

18.6.2. Updating entities

Update and UpdateBuilder method extensions expect a NHibernate queryable defining the entities to update. Then they allow specifying which properties should be updated with which values. As for insertion, three forms of target specification exist.

Using projection to updated entity:

```
session.Query<Cat>()
    .Where(c => c.BodyWeight > 20)
    .Update(c => new Cat { BodyWeight = c.BodyWeight / 2 });
```

Projections can be done with an anonymous object too:

```
session.Query<Cat>()
    .Where(c => c.BodyWeight > 20)
    .Update(c => new { BodyWeight = c.BodyWeight / 2 });
```

Or using assignments:

```
session.Query<Cat>()
    .Where(c => c.BodyWeight > 20)
    .UpdateBuilder()
    .Set(c => c.BodyWeight, c => c.BodyWeight / 2)
    .Update();
```

In all cases, unspecified properties are not included in the resulting SQL update. This could be changed for version and timestamp properties: using UpdateVersioned instead of update allows incrementing the version. Custom version types (NHibernate.UserType.IUserVersionType) are not supported.

When using projection to updated entity, please note that the constructed entity must have the exact same type than the underlying queryable source type. Attempting to project to any other class (anonymous projections excepted) will fail.

18.6.3. Deleting entities

Delete method extension expects a queryable defining the entities to delete. It immediately deletes them.

```
session.Query<Cat>()
    .Where(c => c.BodyWeight > 20)
    .Delete();
```

18.7. Query cache

The Linq provider can use the query cache if it is setup. Refer to Section 21.4, “The Query Cache” for more details on how to set it up.

SetOptions extension method allows to enable the cache for the query.

```
IList<Cat> oldCats =
    session.Query<Cat>()
        .Where(c => c.BirthDate.Year < 2010)
        .SetOptions(o => o.SetCacheable(true))
        .ToList();
```

The cache mode and cache region can be specified too.

```
IList<Cat> cats =
    session.Query<Cat>()
        .Where(c => c.Name == "Max")
        .SetOptions(o => o
            .SetCacheable(true)
            .SetCacheRegion("catNames")
            .SetCacheMode(CacheMode.Put))
        .ToList();
```

18.8. Extending the Linq to NHibernate provider

The Linq to NHibernate provider can be extended for supporting additional SQL functions or translating additional methods or properties to a SQL query.

18.8.1. Adding SQL functions

NHibernate Linq provider feature a LinqExtensionMethod attribute. It allows using an arbitrary, built-in or user defined, SQL function. It should be applied on a method having the same arguments than the SQL function.

```
public static class CustomLinqExtensions
{
    [LinqExtensionMethod()]
    public static string Checksum(this double input)
    {
        // No need to implement it in .Net, unless you wish to call it
        // outside IQueryable context too.
        throw new NotImplementedException("This call should be translated " +
            "to SQL and run db side, but it has been run with .Net runtime");
    }
}
```

Then it can be used in a Linq to NHibernate query.

```
var rnd = (new Random()).NextDouble();
IList<Cat> cats =
    session.Query<Cat>()
        // Pseudo random order
        .OrderBy(c => (c.Id * rnd).Checksum())
        .ToList();
```

The function name is inferred from the method name. If needed, another name can be provided.

```
public static class CustomLinqExtensions
{
    [LinqExtensionMethod("dbo.aCustomFunction")]
    public static string ACustomFunction(this string input, string otherInput)
    {
        throw new NotImplementedException();
    }
}
```

Since NHibernate v5.0, the Linq provider will no more evaluate in-memory the method call even when it does not depend on the queried data. If you wish to have the method call evaluated before querying whenever possible, and then replaced in the query by its resulting value, specify `LinqExtensionPreEvaluation.AllowPreEvaluation` on the attribute.

```
public static class CustomLinqExtensions
{
    [LinqExtensionMethod("dbo.aCustomFunction",
        LinqExtensionPreEvaluation.AllowPreEvaluation)]
    public static string ACustomFunction(this string input, string otherInput)
    {
        // In-memory evaluation implementation.
        return input.Replace(otherInput, "blah");
    }
}
```

18.8.2. Adding a custom generator

Generators are responsible for translating .Net method calls found in lambdas to the proper HQL constructs. Adding support for a new method call can be achieved by registering an additional generator in the Linq to NHibernate provider.

If the purpose of the added method is to simply call some SQL function, using Section 18.8.1, “Adding SQL functions” will be easier.

As an example, here is how to add support for an `AsNullable` method which would allow to call aggregates which may yield `null` without to explicitly cast to the nullable type of the aggregate.

```
public static class NullableExtensions
{
    public static T? AsNullable<T>(this T value) where T : struct
    {
        // Allow runtime use.
        // Not useful for linq-to-nhibernate, could be:
        // throw NotSupportedException();
        return value;
    }
}
```

Adding support in Linq to NHibernate for a custom method requires a generator. For this AsNullable method, we need a method generator, declaring statically its supported method.

```
public class AsNullableGenerator : BaseHqlGeneratorForMethod
{
    public AsNullableGenerator()
    {
        SupportedMethods = new[]
        {
            ReflectHelper.GetMethodDefinition(() => NullableExtensions.AsNullable(0))
        };
    }

    public override HqlTreeNode BuildHql(MethodInfo method,
        Expression targetObject,
        ReadOnlyCollection<Expression> arguments,
        HqlTreeBuilder treeBuilder,
        IHqlExpressionVisitor visitor)
    {
        // This has just to transmit the argument "as is", HQL does not need
        // a specific call for null conversion.
        return visitor.Visit(arguments[0]).AsExpression();
    }
}
```

There are property generators too, and the supported methods or properties can be dynamically declared. Check NHibernate NHibernate.Linq.Functions namespace classes's sources for more examples. CompareGenerator and DateTimePropertiesHqlGenerator are examples of those other cases.

For adding AsNullableGenerator in Linq to NHibernate provider, a new generators registry should be used. Derive from the default one and merge it. (Here we have a static declaration of method support case.)

```
public class ExtendedLinqToHqlGeneratorsRegistry :
DefaultLinqToHqlGeneratorsRegistry
{
    public ExtendedLinqToHqlGeneratorsRegistry()
        : base()
    {
        this.Merge(new AsNullableGenerator());
    }
}
```

In the case of dynamic declaration of method support, another call is required instead of the merge: RegisterGenerator. CompareGenerator illustrates this.

The last step is to instruct NHibernate to use this extended registry. It can be achieved through xml configuration under session-factory node, or by code before building the session factory. Use one of them.

```
<property name="linqtohql.generatorsregistry">
    YourNameSpace.ExtendedLinqToHqlGeneratorsRegistry, YourAssemblyName
</property>
```

```
using NHibernate.Cfg;
// ...

var cfg = new Configuration();
cfg.LinqToHqlGeneratorsRegistry<ExtendedLinqToHqlGeneratorsRegistry>();
// And build the session factory with this configuration.
```

Now the following query could be executed, without failing if no Max cat exists.

```
var oldestMaxBirthDate =
    session.Query<Cat>()
```

```
.Where(c => c.Name == "Max")  
.Select(c => c.BirthDate.AsNullable())  
.Min();
```

(Of course, the same result could be obtained with `(DateTime?)(c.BirthDate)`.)

By default, the Linq provider will try to evaluate the method call with .Net runtime whenever possible, instead of translating it to SQL. It will not do it if at least one of the parameters of the method call has its value originating from an entity, or if the method is marked with the `NoPreEvaluation` attribute (available since NHibernate 5.0).

Chapter 19. Native SQL

You may also express queries in the native SQL dialect of your database. This is useful if you want to utilize database specific features such as query hints or the `CONNECT` keyword in Oracle. It also provides a clean migration path from a direct SQL/ADO.NET based application to NHibernate.

NHibernate allows you to specify handwritten SQL (including stored procedures) for all create, update, delete, and load operations.

19.1. Using an `ISQLQuery`

Execution of native SQL queries is controlled via the `ISQLQuery` interface, which is obtained by calling `ISession.CreateSQLQuery()`. The following describes how to use this API for querying.

19.1.1. Scalar queries

The most basic SQL query is to get a list of scalars (values).

```
sess.CreateSQLQuery("SELECT * FROM CATS")
    .AddScalar("ID", NHibernateUtil.Int64)
    .AddScalar("NAME", NHibernateUtil.String)
    .AddScalar("BIRTHDATE", NHibernateUtil.Date)
```

This query specified:

- the SQL query string
- the columns and types to return

This will return an `IList` of `Object` arrays (`object[]`) with scalar values for each column in the `CATS` table. Only these three columns will be returned, even though the query is using `*` and could return more than the three listed columns.

19.1.2. Entity queries

The above query was about returning scalar values, basically returning the "raw" values from the result set. The following shows how to get entity objects from a native SQL query via `AddEntity()`.

```
sess.CreateSQLQuery("SELECT * FROM CATS").AddEntity(typeof(Cat));
sess.CreateSQLQuery("SELECT ID, NAME, BIRTHDATE FROM CATS").AddEntity(typeof(Cat));
```

This query specified:

- the SQL query string
- the entity returned by the query

Assuming that `Cat` is mapped as a class with the columns `ID`, `NAME` and `BIRTHDATE` the above queries will both return an `IList` where each element is a `Cat` entity.

If the entity is mapped with a many-to-one to another entity it is required to also return its identifier when per-

forming the native query, otherwise a database specific "column not found" error will occur. The additional columns will automatically be returned when using the * notation, but we prefer to be explicit as in the following example for a many-to-one to a Dog:

```
sess.CreateSQLQuery("SELECT ID, NAME, BIRTHDATE, DOG_ID FROM CATS")
    .AddEntity(typeof(Cat));
```

This will allow `cat.Dog` property access to function properly.

19.1.3. Handling associations and collections

It is possible to eagerly join in the Dog to avoid the possible extra round-trip for initializing the proxy. This is done via the `AddJoin()` method, which allows you to join in an association or collection.

```
sess
    .CreateSQLQuery(
        "SELECT cat.ID, NAME, BIRTHDATE, DOG_ID, D_ID, D_NAME " +
        "FROM CATS cat, DOGS d WHERE cat.DOG_ID = d.D_ID")
    .AddEntity("cat", typeof(Cat))
    .AddJoin("cat.Dog");
```

In this example the returned Cat's will have their Dog property fully initialized without any extra round-trip to the database. Notice that we added a alias name ("cat") to be able to specify the target property path of the join. It is possible to do the same eager joining for collections, e.g. if the Cat had a one-to-many to Dog instead.

```
sess
    .CreateSQLQuery(
        "SELECT ID, NAME, BIRTHDATE, D_ID, D_NAME, CAT_ID " +
        "FROM CATS cat, DOGS d WHERE cat.ID = d.CAT_ID")
    .AddEntity("cat", typeof(Cat))
    .AddJoin("cat.Dogs");
```

At this stage we are reaching the limits of what is possible with native queries without starting to enhance the SQL queries to make them usable in NHibernate; the problems start to arise when returning multiple entities of the same type or when the default alias/column names are not enough.

19.1.4. Returning multiple entities

Until now the result set column names are assumed to be the same as the column names specified in the mapping document. This can be problematic for SQL queries which join multiple tables, since the same column names may appear in more than one table.

Column alias injection is needed in the following query (which most likely will fail):

```
sess
    .CreateSQLQuery(
        "SELECT cat.*, mother.* " +
        "FROM CATS cat, CATS mother WHERE cat.MOTHER_ID = mother.ID")
    .AddEntity("cat", typeof(Cat))
    .AddEntity("mother", typeof(Cat))
```

The intention for this query is to return two Cat instances per row, a cat and its mother. This will fail since there is a conflict of names since they are mapped to the same column names and on some databases the returned column aliases will most likely be on the form "c.ID", "c.NAME", etc. which are not equal to the columns specified in the mappings ("ID" and "NAME").

The following form is not vulnerable to column name duplication:

```

sess
    .CreateSQLQuery(
        "SELECT {cat.*}, {mother.*} " +
        "FROM CATS cat, CATS mother WHERE cat.MOTHER_ID = mother.ID")
    .AddEntity("cat", typeof(Cat))
    .AddEntity("mother", typeof(Cat))

```

This query specified:

- the SQL query string, with placeholders for NHibernate to inject column aliases
- the entities returned by the query

The {cat.*} and {mother.*} notation used above is a shorthand for "all properties". Alternatively, you may list the columns explicitly, but even in this case we let NHibernate inject the SQL column aliases for each property. The placeholder for a column alias is just the property name qualified by the table alias. In the following example, we retrieve Cats and their mothers from a different table (cat_log) to the one declared in the mapping metadata. Notice that we may even use the property aliases in the where clause if we like.

```

String sql = "SELECT c.ID as {c.Id}, c.NAME as {c.Name}, " +
    "c.BIRTHDATE as {c.BirthDate}, c.MOTHER_ID as {c.Mother}, {mother.*} " +
    "FROM CAT_LOG c, CAT_LOG m WHERE {c.Mother} = m.ID";

var loggedCats = sess.CreateSQLQuery(sql)
    .AddEntity("c", typeof(Cat))
    .AddEntity("m", typeof(Cat)).List<object[]>();

```

19.1.4.1. Alias and property references

For most cases the above alias injection is needed, but for queries relating to more complex mappings like composite properties, inheritance discriminators, collections etc. there are some specific aliases to use to allow NHibernate to inject the proper aliases.

The following table shows the different possibilities of using the alias injection. Note: the alias names in the result are examples, each alias will have a unique and probably different name when used.

Table 19.1. Alias injection names

Description	Syntax	Example
A simple property	{[aliasname].[propertyname]}	A_NAME as {item.Name}
A composite property	{[aliasname].[component-name].[propertyname]}	CURRENCY as {item.Amount.Currency}, VALUE as {item.Amount.Value}
Discriminator of an entity	{[aliasname].class}	DISC as {item.class}
All properties of an entity	{[aliasname].*}	{item.*}
A collection key	{[aliasname].key}	ORGID as {coll.key}
The id of an collection	{[aliasname].id}	EMPID as {coll.id}

Description	Syntax	Example
The element of an collection	<code>{{[aliasname].element}}</code>	<code>XID as {coll.element}</code>
property of the element in the collection	<code>{{[aliasname].element.[propertyname]}}</code>	<code>NAME as {coll.element.Name}</code>
All properties of the element in the collection	<code>{{[aliasname].element.*}}</code>	<code>{coll.element.*}</code>
All properties of the collection	<code>{{[aliasname].*}}</code>	<code>{coll.*}</code>

19.1.5. Returning non-managed entities

It is possible to apply an `IResultTransformer` to native sql queries. Allowing it to e.g. return non-managed entities.

```
sess.CreateSQLQuery("SELECT NAME, BIRTHDATE FROM CATS")
    .SetResultTransformer(Transformers.AliasToBean(typeof(CatDTO)))
```

This query specified:

- the SQL query string
- a result transformer

The above query will return a list of `CatDTO` which has been instantiated and injected the values of `NAME` and `BIRTHNAME` into its corresponding properties or fields.

IMPORTANT: The custom `IResultTransformer` should override `Equals` and `GetHashCode`, otherwise the query translation won't be cached. This also will result in memory leak.

19.1.6. Handling inheritance

Native SQL queries which query for entities that are mapped as part of an inheritance hierarchy must include all properties for the base class and all its subclasses.

19.1.7. Parameters

Native SQL queries support positional as well as named parameters:

```
var query = sess
    .CreateSQLQuery("SELECT * FROM CATS WHERE NAME like ?")
    .AddEntity(typeof(Cat));
var pusList = query.SetString(0, "Pus%").List<Cat>();

query = sess
    .createSQLQuery("SELECT * FROM CATS WHERE NAME like :name")
    .AddEntity(typeof(Cat));
var pusList = query.SetString("name", "Pus%").List<Cat>();
```

19.2. Named SQL queries

Named SQL queries may be defined in the mapping document and called in exactly the same way as a named HQL query. In this case, we do *not* need to call `AddEntity()`.

```
<sql-query name="persons">
  <return alias="person" class="eg.Person"/>
  SELECT person.NAME AS {person.Name},
         person.AGE AS {person.Age},
         person.SEX AS {person.Sex}
  FROM PERSON person
  WHERE person.NAME LIKE :namePattern
</sql-query>
```

```
var people = sess.GetNamedQuery("persons")
    .SetString("namePattern", namePattern)
    .SetMaxResults(50)
    .List<Person>();
```

The `<return-join>` and `<load-collection>` elements are used to join associations and define queries which initialize collections, respectively.

```
<sql-query name="personsWith">
  <return alias="person" class="eg.Person"/>
  <return-join alias="address" property="person.MailingAddress"/>
  SELECT person.NAME AS {person.Name},
         person.AGE AS {person.Age},
         person.SEX AS {person.Sex},
         address.STREET AS {address.Street},
         address.CITY AS {address.City},
         address.STATE AS {address.State},
         address.ZIP AS {address.Zip}
  FROM PERSON person
  JOIN ADDRESS address
    ON person.ID = address.PERSON_ID AND address.TYPE='MAILING'
  WHERE person.NAME LIKE :namePattern
</sql-query>
```

A named SQL query may return a scalar value. You must declare the column alias and NHibernate type using the `<return-scalar>` element:

```
<sql-query name="mySqlQuery">
  <return-scalar column="name" type="String"/>
  <return-scalar column="age" type="Int64"/>
  SELECT p.NAME AS name,
         p.AGE AS age,
  FROM PERSON p WHERE p.NAME LIKE 'Hiber%'
</sql-query>
```

You can externalize the resultset mapping information in a `<resultset>` element to either reuse them across several named queries or through the `SetResultSetMapping()` API.

```
<resultset name="personAddress">
  <return alias="person" class="eg.Person"/>
  <return-join alias="address" property="person.MailingAddress"/>
</resultset>

<sql-query name="personsWith" resultset-ref="personAddress">
  SELECT person.NAME AS {person.Name},
         person.AGE AS {person.Age},
         person.SEX AS {person.Sex},
         address.STREET AS {address.Street},
         address.CITY AS {address.City},
```

```

        address.STATE AS {address.State},
        address.ZIP AS {address.Zip}
FROM PERSON person
JOIN ADDRESS address
    ON person.ID = address.PERSON_ID AND address.TYPE='MAILING'
WHERE person.NAME LIKE :namePattern
</sql-query>

```

You can alternatively use the resultset mapping information in your .hbm.xml files directly in code.

```

var cats = sess.CreateSQLQuery(
    "select {cat.*}, {kitten.*} " +
    "from cats cat, cats kitten " +
    "where kitten.mother = cat.id")
    .SetResultSetMapping("catAndKitten")
    .List<Cat>();

```

Like HQL named queries, SQL named queries accepts a number of attributes matching settings available on the `ISQLQuery` interface.

- `flush-mode` - override the session flush mode just for this query.
- `cacheable` - allow the query results to be cached by the second level cache. See Chapter 27, *NHibernate.Caches*.
- `cache-region` - specify the cache region of the query.
- `cache-mode` - specify the cache mode of the query.
- `fetch-size` - set a fetch size for the underlying ADO query.
- `timeout` - set the query timeout in seconds.
- `read-only` - true switches yielded entities to read-only. See Chapter 11, *Read-only entities*.
- `comment` - add a custom comment to the SQL.

19.2.1. Using return-property to explicitly specify column/alias names

With `<return-property>` you can explicitly tell NHibernate what column aliases to use, instead of using the `{}`-syntax to let NHibernate inject its own aliases.

```

<sql-query name="mySqlQuery">
    <return alias="person" class="eg.Person">
        <return-property name="Name" column="myName"/>
        <return-property name="Age" column="myAge"/>
        <return-property name="Sex" column="mySex"/>
    </return>
    SELECT person.NAME AS myName,
           person.AGE AS myAge,
           person.SEX AS mySex,
    FROM PERSON person WHERE person.NAME LIKE :name
</sql-query>

```

`<return-property>` also works with multiple columns. This solves a limitation with the `{}`-syntax which can not allow fine grained control of multi-column properties.

```

<sql-query name="organizationCurrentEmployments">
    <return alias="emp" class="Employment">
        <return-property name="Salary">
            <return-column name="VALUE"/>
            <return-column name="CURRENCY"/>
        </return-property>
        <return-property name="EndDate" column="myEndDate"/>
    </return>
    SELECT EMPLOYEE AS {emp.Employee}, EMPLOYER AS {emp.Employer},
    STARTDATE AS {emp.StartDate}, ENDDATE AS {emp.EndDate},
    REGIONCODE as {emp.RegionCode}, EID AS {emp.Id}, VALUE, CURRENCY

```

```
FROM EMPLOYMENT
WHERE EMPLOYER = :id AND ENDDATE IS NULL
ORDER BY STARTDATE ASC
</sql-query>
```

Notice that in this example we used `<return-property>` in combination with the `{}`-syntax for injection, allowing users to choose how they want to refer column and properties.

If your mapping has a discriminator you must use `<return-discriminator>` to specify the discriminator column.

19.2.2. Using stored procedures for querying

NHibernate introduces support for queries via stored procedures and functions. Most of the following documentation is equivalent for both. The stored procedure/function must return a resultset to be able to work with NHibernate. An example of such a stored function in MS SQL Server 2000 and higher is as follows:

```
CREATE PROCEDURE selectAllEmployments AS
SELECT EMPLOYEE, EMPLOYER, STARTDATE, ENDDATE,
REGIONCODE, EMPID, VALUE, CURRENCY
FROM EMPLOYMENT
```

To use this query in NHibernate you need to map it via a named query.

```
<sql-query name="selectAllEmployments_SP">
  <return alias="emp" class="Employment">
    <return-property name="employee" column="EMPLOYEE"/>
    <return-property name="employer" column="EMPLOYER"/>
    <return-property name="startDate" column="STARTDATE"/>
    <return-property name="endDate" column="ENDDATE"/>
    <return-property name="regionCode" column="REGIONCODE"/>
    <return-property name="id" column="EID"/>
    <return-property name="salary">
      <return-column name="VALUE"/>
      <return-column name="CURRENCY"/>
    </return-property>
  </return>
  exec selectAllEmployments
</sql-query>
```

Notice that stored procedures currently only return scalars and entities. `<return-join>` and `<load-collection>` are not supported.

19.2.2.1. Rules/limitations for using stored procedures

To use stored procedures with NHibernate the procedures/functions have to follow some rules. If they do not follow those rules they are not usable with NHibernate. If you still want to use these procedures you have to execute them via `session.Connection`. The rules are different for each database, since database vendors have different stored procedure semantics/syntax.

Stored procedure queries can't be paged with `SetFirstResult()/SetMaxResults()`.

Recommended call form is dependent on your database. For MS SQL Server use `exec functionName <parameters>`.

For Oracle the following rules apply:

- A function must return a result set. The first parameter of a procedure must be an OUT that returns a result

set. This is done by using a `SYS_REFCURSOR` type in Oracle 9i or later. In Oracle you need to define a `REFCURSOR` type, see Oracle literature.

For MS SQL server the following rules apply:

- The procedure must return a result set. NHibernate will use `DbCommand.ExecuteReader()` to obtain the results.
- If you can enable `SET NOCOUNT ON` in your procedure it will probably be more efficient, but this is not a requirement.

19.3. Custom SQL for create, update and delete

NHibernate can use custom SQL statements for create, update, and delete operations. The class and collection persisters in NHibernate already contain a set of configuration time generated strings (`insertsql`, `deletesql`, `updatesql` etc.). The mapping tags `<sql-insert>`, `<sql-delete>`, and `<sql-update>` override these strings:

```
<class name="Person">
  <id name="id">
    <generator class="increment"/>
  </id>
  <property name="name" not-null="true"/>
  <sql-insert>INSERT INTO PERSON (NAME, ID) VALUES ( UPPER(?), ? )</sql-insert>
  <sql-update>UPDATE PERSON SET NAME=UPPER(?) WHERE ID=?</sql-update>
  <sql-delete>DELETE FROM PERSON WHERE ID=?</sql-delete>
</class>
```

Note that the custom `sql-insert` will not be used if you use identity to generate identifier values for the class.

The SQL is directly executed in your database, so you are free to use any dialect you like. This will of course reduce the portability of your mapping if you use database specific SQL.

Stored procedures are supported if the database-native syntax is used:

```
<class name="Person">
  <id name="id">
    <generator class="increment"/>
  </id>
  <property name="name" not-null="true"/>
  <sql-insert>exec createPerson ?, ?</sql-insert>
  <sql-delete>exec deletePerson ?</sql-delete>
  <sql-update>exec updatePerson ?, ?</sql-update>
</class>
```

The order of the positional parameters is currently vital, as they must be in the same sequence as NHibernate expects them.

You can see the expected order by enabling debug logging for the `NHibernate.Persister.Entity` level. With this level enabled NHibernate will print out the static SQL that is used to create, update, delete etc. entities. (To see the expected sequence, remember to not include your custom SQL in the mapping files as that will override the NHibernate generated static sql.)

The stored procedures are by default required to affect the same number of rows as NHibernate-generated SQL would. NHibernate uses `DbCommand.ExecuteNonQuery` to retrieve the number of rows affected. This check can be disabled by using `check="none"` attribute in `sql-insert` element.

19.4. Custom SQL for loading

You may also declare your own SQL (or HQL) queries for entity loading:

```
<sql-query name="person">
  <return alias="pers" class="Person" lock-mode="upgrade"/>
  SELECT NAME AS {pers.Name}, ID AS {pers.Id}
  FROM PERSON
  WHERE ID=?
  FOR UPDATE
</sql-query>
```

This is just a named query declaration, as discussed earlier. You may reference this named query in a class mapping:

```
<class name="Person">
  <id name="Id">
    <generator class="increment"/>
  </id>
  <property name="Name" not-null="true"/>
  <loader query-ref="person"/>
</class>
```

This even works with stored procedures.

You may even define a query for collection loading:

```
<set name="Employments" inverse="true">
  <key/>
  <one-to-many class="Employment"/>
  <loader query-ref="employments"/>
</set>
```

```
<sql-query name="employments">
  <load-collection alias="emp" role="Person.Employments"/>
  SELECT {emp.*}
  FROM EMPLOYMENT emp
  WHERE EMPLOYER = :id
  ORDER BY STARTDATE ASC, EMPLOYEE ASC
</sql-query>
```

You could even define an entity loader that loads a collection by join fetching:

```
<sql-query name="person">
  <return alias="pers" class="Person"/>
  <return-join alias="emp" property="pers.Employments"/>
  SELECT NAME AS {pers.*}, {emp.*}
  FROM PERSON pers
  LEFT OUTER JOIN EMPLOYMENT emp
    ON pers.ID = emp.PERSON_ID
  WHERE ID=?
</sql-query>
```

Chapter 20. Filtering data

NHibernate provides an innovative new approach to handling data with "visibility" rules. A *NHibernate filter* is a global, named, parameterized filter that may be enabled or disabled for a particular NHibernate session.

20.1. NHibernate filters

NHibernate adds the ability to pre-define filter criteria and attach those filters at both a class and a collection level. A filter criteria is the ability to define a restriction clause very similar to the existing "where" attribute available on the class and various collection elements. Except these filter conditions can be parameterized. The application can then make the decision at runtime whether given filters should be enabled and what their parameter values should be. Filters can be used like database views, but parameterized inside the application.

In order to use filters, they must first be defined and then attached to the appropriate mapping elements. To define a filter, use the `<filter-def/>` element within a `<hibernate-mapping/>` element:

```
<filter-def name="myFilter">
  <filter-param name="myFilterParam" type="String"/>
</filter-def>
```

Then, this filter can be attached to a class:

```
<class name="MyClass" ...>
  ...
  <filter name="myFilter" condition=":myFilterParam = MY_FILTERED_COLUMN"/>
</class>
```

or, to a collection:

```
<set ...>
  <filter name="myFilter" condition=":myFilterParam = MY_FILTERED_COLUMN"/>
</set>
```

or, even to both (or multiples of each) at the same time.

The methods on `ISession` are: `EnableFilter(string filterName)`, `GetEnabledFilter(string filterName)`, and `DisableFilter(string filterName)`. By default, filters are *not* enabled for a given session; they must be explicitly enabled through use of the `ISession.EnableFilter()` method, which returns an instance of the `IFilter` interface. Using the simple filter defined above, this would look like:

```
session.EnableFilter("myFilter").SetParameter("myFilterParam", "some-value");
```

Note that methods on the `NHibernate.IFilter` interface do allow the method-chaining common to much of NHibernate.

A full example, using temporal data with an effective record date pattern:

```
<class name="Employee" ...>
  ...
  <many-to-one name="Department" column="dept_id" class="Department"/>
  <property name="EffectiveStartDate" type="date" column="eff_start_dt"/>
  <property name="EffectiveEndDate" type="date" column="eff_end_dt"/>
  ...
  <!--
    Note that this assumes non-terminal records have an eff_end_dt set to
    a max db date for simplicity-sake
  -->
```

```

-->
<filter name="effectiveDate"
      condition=":asOfDate BETWEEN eff_start_dt and eff_end_dt"/>
</class>

<class name="Department" ...>
...
  <set name="Employees" lazy="true">
    <key column="dept_id"/>
    <one-to-many class="Employee"/>
    <filter name="effectiveDate"
          condition=":asOfDate BETWEEN eff_start_dt and eff_end_dt"/>
  </set>
</class>

<filter-def name="effectiveDate">
  <filter-param name="asOfDate" type="date"/>
</filter-def>

```

Then, in order to ensure that you always get back currently effective records, simply enable the filter on the session prior to retrieving employee data:

```

ISession session = ...;
session.EnableFilter("effectiveDate").SetParameter("asOfDate", DateTime.Today);
var results = session.CreateQuery("from Employee as e where e.Salary > :targetSalary")
    .SetInt64("targetSalary", 1000000L)
    .List<Employee>();

```

In the HQL above, even though we only explicitly mentioned a salary constraint on the results, because of the enabled filter the query will return only currently active employees who have a salary greater than a million dollars.

Note: if you plan on using filters with outer joining (either through HQL or load fetching) be careful of the direction of the condition expression. It's safest to set this up for left outer joining; in general, place the parameter first followed by the column name(s) after the operator.

Default all filter definitions are applied to <many-to-one/> and <one-to-one/> elements. You can turn off this behaviour by using use-many-to-one attribute on <filter-def/> element.

```

<filter-def name="effectiveDate" use-many-to-one="false"/>

```

Chapter 21. Improving performance

21.1. Fetching strategies

A *fetching strategy* is the strategy NHibernate will use for retrieving associated objects if the application needs to navigate the association. Fetch strategies may be declared in the O/R mapping metadata, or overridden by a particular HQL or Criteria query.

NHibernate defines the following fetching strategies:

- *Join fetching* - NHibernate retrieves the associated instance or collection in the same SELECT, using an OUTER JOIN.
- *Select fetching* - a second SELECT is used to retrieve the associated entity or collection. Unless you explicitly disable lazy fetching by specifying `lazy="false"`, this second select will only be executed when you actually access the association.
- *Subselect fetching* - a second SELECT is used to retrieve the associated collections for all entities retrieved in a previous query or fetch. Unless you explicitly disable lazy fetching by specifying `lazy="false"`, this second select will only be executed when you actually access the association.
- *"Extra-lazy" collection fetching* - individual elements of the collection are accessed from the database as needed. NHibernate tries not to fetch the whole collection into memory unless absolutely needed (suitable for very large collections)
- *Batch fetching* - an optimization strategy for select fetching - NHibernate retrieves a batch of entity instances or collections in a single SELECT, by specifying a list of primary keys or foreign keys.

NHibernate also distinguishes between:

- *Immediate fetching* - an association, collection or attribute is fetched immediately, when the owner is loaded.
- *Lazy collection fetching* - a collection is fetched when the application invokes an operation upon that collection. (This is the default for collections.)
- *Proxy fetching* - a single-valued association is fetched when a method other than the identifier getter is invoked upon the associated object.

We have two orthogonal notions here: *when* is the association fetched, and *how* is it fetched (what SQL is used). Don't confuse them! We use `fetch` to tune performance. We may use `lazy` to define a contract for what data is always available in any detached instance of a particular class.

21.1.1. Working with lazy associations

By default, NHibernate uses lazy select fetching for collections and lazy proxy fetching for single-valued associations. These defaults make sense for almost all associations in almost all applications.

However, lazy fetching poses one problem that you must be aware of. Access to a lazy association outside of the context of an open NHibernate session will result in an exception. For example:

```

IDictionary<string, int> permissions;
using (var s = sessions.OpenSession())
using (Transaction tx = s.BeginTransaction())
{
    User u = s.CreateQuery("from User u where u.Name=:userName")
        .SetString("userName", userName).UniqueResult<User>();
    permissions = u.Permissions;

    tx.Commit();
}

int accessLevel = permissions["accounts"]; // Error!

```

Since the permissions collection was not initialized when the `ISession` was closed, the collection will not be able to load its state. *NHibernate does not support lazy initialization for detached objects.* The fix is to move the code that reads from the collection to just before the transaction is committed.

Alternatively, we could use a non-lazy collection or association, by specifying `lazy="false"` for the association mapping. However, it is intended that lazy initialization be used for almost all collections and associations. If you define too many non-lazy associations in your object model, NHibernate will end up needing to fetch the entire database into memory in every transaction!

On the other hand, we often want to choose join fetching (which is non-lazy by nature) instead of select fetching in a particular transaction. We'll now see how to customize the fetching strategy. In NHibernate, the mechanisms for choosing a fetch strategy are identical for single-valued associations and collections.

21.1.2. Tuning fetch strategies

Select fetching (the default) is extremely vulnerable to N+1 selects problems, so we might want to enable join fetching in the mapping document:

```

<set name="Permissions"
    fetch="join">
    <key column="userId"/>
    <one-to-many class="Permission"/>
</set>

```

```

<many-to-one name="Mother" class="Cat" fetch="join"/>

```

The fetch strategy defined in the mapping document affects:

- retrieval via `Get()` or `Load()`
- retrieval that happens implicitly when an association is navigated
- `ICriteria` queries
- HQL queries if subselect fetching is used

No matter what fetching strategy you use, the defined non-lazy graph is guaranteed to be loaded into memory. Note that this might result in several immediate selects being used to execute a particular HQL query.

Usually, we don't use the mapping document to customize fetching. Instead, we keep the default behavior, and override it for a particular transaction, using `left join fetch` in HQL. This tells NHibernate to fetch the association eagerly in the first select, using an outer join. In the `ICriteria` query API, you would use `Fetch()`.

If you ever feel like you wish you could change the fetching strategy used by `Get()` or `Load()`, simply use a

ICriteria query, for example:

```
User user = session.CreateCriteria(typeof(User))
    .Fetch(SelectMode.Fetch, "Permissions")
    .Add( Expression.Eq("Id", userId) )
    .UniqueResult<User>();
```

(This is NHibernate's equivalent of what some *ORM* solutions call a "fetch plan".)

A completely different way to avoid problems with N+1 selects is to use the second-level cache, or to enable batch fetching.

21.1.3. Single-ended association proxies

Lazy fetching for collections is implemented using NHibernate's own implementation of persistent collections. However, a different mechanism is needed for lazy behavior in single-ended associations. The target entity of the association must be proxied. NHibernate implements lazy initializing proxies for persistent objects using runtime bytecode enhancement.

By default, NHibernate generates proxies (at startup) for all persistent classes and uses them to enable lazy fetching of many-to-one and one-to-one associations.

The mapping file may declare an interface to use as the proxy interface for that class, with the proxy attribute. By default, NHibernate uses a subclass of the class. *Note that the proxied class must implement a non-private default constructor. We recommend this constructor for all persistent classes!*

There are some gotchas to be aware of when extending this approach to polymorphic classes, eg.

```
<class name="Cat" proxy="Cat">
    .....
    <subclass name="DomesticCat">
        .....
    </subclass>
</class>
```

Firstly, instances of Cat will never be castable to DomesticCat, even if the underlying instance is an instance of DomesticCat:

```
// instantiate a proxy (does not hit the db)
Cat cat = session.Load<Cat>(id);
// hit the db to initialize the proxy
if ( cat.IsDomesticCat ) {
    DomesticCat dc = (DomesticCat) cat; // Error!
    ....
}
```

Secondly, it is possible to break proxy ==.

```
// instantiate a Cat proxy
Cat cat = session.Load<Cat>(id);
DomesticCat dc =
    // acquire new DomesticCat proxy!
    session.Load<DomesticCat>(id);
Console.WriteLine(cat == dc); // false
```

However, the situation is not quite as bad as it looks. Even though we now have two references to different proxy objects, the underlying instance will still be the same object:

```
cat.Weight = 11.0; // hit the db to initialize the proxy
Console.WriteLine( dc.Weight ); // 11.0
```

Third, you may not use a proxy for a sealed class or a class with any non-overridable public members.

Finally, if your persistent object acquires any resources upon instantiation (eg. in initializers or default constructor), then those resources will also be acquired by the proxy. The proxy class is an actual subclass of the persistent class.

These problems are all due to fundamental limitations in .NET's single inheritance model. If you wish to avoid these problems your persistent classes must each implement an interface that declares its business methods. You should specify these interfaces in the mapping file. eg.

```
<class name="CatImpl" proxy="ICat">
    .....
    <subclass name="DomesticCatImpl" proxy="IDomesticCat">
        .....
    </subclass>
</class>
```

where `CatImpl` implements the interface `ICat` and `DomesticCatImpl` implements the interface `IDomesticCat`. Then proxies for instances of `ICat` and `IDomesticCat` may be returned by `Load()` or `Enumerable()`. (Note that `List()` does not usually return proxies.)

```
ICat cat = session.Load<CatImpl>(catid);
using(var iter = session
    .CreateQuery("from CatImpl as cat where cat.Name='fritz'")
    .Enumerable<CatImpl>()
    .GetEnumerator())
{
    iter.MoveNext();
    ICat fritz = iter.Current;
}
```

Relationships are also lazily initialized. This means you must declare any properties to be of type `ICat`, not `CatImpl`.

Certain operations do *not* require proxy initialization

- `Equals()`, if the persistent class does not override `Equals()`
- `GetHashCode()`, if the persistent class does not override `GetHashCode()`
- The identifier getter method

NHibernate will detect persistent classes that override `Equals()` or `GetHashCode()`.

21.1.4. Initializing collections and proxies

A `LazyInitializationException` will be thrown by NHibernate if an uninitialized collection or proxy is accessed outside of the scope of the `ISession`, ie. when the entity owning the collection or having the reference to the proxy is in the detached state.

Sometimes we need to ensure that a proxy or collection is initialized before closing the `ISession`. Of course, we can always force initialization by calling `cat.Sex` or `cat.Kittens.Count`, for example. But that is confusing to readers of the code and is not convenient for generic code.

The static methods `NHibernateUtil.Initialize()` and `NHibernateUtil.IsInitialized()` provide the application with a convenient way of working with lazily initialized collections or proxies. `NHibernateUtil.Initialize(cat)` will force the initialization of a proxy, `cat`, as long as its `ISession` is still open. `NHibernateUtil.Initialize( cat.Kittens )` has a similar effect for the collection of kittens.

Another option is to keep the `ISession` open until all needed collections and proxies have been loaded. In some application architectures, particularly where the code that accesses data using NHibernate, and the code that uses it are in different application layers or different physical processes, it can be a problem to ensure that the `ISession` is open when a collection is initialized. There are two basic ways to deal with this issue:

- In a web-based application, a `HttpModule` can be used to close the `ISession` only at the very end of a user request, once the rendering of the view is complete (the *Open Session in View* pattern). Of course, this places heavy demands on the correctness of the exception handling of your application infrastructure. It is vitally important that the `ISession` is closed and the transaction ended before returning to the user, even when an exception occurs during rendering of the view. See the NHibernate Wiki for examples of this "Open Session in View" pattern.
- In an application with a separate business tier, the business logic must "prepare" all collections that will be needed by the web tier before returning. This means that the business tier should load all the data and return all the data already initialized to the presentation/web tier that is required for a particular use case. Usually, the application calls `NHibernateUtil.Initialize()` for each collection that will be needed in the web tier (this call must occur before the session is closed) or retrieves the collection eagerly using a NHibernate query with a `FETCH` clause or a `SelectMode.Fetch` in `ICriteria`. This is usually easier if you adopt the *Command* pattern instead of a *Session Facade*.
- You may also attach a previously loaded object to a new `ISession` with `Merge()` or `Lock()` before accessing uninitialized collections (or other proxies). No, NHibernate does not, and certainly *should* not do this automatically, since it would introduce ad hoc transaction semantics!

Sometimes you don't want to initialize a large collection, but still need some information about it (like its size) or a subset of the data.

You can use a collection filter to get the size of a collection without initializing it:

```
s.CreateFilter(collection, "select count(*)").UniqueResult<long>()
```

The `CreateFilter()` method is also used to efficiently retrieve subsets of a collection without needing to initialize the whole collection:

```
s.CreateFilter(lazyCollection, "").SetFirstResult(0).SetMaxResults(10).List<Entity>();
```

21.1.5. Using batch fetching

NHibernate can make efficient use of batch fetching, that is, NHibernate can load several uninitialized proxies if one proxy is accessed (or collections). Batch fetching is an optimization of the lazy select fetching strategy. There are two ways you can tune batch fetching: on the class and the collection level.

Batch fetching for classes/entities is easier to understand. Imagine you have the following situation at runtime: You have 25 `Cat` instances loaded in an `ISession`, each `Cat` has a reference to its `Owner`, a `Person`. The `Person` class is mapped with a proxy, `lazy="true"`. If you now iterate through all cats and call `cat.Owner` on each, NHibernate will by default execute 25 `SELECT` statements, to retrieve the proxied owners. You can tune this behavior by specifying a batch-size in the mapping of `Person`:

```
<class name="Person" batch-size="10">...</class>
```

NHibernate will now execute only three queries, the pattern is 10, 10, 5.

You may also enable batch fetching of collections. For example, if each `Person` has a lazy collection of `Cats`, and 10 persons are currently loaded in the `ISession`, iterating through all persons will generate 10 `SELECT`s, one for every call to `person.Cats`. If you enable batch fetching for the `Cats` collection in the mapping of `Person`, NHibernate can pre-fetch collections:

```
<class name="Person">
  <set name="Cats" batch-size="3">
    ...
  </set>
</class>
```

With a batch-size of 3, NHibernate will load 3, 3, 3, 1 collections in four `SELECT`s. Again, the value of the attribute depends on the expected number of uninitialized collections in a particular `Session`.

Batch fetching of collections is particularly useful if you have a nested tree of items, ie. the typical bill-of-materials pattern. (Although a *nested set* or a *materialized path* might be a better option for read-mostly trees.)

Note: if you set `default_batch_fetch_size` in configuration, NHibernate will configure the batch fetch optimization for lazy fetching globally. Batch sizes specified at more granular level take precedence.

21.1.6. Using subselect fetching

If one lazy collection or single-valued proxy has to be fetched, NHibernate loads all of them, re-running the original query in a subselect. This works in the same way as batch-fetching, without the piecemeal loading.

21.2. The Second Level Cache

A NHibernate `ISession` is a transaction-level cache of persistent data. It is possible to configure a cluster or process-level (`ISessionFactory`-level) cache on a class-by-class and collection-by-collection basis. You may even plug in a clustered cache. Be careful. Caches are never aware of changes made to the persistent store by another application (though they may be configured to regularly expire cached data). *In NHibernate 1.x the second level cache does not work correctly in combination with distributed transactions.*

The second level cache requires the use of transactions, be it through transaction scopes or NHibernate transactions. Interacting with the data store without an explicit transaction is discouraged, and will not allow the second level cache to work as intended.

By default, NHibernate uses `HashtableCache` for process-level caching. You may choose a different implementation by specifying the name of a class that implements `NHibernate.Cache.ICacheProvider` using the property `cache.provider_class`.

Table 21.1. Cache Providers

Cache	Provider class	Type	Cluster Safe	Query Cache Supported
Hashtable (not intended for production use)	NHibernate.Cache.HashtableCacheProvider	memory		yes
ASP.NET	NHibernate-	memory		yes

Cache	Provider class	Type	Cluster Safe	Query Cache Supported
Cache (System.Web. Cache)	ate.Caches.SysCache.SysCacheProvider, NHibernate.Caches.SysCache			

21.2.1. Cache mappings

The <cache> element of a class or collection mapping has the following form:

```
<cache
  usage="read-write|nonstrict-read-write|read-only|never"      (1)
  region="RegionName"                                         (2)
/>
```

- (1) usage specifies the caching strategy: read-write, nonstrict-read-write, read-only or never
- (2) region (optional, defaults to the class or collection role name) specifies the name of the second level cache region

Alternatively (preferably?), you may specify <class-cache> and <collection-cache> elements in hibernate.cfg.xml.

The usage attribute specifies a *cache concurrency strategy*.

21.2.2. Strategy: read only

If your application needs to read but never modify instances of a persistent class, a read-only cache may be used. This is the simplest and best performing strategy. Its even perfectly safe for use in a cluster.

```
<class name="Eg.Immutable" mutable="false">
  <cache usage="read-only"/>
  ....
</class>
```

21.2.3. Strategy: read/write

If the application needs to update data, a read-write cache might be appropriate. This cache strategy should never be used if serializable transaction isolation level is required. You should ensure that the transaction is completed when `ISession.Close()` or `ISession.Disconnect()` is called. If you wish to use this strategy in a cluster, you should ensure that the underlying cache implementation supports locking. The built-in cache providers do *not*.

```
<class name="eg.Cat" .... >
  <cache usage="read-write"/>
  ....
  <set name="Kittens" ... >
    <cache usage="read-write"/>
    ....
  </set>
</class>
```

21.2.4. Strategy: nonstrict read/write

If the application only occasionally needs to update data (ie. if it is extremely unlikely that two transactions would try to update the same item simultaneously) and strict transaction isolation is not required, a `nonstrict-read-write` cache might be appropriate. When using this strategy you should ensure that the transaction is completed when `ISession.Close()` or `ISession.Disconnect()` is called.

21.2.5. Strategy: never

By default, without a cache configuration, entities are not cacheable. But they may still be referenced in cacheable queries, which results will then be cached according to the values these non cacheable entities have. So, their data may be indirectly cached through cacheable queries.

By using the cache strategy `never`, such indirect caching of these entities data will be forbidden by NHibernate. Setting as cacheable a query referencing entities with strategy `never` will be treated as an error by default. Alternatively, the `query.throw_never_cached` setting can be set to `false`: instead of raising an error, it will disable the query cache on such queries, and log a warning.

The following table shows which providers are compatible with which concurrency strategies.

Table 21.2. Cache Concurrency Strategy Support

Cache	read-only	nonstrict-read-write	read-write
Hashtable (not intended for production use)	yes	yes	yes
SysCache	yes	yes	yes

Refer to Chapter 27, *NHibernate.Caches* for more details.

21.3. Managing the caches

Whenever you pass an object to `Save()`, `Update()` or `SaveOrUpdate()` and whenever you retrieve an object using `Load()`, `Get()`, `List()`, or `Enumerable()`, that object is added to the internal cache of the `ISession`.

When `Flush()` is subsequently called, the state of that object will be synchronized with the database. If you do not want this synchronization to occur or if you are processing a huge number of objects and need to manage memory efficiently, the `Evict()` method may be used to remove the object and its collections from the first-level cache.

```

IEnumerable<Cat> cats = sess
    .CreateQuery("from Eg.Cat as cat")
    .List<Cat>(); //a huge result set
foreach (Cat cat in cats)
{
    DoSomethingWithACat(cat);
    sess.Evict(cat);
}

```

NHibernate will evict associated entities automatically if the association is mapped with `cascade="all"` or `cascade="all-delete-orphan"`.

The `ISession` also provides a `Contains()` method to determine if an instance belongs to the session cache.

To completely evict all objects from the session cache, call `ISession.Clear()`

For the second-level cache, there are methods defined on `ISessionFactory` for evicting the cached state of an instance, entire class, collection instance or entire collection role.

```
//evict a particular Cat
sessionFactory.Evict(typeof(Cat), catId);
//evict all Cats
sessionFactory.Evict(typeof(Cat));
//evict a particular collection of kittens
sessionFactory.EvictCollection("Eg.Cat.Kittens", catId);
//evict all kitten collections
sessionFactory.EvictCollection("Eg.Cat.Kittens");
```

21.4. The Query Cache

Query result sets may also be cached. This is only useful for queries that are run frequently with the same parameters. To use the query cache you must first enable it:

```
<property name="cache.use_query_cache">true</property>>
```

This setting causes the creation of two new cache regions - one holding cached query result sets (`NHibernate.Cache.StandardQueryCache`), the other holding timestamps of the most recent updates to queryable tables (`UpdateTimestampsCache`). Those region names will be prefixed by the cache region prefix if `cache.region_prefix` setting is configured.

If you use a cache provider handling an expiration for cached entries, you should set the `UpdateTimestampsCache` region expiration to a value greater than the expiration of query cache regions. (Or disable its expiration.) Otherwise the query cache may yield stale data.

Note that the query cache does not cache the state of any entities in the result set; it caches only identifier values and results of value type. So the query cache should always be used in conjunction with the second-level cache.

Most queries do not benefit from caching, so by default queries are not cached. To enable caching, call `IQuery.SetCacheable(true)`. This call allows the query to look for existing cache results or add its results to the cache when it is executed.

If you require fine-grained control over query cache expiration policies, you may specify a named cache region for a particular query by calling `IQuery.SetCacheRegion()`.

```
var blogs = sess.CreateQuery("from Blog blog where blog.Blogger = :blogger")
    .SetEntity("blogger", blogger)
    .SetMaxResults(15)
    .SetCacheable(true)
    .SetCacheRegion("frontpages")
    .List<Blog>();
```

If the query should force a refresh of its query cache region, you may call `IQuery.SetForceCacheRefresh()` to `true`. This is particularly useful in cases where underlying data may have been updated via a separate process (i.e., not modified through `NHibernate`) and allows the application to selectively refresh the query cache regions based on its knowledge of those events. This is a more efficient alternative to eviction of a query cache region via `ISessionFactory.EvictQueries()`.

21.5. Understanding Collection performance

We've already spent quite some time talking about collections. In this section we will highlight a couple more issues about how collections behave at runtime.

21.5.1. Taxonomy

NHibernate defines three basic kinds of collections:

- collections of values
- one to many associations
- many to many associations

This classification distinguishes the various table and foreign key relationships but does not tell us quite everything we need to know about the relational model. To fully understand the relational structure and performance characteristics, we must also consider the structure of the primary key that is used by NHibernate to update or delete collection rows. This suggests the following classification:

- indexed collections
- sets
- bags

All indexed collections (maps, lists, arrays) have a primary key consisting of the `<key>` and `<index>` columns. In this case collection updates are usually extremely efficient - the primary key may be efficiently indexed and a particular row may be efficiently located when NHibernate tries to update or delete it.

Sets have a primary key consisting of `<key>` and `element` columns. This may be less efficient for some types of collection element, particularly composite elements or large text or binary fields; the database may not be able to index a complex primary key as efficiently. On the other hand, for one to many or many to many associations, particularly in the case of synthetic identifiers, it is likely to be just as efficient. (Side-note: if you want `SchemaExport` to actually create the primary key of a `<set>` for you, you must declare all columns as `not-null="true"`.)

`<idbag>` mappings define a surrogate key, so they are always very efficient to update. In fact, they are the best case.

Bags are the worst case. Since a bag permits duplicate element values and has no index column, no primary key may be defined. NHibernate has no way of distinguishing between duplicate rows. NHibernate resolves this problem by completely removing (in a single `DELETE`) and recreating the collection whenever it changes. This might be very inefficient.

Note that for a one-to-many association, the "primary key" may not be the physical primary key of the database table - but even in this case, the above classification is still useful. (It still reflects how NHibernate "locates" individual rows of the collection.)

21.5.2. Lists, maps, idbags and sets are the most efficient collections to update

From the discussion above, it should be clear that indexed collections and (usually) sets allow the most efficient operation in terms of adding, removing and updating elements.

There is, arguably, one more advantage that indexed collections have over sets for many to many associations or collections of values. Because of the structure of an `ISet`, NHibernate doesn't ever `UPDATE` a row when an element is "changed". Changes to an `ISet` always work via `INSERT` and `DELETE` (of individual rows). Once again, this consideration does not apply to one to many associations.

After observing that arrays cannot be lazy, we would conclude that lists, maps and idbags are the most performant (non-inverse) collection types, with sets not far behind. Sets are expected to be the most common kind of collection in NHibernate applications. This is because the "set" semantics are most natural in the relational model.

However, in well-designed NHibernate domain models, we usually see that most collections are in fact one-to-many associations with `inverse="true"`. For these associations, the update is handled by the many-to-one end of the association, and so considerations of collection update performance simply do not apply.

21.5.3. Bags and lists are the most efficient inverse collections

Just before you ditch bags forever, there is a particular case in which bags (and also lists) are much more performant than sets. For a collection with `inverse="true"` (the standard bidirectional one-to-many relationship idiom, for example) we can add elements to a bag or list without needing to initialize (fetch) the bag elements! This is because `IList.Add()` must always succeed for a bag or `IList` (unlike an `ISet`). This can make the following common code much faster.

```
Parent p = sess.Load<Parent>(id);
Child c = new Child();
c.Parent = p;
p.Children.Add(c); //no need to fetch the collection!
sess.Flush();
```

21.5.4. One shot delete

Occasionally, deleting collection elements one by one can be extremely inefficient. NHibernate isn't completely stupid, so it knows not to do that in the case of an newly-empty collection (if you called `list.Clear()`, for example). In this case, NHibernate will issue a single `DELETE` and we are done!

Suppose we add a single element to a collection of size twenty and then remove two elements. NHibernate will issue one `INSERT` statement and two `DELETE` statements (unless the collection is a bag). This is certainly desirable.

However, suppose that we remove eighteen elements, leaving two and then add three new elements. There are two possible ways to proceed:

- Delete eighteen rows one by one and then insert three rows
- Remove the whole collection (in one SQL `DELETE`) and insert all five current elements (one by one)

NHibernate isn't smart enough to know that the second option is probably quicker in this case. (And it would probably be undesirable for NHibernate to be that smart; such behaviour might confuse database triggers, etc.)

Fortunately, you can force this behaviour (ie. the second strategy) at any time by discarding (ie. dereferencing) the original collection and returning a newly instantiated collection with all the current elements. This can be

very useful and powerful from time to time.

Of course, one-shot-delete does not apply to collections mapped `inverse="true"`.

21.6. Batch updates

NHibernate supports batching SQL update commands (INSERT, UPDATE, DELETE) with the following limitations:

- the NHibernate's driver used for your RDBMS may not support batching,
- since the implementation uses reflection to access members and types in System.Data assembly which are not normally visible, it may not function in environments where necessary permissions are not granted,
- optimistic concurrency checking may be impaired since ADO.NET 2.0 does not return the number of rows affected by each statement in the batch, only the total number of rows affected by the batch.

Update batching is enabled by setting `adonet.batch_size` to a non-zero value.

21.7. Query batch

This functionality allows you to execute several queries in one round-trip against the database server. A simple use case is executing a paged query while also getting the total count of results, in a single round-trip. Here is an example:

```
using NHibernate.Multi;

...

IQueryBatch queries = s.CreateQueryBatch()
    .Add<Item>()
        .s.CreateQuery("from Item i where i.Id > ?")
        .SetInt32(0, 50).SetFirstResult(10))
    .Add<long>()
        .s.CreateQuery("select count(*) from Item i where i.Id > :id")
        .SetInt32("id", 50));
IList<Item> items = queries.GetResult<Item>(0);
long count = queries.GetResult<long>(1).Single();
```

The results are got by index, ordered according to the order of queries added to the query batch. Instead of relying on this ordering, a key can be associated with each query for later retrieval:

```
using NHibernate.Multi;

...

var queries = s.CreateQueryBatch()
    .Add("list", s.Query<Item>().Where(i => i.Id > 50))
    .Add("count", s.Query<Item>().Where(i => i.Id > 50), q => q.Count());
var count = queries.GetResult<int>("count").Single();
var items = queries.GetResult<Item>("list");
```

The namespace `NHibernate.Multi` has to be imported since most query batch methods are extension methods.

Criteria queries are also supported by the query batch:

```
var queries = s.CreateQueryBatch()
    .Add<Item>()
```

```

        s.CreateCriteria(typeof(Item))
        .Add(Expression.Gt("Id", 50))
        .SetFirstResult(10))
    .Add<long>(
        s.CreateCriteria(typeof(Item))
        .Add(Expression.Gt("Id", 50))
        .SetProject(Projections.RowCount()));
var items = queries.GetResult<Item>(0);
var count = queries.GetResult<long>(1).Single();

```

You can add `ICriteria` or `DetachedCriteria` to the query batch. In fact, using `DetachedCriteria` in this fashion has some interesting implications.

```

DetachedCriteria customersCriteria = AuthorizationService.GetAssociatedCustomersQuery();
IQueryBatch queries = session.CreateQueryBatch()
    .Add<Customer>(customersCriteria)
    .Add<Policy>(DetachedCriteria.For<Policy>()
        .Add(Subqueries.PropertyIn("id",
            CriteriaTransformer.Clone(customersCriteria)
                .SetProjection(Projections.Id())
        )));

IList<Customer> customers = queries.GetResult<Customer>(0);
IList<Policy> policies = queries.GetResult<Policy>(1);

```

We get a query that represents the customers we can access, and then we can utilize this query further in order to perform additional logic (getting the policies of the customers we are associated with), all in a single database round-trip.

The query batch also supports `QueryOver` and `sql` queries. All kind of queries can be mixed in the same batch.

```

using NHibernate.Multi;

...

var queries = s.CreateQueryBatch()
    .Add("queryOverList", s.QueryOver<Item>().Where(i => i.Category == "Food"))
    .Add<long>("sqlCount",
        s.CreateSQLQuery("select count(*) as itemCount from Item i where i.Category = :cat")
        .AddScalar("itemCount", NHibernateUtil.Int64)
        .SetString("cat", "Food"));
var count = queries.GetResult<long>("sqlCount").Single();
var items = queries.GetResult<Item>("queryOverList");

```

Second level cache is supported by the query batch. Queries flagged as cacheable will be retrieved from the cache if already cached, otherwise their results will be put in the cache.

```

using NHibernate.Multi;

...

var queries = s.CreateQueryBatch()
    .Add("list",
        s.Query<Item>()
        .Where(i => i.Id > 50)
        .WithOptions(o => o.SetCacheable(true)))
    .Add<long>("count",
        s.CreateQuery("select count(*) from Item i where i.Id > :id")
        .SetInt32("id", 50)
        .SetCacheable(true));
var count = queries.GetResult<long>("count").Single();
var items = queries.GetResult<Item>("list");

```

Multi query is executed by concatenating the queries and sending the query to the database as a single string.

This means that the database should support returning several result sets in a single query. Otherwise each query will be individually executed instead.

The first `GetResult` call triggers execution of the whole query batch, which then stores all results. Later calls only retrieve the stored results. A query batch can be re-executed by calling its `Execute` method. Once executed, no new query can be added to the batch.

Note that the database server is likely to enforce a limit on the maximum number of parameters in a query, in which case the limit applies to the query batch as a whole. Queries using `in` with a large number of arguments passed as parameters may easily exceed this limit. For example, SQL Server has a limit of 2,100 parameters per round-trip, and will throw an exception executing this query:

```
int[] allEmployeesId = ...; // 1,500 items
var queries = s.CreateQueryBatch()
    .Add<Employee>(
        s.CreateQuery("from Employee e where e.Id in :empIds")
        .SetParameterList("empIds", allEmployeesId)
        .SetFirstResult(10))
    .Add<long>(
        s.CreateQuery("select count(*) from Employee e where e.Id in :empIds")
        .SetParameterList("empIds", allEmployeesId));
queries.Execute(); // will throw an exception from SQL Server
```

An interesting usage of the query batch is to load several collections of an object in one round-trip, without an expensive cartesian product (`blog * users * posts`).

```
Blog blog = s.CreateQueryBatch()
    .Add(
        s.CreateQuery("select b from Blog b left join fetch b.Users where b.Id = :id")
        .SetInt32("id", 123))
    .Add(
        s.CreateQuery("select b from Blog b left join fetch b.Posts where b.Id = :id")
        .SetInt32("id", 123))
    .GetResult<Blog>(0).FirstOrDefault();
```

You can also add queries as future queries to a query batch:

```
using NHibernate.Multi;

...

var queries = s.CreateQueryBatch();
var list = queries.AddAsFuture(s.Query<Item>().Where(i => i.Id > 50));
var countValue = queries.AddAsFutureValue<long>(
    s.CreateQuery("select count(*) from Item i where i.Id > :id")
    .SetInt32("id", 50));
var count = countValue.Value;
var items = list.GetEnumerable();
```

Futures built from a query batch are executed together the first time the result of one of them is accessed. They are independent of futures obtained directly from the queries.

21.8. Future results

Queries can be converted to future results instead of being directly executed. Future results are not evaluated till one gets executed. At that point, all defined future results are evaluated in one single round-trip to the database.

Future results are an alternative to using Section 21.7, “Query batch”. They avoid the need to explicitly regroup queries, but they also hide which queries will get executed: any pending future results of the session will be

batched together, no matter where they were defined, included out-of-scope pending future results.

Future results are obtained by calling `Future` or `FutureValue` methods of a HQL, Criteria, QueryOver or SQL query. For LINQ queries, the methods are named `ToFuture` and `ToFutureValue`, see Section 18.4, “Future results” for an example.

```
// Define queries
IFutureEnumerable<Cat> cats =
    session.CreateQuery("from Cat c where c.Color = :color")
        .SetString("color", "black")
        .Future();
IFutureValue<int> catCount =
    session.QueryOver<Cat>()
        .ToRowCountQuery()
        .FutureValue<int>();
// Execute them
foreach(Cat cat in cats.GetEnumerable())
{
    // Do something
}
if (catCount.Value > 10)
{
    // Do something
}
```

In the above example, accessing `catCount.Value` does not trigger a round-trip to the database: it has been evaluated with `cats.GetEnumerable()` call. If instead `catCount.Value` was accessed first, it would have executed both future results and `cats.GetEnumerable()` would not have triggered a round-trip to the database.

As showcased in the previous example, `Future` allows to get a future enumerable result, and `FutureValue` is meant to obtain a single value result.

Note: in NHibernate v5.1 and previous versions, Criteria/QueryOver future results were batched separately. Since NHibernate v5.2, they are batched with other querying API future results.

Chapter 22. Toolset Guide

Roundtrip engineering with NHibernate is possible using a set of commandline tools maintained as part of the NHibernate project, along with NHibernate support built into various code generation tools (MyGeneration, CodeSmith, ObjectMapper, AndroMDA).

The NHibernate main package comes bundled with the most important tool (it can even be used from "inside" NHibernate on-the-fly):

- DDL schema generation from a mapping file (aka SchemaExport, hbm2ddl)

Other tools directly provided by the NHibernate project are delivered with a separate package, *NHibernateContrib*. This package includes tools for the following tasks:

- mapping file generation from .NET classes marked with attributes (NHibernate.Mapping.Attributes, or NHMA for short)

Third party tools with NHibernate support are:

- CodeSmith, MyGeneration, and ObjectMapper (mapping file generation from an existing database schema)
- AndroMDA (MDA (Model-Driven Architecture) approach generating code for persistent classes from UML diagrams and their XML/XMI representation)

These 3rd party tools are not documented in this reference. Please refer to the NHibernate website for up-to-date information.

22.1. Schema Generation

The generated schema includes referential integrity constraints (primary and foreign keys) for entity and collection tables. Tables and sequences are also created for mapped identifier generators.

You *must* specify a SQL Dialect via the dialect configuration property when using this tool.

22.1.1. Customizing the schema

Many NHibernate mapping elements define an optional attribute named `length`. You may set the length of a column with this attribute. (Or, for numeric/decimal data types, the precision.)

Some tags also accept a `not-null` attribute (for generating a NOT NULL constraint on table columns) and a `unique` attribute (for generating UNIQUE constraint on table columns).

Some tags accept an `index` attribute for specifying the name of an index for that column. A `unique-key` attribute can be used to group columns in a single unit key constraint. Currently, the specified value of the `unique-key` attribute is *not* used to name the constraint, only to group the columns in the mapping file.

Examples:

```
<property name="Foo" type="String" length="64" not-null="true"/>
<many-to-one name="Bar" foreign-key="fk_foo_bar" not-null="true"/>
```

```
<element column="serial_number" type="Int64" not-null="true" unique="true"/>
```

Alternatively, these elements also accept a child `<column>` element. This is particularly useful for multi-column types:

```
<property name="Foo" type="String">
  <column name="foo" length="64" not-null="true" sql-type="text"/>
</property>

<property name="Bar" type="My.CustomTypes.MultiColumnType, My.CustomTypes"/>
  <column name="fee" not-null="true" index="bar_idx"/>
  <column name="fi" not-null="true" index="bar_idx"/>
  <column name="fo" not-null="true" index="bar_idx"/>
</property>
```

The `sql-type` attribute allows the user to override the default mapping of NHibernate type to SQL data type.

The `check` attribute allows you to specify a check constraint.

```
<property name="Foo" type="Int32">
  <column name="foo" check="foo > 10"/>
</property>

<class name="Foo" table="foos" check="bar < 100.0">
  ...
  <property name="Bar" type="Single"/>
</class>
```

Table 22.1. Summary

Attribute	Values	Interpretation
length	number	column length/decimal precision
not-null	true false	specifies that the column should be non-nullable
unique	true false	specifies that the column should have a unique constraint
index	index_name	specifies the name of a (multi-column) index
unique-key	unique_key_name	specifies the name of a multi-column unique constraint
foreign-key	foreign_key_name	specifies the name of the foreign key constraint generated for an association, use it on <code><one-to-one></code> , <code><many-to-one></code> , <code><key></code> , and <code><many-to-many></code> mapping elements. Note that <code>inverse="true"</code> sides will not be considered by SchemaExport.
sql-type	column_type	overrides the default column type (attribute of <code><column></code> element only)
check	SQL expression	create an SQL check constraint on either column or table

22.1.2. Running the tool

The SchemaExport tool writes a DDL script to standard out and/or executes the DDL statements.

You may embed SchemaExport in your application:

```
Configuration cfg = ....;  
new SchemaExport(cfg).Create(false, true);
```

Chapter 23. Example: Parent/Child

One of the first things that new users want to do with NHibernate is to model a parent/child type relationship. There are two different approaches to this. The most convenient approach, especially for new users, is to model both `Parent` and `Child` as entity classes with a <one-to-many> association from `Parent` to `Child`. The alternative approach is to declare the `Child` as a <composite-element>. The default semantics of a one to many association in NHibernate are much less close to the usual semantics of a parent/child relationship than those of a composite element mapping. We will explain how to use a *bidirectional one to many association with cascades* to model a parent/child relationship efficiently and elegantly.

23.1. A note about collections

NHibernate collections are considered to be a logical part of their owning entity and not of the contained entities. Be aware that this is a critical distinction that has the following consequences:

- When you remove/add an object from/to a collection, the version number of the collection owner is incremented.
- If an object that was removed from a collection is an instance of a value type (e.g., a composite element), that object will cease to be persistent and its state will be completely removed from the database. Likewise, adding a value type instance to the collection will cause its state to be immediately persistent.
- Conversely, if an entity is removed from a collection (a one-to-many or many-to-many association), it will not be deleted by default. This behavior is completely consistent; a change to the internal state of another entity should not cause the associated entity to vanish. Likewise, adding an entity to a collection does not cause that entity to become persistent, by default.

Adding an entity to a collection, by default, merely creates a link between the two entities. Removing the entity will remove the link. This is appropriate for all sorts of cases. However, it is not appropriate in the case of a parent/child relationship. In this case, the life of the child is bound to the life cycle of the parent.

23.2. Bidirectional one-to-many

Suppose we start with a simple <one-to-many> association from `Parent` to `Child`.

```
<set name="Children">
  <key column="parent_id" />
  <one-to-many class="Child" />
</set>
```

If we were to execute the following code:

```
Parent p = ...;
Child c = new Child();
p.Children.Add(c);
session.Save(c);
session.Flush();
```

NHibernate would issue two SQL statements:

- an `INSERT` to create the record for `c`

- an UPDATE to create the link from p to c

This is not only inefficient, but also violates any NOT NULL constraint on the parent_id column. You can fix the nullability constraint violation by specifying not-null="true" in the collection mapping:

```
<set name="Children">
  <key column="parent_id" not-null="true"/>
  <one-to-many class="Child"/>
</set>
```

However, this is not the recommended solution.

The underlying cause of this behavior is that the link (the foreign key parent_id) from p to c is not considered part of the state of the Child object and is therefore not created in the INSERT. The solution is to make the link part of the Child mapping.

```
<many-to-one name="Parent" column="parent_id" not-null="true"/>
```

You also need to add the Parent property to the Child class.

Now that the child entity is managing the state of the link, we tell the collection not to update the link. We use the inverse attribute to do this:

```
<set name="Children" inverse="true">
  <key column="parent_id"/>
  <one-to-many class="Child"/>
</set>
```

The following code would be used to add a new Child:

```
Parent p = session.Load<Parent>(pid);
Child c = new Child();
c.Parent = p;
p.Children.Add(c);
session.Save(c);
session.Flush();
```

Only one SQL INSERT would now be issued.

You could also create an AddChild() method of Parent.

```
public void AddChild(Child c)
{
    c.Parent = this;
    children.Add(c);
}
```

The code to add a Child looks like

```
Parent p = session.Load<Parent>(pid);
Child c = new Child();
p.AddChild(c);
session.Save(c);
session.Flush();
```

23.3. Cascading lifecycle

You can address the frustrations of the explicit call to `Save()` by using cascades.

```
<set name="Children" inverse="true" cascade="all">
  <key column="parent_id"/>
  <one-to-many class="Child"/>
</set>
```

This simplifies the code above to:

```
Parent p = session.Load<Parent>(pid);
Child c = new Child();
p.AddChild(c);
session.Flush();
```

Similarly, we do no more need to iterate over the children when saving or deleting a `Parent`. The following removes `p` and all its children from the database.

```
Parent p = session.Load<Parent>(pid);
session.Delete(p);
session.Flush();
```

However, the following code:

```
Parent p = session.Load<Parent>(pid);
// Get one child out of the set
Child c = p.Children.First();

p.Children.Remove(c);
c.Parent = null;
session.Flush();
```

will not remove `c` from the database. In this case, it will only remove the link to `p` and cause a `NOT NULL` constraint violation. You need to explicitly `Delete()` the `Child`.

```
Parent p = session.Load<Parent>(pid);
// Get one child out of the set
Child c = p.Children.First();

p.Children.Remove(c);
session.Delete(c);
session.Flush();
```

In our case, a `Child` cannot exist without its parent. So if we remove a `Child` from the collection, we do want it to be deleted. To do this, we must use `cascade="all-delete-orphan"`.

```
<set name="Children" inverse="true" cascade="all-delete-orphan">
  <key column="parent_id"/>
  <one-to-many class="Child"/>
</set>
```

Even though the collection mapping specifies `inverse="true"`, cascades are still processed by iterating the collection elements. If you need an object be saved, deleted or updated by cascade, you must add it to the collection. It is not enough to simply set its parent.

23.4. Using cascading update()

Suppose we loaded up a `Parent` in one `ISession`, made some changes in a UI action and wanted to persist these changes in a new `ISession` by calling `update()`. The `Parent` will contain a collection of children and,

since the cascading update is enabled, NHibernate needs to know which children are newly instantiated and which represent existing rows in the database. We will also assume that both `Parent` and `Child` have generated identifier properties of type `long`. NHibernate will use the identifier and version/timestamp property value to determine which of the children are new. (See Section 10.4.2, “Updating detached objects”.)

The `unsaved-value` attribute is used to specify the identifier value of a newly instantiated instance. *In NHibernate it is not necessary to specify `unsaved-value` explicitly.*

The following code will update parent and child and insert `newChild`.

```
//parent and child were both loaded in a previous session
parent.AddChild(child);
Child newChild = new Child();
parent.AddChild(newChild);
session.Update(parent);
session.Flush();
```

This may be suitable for the case of a generated identifier, but what about assigned identifiers and composite identifiers? This is more difficult, since NHibernate cannot use the identifier property to distinguish between a newly instantiated object, with an identifier assigned by the user, and an object loaded in a previous session. In this case, NHibernate will either use the timestamp or version property, or will actually query the second-level cache or, worst case, the database, to see if the row exists.

To avoid the worst case, either:

- define an `unsaved-value` on a `<version>` or `<timestamp>` property mapping for the class.
- set `unsaved-value="none"` and explicitly `Save()` newly instantiated children before calling `update(parent)`.
- set `unsaved-value="any"` and explicitly `Update()` previously persistent children before calling `update(parent)`.
- implement `IInterceptor.IsTransient()` for providing your own strategy for distinguishing newly instantiated objects.

For the `IInterceptor` solution, you could by example define a base class for your persistent classes:

```
public class Persistent
{
    private bool _saved = false;

    public virtual void OnSave()
    {
        _saved = true;
    }

    public virtual void OnLoad()
    {
        _saved = true;
    }

    public virtual void OnDelete()
    {
        _saved = false;
    }

    ...

    public virtual bool IsSaved
    {
        get { return _saved; }
    }
}
```

```

    }
}

```

(The saved property is non-persistent.) Then implement in you interceptor class `IsTransient()`, along with `OnLoad()`, `OnSave()` and `OnDelete()` as follows:

```

public object IsTransient(object entity)
{
    if (entity is Persistent)
    {
        return !((Persistent) entity).IsSaved;
    }
    else
    {
        return null;
    }
}

public bool OnLoad(object entity,
    object id,
    object[] state,
    string[] propertyNames,
    IType[] types)
{
    if (entity is Persistent)
        ((Persistent) entity).OnLoad();
    return false;
}

public boolean OnSave(object entity,
    object id,
    object[] state,
    string[] propertyNames,
    IType[] types)
{
    if (entity is Persistent)
        ((Persistent) entity).OnSave();
    return false;
}

public virtual void OnDelete(object entity,
    object id,
    object[] state,
    string[] propertyNames,
    IType[] types)
{
    if (entity is Persistent)
        ((Persistent) entity).OnDelete();
}

```

See Section 13.1, “Interceptors” for more information.

23.5. Conclusion

There is quite a bit to digest here and it might look confusing first time around. However, in practice, it all works out quite nicely. Most NHibernate applications use the parent / child pattern in many places.

We mentioned an alternative in the first paragraph. None of the above issues exist in the case of `<composite-element>` mappings, which have exactly the semantics of a parent/child relationship. Unfortunately, there are two big limitations to composite element classes: composite elements cannot own collections, and they should not be the child of any entity other than the unique parent. (However, they *may* have a surrogate primary key, using an `<idbag>` mapping.)

Chapter 24. Example: Weblog Application

24.1. Persistent Classes

The persistent classes represent a weblog, and an item posted in a weblog. They are to be modelled as a standard parent/child relationship, but we will use an ordered bag, instead of a set.

```
using System;
using System.Collections.Generic;

namespace Eg
{
    public class Blog
    {
        public virtual long Id { get; set;}

        public virtual IList<BlogItem> Items { get; set;}

        public virtual string Name { get; set;}
    }
}
```

```
using System;

namespace Eg
{
    public class BlogItem
    {
        public virtual Blog Blog { get; set;}

        public virtual DateTime DateTime { get; set;}

        public virtual long Id { get; set;}

        public virtual string Text { get; set;}

        public virtual string Title { get; set;}
    }
}
```

24.2. NHibernate Mappings

The XML mappings should now be quite straightforward.

```
<?xml version="1.0" encoding="utf-8"?>
<hibernate-mapping xmlns="urn:hibernate-mapping-2.2"
    assembly="Eg" namespace="Eg">

    <class
        name="Blog"
        table="BLOGS"
        lazy="true">

        <id
            name="Id"
            column="BLOG_ID">

            <generator class="native"/>

        </id>
```

```

    <property
      name="Name"
      column="NAME"
      not-null="true"
      unique="true"/>

    <bag
      name="Items"
      inverse="true"
      lazy="true"
      order-by="DATE_TIME"
      cascade="all">

      <key column="BLOG_ID"/>
      <one-to-many class="BlogItem"/>

    </bag>

  </class>

</hibernate-mapping>

```

```

<?xml version="1.0" encoding="utf-8"?>
<hibernate-mapping xmlns="urn:hibernate-mapping-2.2"
  assembly="Eg" namespace="Eg">

  <class
    name="BlogItem"
    table="BLOG_ITEMS"
    dynamic-update="true">

    <id
      name="Id"
      column="BLOG_ITEM_ID">

      <generator class="native"/>

    </id>

    <property
      name="Title"
      column="TITLE"
      not-null="true"/>

    <property
      name="Text"
      column="TEXT"
      not-null="true"/>

    <property
      name="DateTime"
      column="DATE_TIME"
      not-null="true"/>

    <many-to-one
      name="Blog"
      column="BLOG_ID"
      not-null="true"/>

  </class>

</hibernate-mapping>

```

24.3. NHibernate Code

The following class demonstrates some of the kinds of things we can do with these classes, using NHibernate.

```
using System;
using System.Collections.Generic;
using NHibernate;
using NHibernate.Cfg;
using NHibernate.Tool.hbm2ddl;

namespace Eg
{
    public class BlogMain
    {
        private ISessionFactory _sessions;

        public void Configure()
        {
            _sessions = new Configuration().Configure()
                .BuildSessionFactory();
        }

        public void ExportTables()
        {
            var cfg = new Configuration().Configure();
            new SchemaExport(cfg).Create(true, true);
        }

        public Blog CreateBlog(string name)
        {
            var blog = new Blog
            {
                Name = name,
                Items = new List<BlogItem>()
            };

            using (var session = _sessions.OpenSession())
            using (var tx = session.BeginTransaction())
            {
                session.Save(blog);
                tx.Commit();
            }

            return blog;
        }

        public BlogItem CreateBlogItem(Blog blog, string title, string text)
        {
            var item = new BlogItem
            {
                Title = title,
                Text = text,
                Blog = blog,
                DateTime = DateTime.Now
            };
            blog.Items.Add(item);

            using (var session = _sessions.OpenSession())
            using (var tx = session.BeginTransaction())
            {
                session.Update(blog);
                tx.Commit();
            }

            return item;
        }

        public BlogItem CreateBlogItem(long blogId, string title, string text)
        {
            var item = new BlogItem
            {
                Title = title,
                Text = text,
                DateTime = DateTime.Now
            };
        }
    }
}
```

```
        using (var session = _sessions.OpenSession())
        using (var tx = session.BeginTransaction())
        {
            var blog = session.Load<Blog>(blogId);
            item.Blog = blog;
            blog.Items.Add(item);
            tx.Commit();
        }

        return item;
    }

    public void UpdateBlogItem(BlogItem item, string text)
    {
        item.Text = text;

        using (var session = _sessions.OpenSession())
        using (var tx = session.BeginTransaction())
        {
            session.Update(item);
            tx.Commit();
        }
    }

    public void UpdateBlogItem(long itemId, string text)
    {
        using (var session = _sessions.OpenSession())
        using (var tx = session.BeginTransaction())
        {
            var item = session.Load<BlogItem>(itemId);
            item.Text = text;
            tx.Commit();
        }
    }

    public IList<object[]> ListAllBlogNamesAndItemCounts(int max)
    {
        IList<object[]> result;

        using (var session = _sessions.OpenSession())
        using (var tx = session.BeginTransaction())
        {
            var q = session.CreateQuery(
                "select blog.id, blog.Name, count(blogItem) " +
                "from Blog as blog " +
                "left outer join blog.Items as blogItem " +
                "group by blog.Name, blog.id " +
                "order by max(blogItem.DateTime)"
            );
            q.SetMaxResults(max);
            result = q.List<object[]>();
            tx.Commit();
        }

        return result;
    }

    public Blog GetBlogAndAllItems(long blogId)
    {
        Blog blog = null;

        using (var session = _sessions.OpenSession())
        using (var tx = session.BeginTransaction())
        {
            var q = session.CreateQuery(
                "from Blog as blog " +
                "left outer join fetch blog.Items " +
                "where blog.id = :blogId"
            );
            q.SetParameter("blogId", blogId);
        }
    }
}
```

```

        blog = q.UniqueResult<Blog>();
        tx.Commit();
    }

    return blog;
}

public IList<object[]> ListBlogsAndRecentItems()
{
    IList<object[]> result = null;

    using (var session = _sessions.OpenSession())
    using (var tx = session.BeginTransaction())
    {
        var q = session.CreateQuery(
            "from Blog as blog " +
            "inner join blog.Items as blogItem " +
            "where blogItem.DateTime > :minDate"
        );

        var date = DateTime.Now.AddMonths(-1);
        q.SetDateTime("minDate", date);

        result = q.List<object[]>();
        tx.Commit();
    }

    return result;
}
}
}

```

It requires some configuration settings in `web.config`, such as:

```

<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <!-- Add this element -->
  <configSections>
    <section
      name="hibernate-configuration"
      type="NHibernate.Cfg.ConfigurationSectionHandler, NHibernate" />
  </configSections>

  <!-- Add this element -->
  <hibernate-configuration xmlns="urn:nhibernate-configuration-2.2">
    <session-factory>
      <property name="dialect">NHibernate.Dialect.MsSql2012Dialect</property>
      <property name="connection.connection_string">
        Server=localhost\SQLEXPRESS;initial catalog=Eg;Integrated Security=True
      </property>

      <mapping assembly="Eg" />
    </session-factory>
  </hibernate-configuration>

  <!-- Leave the other sections unchanged -->
  <system.web>
    ...
  </system.web>
</configuration>

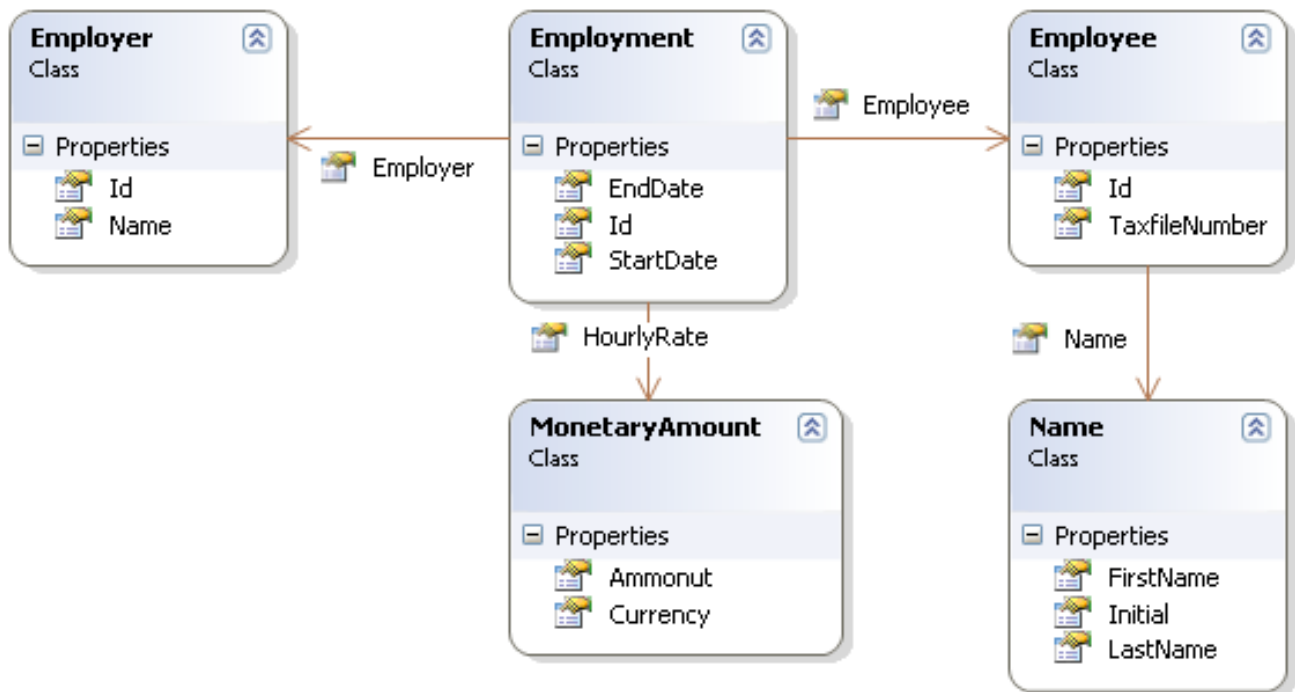
```

Chapter 25. Example: Various Mappings

This chapter shows off some more complex association mappings.

25.1. Employer/Employee

The following model of the relationship between Employer and Employee uses an actual entity class (Employment) to represent the association. This is done because there might be more than one period of employment for the same two parties. Components are used to model monetary values and employee names.



Here's a possible mapping document:

```
<hibernate-mapping xmlns="urn:hibernate-mapping-2.2"
  assembly="..." namespace="...">

  <class name="Employer" table="employers">
    <id name="Id">
      <generator class="sequence">
        <param name="sequence">employer_id_seq</param>
      </generator>
    </id>
    <property name="Name"/>
  </class>

  <class name="Employment" table="employment_periods">
    <id name="Id">
      <generator class="sequence">
        <param name="sequence">employment_id_seq</param>
      </generator>
    </id>
    <property name="StartDate" column="start_date"/>
    <property name="EndDate" column="end_date"/>

    <component name="HourlyRate" class="MonetaryAmount">
      <property name="Amount">
        <column name="hourly_rate" sql-type="NUMERIC(12, 2)"/>
      </property>
      <property name="Currency" length="12"/>
    </component>
  </class>

  <class name="Employee" table="employees">
    <id name="Id">
      <generator class="sequence">
        <param name="sequence">employee_id_seq</param>
      </generator>
    </id>
    <property name="TaxfileNumber"/>
  </class>

  <class name="Name" table="names">
    <id name="Id">
      <generator class="sequence">
        <param name="sequence">name_id_seq</param>
      </generator>
    </id>
    <property name="FirstName"/>
    <property name="Initial"/>
    <property name="LastName"/>
  </class>

  <many-to-many class="Employment" class="Employer">
    <join class="Employment" column="employer_id"/>
    <join class="Employer" column="employment_id"/>
  </many-to-many>

  <many-to-many class="Employment" class="Employee">
    <join class="Employment" column="employee_id"/>
    <join class="Employee" column="employment_id"/>
  </many-to-many>

  <many-to-many class="Employee" class="Name">
    <join class="Employee" column="name_id"/>
    <join class="Name" column="employee_id"/>
  </many-to-many>

</hibernate-mapping>
```

```

        </component>

        <many-to-one name="Employer" column="employer_id" not-null="true"/>
        <many-to-one name="Employee" column="employee_id" not-null="true"/>

    </class>

    <class name="Employee" table="employees">
        <id name="Id">
            <generator class="sequence">
                <param name="sequence">employee_id_seq</param>
            </generator>
        </id>
        <property name="TaxfileNumber"/>
        <component name="Name" class="Name">
            <property name="FirstName"/>
            <property name="Initial"/>
            <property name="LastName"/>
        </component>
    </class>

</hibernate-mapping>

```

And here's the table schema generated by SchemaExport.

```

create table employers (
    Id BIGINT not null,
    Name VARCHAR(255),
    primary key (Id)
)

create table employment_periods (
    Id BIGINT not null,
    hourly_rate NUMERIC(12, 2),
    Currency VARCHAR(12),
    employee_id BIGINT not null,
    employer_id BIGINT not null,
    end_date TIMESTAMP,
    start_date TIMESTAMP,
    primary key (Id)
)

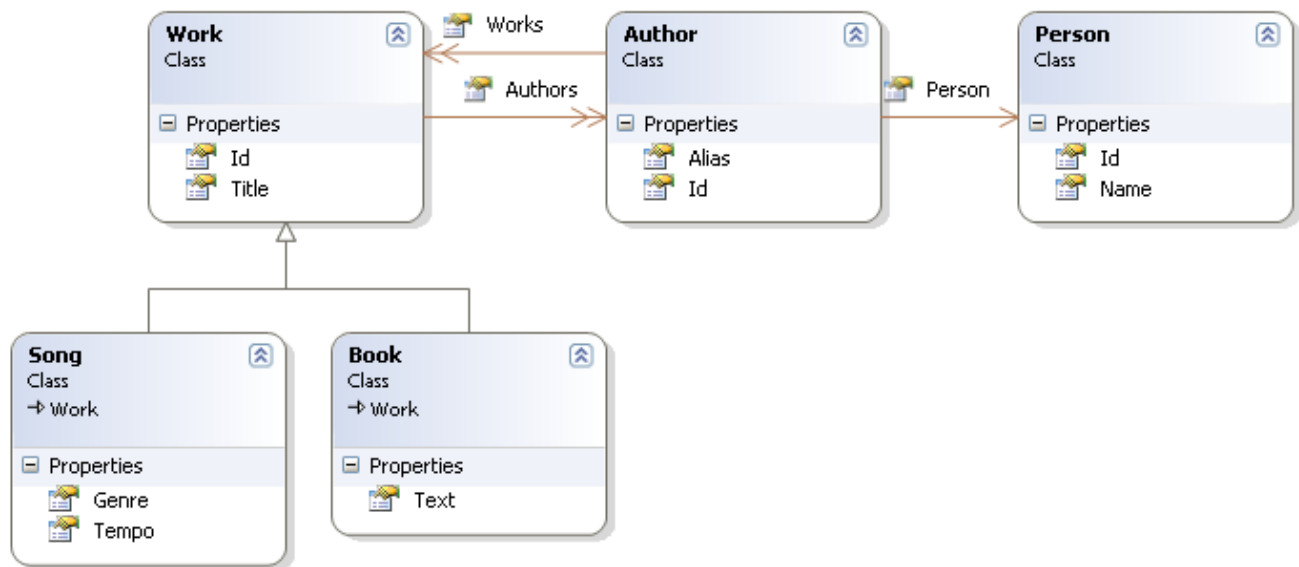
create table employees (
    Id BIGINT not null,
    FirstName VARCHAR(255),
    Initial CHAR(1),
    LastName VARCHAR(255),
    TaxfileNumber VARCHAR(255),
    primary key (Id)
)

alter table employment_periods
    add constraint employment_periodsFK0 foreign key (employer_id) references employers
alter table employment_periods
    add constraint employment_periodsFK1 foreign key (employee_id) references employees
create sequence employee_id_seq
create sequence employment_id_seq
create sequence employer_id_seq

```

25.2. Author/Work

Consider the following model of the relationships between work, Author and Person. We represent the relationship between work and Author as a many-to-many association. We choose to represent the relationship between Author and Person as one-to-one association. Another possibility would be to have Author extend Person.



The following mapping document correctly represents these relationships:

```

<hibernate-mapping xmlns="urn:nhibernate-mapping-2.2"
  assembly="..." namespace="...">

  <class name="Work" table="works" discriminator-value="W">

    <id name="Id" column="id" generator="native" />

    <discriminator column="type" type="character"/>

    <property name="Title"/>
    <set name="Authors" table="author_work" lazy="true">
      <key>
        <column name="work_id" not-null="true"/>
      </key>
      <many-to-many class="Author">
        <column name="author_id" not-null="true"/>
      </many-to-many>
    </set>

    <subclass name="Book" discriminator-value="B">
      <property name="Text" column="text" />
    </subclass>

    <subclass name="Song" discriminator-value="S">
      <property name="Tempo" column="tempo" />
      <property name="Genre" column="genre" />
    </subclass>

  </class>

  <class name="Author" table="authors">

    <id name="Id" column="id">
      <!-- The Author must have the same identifier as the Person -->
      <generator class="assigned"/>
    </id>

    <property name="Alias" column="alias" />
    <one-to-one name="Person" constrained="true"/>

    <set name="Works" table="author_work" inverse="true" lazy="true">
      <key column="author_id"/>
      <many-to-many class="Work" column="work_id"/>
    </set>

  </class>

```

```
<class name="Person" table="persons">
  <id name="Id" column="id">
    <generator class="native"/>
  </id>
  <property name="Name" column="name" />
</class>

</hibernate-mapping>
```

There are four tables in this mapping. works, authors and persons hold work, author and person data respectively. author_work is an association table linking authors to works. Here is the table schema, as generated by SchemaExport.

```
create table works (
  id BIGINT not null generated by default as identity,
  tempo FLOAT,
  genre VARCHAR(255),
  text INTEGER,
  title VARCHAR(255),
  type CHAR(1) not null,
  primary key (id)
)

create table author_work (
  author_id BIGINT not null,
  work_id BIGINT not null,
  primary key (work_id, author_id)
)

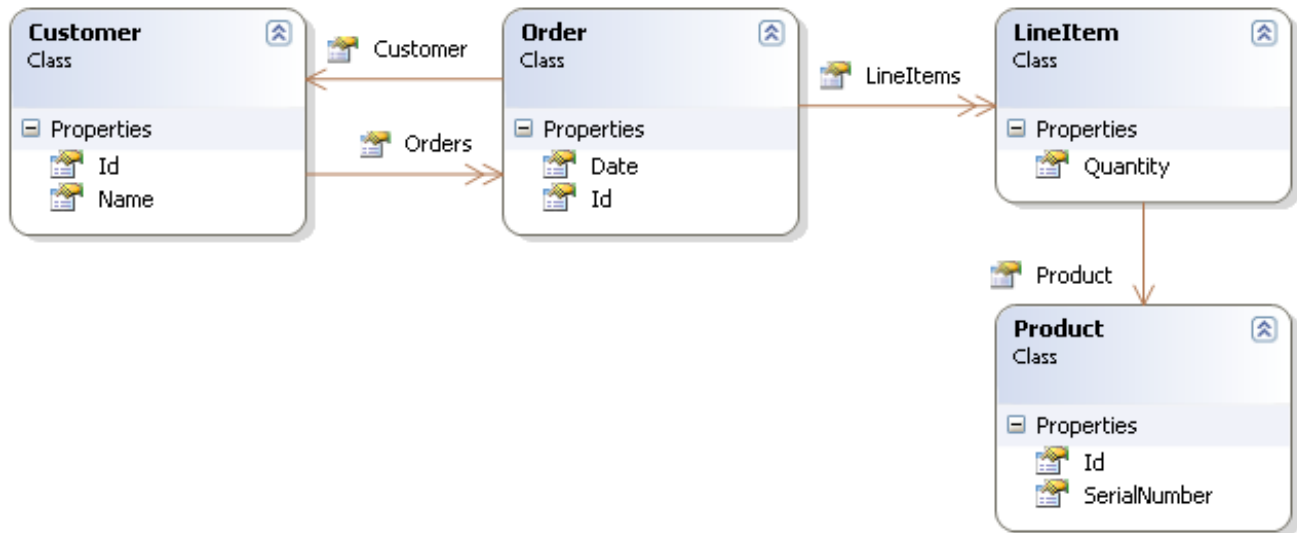
create table authors (
  id BIGINT not null generated by default as identity,
  alias VARCHAR(255),
  primary key (id)
)

create table persons (
  id BIGINT not null generated by default as identity,
  name VARCHAR(255),
  primary key (id)
)

alter table authors
  add constraint authorsFK0 foreign key (id) references persons
alter table author_work
  add constraint author_workFK0 foreign key (author_id) references authors
alter table author_work
  add constraint author_workFK1 foreign key (work_id) references works
```

25.3. Customer/Order/Product

In this section we consider a model of the relationships between Customer, Order, LineItem and Product. There is a one-to-many association between Customer and Order, but how can you represent Order / LineItem / Product? In the example, LineItem is mapped as an association class representing the many-to-many association between Order and Product. In NHibernate, this is called a composite element.



The mapping document:

```

<hibernate-mapping xmlns="urn:hibernate-mapping-2.2"
  assembly="..." namespace="...">

  <class name="Customer" table="customers">
    <id name="Id" column="id" generator="native" />
    <property name="Name" column="name"/>
    <set name="Orders" inverse="true" lazy="true">
      <key column="customer_id"/>
      <one-to-many class="Order"/>
    </set>
  </class>

  <class name="Order" table="orders">
    <id name="Id" column="id" generator="native" />
    <property name="Date" column="date"/>
    <many-to-one name="Customer" column="customer_id"/>
    <list name="LineItems" table="line_items" lazy="true">
      <key column="order_id"/>
      <list-index column="line_number"/>
      <composite-element class="LineItem">
        <property name="Quantity" column="quantity"/>
        <many-to-one name="Product" column="product_id"/>
      </composite-element>
    </list>
  </class>

  <class name="Product" table="products">
    <id name="Id" column="id">
      <generator class="native"/>
    </id>
    <property name="SerialNumber" column="serial_number" />
  </class>

</hibernate-mapping>

```

customers, orders, line_items and products hold customer, order, order line item and product data respectively. line_items also acts as an association table linking orders with products.

```

create table customers (
  id BIGINT not null generated by default as identity,
  name VARCHAR(255),
  primary key (id)
)

create table orders (
  id BIGINT not null generated by default as identity,

```

```

    customer_id BIGINT,
    date TIMESTAMP,
    primary key (id)
)

create table line_items (
    line_number INTEGER not null,
    order_id BIGINT not null,
    product_id BIGINT,
    quantity INTEGER,
    primary key (order_id, line_number)
)

create table products (
    id BIGINT not null generated by default as identity,
    serial_number VARCHAR(255),
    primary key (id)
)

alter table orders
    add constraint ordersFK0 foreign key (customer_id) references customers
alter table line_items
    add constraint line_itemsFK0 foreign key (product_id) references products
alter table line_items
    add constraint line_itemsFK1 foreign key (order_id) references orders

```

25.4. Miscellaneous example mappings

These examples have been elaborated from the Hibernate test suite. You will find many other useful example mappings in NHibernate own test suite by searching in the test folder of the NHibernate sources.

25.4.1. "Typed" one-to-one association

```

<class name="Person">
    <id name="Name"/>
    <one-to-one name="Address" cascade="all">
        <formula>Name</formula>
        <formula>'HOME'</formula>
    </one-to-one>
    <one-to-one name="MailingAddress" cascade="all">
        <formula>Name</formula>
        <formula>'MAILING'</formula>
    </one-to-one>
</class>

<class name="Address" batch-size="2"
    check="addressType in ('MAILING', 'HOME', 'BUSINESS')">
    <composite-id>
        <key-many-to-one name="Person" column="personName"/>
        <key-property name="Type" column="addressType"/>
    </composite-id>
    <property name="Street" type="text"/>
    <property name="State"/>
    <property name="Zip"/>
</class>

```

25.4.2. Composite key example

```

<class name="Customer">
    <id name="CustomerId" length="10">
        <generator class="assigned"/>
    </id>

```

```

<property name="Name" not-null="true" length="100"/>
<property name="Address" not-null="true" length="200"/>

<list name="Orders" inverse="true" cascade="save-update">
  <key column="CustomerId"/>
  <list-index column="OrderNumber"/>
  <one-to-many class="Order"/>
</list>
</class>

<class name="Order" table="CustomerOrder">
  <synchronize table="LineItem"/>
  <synchronize table="Product"/>

  <composite-id name="Id" class="Order+Id">
    <key-property name="CustomerId" length="10"/>
    <key-property name="OrderNumber"/>
  </composite-id>

  <property name="OrderDate" type="date" not-null="true"/>

  <property name="Total">
    <formula>
      ( select sum(li.quantity * p.price)
        from LineItem li, Product p
        where li.productId = p.productId
          and li.customerId = customerId
          and li.orderNumber = orderNumber )
    </formula>
  </property>

  <many-to-one name="Customer"
    column="CustomerId"
    insert="false"
    update="false"
    not-null="true"/>

  <bag name="LineItems"
    fetch="join"
    inverse="true"
    cascade="save-update">
    <key>
      <column name="CustomerId"/>
      <column name="OrderNumber"/>
    </key>
    <one-to-many class="LineItem"/>
  </bag>
</class>

<class name="LineItem">
  <composite-id name="id" class="LineItem+Id">
    <key-property name="CustomerId" length="10"/>
    <key-property name="OrderNumber"/>
    <key-property name="ProductId" length="10"/>
  </composite-id>

  <property name="Quantity"/>

  <many-to-one name="Order" insert="false" update="false" not-null="true">
    <column name="CustomerId"/>
    <column name="OrderNumber"/>
  </many-to-one>

  <many-to-one name="Product"
    insert="false"
    update="false"
    not-null="true"
    column="ProductId"/>
</class>

<class name="Product">

```

```

<synchronize table="LineItem"/>

<id name="ProductId" length="10">
  <generator class="assigned"/>
</id>

<property name="Description" not-null="true" length="200"/>
<property name="Price" length="3"/>
<property name="NumberAvailable"/>

<property name="NumberOrdered">
  <formula>
    ( select sum(li.quantity)
      from LineItem li
      where li.productId = productId )
  </formula>
</property>
</class>

```

25.4.3. Many-to-many with shared composite key attribute

```

<class name="User" table="`User`">
  <composite-id>
    <key-property name="Name"/>
    <key-property name="Org"/>
  </composite-id>
  <set name="Groups" table="UserGroup">
    <key>
      <column name="UserName"/>
      <column name="Org"/>
    </key>
    <many-to-many class="Group">
      <column name="GroupName"/>
      <formula>Org</formula>
    </many-to-many>
  </set>
</class>

<class name="Group" table="`Group`">
  <composite-id>
    <key-property name="Name"/>
    <key-property name="Org"/>
  </composite-id>
  <property name="Description"/>
  <set name="Users" table="UserGroup" inverse="true">
    <key>
      <column name="GroupName"/>
      <column name="Org"/>
    </key>
    <many-to-many class="User">
      <column name="UserName"/>
      <formula>Org</formula>
    </many-to-many>
  </set>
</class>

```

This case requires mixing column and formula under the `<many-to-many>` element, which is supported only since NHibernate 5.2.

25.4.4. Content based discrimination

```

<class name="Person" discriminator-value="P">
  <id name="Id" column="person_id">
    <generator class="native"/>
  </id>

```

```

<discriminator type="character">
  <formula>
    case
      when Title is not null then 'E'
      when Salesperson is not null then 'C'
      else 'P'
    end
  </formula>
</discriminator>

<property name="Name" not-null="true" length="80"/>
<property name="Sex" not-null="true" update="false"/>

<component name="Address">
  <property name="Address"/>
  <property name="Zip"/>
  <property name="Country"/>
</component>

<subclass name="Employee" discriminator-value="E">
  <property name="Title" length="20"/>
  <property name="Salary"/>
  <many-to-one name="Manager"/>
</subclass>

<subclass name="Customer" discriminator-value="C">
  <property name="Comments"/>
  <many-to-one name="Salesperson"/>
</subclass>
</class>

```

25.4.5. Associations on alternate keys

```

<class name="Person">
  <id name="Id">
    <generator class="hilo"/>
  </id>

  <property name="Name" length="100"/>

  <one-to-one name="Address"
    property-ref="Person"
    cascade="all"
    fetch="join"/>

  <set name="accounts" inverse="true">
    <key column="UserId" property-ref="User"/>
    <one-to-many class="Account"/>
  </set>

  <property name="UserId" length="8"/>
</class>

<class name="Address">
  <id name="Id">
    <generator class="hilo"/>
  </id>

  <property name="Address" length="300"/>
  <property name="Zip" length="5"/>
  <property name="Country" length="25"/>
  <many-to-one name="Person" unique="true" not-null="true"/>
</class>

<class name="Account">
  <id name="AccountId" length="32">
    <generator class="uuid"/>
  </id>

```

```
</id>

<many-to-one name="User"
  column="UserId"
  property-ref="UserId"/>

<property name="Type" not-null="true"/>
</class>
```

Chapter 26. Best Practices

Write fine-grained classes and map them using `<component>`.

Use an `Address` class to encapsulate street, suburb, state, postcode. This encourages code reuse and simplifies refactoring.

Declare identifier properties on persistent classes.

NHibernate makes identifier properties optional. There are all sorts of reasons why you should use them. We recommend that identifiers be 'synthetic' (generated, with no business meaning) and of a primitive type. For maximum flexibility, use `Int64` or `String`.

Place each class mapping in its own file.

Don't use a single monolithic mapping document. Map `Eg.Foo` in the file `Eg/Foo.hbm.xml`. This makes particularly good sense in a team environment.

Embed mappings in assemblies.

Place mapping files along with the classes they map and declare them as `Embedded Resources` in Visual Studio.

Consider externalising query strings.

This is a good practice if your queries call non-ANSI-standard SQL functions. Externalising the query strings to mapping files will make the application more portable.

Use parameters.

As in ADO.NET, always replace non-constant values by `"?"`. Never use string manipulation to bind a non-constant value in a query! Even better, consider using named parameters in queries.

Don't manage your own ADO.NET connections.

NHibernate lets the application manage ADO.NET connections. This approach should be considered a last-resort. If you can't use the built-in connections providers, consider providing your own implementation of `NHibernate.Connection.IConnectionProvider`.

Consider using a custom type.

Suppose you have a type, say from some library, that needs to be persisted but doesn't provide the accessors needed to map it as a component. You should consider implementing `NHibernate.UserTypes.IUserType`. This approach frees the application code from implementing transformations to / from an `NHibernate` type.

Use hand-coded ADO.NET in bottlenecks.

In performance-critical areas of the system, some kinds of operations (eg. mass update / delete) might benefit from direct ADO.NET. But please, wait until you *know* something is a bottleneck. And don't assume that direct ADO.NET is necessarily faster. If need to use direct ADO.NET, it might be worth opening a `NHibernate ISession` and using that SQL connection. That way you can still use the same transaction strategy and underlying connection provider.

Understand `ISession` flushing.

From time to time the `ISession` synchronizes its persistent state with the database. Performance will be affected if this process occurs too often. You may sometimes minimize unnecessary flushing by disabling automatic flushing or even by changing the order of queries and other operations within a particular transaction.

In a three tiered architecture, consider using `SaveOrUpdate()`.

When using a distributed architecture, you could pass persistent objects loaded in the middle tier to and from the user interface tier. Use a new session to service each request. Use `ISession.Update()` or `ISes-`

`session.SaveOrUpdate()` to update the persistent state of an object.

In a two tiered architecture, consider using session disconnection.

Database Transactions have to be as short as possible for best scalability. However, it is often necessary to implement long running Application Transactions, a single unit-of-work from the point of view of a user. This Application Transaction might span several client requests and response cycles. Either use Detached Objects or, in two tiered architectures, simply disconnect the NHibernate Session from the ADO.NET connection and reconnect it for each subsequent request. Never use a single Session for more than one Application Transaction use-case, otherwise, you will run into stale data.

Don't treat exceptions as recoverable.

This is more of a necessary practice than a "best" practice. When an exception occurs, roll back the `ITransaction` and close the `ISession`. If you don't, NHibernate can't guarantee that in-memory state accurately represents persistent state. As a special case of this, do not use `ISession.Load()` to determine if an instance with the given identifier exists on the database; use `Get()` or a query instead.

Prefer lazy fetching for associations.

Use eager (outer-join) fetching sparingly. Use proxies and/or lazy collections for most associations to classes that are not cached in the second-level cache. For associations to cached classes, where there is a high probability of a cache hit, explicitly disable eager fetching using `fetch="select"`. When an outer-join fetch is appropriate to a particular use case, use a query with a `left join fetch`.

Consider abstracting your business logic from NHibernate.

Hide (NHibernate) data-access code behind an interface. Combine the *DAO* and *Thread Local Session* patterns. You can even have some classes persisted by hand-coded ADO.NET, associated to NHibernate via an `IUserType`. (This advice is intended for "sufficiently large" applications; it is not appropriate for an application with five tables!)

Implement `Equals()` and `GetHashCode()` using a unique business key.

If you compare objects outside of the `ISession` scope, you have to implement `Equals()` and `GetHashCode()`. Inside the `ISession` scope, object identity is guaranteed. If you implement these methods, never ever use the database identifier! A transient object doesn't have an identifier value and NHibernate would assign a value when the object is saved. If the object is in an `ISet` while being saved, the hash code changes, breaking the contract. To implement `Equals()` and `GetHashCode()`, use a unique business key, that is, compare a unique combination of class properties. Remember that this key has to be stable and unique only while the object is in an `ISet`, not for the whole lifetime (not as stable as a database primary key). Never use collections in the `Equals()` comparison (lazy loading) and be careful with other associated classes that might be proxied.

Don't use exotic association mappings.

Good use-cases for a real many-to-many associations are rare. Most of the time you need additional information stored in the "link table". In this case, it is much better to use two one-to-many associations to an intermediate link class. In fact, we think that most associations are one-to-many and many-to-one, you should be careful when using any other association style and ask yourself if it is really necessary.

Part I. NHibernateContrib Documentation

Preface

The NHibernateContrib is various programs contributed to NHibernate by members of the NHibernate Team or by the end users. The projects in here are not considered core pieces of NHibernate but they extend it in a useful way.

Chapter 27. NHibernate.Caches

What is NHibernate.Caches?

NHibernate.Caches namespace contains several second-level cache providers for NHibernate. A cache is a place where entities are kept after being loaded from the database; once cached, they can be retrieved without going to the database. This means that they are faster to (re)load.

An NHibernate session has an internal (first-level) cache where it keeps its entities. There is no sharing between these caches - a first-level cache belongs to a given session and is destroyed with it. NHibernate provides a *second-level cache* system; it works at the session factory level. A second-level cache is shared by all sessions created by the same session factory.

An important point is that the second-level cache *does not* cache instances of the object type being cached; instead it caches the individual values of the properties of that object. This provides two benefits. One, NHibernate doesn't have to worry that your client code will manipulate the objects in a way that will disrupt the cache. Two, the relationships and associations do not become stale, and are easy to keep up-to-date because they are simply identifiers. The cache is not a tree of objects but rather a map of arrays.

With the *session-per-request* model, a high number of sessions can concurrently access the same entity without hitting the database each time; hence the performance gain.

Depending on the chosen cache provider, the second level cache may be actually shared between different session factories. If you need to avoid this for some session factories, configure each of them with a different `cache.region_prefix`. See Section 3.6, “Optional configuration properties”.

Several cache providers have been contributed by NHibernate users:

NHibernate.Caches.SysCache

Uses `System.Web.Caching.Cache` as the cache provider. This means that you can rely on ASP.NET caching feature to understand how it works. For more information, read (on the MSDN): Caching Application Data [<https://msdn.microsoft.com/en-us/library/6hbbsfk6.aspx>]. This provider is available for the .Net Framework only. Also see Section 27.3, “SysCache Configuration”.

NHibernate.Caches.SysCache2

Similar to `NHibernate.Caches.SysCache`, uses ASP.NET cache. This provider also supports SQL dependency-based expiration, meaning that it is possible to configure certain cache regions to automatically expire when the relevant data in the database changes.

`SysCache2` requires Microsoft SQL Server 2000 or higher. This provider is available for the .Net Framework only.

See Section 27.4, “SysCache2 Configuration”.

NHibernate.Caches.EnyimMemcached

Uses `Memcached`. See `memcached` homepage [<https://memcached.org/>] for more information on `Memcached`. This provider is available for the .Net Framework only. Also see Section 27.5, “EnyimMemcached Configuration”.

NCache provider for NHibernate

Uses `NCache`. `NCache` is a commercial distributed caching system with a provider for NHibernate. The `NCache Express` version is free for use, see `NCache Express` homepage [<http://www.alachisoft.com/ncache/>] for more information.

NHibernate.Caches.RtMemoryCache

Uses `System.Runtime.Caching.MemoryCache.Default` as the cache provider. This provider is available as a .Net Standard NuGet package. See Section 27.6, “RtMemoryCache Configuration”.

NHibernate.Caches.StackExchangeRedis

Uses `StackExchange.Redis`. This provider is available as a .Net Standard NuGet package. It can batch together puts and reads, reducing incurred IOs. See Section 27.7, “NHibernate.Caches.StackExchangeRedis Configuration”.

NHibernate.Caches.CoreMemoryCache

Uses `Microsoft.Extensions.Caching.Memory.MemoryCache` as the cache provider. This provider is available as a .Net Standard NuGet package. See Section 27.8, “CoreMemoryCache Configuration”.

NHibernate.Caches.CoreDistributedCache

Uses `Microsoft.Extensions.Caching.Abstractions.IDistributedCache` implementations as the cache provider. The implementation has to be provided through an `IDistributedCacheFactory`. Distributed cache factories for Memcached, Redis, SqlServer and Memory caches are available through their own package, prefixed by `NHibernate.Caches.CoreDistributedCache..`

This provider is available as a .Net Standard NuGet package. See Section 27.9, “CoreDistributedCache Configuration”.

27.1. How to use a cache?

Here are the steps to follow to enable the second-level cache in NHibernate:

- Choose the cache provider you want to use and copy its assembly in your assemblies directory. (For example, `NHibernate.Caches.SysCache.dll`.)
- To tell NHibernate which cache provider to use, add in your NHibernate configuration file (can be `YourAssembly.exe.config` or `web.config` or a `.cfg.xml` file):

```
<property name="cache.provider_class">xxx</property>(1)
<property name="cache.default_expiration">120</property>(2)
<property name="cache.use_sliding_expiration">true</property>(3)
```

- (1) "xxx" is the assembly-qualified class name of a class implementing `ICacheProvider`, eg. "`NHibernate.Caches.SysCache.SysCacheProvider`", `NHibernate.Caches.SysCache`".
 - (2) The expiration value is the number of seconds you wish to cache each entry (here two minutes). Not all providers support this setting, it may be ignored. Check their respective documentation.
 - (3) The `use_sliding_expiration` value is whether you wish to use a sliding expiration or not. Defaults to false. Not all providers support this setting, it may be ignored. Check their respective documentation.
- Add `<cache usage="read-write|nonstrict-read-write|read-only"/>` (just after `<class>`) in the mapping of the entities you want to cache. It also works for collections (bag, list, map, set, ...).

Be careful.

- Most caches are never aware of changes made to the persistent store by another process (though they may

be configured to regularly expire cached data). As the caches are created at the session factory level, they are destroyed with the `SessionFactory` instance; so you must keep them alive as long as you need them.

- The second level cache requires the use of transactions, be it through transaction scopes or NHibernate transactions. Interacting with the data store without an explicit transaction is discouraged, and will not allow the second level cache to work as intended.
- To avoid issues with composite ids and some cache providers, `ToString()` needs to be overridden on composite id classes. It should yield a unique string representing the id. If the composite id is mapped as a component, overriding the component `ToString()` is enough. See Section 8.4, “Components as composite identifiers”.

See also Section 21.2, “The Second Level Cache”.

27.2. Using a custom configuration provider

Most caches have their own configuration section. By default, the cache attempts to read its configuration section through the `.Net ConfigurationManager`. Some environments do not support it, especially when running under `.Net Core`. The `.Net Core` compatible caches provide a way to set a custom configuration provider, through their `NHibernate.Caches.cache specific namespace .ConfigurationProvider` class.

To disable the configuration provider, in case you configure the cache entirely programmatically, set its `ConfigurationProvider.Current` property to `null`.

```
ConfigurationProvider.Current = null;
```

To provide directly the `System.Configuration.Configuration` instance to use, call the cache `ConfigurationProvider.SetConfiguration` method.

```
ConfigurationProvider.SetConfiguration(yourConfig);
```

You may also derive a custom provider from the `ConfigurationProvider` of the cache, implements its abstract method `GetConfiguration`, and assign an instance of your custom provider to the cache `ConfigurationProvider.Current` property.

Changes of the `ConfigurationProvider.Current` property value are to be done very early in the application lifecycle, before building any session factory using the cache provider or before any other call on the cache API. Otherwise they will not be taken into account.

27.3. SysCache Configuration

`SysCache` relies on `System.Web.Caching.Cache` for the underlying implementation. This is a `.Net Framework` only library, since `System.Web.Caching` is available neither in the `.Net Standard` nor in `.Net Core`.

The following NHibernate configuration settings are available:

`cache.default_expiration`

Number of seconds to wait before expiring each item. Defaults to 300. It can also be set programmatically on the NHibernate configuration object under the name `expiration`, which then takes precedence over `cache.default_expiration`.

`cache.use_sliding_expiration`

Should the expiration be sliding? A sliding expiration is reinitialized at each get. Defaults to `false`.

`priority`

A numeric cost of expiring each item, where 1 is a low cost, 5 is the highest, and 3 is normal. Only values 1 through 6 are valid. 6 is a special value corresponding to `NotRemovable`. This setting can only be set programmatically through on the `NHibernate` configuration object, by example with `Configuration.SetProperty`.

`SysCache` has a config file section handler to allow configuring different expirations and priorities for different regions. Here is an example:

Example 27.1.

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <configSections>
    <section name="syscache"
      type="NHibernate.Caches.SysCache.SysCacheSectionHandler,NHibernate.Caches.SysCache" />
  </configSections>

  <syscache>
    <cache region="foo" expiration="500" priority="4" />
    <cache region="bar" expiration="300" priority="3" sliding="true" />
  </syscache>
</configuration>
```

27.4. SysCache2 Configuration

`SysCache2` uses `System.Web.Caching` like `SysCache`, but can use `SqlCacheDependencies` to invalidate cache regions when data in an underlying SQL Server table or query changes. Query dependencies are only available for SQL Server 2005 or higher. To use the cache provider, the application must be setup and configured to support SQL notifications as described in the MSDN documentation.

This is a .Net Framework only library, since `System.Web.Caching` is available neither in the .Net Standard nor in .Net Core.

The following `NHibernate` configuration settings are available:

`cache.default_expiration`

Number of seconds to wait before expiring each item. Defaults to 300. It can also be set programmatically on the `NHibernate` configuration object under the name `expiration`, which then takes precedence over `cache.default_expiration`.

`cache.use_sliding_expiration`

Should the expiration be sliding? A sliding expiration is reinitialized at each get. Defaults to `false`.

To configure cache regions with `SqlCacheDependencies` a `syscache2` config section must be defined in the application's configuration file. See the sample below.

Example 27.2.

```
<configSections>
  <section name="syscache2"
    type="NHibernate.Caches.SysCache2.SysCacheSection, NHibernate.Caches.SysCache2"/>
</configSections>
```

27.4.1. Table-based Dependency

A table-based dependency will monitor the data in a database table for changes. Table-based dependencies are generally used for a SQL Server 2000 database but will work with SQL Server 2005 or superior as well. Before you can use SQL Server cache invalidation with table based dependencies, you need to enable notifications for the database. This task is performed with the **aspnet_regsql** command. With table-based notifications, the application will poll the database for changes at a predefined interval. A cache region will not be invalidated immediately when data in the table changes. The cache will be invalidated the next time the application polls the database for changes.

To configure the data in a cache region to be invalidated when data in an underlying table is changed, a cache region must be configured in the application's configuration file. See the sample below.

Example 27.3.

```
<syscache2>
  <cacheRegion name="Product">
    <dependencies>
      <tables>
        <add name="price"
          databaseEntryName="Default"
          tableName="VideoTitle" />
      </tables>
    </dependencies>
  </cacheRegion>
</syscache2>
```

Table-based Dependency Configuration Properties

name

Unique name for the dependency

tableName

The name of the database table that the dependency is associated with. The table must be enabled for notification support with the `AspNet_SqlCacheRegisterTableStoredProcedure`.

databaseEntryName

The name of a database defined in the `databases` element for `sqlCacheDependency` for caching (ASP.NET Settings Schema) element of the application's `web.config` file.

27.4.2. Command-Based Dependencies

A command-based dependency will use a SQL command to identify records to monitor for data changes. Command-based dependencies work only with SQL Server 2005.

Before you can use SQL Server cache invalidation with command-based dependencies, you need to enable the

Service Broker for query notifications. The application must also start the listener for receiving change notifications from SQL Server. With command based notifications, SQL Server will notify the application when the data of a record returned in the results of a SQL query has changed. Note that a change will be indicated if the data in any of the columns of a record change, not just the columns returned by a query. The query is a way to limit the number of records monitored for changes, not the columns. As soon as data in one of the records is modified, the data in the cache region will be invalidated immediately.

To configure the data in a cache region to be invalidated based on a SQL command, a cache region must be configured in the application's configuration file. See the samples below.

Example 27.4. Stored Procedure

```
<cacheRegion name="Product" priority="High" >
  <dependencies>
    <commands>
      <add name="price"
        command="ActiveProductsStoredProcedure"
        isStoredProcedure="true"/>
    </commands>
  </dependencies>
</cacheRegion>
```

Example 27.5. SELECT Statement

```
<cacheRegion name="Product" priority="High">
  <dependencies>
    <commands>
      <add name="price"
        command="Select VideoTitleId from dbo.VideoTitle where Active = 1"
        connectionName="default"
        connectionStringProviderType="NHibernate.Caches.SysCache2.ConfigConnectionStringProvider, NHil
      </commands>
    </dependencies>
  </cacheRegion>
```

Command Configuration Properties

name

Unique name for the dependency

command (required)

SQL command that returns results which should be monitored for data changes

isStoredProcedure (optional)

Indicates if command is a stored procedure. The default is false.

connectionName (optional)

The name of the connection in the applications configuration file to use for registering the cache dependency for change notifications. If no value is supplied for connectionName or connectionStringProviderType, the connection properties from the NHibernate configuration will be used.

connectionStringProviderType (optional)

IConnectionStringProvider to use for retrieving the connection string to use for registering the cache de-

pendency for change notifications. If no value is supplied for `connectionName`, the unnamed connection supplied by the provider will be used.

27.4.3. Aggregate Dependencies

Multiple cache dependencies can be specified. If any of the dependencies triggers a change notification, the data in the cache region will be invalidated. See the samples below.

Example 27.6. Multiple commands

```
<cacheRegion name="Product">
  <dependencies>
    <commands>
      <add name="price"
        command="ActiveProductsStoredProcedure"
        isStoredProcedure="true"/>
      <add name="quantity"
        command="Select quantityAvailable from dbo.VideoAvailability"/>
    </commands>
  </dependencies>
</cacheRegion>
```

Example 27.7. Mixed

```
<cacheRegion name="Product">
  <dependencies>
    <commands>
      <add name="price"
        command="ActiveProductsStoredProcedure"
        isStoredProcedure="true"/>
    </commands>
    <tables>
      <add name="quantity"
        databaseEntryName="Default"
        tableName=" VideoAvailability" />
    </tables>
  </dependencies>
</cacheRegion>
```

27.4.4. Additional Settings

In addition to data dependencies for the cache regions, time based expiration policies can be specified for each item added to the cache. Time based expiration policies will not invalidate the data dependencies for the whole cache region, but serve as a way to remove items from the cache after they have been in the cache for a specified amount of time. See the samples below.

Example 27.8. Relative Expiration

```
<cacheRegion name="Product" relativeExpiration="300" priority="High" useSlidingExpiration="true" />
```

Example 27.9. Time of Day Expiration

```
<cacheRegion name="Product" timeOfDayExpiration="2:00:00" priority="High" />
```

Additional Configuration Properties

relativeExpiration

Number of seconds that an individual item will exist in the cache before being removed. Defaults to 300 if neither `relativeExpiration` nor `timeOfDayExpiration` are defined, and if no expiration settings are defined in NHibernate configuration.

useSlidingExpiration

Should the expiration be sliding? A sliding expiration is reinitialized at each get. Defaults to `false` if not defined in NHibernate configuration.

timeOfDayExpiration

24 hour based time of day that an item will exist in the cache until. 12am is specified as 00:00:00; 10pm is specified as 22:00:00. Only valid if `relativeExpiration` is not specified. Time of Day Expiration is useful for scenarios where items should be expired from the cache after a daily process completes.

priority

`System.Web.Caching.CacheItemPriority` that identifies the relative priority of items stored in the cache.

27.5. EnyimMemcached Configuration

Its configuration relies on the EnyimMemcached library own configuration, through its enyim.com/memcached configuration section. See project site [<https://github.com/enyim/EnyimMemcached>]. This is a .Net Framework only library.

27.6. RtMemoryCache Configuration

`RtMemoryCache` relies on `System.Runtime.Caching.MemoryCache` for the underlying implementation.

The following NHibernate configuration settings are available:

cache.default_expiration

Number of seconds to wait before expiring each item. Defaults to 300. It can also be set programmatically on the NHibernate configuration object under the name `expiration`, which then takes precedence over `cache.default_expiration`.

cache.use_sliding_expiration

Should the expiration be sliding? A sliding expiration is reinitialized at each get. Defaults to `false`.

`RtMemoryCache` has a config file section handler to allow configuring different expirations for different regions. Here is an example:

Example 27.10.

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <configSections>
```

```

<section name="rtmemorycache"
  type="NHibernate.Caches.RtMemoryCache.RtMemoryCacheSectionHandler,NHibernate.Caches.RtMemoryCache" />
</configSections>

<rtmemorycache>
  <cache region="foo" expiration="500" />
  <cache region="bar" expiration="300" sliding="true" />
</rtmemorycache>
</configuration>

```

The loading of this section can be customized with the `NHibernate.Caches.RtMemoryCache.ConfigurationProvider` class. See Section 27.2, “Using a custom configuration provider”.

27.7. NHibernate.Caches.StackExchangeRedis Configuration

`NHibernate.Caches.StackExchangeRedis` relies on `StackExchange.Redis` for the underlying implementation.

The following `NHibernate` configuration settings are available (also defined in `NHibernate.Caches.StackExchangeRedis.RedisEnvironment`):

`cache.default_expiration`

Number of seconds to wait before expiring each item. Defaults to 300. It can also be set programmatically on the `NHibernate` configuration object under the name `expiration`, which then takes precedence over `cache.default_expiration`.

`cache.use_sliding_expiration`

Should the expiration be sliding? A sliding expiration is reinitialized at each get. Can be overridden for each region by using `sliding` attribute. Defaults to `false`.

`cache.database`

The default Redis database index, that can be overridden for each region by using `database` attribute. Defaults to -1.

`cache.strategy`

The assembly qualified name of the region strategy, that can be overridden for each region by using `strategy` attribute. `NHibernate.Caches.StackExchangeRedis` provides the following strategies:

- `NHibernate.Caches.StackExchangeRedis.DefaultRegionStrategy`

Uses a special key that contains the region current version number which is appended after the region prefix. Each time a clear operation is performed the version number is increased and an event is sent to all clients so that they can update their local versions. Even if the event was not sent to all clients, each operation has a version check in order to prevent working with stale data. This strategy has additional settings:

<varlistentry>cache.region_strategy.default.max_allowed_version The max allowed version number. When the max value is reached, the next value will be reset to zero. Defaults to 10000.</varlistentry>

<varlistentry>cache.region_strategy.default.use_pubsub Whether to use Redis pub/sub mechanism in order to notify other cache instances when the clear operation was performed. Defaults to true.</varlistentry>

<varlistentry>cache.region_strategy.default.retry_times Total retry times for read and lock oper-

ations, when concurrent clear operations are performed. Defaults to 1. </varlistentry>

- `NHibernate.Caches.StackExchangeRedis.FastRegionStrategy`

Uses very simple read/write operations but does not support `ICache.Clear` operation.

- `NHibernate.Caches.StackExchangeRedis.TwoLayerCacheRegionStrategy`

Extends `NHibernate.Caches.StackExchangeRedis.DefaultRegionStrategy` and uses an additional local memory cache for faster readings. The local caches are invalidated by using Redis pub/sub mechanism. This strategy should be used only for regions that have few write operations and a high expiration time.

In order to use this strategy a custom `ICacheRegionStrategyFactory` has to be provided (see `cache.region_strategy_factory` setting), where the strategy is created with a custom `RegionMemoryCacheBase` implementation. See the source of `NHibernate.Caches.StackExchangeRedis.Tests.CacheRegionStrategyFactory` for an example.

This strategy inherits additional settings from `DefaultRegionStrategy` and also has its own settings:

<varlistentry>`cache.region_strategy.two_layer_cache.use_pipelining` Whether to use `StackExchange.Redis` pipelining feature. Defaults to false. </varlistentry>

<varlistentry>`cache.region_strategy.two_layer_cache.client_id` The client id used for cache invalidation. Defaults to a random number. </varlistentry>

<varlistentry>`cache.region_strategy.two_layer_cache.max_synchronization_time` The max synchronization time between caches in seconds. Defaults to 10. </varlistentry>

- `NHibernate.Caches.StackExchangeRedis.FastTwoLayerCacheRegionStrategy`

Extends `NHibernate.Caches.StackExchangeRedis.FastRegionStrategy` and uses an additional local memory cache for faster readings. The local caches are invalidated by using Redis pub/sub mechanism. This strategy does not support `ICache.Clear` operation and should be used only for regions that have few write operations and a high expiration time.

In order to use this strategy a custom `ICacheRegionStrategyFactory` has to be provided (see `cache.region_strategy_factory` setting), where the strategy is created with a custom `RegionMemoryCacheBase` implementation. See the source of `NHibernate.Caches.StackExchangeRedis.Tests.CacheRegionStrategyFactory` for an example.

This strategy has additional settings:

<varlistentry>`cache.region_strategy.fast_two_layer_cache.use_pipelining` Whether to use `StackExchange.Redis` pipelining feature. Defaults to false. </varlistentry>

<varlistentry>`cache.region_strategy.fast_two_layer_cache.client_id` The client id used for cache invalidation. Defaults to a random number. </varlistentry>

<varlistentry>`cache.region_strategy.fast_two_layer_cache.max_synchronization_time` The max synchronization time between caches in seconds. Defaults to 10. </varlistentry>

- `NHibernate.Caches.StackExchangeRedis.DistributedLocalCacheRegionStrategy`

Uses only a memory cache to store the values and uses Redis pub/sub mechanism to synchronize data between other local caches. The synchronization between caches is done by comparing the `DateTime.Ticks`, which represent when the operation was performed. When two operations have the same `DateTime.Ticks`, then the client with the highest id wins. This strategy should be used only for regions that have few write operations and a high expiration time. It is recommended to use `NHibernate.Caches.StackExchangeRedis.TwoLayerCacheRegionStrategy`, when the instances where the

strategy would run are often restarted/recycled.

In order to use this strategy a custom `ICacheRegionStrategyFactory` has to be provided (see `cache.region_strategy_factory` setting), where the strategy is created with a custom `RegionMemoryCacheBase` implementation. See the source of `NHibernate.Caches.StackExchangeRedis.Tests.CacheRegionStrategyFactory` for an example.

This strategy has additional settings:

`<varlistentry>cache.region_strategy.distributed_local_cache.use_pipelining` Whether to use `StackExchange.Redis` pipelining feature. Defaults to `false`. `</varlistentry>`

`<varlistentry>cache.region_strategy.distributed_local_cache.client_id` The client id used for cache invalidation. Defaults to a random number. `</varlistentry>`

`<varlistentry>cache.region_strategy.distributed_local_cache.max_synchronization_time` The max synchronization time between caches in seconds. Defaults to `10`. `</varlistentry>`

Defaults to `NHibernate.Caches.StackExchangeRedis.DefaultRegionStrategy`.

`cache.append_hashcode`

Whether the hash code of the key should be added to the cache key. Can be overridden for each region by using `append-hashcode` attribute. Defaults to `false`.

`cache.key_prefix`

The prefix that will be prepended before each cache key in order to avoid having collisions when multiple clients uses the same Redis database. Defaults to `NHibernate-Cache:`.

`cache.environment_name`

The name of the environment that will be prepended before each cache key in order to allow having multiple environments on the same Redis database. Defaults to `null`.

`cache.serializer`

The assembly qualified name of the serializer that is used to serialize/deserialize the key values. Optionally, a faster json serializer can be used by installing `NHibernate.Caches.Util.JsonSerializer` package and setting `NHibernate.Caches.Util.JsonSerializer.JsonCacheSerializer`, `NHibernate.Caches.Util.JsonSerializer` value instead. Defaults to `NHibernate.Caches.Common.BinaryCacheSerializer`, `NHibernate.Caches.Common`.

`cache.region_strategy_factory`

The assembly qualified name of the region strategy factory. Defaults to `NHibernate.Caches.StackExchangeRedis.DefaultCacheRegionStrategyFactory`.

`cache.connection_multiplexer_provider`

The assembly qualified name of the connection multiplexer provider. Defaults to `NHibernate.Caches.StackExchangeRedis.DefaultConnectionMultiplexerProvider`.

`cache.database_provider`

The assembly qualified name of the database provider. Defaults to `NHibernate.Caches.StackExchangeRedis.DefaultDatabaseProvider`.

`cache.lock.key_timeout`

The timeout for a lock key to expire in seconds. Defaults to `5`.

`cache.lock.acquire_timeout`

The time limit to acquire the lock in seconds. Defaults to `5`.

`cache.lock.retry_times`

The number of retries for acquiring the lock. Defaults to 3.

`cache.lock.max_retry_delay`

The maximum delay before retrying to acquire the lock in milliseconds. Defaults to 400.

`cache.lock.min_retry_delay`

The minimum delay before retrying to acquire the lock in milliseconds. Defaults to 10.

`cache.lock.value_provider`

The assembly qualified name of the lock value provider. Defaults to `NHibernate.Caches.StackExchangeRedis.DefaultCacheLockValueProvider`.

`cache.lock.retry_delay_provider`

The assembly qualified name of the lock retry delay provider. Defaults to `NHibernate.Caches.StackExchangeRedis.DefaultCacheLockRetryDelayProvider`.

`cache.lock.key_suffix`

The suffix for the lock key. Defaults to `:lock`.

`NHibernate.Caches.StackExchangeRedis` has a config file section handler to allow configuring different expirations for different regions. Here is an example:

Example 27.11.

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <configSections>
    <section name="redis"
      type="NHibernate.Caches.StackExchangeRedis.RedisSectionHandler, NHibernate.Caches.StackExchangeRedis" />
  </configSections>

  <redis>
    <cache region="foo" expiration="500" database="1" />
    <cache region="bar" sliding="true" append-hashcode="true" />
    <cache region="baz"
      strategy="NHibernate.Caches.StackExchangeRedis.FastRegionStrategy, NHibernate.Caches" />
  </redis>
</configuration>
```

The loading of this section can be customized with the `NHibernate.Caches.StackExchangeRedis.ConfigurationProvider` class. See Section 27.2, “Using a custom configuration provider”.

27.8. CoreMemoryCache Configuration

`CoreMemoryCache` relies on `Microsoft.Extensions.Caching.Memory.MemoryCache` for the underlying implementation.

The following `NHibernate` configuration settings are available:

`cache.default_expiration`

Number of seconds to wait before expiring each item. Defaults to 300. It can also be set programmatically on the `NHibernate` configuration object under the name `expiration`, which then takes precedence over `cache.default_expiration`.

cache.use_sliding_expiration

Should the expiration be sliding? A sliding expiration is reinitialized at each get. Defaults to false.

CoreMemoryCache has a config file section handler to allow configuring different expirations for different regions, and configuring the MemoryCache expiration scan frequency. Here is an example:

Example 27.12.

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <configSections>
    <section name="corememorycache"
      type="NHibernate.Caches.CoreMemoryCache.CoreMemoryCacheSectionHandler,NHibernate.Caches.CoreMem
    />
  </configSections>

  <corememorycache expiration-scan-frequency="00:05:00">
    <cache region="foo" expiration="500" />
    <cache region="bar" expiration="300" sliding="true" />
  </corememorycache>
</configuration>
```

The loading of this section can be customized with the `NHibernate.Caches.CoreMemoryCache.ConfigurationProvider` class. See Section 27.2, “Using a custom configuration provider”.

27.9. CoreDistributedCache Configuration

CoreDistributedCache relies on `Microsoft.Extensions.Caching.Abstractions.IDistributedCache` implementations. The implementation has to be provided through an `IDistributedCacheFactory`, either supplied through configuration or programmatically by affecting `CoreDistributedCacheProvider.CacheFactory` before building a session factory.

The following NHibernate configuration settings are available:

cache.default_expiration

Number of seconds to wait before expiring each item. Defaults to 300. It can also be set programmatically on the NHibernate configuration object under the name `expiration`, which then takes precedence over `cache.default_expiration`.

cache.use_sliding_expiration

Should the expiration be sliding? A sliding expiration is reinitialized at each get. Defaults to false.

cache.serializer

The assembly qualified name of the serializer that is used to serialize/deserialize the key values. Optionally, a faster json serializer can be used by installing `NHibernate.Caches.Util.JsonSerializer` package and setting `NHibernate.Caches.Util.JsonSerializer.JsonCacheSerializer`, `NHibernate.Caches.Util.JsonSerializer` value instead. Defaults to `NHibernate.Caches.Common.BinaryCacheSerializer`, `NHibernate.Caches.Common`. This setting is available since the 5.7 version of `CoreDistributedCache`.

CoreDistributedCache has a config file section handler to allow configuring different expirations for different regions, configuring the `IDistributedCacheFactory` to use, and configuring additional properties specific to

the chosen `IDistributedCache` implementation. Here is an example:

Example 27.13.

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <configSections>
    <section name="coredistributedcache"
      type="NHibernate.Caches.CoreDistributedCache.CoreDistributedCacheSectionHandler,
        NHibernate.Caches.CoreDistributedCache" />
  </configSections>

  <coredistributedcache
    factory-class="NHibernate.Caches.CoreDistributedCache.Memory.MemoryFactory,
      NHibernate.Caches.CoreDistributedCache.Memory">
    <properties>
      <property name="expiration-scan-frequency">00:10:00</property>
      <property name="size-limit">1048576</property>
      <property name="cache.serializer"
        >NHibernate.Caches.Util.JsonSerializer.JsonCacheSerializer, NHibernate.Caches.Util.JsonSeriali
      </property>
    </properties>
    <cache region="foo" expiration="500" sliding="true" />
    <cache region="noExplicitExpiration" sliding="true" />
    <cache region="specificSerializer"
      serializer="NHibernate.Caches.Common.BinaryCacheSerializer, NHibernate.Caches.Common" />
    </coredistributedcache>
  </configuration>
```

The loading of this section can be customized with the `NHibernate.Caches.CoreDistributedCache.ConfigurationProvider` class. See Section 27.2, “Using a custom configuration provider”.

`CoreDistributedCache` does not support `NHibernate.Cache.ICache.Clear`. Clearing the `NHibernate` cache has no effects with `CoreDistributedCache`.

27.9.1. Memcached distributed cache factory

`NHibernate.Caches.CoreDistributedCache.Memcached` provides a `Memcached` distributed cache factory. This factory yields a `Enyim.Caching.MemcachedClient` from `EnyimMemcachedCore`. For using it, reference the cache factory package and set the `factory-class` attribute of the `coredistributedcache` configuration section to `NHibernate.Caches.CoreDistributedCache.Memcached.MemcachedFactory`, `NHibernate.Caches.CoreDistributedCache.Memcached`.

`Memcached` does not support sliding expirations. `cache.use_sliding_expiration` setting or `sliding` region setting do not have any effect with `Memcached`.

The following additional properties can be configured:

configuration

The JSON configuration of `EnyimMemcachedCore`, see its project website [<https://github.com/cnblogs/EnyimMemcachedCore>]. It has to be structured like the value part of the "enyimMemcached" property in an `appsettings.json` file.

Example 27.14.

```
{
```

```
"Servers": [  
  {  
    "Address": "localhost",  
    "Port": 11211  
  }  
]
```

27.9.2. Redis distributed cache factory

`NHibernate.Caches.CoreDistributedCache.Redis` provides a Redis distributed cache factory. This factory yields a `Microsoft.Extensions.Caching.Redis.RedisCache`. For using it, reference the cache factory package and set the `factory-class` attribute of the `coredistributedcache` configuration section to `NHibernate.Caches.CoreDistributedCache.Redis.RedisFactory`, `NHibernate.Caches.CoreDistributedCache.Redis`.

The following additional properties can be configured:

`configuration`

Its value will be used to set the `Configuration` property of the `RedisCache` options (`RedisCacheOptions`).

`instance-name`

Its value will be used to set the `InstanceName` property of the `RedisCache` options (`RedisCacheOptions`).

27.9.3. SQL Server distributed cache factory

`NHibernate.Caches.CoreDistributedCache.SqlServer` provides a SQL Server distributed cache factory. This factory yields a `Microsoft.Extensions.Caching.SqlServer.SqlServerCache`. For using it, reference the cache factory package and set the `factory-class` attribute of the `coredistributedcache` configuration section to `NHibernate.Caches.CoreDistributedCache.SqlServer.SqlServerFactory`, `NHibernate.Caches.CoreDistributedCache.SqlServer`.

The following additional properties can be configured:

`connection-string`

Its value will be used to set the `ConnectionString` property of the `SqlServerCache` options (`SqlServerCacheOptions`).

`schema-name`

Its value will be used to set the `SchemaName` property of the `SqlServerCache` options (`SqlServerCacheOptions`).

`table-name`

Its value will be used to set the `TableName` property of the `SqlServerCache` options (`SqlServerCacheOptions`).

`expired-items-deletion-interval`

Its value will be used to set the `ExpiredItemsDeletionInterval` property of the `SqlServerCache` options (`SqlServerCacheOptions`). It can be provided either as an integer being a number of minutes or as a `TimeSpan` string representation.

27.9.4. Memory distributed cache factory

`NHibernate.Caches.CoreDistributedCache.Memory` provides a memory "distributed" cache factory. This factory yields a `Microsoft.Extensions.Caching.Memory.MemoryDistributedCache`. For using it, reference the cache factory package and set the `factory-class` attribute of the `coredistributedcache` configuration section to `NHibernate.Caches.CoreDistributedCache.Memory.MemoryFactory`, `NHibernate.Caches.CoreDistributedCache.Memory`.

As implied by its name, this cache is not actually distributed. It is meant for testing purpose. For other usages, consider using another memory cache provider, like `CoreMemoryCache`. Due to the distributed cache implementation, using the `MemoryDistributedCache` has some drawbacks compared to most other memory cache providers: it will serialize cached objects, incurring some overhead; it does not support clearing the cache. But due to the serialization of cached objects, it is able of computing its consumed memory size, thus the availability of the `SizeLimit` option.

The following additional properties can be configured:

expiration-scan-frequency

Its value will be used to set the `ExpirationScanFrequency` property of the `MemoryDistributedCache` options (`MemoryDistributedCacheOptions`). It can be provided either as an integer being a number of minutes or as a `TimeSpan` string representation.

size-limit

Its value will be used to set the `SizeLimit` property of the `MemoryDistributedCache` options (`MemoryDistributedCacheOptions`). Its value is an integer, representing the maximal bytes count to be stored in the cache.

Chapter 28. NHibernate.Mapping.Attributes

What is NHibernate.Mapping.Attributes?

NHibernate.Mapping.Attributes is an add-in for NHibernate [<https://nhibernate.info/>] contributed by **Pierre Henri Kuate** (aka *KPixel*); the former implementation was made by **John Morris**. NHibernate require mapping streams to bind your domain model to your database. Usually, they are written (and maintained) in separated hbm.xml files.

With NHibernate.Mapping.Attributes, you can use .NET attributes to decorate your entities and these attributes will be used to generate these mapping .hbm.xml (as files or streams). So you will no longer have to bother with these *nasty* xml files ;).

Content of this library project [<https://github.com/nhibernate/NHibernate.Mapping.Attributes>]:

1. *NHibernate.Mapping.Attributes*: that is the only assembly you need (as an end-user).
2. *Test*: a working sample using attributes and HbmSerializer for a NUnit TestFixture.
3. *Generator*: the program used to generate the attributes and the HbmWriter of the end-user assembly.
4. *Refly* [<http://mbunit.tigris.org/>]: thanks to Jonathan de Halleux [<http://www.dotnetwiki.org/>] for this library which makes it so easy to generate code.

Important

This library is generated using the file `/src/NHibernate.Mapping.Attributes/nhibernate-mapping.xsd` (which is embedded in the assembly to be able to validate generated XML streams). As this file can change at each new release of NHibernate, a new release of NHibernate.Mapping.Attributes should be regenerated before using it with a different version. It can be done by opening the Generator solution, compiling and running the Generator project.

28.1. What's new?

1. It is possible to import classes by simply decorating them with `[Import] class ImportedClass1 {}`. Note that you must use `HbmSerializer.Serialize(assembly)`; The `<import/>` mapping will be added before the classes mapping. If you prefer to keep these imports in the class using them, you can specify them all on the class: `[Import(ClassType=typeof(ImportedClass1))] class Query {}`.
2. `[RawXmlAttribute]` is a new attribute allowing to insert xml as-is in the mapping. This feature can be very useful to do complex mapping (eg: components). It may also be used to quickly move the mapping from xml files to attributes. Usage: `[RawXml(After=typeof(ComponentAttribute), Content="<component name= \"...\">...</component>")]`. `After` tells after which kind of mapping the xml should be inserted (generally, it is the type of the mapping you are inserting); it is optional (in which case the xml is inserted on the top of the mapping). Note: At the moment, all raw xmls are prefixed by a `<! - - - ->` (in the generated stream); this is a known side-effect.
3. `[AttributeIdentifierAttribute]` is a new attribute allowing to provide the value of a defined "place

holder". Eg:

```
public class Base {
    [Id(..., Column="{{Id.Column}}")]
    [AttributeIdentifier(Name="Id.Column", Value="ID")] // Default value
    public int Id { ... }
}

[AttributeIdentifier(Name="Id.Column", Value="SUB_ID")]
[Class]
public class MappedSubClass : Base { }
```

The idea is that, when you have a mapping which is shared by many subclasses but which has minor differences (like different column names), you can put the mapping in the base class with place holders on these fields and give their values in subclasses. Note that this is possible for any mapping field taking a string (column, name, type, access, etc.). And, instead of Value, you can use ValueType or ValueObject (if you use an enum, you can control its formatting with ValueObject).

The "place holder" is defined like this: {{XXX}}. If you don't want to use these double curly brackets, you can change them using the properties StartQuote and EndQuote of the class HbmWriter.

4. It is possible to register patterns (using Regular Expressions) to automatically transform fully qualified names of properties types into something else. Eg: HbmSerializer.Default.HbmWriter.Patterns.Add(@"Namespace\.(\S+), Assembly", "\$1"); will map all properties with a not-qualified type name.
5. Two methods have been added to the HbmSerializer class, allowing generating mappings from a type or an assembly: HbmSerializer.Default.Serialize(typeof(XXX)) and HbmSerializer.Default.Serialize(typeof(XXX).Assembly). So it is no longer required to create a MemoryStream for these simple cases. The output of these call can be directly added to your NHibernate Configuration instance: cfg.AddInputStream(HbmSerializer.Default.Serialize(typeof(XXX))).
6. Two WriteUserDefinedContent() methods have been added to HbmWriter. They improve the extensibility of this library; it is now very easy to create a .NET attribute and integrate it in the mapping.
7. Attributes [(Jcs)Cache], [Discriminator] and [Key] can be specified at class-level.
8. Interfaces can be mapped (just like classes and structs).
9. A notable "bug" fix is the re-ordering of (joined-)subclasses. This operation may be required when a subclass extends another subclass. In this case, the extended class mapping must come before the extending class mapping. Note that the re-ordering takes place only for "top-level" classes (that is not nested in other mapped classes). Anyway, it is quite unusual to put an interdependent mapped subclasses in a mapped class.
10. There are also many other little changes: refer to the release notes for more details.

28.2. How to use it?

The *end-user class* is `NHibernate.Mapping.Attributes.HbmSerializer`. This class *serialize* your domain model to mapping streams. You can either serialize classes one by one, or serialize a whole assembly. Look at `NHibernate.Mapping.Attributes.Test` project for a working sample.

The first step is to decorate your entities with attributes. You can use: `[Class]`, `[Subclass]`, `[JoinedSubclass]` or `[Component]`. Then, you decorate your members (fields/properties); they can take as

many attributes as required by your mapping. Eg:

```
[NHibernate.Mapping.Attributes.Class]
public class Example
{
    [NHibernate.Mapping.Attributes.Property]
    public string Name;
}
```

After this step, you use `NHibernate.Mapping.Attributes.HbmSerializer` (here, we use its `Default` property, which is an instance you can use if you don't need/want to create it yourself):

```
var cfg = new NHibernate.Cfg.Configuration();
cfg.Configure();
// Enable validation (optional)
HbmSerializer.Default.Validate = true;
// Here, we serialize all decorated classes (but you can also do it class by class)
cfg.AddInputStream(HbmSerializer.Default.Serialize(
    System.Reflection.Assembly.GetExecutingAssembly()));
// Now you can use this configuration to build your SessionFactory...
```

Note

As you can see here, `NHibernate.Mapping.Attributes` is *not* (really) intrusive. Setting attributes on your objects doesn't force you to use them with `NHibernate` and doesn't break any constraint on your architecture. Attributes are purely informative (like documentation)!

28.3. Tips

1. In production, you may want to generate a XML mapping file from `NHibernate.Mapping.Attributes` and use this file each time the `SessionFactory` need to be built. Use: `HbmSerializer.Default.Serialize(typeof(XXX).Assembly, "DomainModel.hbm.xml");`. It is slightly faster.
2. Use `HbmSerializer.Validate` to enable/disable the validation of generated xml streams (against `NHibernate` mapping schema). This is useful to quickly find errors. (They are written in the `StringBuilder` property `HbmSerializer.Error`.) If the error is due to this library, then see if it is a known issue and report it. You are welcome to contribute a solution if you solve the trouble :).
3. Your classes, fields and properties (members) can be private; just make sure that you have the permission to access private members using reflection (`ReflectionPermissionFlag.MemberAccess`).
4. Members of a mapped classes are also seek in its base classes (until we reach *mapped* base class). So you can decorate some members of a (not mapped) base class and use it in its (mapped) sub class(es).
5. For a `Name` taking a `System.Type`, set the type with `Name="xxx"` (as string) or `NameType=typeof(XXX);` (add "Type" to "Name")
6. By default, .NET attributes don't keep the order of attributes; so you need to set it yourself when the order matter (using the first parameter of each attribute); it is *highly* recommended to set it when you have more than one attribute on the same member.
7. As long as there is no ambiguity, you can decorate a member with many unrelated attributes. A good example is to put class-related attributes (like `<discriminator>`) on the identifier member. But don't forget that the order matters (the `<discriminator>` must be after the `<id>`). The order to use comes from the order of elements in the `NHibernate` mapping schema. Personally, I prefer using negative numbers for these

attributes (if they come first!).

8. You can add [HibernateMapping] on your classes to specify <hibernate-mapping> attributes (used when serializing the class in its stream). You can also use HbmSerializer.Hbm* properties (used when serializing an assembly or a type that is not decorated with [HibernateMapping]).
9. Instead of using a string for DiscriminatorValue (in [Class] and [Subclass]), you can use any object you want. Example:

```
[Subclass(DiscriminatorValueEnumFormat="d", DiscriminatorValueObject=DiscEnum.Val1)]
```

Here, the object is an Enum, and you can set the format you want (the default value is "g"). Note that you must put it *before*! For others types, It simply use the ToString() method of the object.

10. Each stream generated by NHibernate.Mapping.Attributes can contain a comment with the date of the generation; You may enable/disable this by using the property HbmSerializer.WriteDateComment.
11. If you forget to provide a required xml attribute, it will obviously throw an exception while generating the mapping.
12. The recommended and easiest way to map [Component] is to use [ComponentProperty]. The first step is to put [Component] on the component class and map its fields/properties. Note that you shouldn't set the Name in [Component]. Then, on each member in your classes, add [ComponentProperty]. But you can't override Access, Update or Insert for each member.

There is a working example in *NHibernate.Mapping.Attributes.Test* (look for the class CompAddress and its usage in others classes).

13. Another way to map [Component] is to use the way this library works: If a mapped class contains a mapped component, then this component will be include in the class. *NHibernate.Mapping.Attributes.Test* contains the classes JoinedBaz and Stuff which both use the component Address.

Basically, it is done by adding

```
[Component(Name = "MyComp")] private class SubComp : Comp {}
```

in each class. One of the advantages is that you can override Access, Update or Insert for each member. But you have to add the component subclass in *each* class (and it can not be inherited). Another advantage is that you can use [AttributeIdentifier].

14. Finally, whenever you think that it is easier to write the mapping in XML (this is often the case for [Component]), you can use [RawXml].
15. **About customization.** HbmSerializer uses HbmWriter to serialize each kind of attributes. Their methods are virtual; so you can create a subclass and override any method you want (to change its default behavior).

Use the property HbmSerializer.HbmWriter to change the writer used (you may set a subclass of HbmWriter).

Example using some of these tips: (0, 1 and 2 are position indexes)

```
// Don't put it after [ManyToOne] !!!
[NHibernate.Mapping.Attributes.Id(0, TypeType=typeof(int))]
[NHibernate.Mapping.Attributes.Generator(1, Class="uuid.hex")]
[NHibernate.Mapping.Attributes.ManyToOne(2,
    ClassType=typeof(Foo), OuterJoin=OuterJoinStrategy.True)]
private Foo Entity;
```

Generates:

```
<id type="Int32">
  <generator class="uuid.hex" />
</id>
<many-to-one name="Entity" class="Namespaces.Foo, SampleAssembly" outer-join="true" />
```

28.4. Known issues and TODOs

First, read TODOs in the source code ;)

A `Position` property has been added to all attributes to order them. But there is still a problem:

When a parent element "p" has a child element "x" that is also the child element of another child element "c" of "p" (preceding "x") :D Illustration:

```
<p>
  <c>
    <x />
  </c>
  <x />
</p>
```

In this case, when writing:

```
[Attributes.P(0)]
[Attributes.C(1)]
    [Attributes.X(2)]
[Attributes.X(3)]
public MyType MyProperty;
```

`X(3)` will always belong to `C(1)` ! (as `X(2)`).

It is the case for `<dynamic-component>` and `<nested-composite-element>`.

Another bad news is that, currently, XML elements coming after this elements can not be included in them. Eg: There is no way put a collection in `<dynamic-component>`. The reason is that the file `nhibernate-mapping.xsd` tells how elements are built and in which order, and `NHibernate.Mapping.Attributes` use this order.

Anyway, the solution would be to add a `int ParentNode` property to `BaseAttribute` so that you can create a real graph...

For now, you can fallback on `[RawXml]`.

Actually, there is no other know issue nor planned modification. This library should be stable and complete. But if you find a bug or think of an useful improvement, contact us!

As a side note, it would be nice to write a better `TestFixture` than `NHibernate.Mapping.Attributes.Test` :D.

28.5. Developer Notes

Any change to the schema (`nhibernate-mapping.xsd`) implies:

1. Checking if there is any change to do in the Generator (like updating `KnowEnums` / `AllowMultipleValue` / `IsRoot` / `IsSystemType` / `IsSystemEnum` / `CanContainItself`).

2. Updating `/src/NHibernate.Mapping.Attributes/nhibernate-mapping.xsd` (copy/paste) and running the Generator again (even if it wasn't modified).
3. Running the Test project and make sure that no exception is thrown. A class/property should be modified/added in this project to be sure that any new breaking change will be caught (=> update the reference `hbm.xml` files and/or the project `NHibernate.Mapping.Attributes.csproj`).

This implementation is based on NHibernate mapping schema. So there is probably lot of "standard schema features" that are not supported...

The version of `NHibernate.Mapping.Attributes` should be the version of the NHibernate schema used to generate it (=> the version of NHibernate library).

In the design of this project, performance is a (very) minor goal :). Easier implementation and maintenance are far more important because you can use this library to generate statically the mapping files and use them instead in production. (Cf. the first tip in Section 28.3, "Tips".)